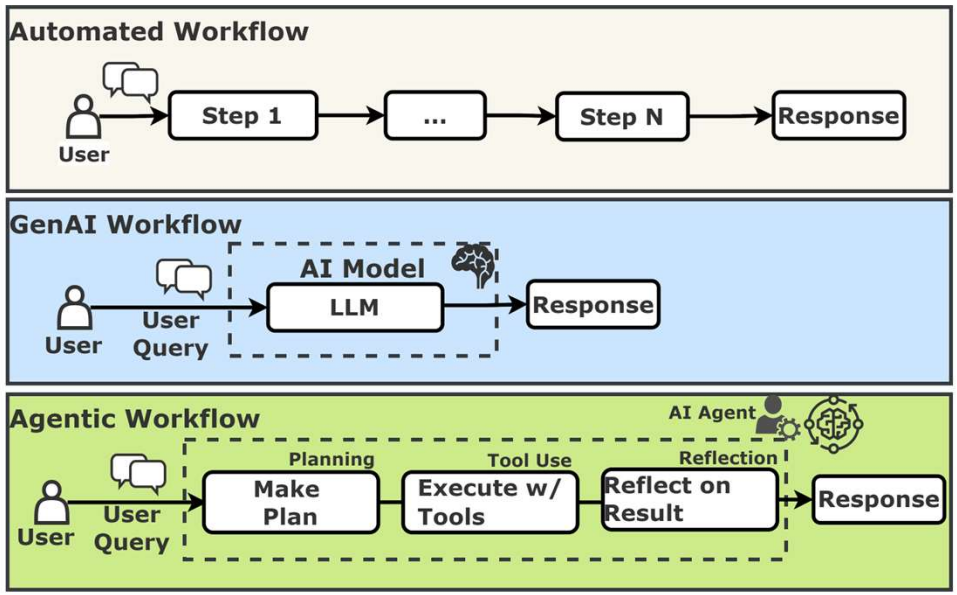
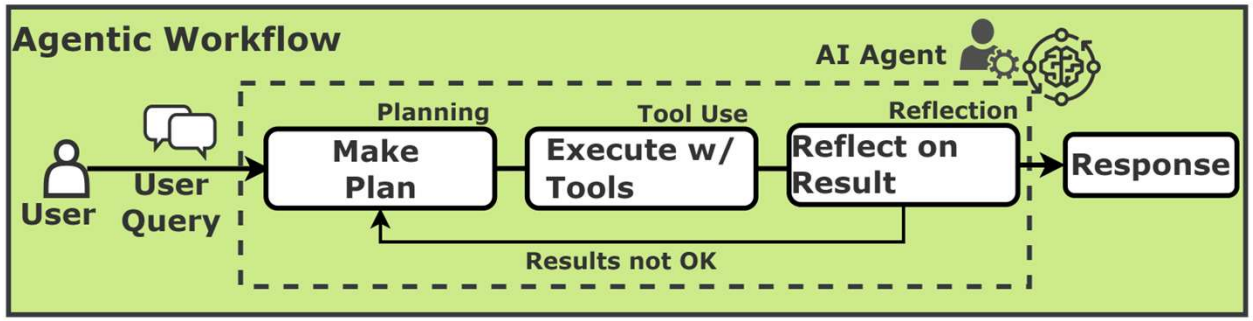
Summary of Workflows

Traditional Workflow:
A static "Flowchart"
Deterministic, rigid and brittle

Agentic Workflow:
An adaptive "Project Manager"
Stateful, LLM-driven, and non-deterministic
It builds the flowchart at runtime



This is the architecture for building complex, reliable AI applications

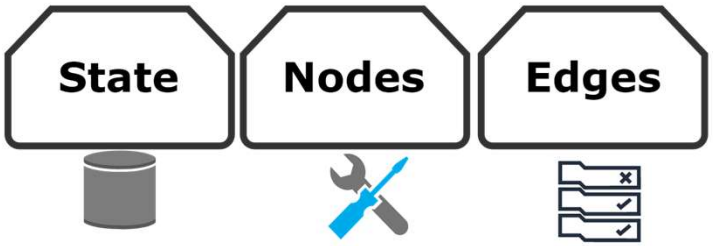


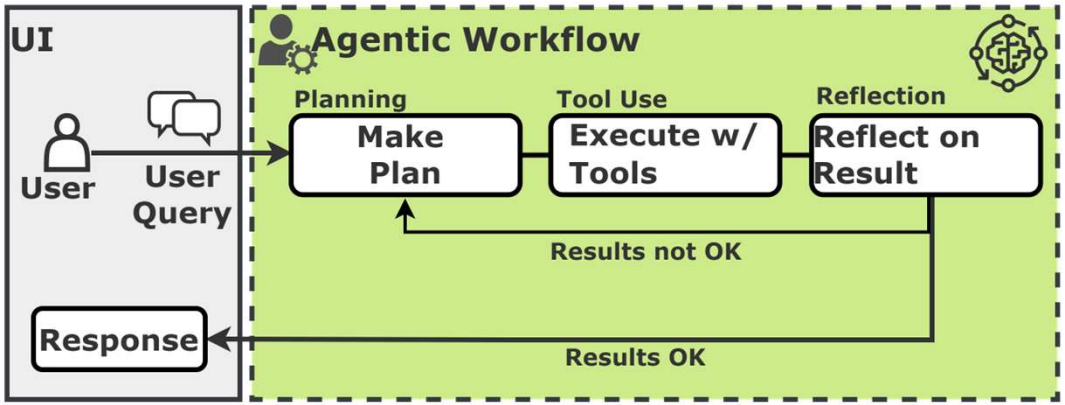
The Anatomy of an Agentic Workflow

Why we need workflows: to build complex, stateful applications by composing simple, modular agents

How are these workflows technically constructed?
What are the actual architectural components—that make them work?

It is composed of **3 fundamental parts**





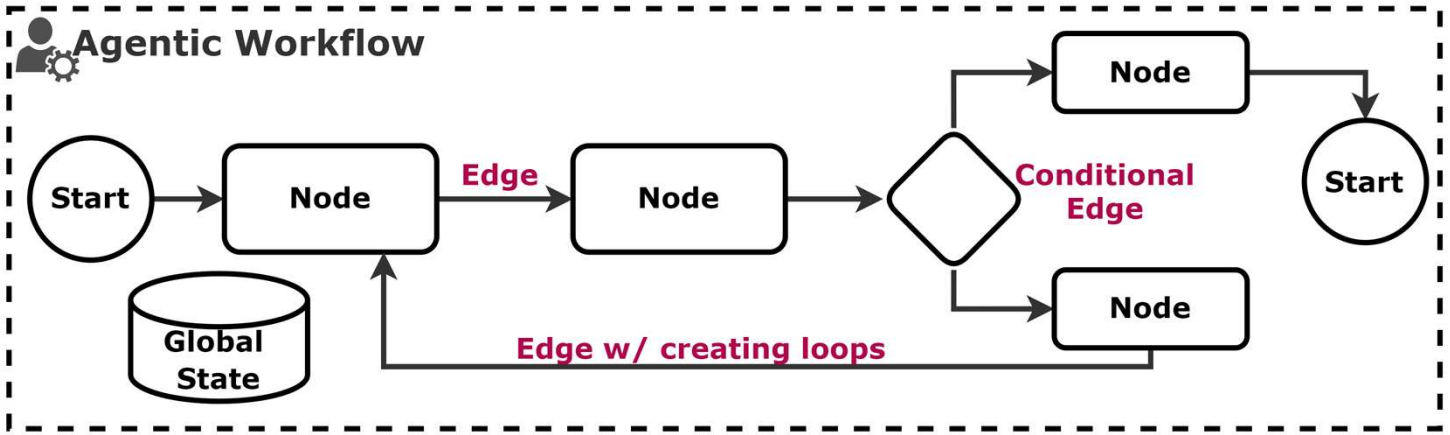
The Three Core Components

Every agentic workflow, regardless of the framework, is built from three parts:

1. The State (The "Scratchpad"):
The 'memory' of the workflow. What is the current status of the job?

2. The Nodes (The "Workers"):
The agents or tools that perform the tasks

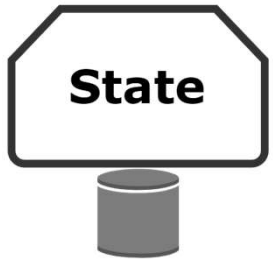
3. The Edges (The "Routing Logic"):
The 'decision-maker' that directs the flow from one node to the next



Component 1: The State (The "Scratchpad")

A stateful object (like a JSON object) that persists and accumulates data throughout the workflow's life

Function: It is the 'Single Source of Truth' for the workflow



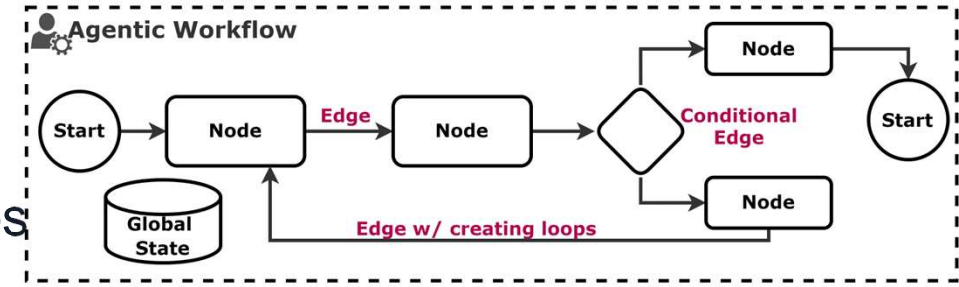
Example (State object):

```
{
  "user_query": "What is Agentic RAG?",
  "research_data": null,
  "summary": null
}
```

After the ResearchAgent (a Node) runs, it modifies the state:

```
{
  "user_query": "What is Agentic RAG?",
  "research_data": "[...]",
  "summary": null
}
```

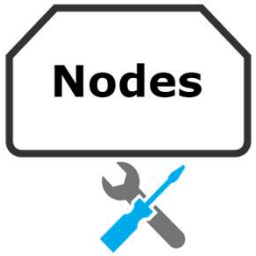
This is the "**memory**" that connects all the steps



Component 2: The Nodes (The "Workers")

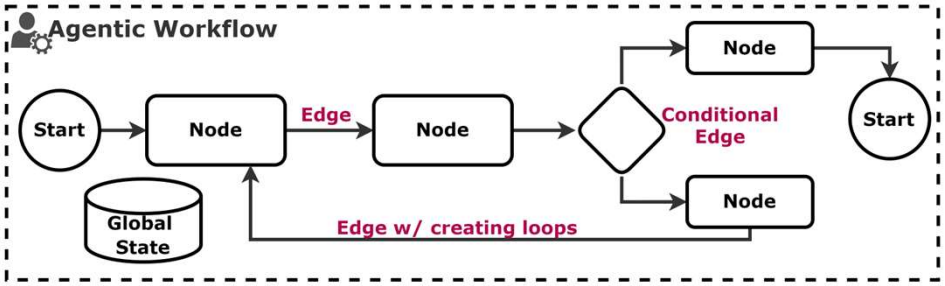
A **function, tool** or **agent** that **performs** a **discrete task**

Function: To read from the State, execute a task, and write back to the State



Key Principle: Modularity (Separation of Concerns)
ResearchNode only knows how to search
SummaryNode only knows how to summarize

Important: A Node is stateless. It receives the current state, does its job, and returns an update. It doesn't know (or care) what happened before or what will happen next

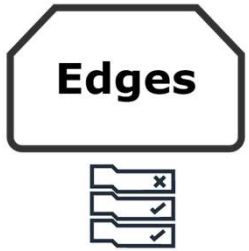


Component 3: The Edges (The "Routing Logic")

The logic that **connects** the **Nodes**. It decides "**where to go next**"

Function: To read from the State and direct the workflow to the next Node

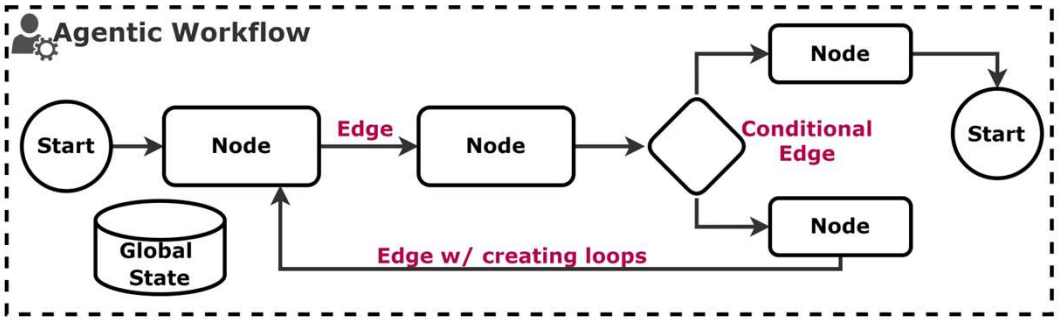
This is where the **LLM's reasoning** is used as a **router**



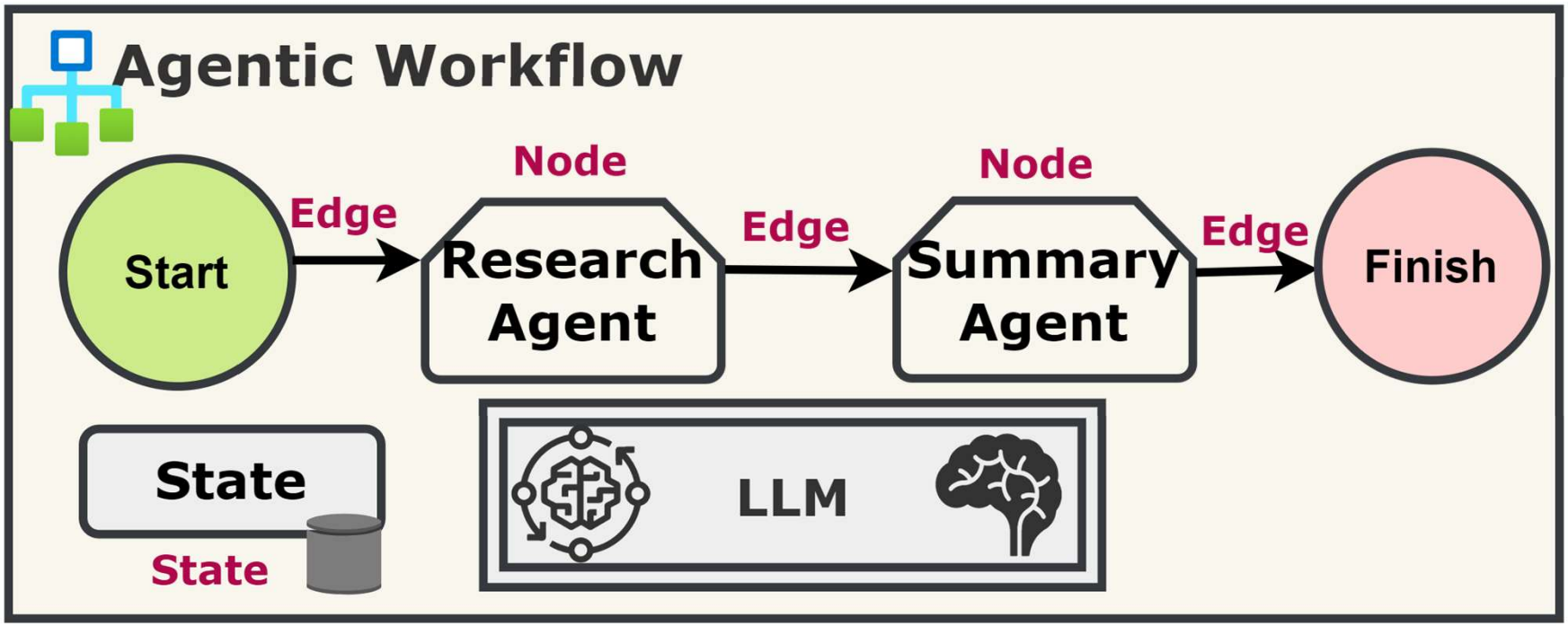
Traditional (Deterministic) Edge:
IF 'research_data' is not null THEN go to 'SummaryNode'

Agentic (Non-Deterministic) Edge:
[LLM CALL]: Given this state, which agent should run next: [Researcher, Summarizer, Coder, Finish]?

This is the 'Project Manager'



Putting It All Together: The Flow



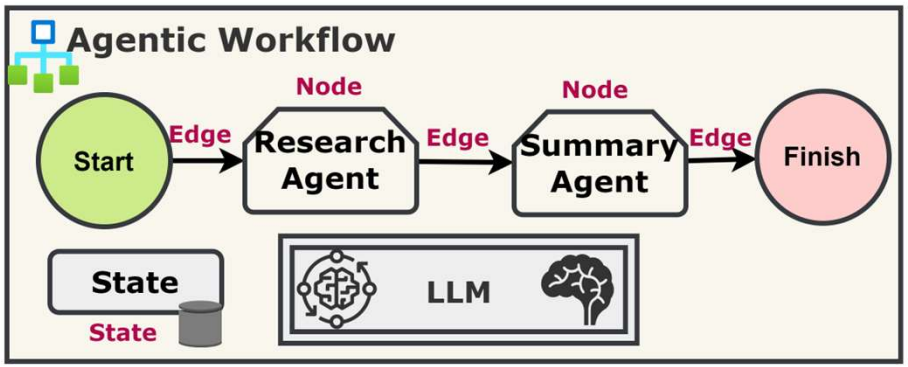
Summary of Agentic Workflow

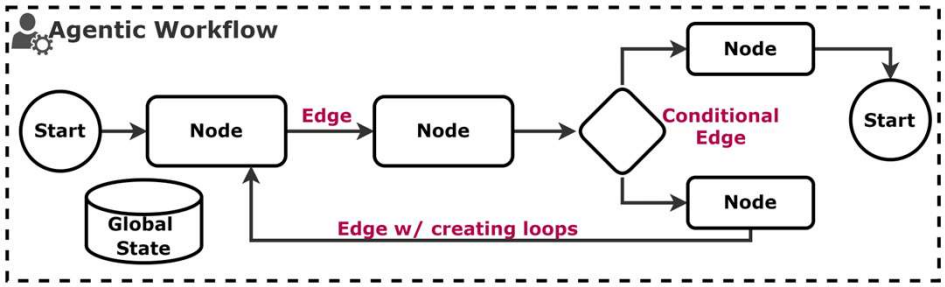
State (The "Scratchpad"): The persistent, shared memory of the job

Nodes (The "Workers"): The stateless, modular agents/tools that perform tasks and modify the State

Edges (The "Routing Logic"): The LLM-driven 'manager' that reads the State and decides which Node to call next

This is the core pattern for building all complex agentic systems





Putting It All Together: The Flow

1. Start: Workflow receives a query, which populates the State

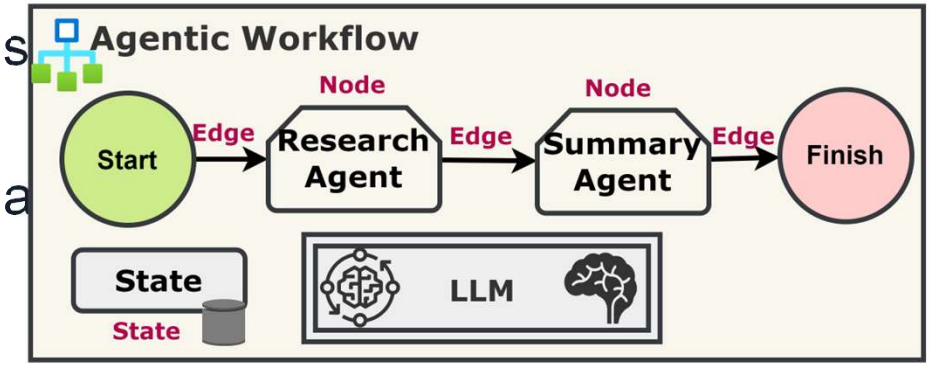
2. Edge (Router): The 'Manager' LLM looks at the State -> "This needs research." -> Routes to ResearchNode

3. Node (Worker): The ResearchAgent reads the query from State, runs its tool, and adds 'research_data' back into the State

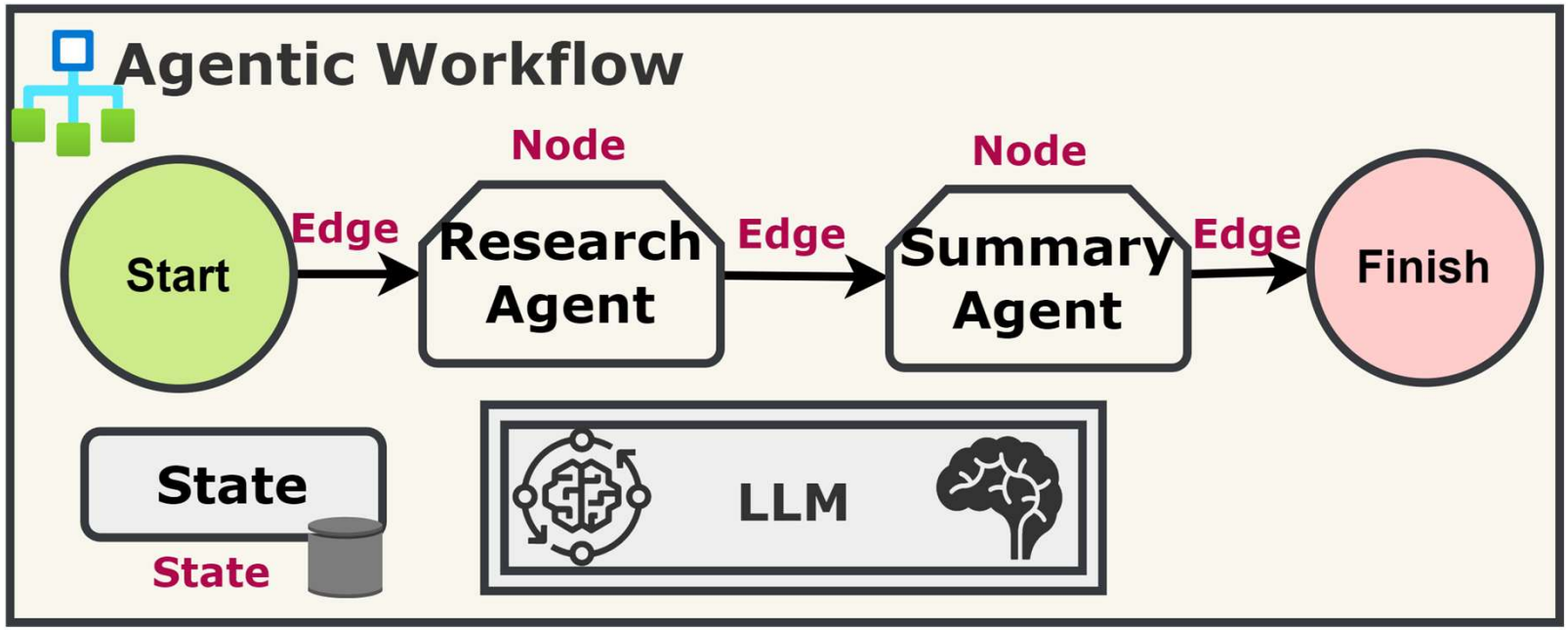
4. Edge (Router): The 'Manager' LLM looks at the State again -> "Now we have data, we need a summary." -> Routes to SummaryNode

5. Node (Worker): The SummaryAgent reads adds 'summary' back into the State

6. Edge (Router): The 'Manager' LLM looks a -> Routes to Finish



Putting It All Together: The Flow



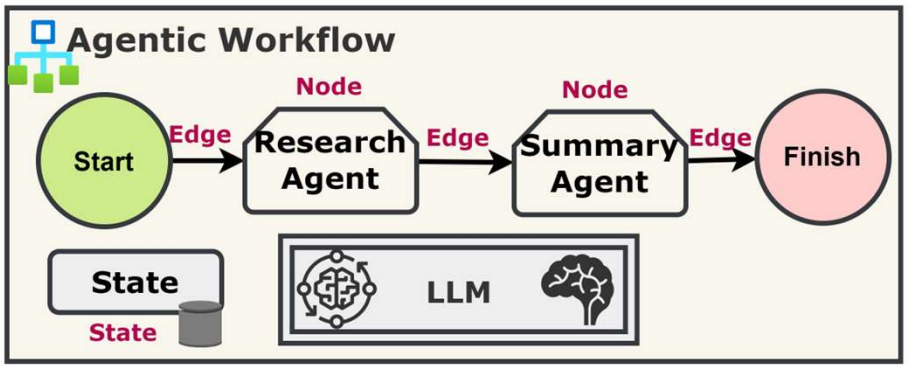
Summary of Agentic Workflow

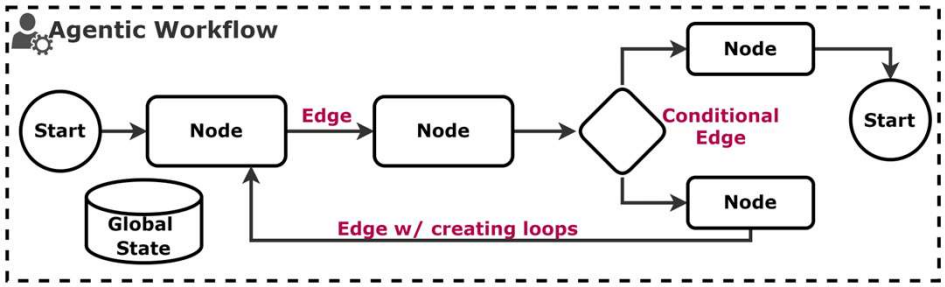
State (The "Scratchpad"): The persistent, shared memory of the job

Nodes (The "Workers"): The stateless, modular agents/tools that perform tasks and modify the State

Edges (The "Routing Logic"): The LLM-driven 'manager' that reads the State and decides which Node to call next

This is the core pattern for building all complex agentic systems



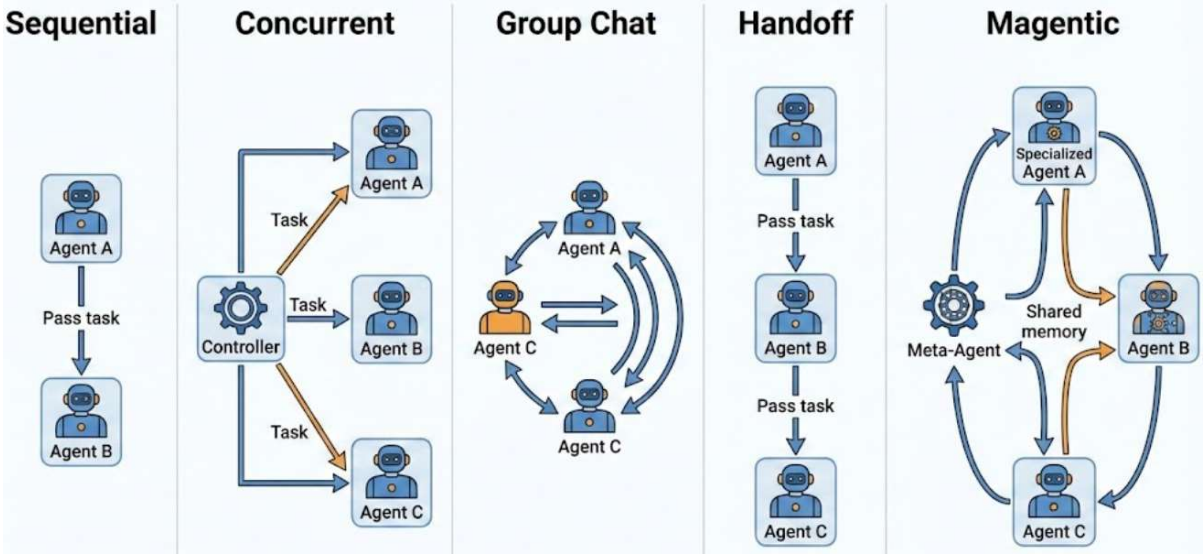


Agentic Orchestration Patterns

Agentic workflow into its 3 core components: the **State** (the scratchpad), the **Nodes** (the workers) and the **Edges** (the routing logic)

What are the **common 'blueprints'** or **'design patterns'** for **arranging blocks**?

How we connect our **nodes** and **edges defines** the **capability** of our **system** ?



Pattern 1: Sequential Handoff (The "Assembly Line")

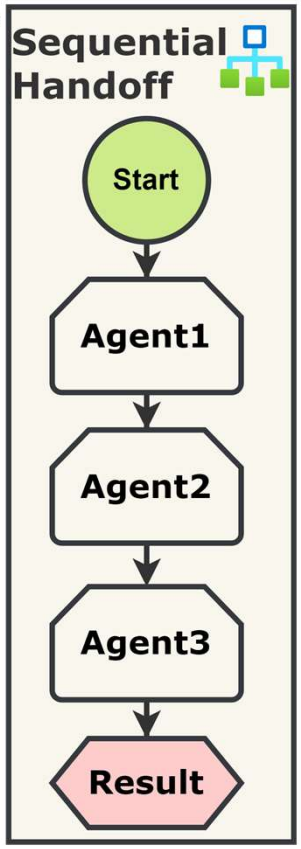
How it Works: The simplest pattern. The workflow executes a fixed, linear sequence of nodes

Edge Logic: The "Edge" is deterministic and hardcoded. (IF Node A is done, ALWAYS go to Node B)

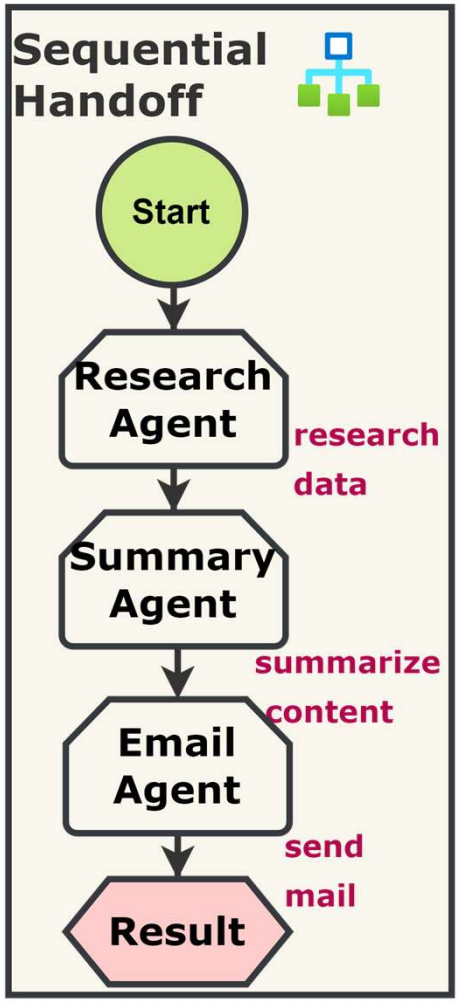
State: The state object is simply passed from one node to the next, like a part on an assembly line

Examples:
1. ResearchNode: Adds 'research_data' to state
2. SummaryNode: Reads 'research_data', adds 'summary' to state
3. EmailNode: Reads 'summary', sends email

Use Case: Simple, predictable, multi-step tasks that do not require adaptation



Pattern 1: Sequential Handoff (Research Example)



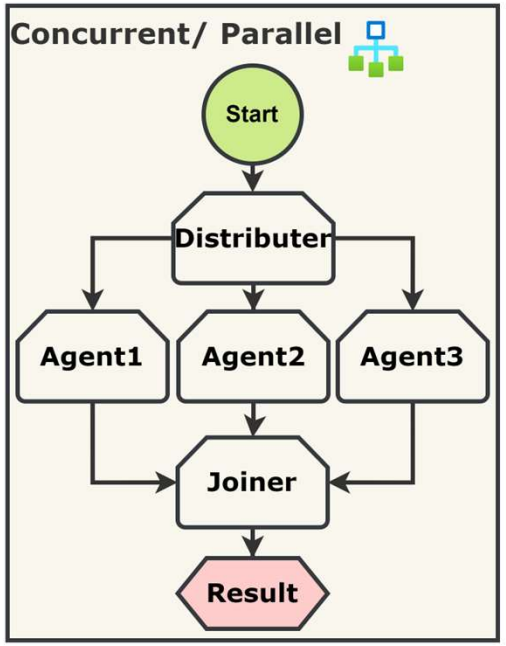
Pattern 2: Concurrent / Parallel (The "Multi-Tasker")

How it Works: The workflow "forks" to run multiple nodes at the same time, and then "joins" back to a single node

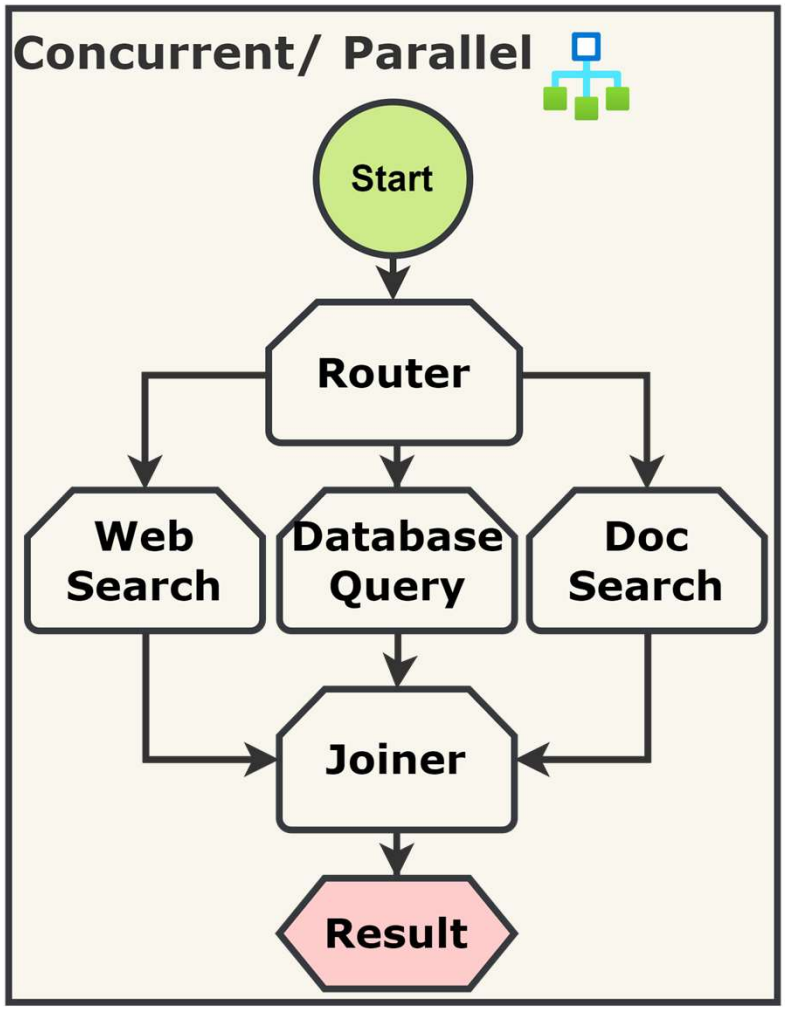
Edge Logic: The Edge from A splits the work. The Edge before D waits for both B and C to be complete

Examples: A query requires two independent sources of information
1. RouterNode (A) starts the job
2. WebSearchNode (B) runs in parallel with...
3. DatabaseQueryNode (C)
4. JoinerNode (D) receives results from both B and C to synthesize a final answer

Use Case: Speed and efficiency. Used when you have multiple tasks that are not dependent on each other



Pattern 2: Concurrent / Parallel (The "Multi-Tasker")



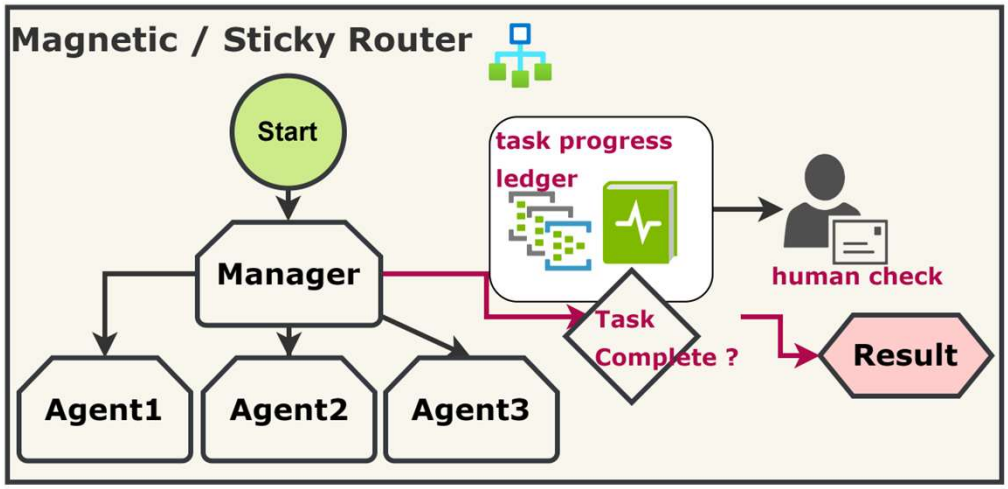
Magnetic / Sticky Router (The "Orchestrator")

A **central 'Manager'** node runs after every worker node to re-evaluate the state and decide the next step

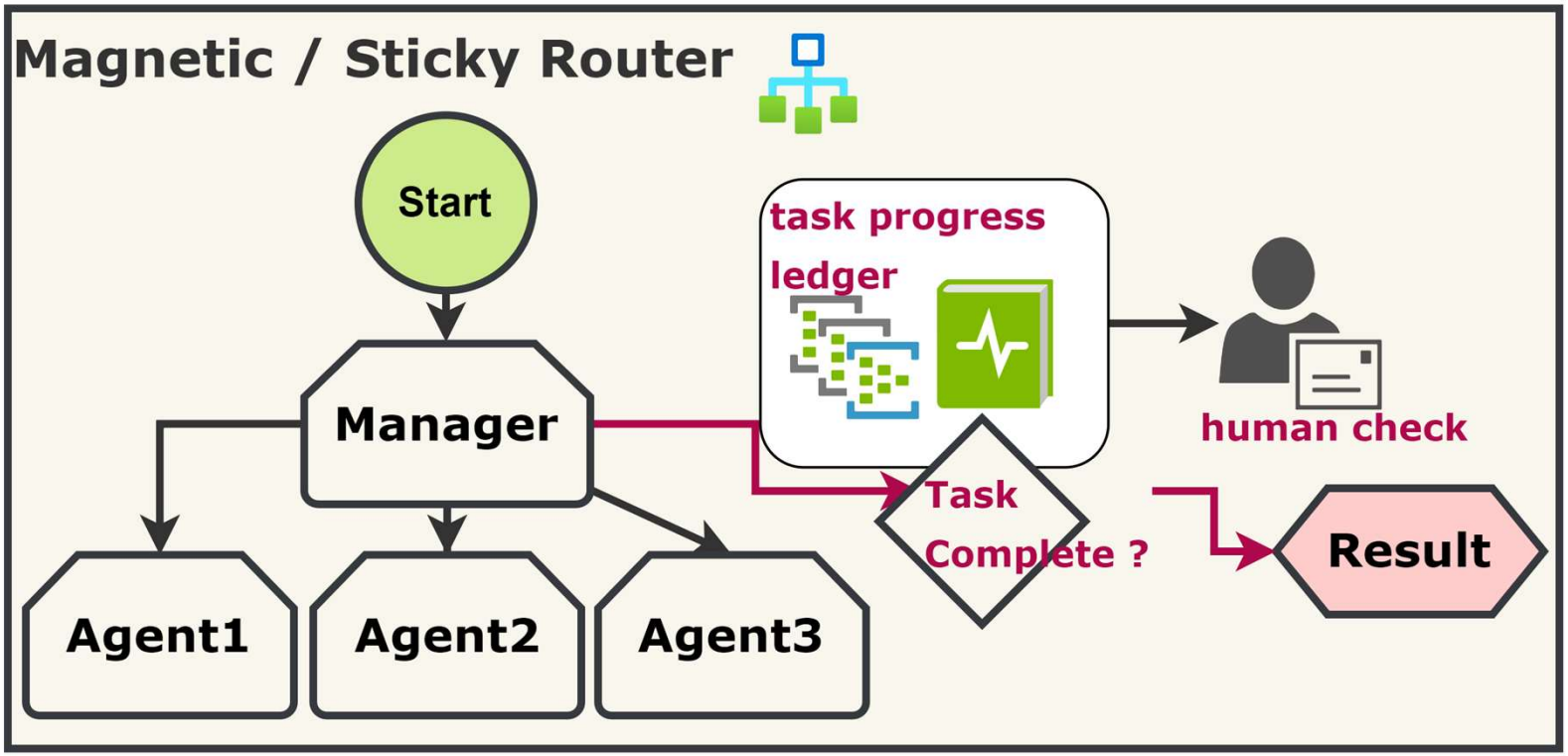
This is the **Agentic Edge**. The **RouterNode** is an **LLM call**. It looks at the entire state and reasons about where to go next (A, B, C, or Finish)

The "**Project Manager**" It's "magnetic" because all work flows back to it

This is the most **common** and **flexible pattern** for building adaptive, autonomous agents



Magnetic / Sticky Router (The "Orchestrator")

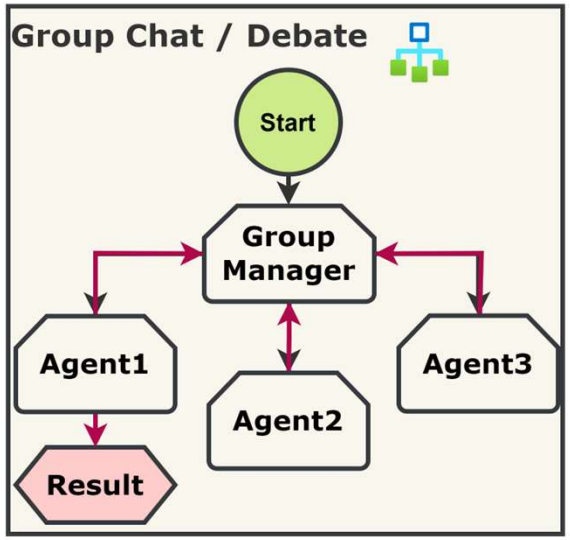


Group Chat / Debate (The "Multi-Agent Loop")

How it Works: An advanced pattern where multiple agents can "talk" to each other in a shared context, modifying the state in a loop

State: The 'State' is often a list of chat messages

Edge Logic: A 'Router' node decides which agent speaks next

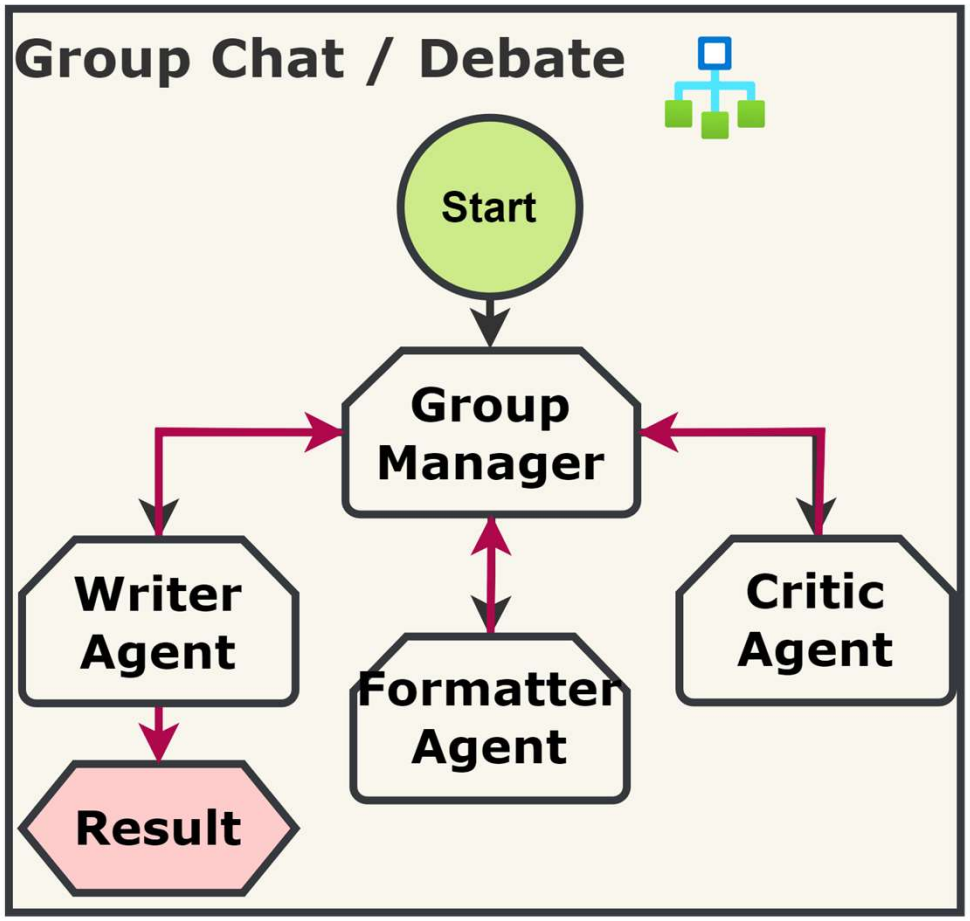


Examples:

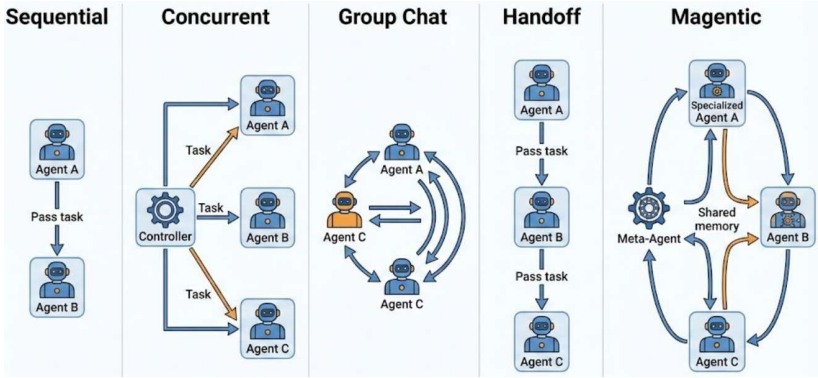
1. WriterAgent drafts a report, adding its message to the state
2. Router passes the state to CriticAgent
3. CriticAgent adds a critique message: "This is too complex"
4. Router passes the entire chat history back to WriterAgent
5. WriterAgent rewrites its report based on the critique

Use Case: High-quality generation, complex problem-solving, and self-correction

Group Chat / Debate (The "Multi-Agent Loop")



Summary: The Core Workflow Patterns



Sequential Handoff:
For simple, linear, and predictable tasks

Concurrent / Parallel:
For speed and efficiency on independent tasks

Magnetic Router (Orchestrator):
For flexible, adaptive, and stateful single-agent workflows

Group Chat / Debate:
For advanced, collaborative, and iterative multi-agent tasks

Key Insight: As an architect, you will choose and combine these patterns to build your application

Frameworks for Building Workflows (Code-First)

We know the **'anatomy'** of a **workflow—State, Nodes, and Edges**

The **'patterns'**—like **Sequential**, **Magnetic Router** and **Group Chat**

How do we, as developers and architects, actually **build these systems** in **code**?

If-then-else block or a complex while loop **unmanageable 'spaghetti code**'

Focus on **'Code-First' frameworks**



The Core Problem: Why Not Just Write Python ?

Building stateful graphs is harder than it looks

1. Managing State:
How do you reliably pass the 'scratchpad' from step to step, ensuring it's always in sync?



2. Handling Cycles (Loops):
The 'Magnetic Router' and 'Group Chat' patterns are cyclic
Simple if/while loops become deeply nested and brittle

3. Non-Determinism:
The LLM's routing choice is non-deterministic. How do you manage this in a standard script?

Solution: We need a framework that natively understands our anatomy (State, Nodes, Edges) and is designed to handle cycles

Introducing LangGraph (The "Graph" Model)

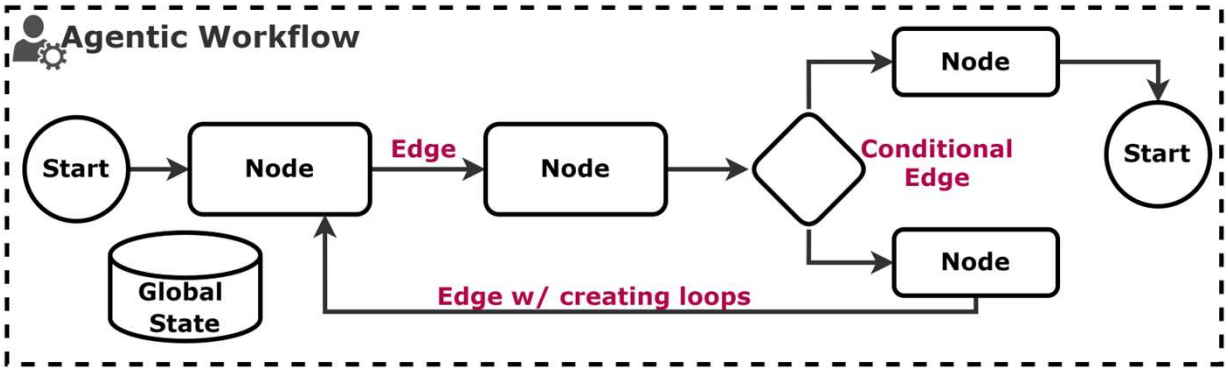
A Python library specifically for building stateful, cyclic agentic workflows

It's not a framework for agents; it's a framework for workflows

State -> StateGraph(MyStateObject):
You formally define your state object as a class. LangGraph manages it

Nodes -> @workflow.add_node:
You define your 'workers' (agents/tools) as standard Python functions

Edges -> @workflow.add_edge:
You explicitly define the connections between your nodes



LangGraph

Platforms for Building Workflows (Low-Code)

We took a deep, **code-first** dive into **frameworks** like **LangGraph**

This **'code-first'** approach **provides maximum power** and **flexibility**

Comes at the **cost** of **development time**, **complexity** and **specialized expertise**

Analyze the role, the **players** and the **critical trade-offs** of using **Low-Code / No-Code platforms** to build your **agentic workflows**



What is the Role of LCNC in Agentic AI ?

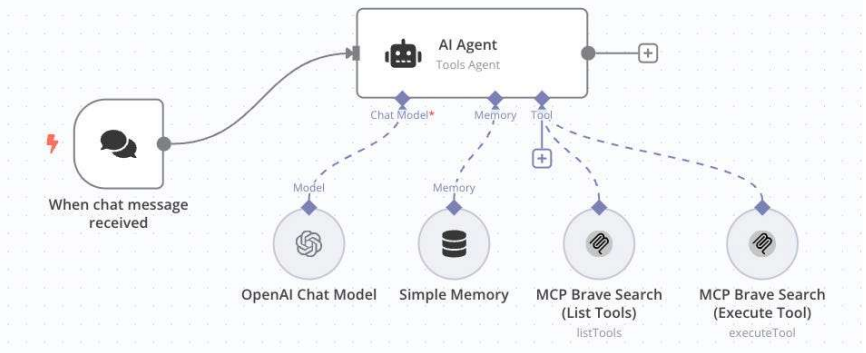
The Goal: To abstract away the code of building a workflow

How: By providing a visual, drag-and-drop interface for connecting components

The View:
Code-First (LangGraph): Writing the 'State', 'Nodes', and 'Edges' in Python
Low-Code (Platforms): Drawing the 'Nodes' on a canvas and connecting them with visual 'Edges'

The Promise: Speed. You are configuring a workflow, not coding one

The "Citizen Developer": Enables non-engineers (like business analysts) to build and manage automations



Exploring the LCNC Landscape

1. Traditional Automation (n8n, Zapier, Make):
Started as "if this, then that" automation
They are "AI-native," adding LLM calls ('Summarize or Classify this')

2. Visual Graph Builders (Flowise, Langflow):
Literally "LangGraph with a UI"
Visual front-ends for LangChain, allow to build and chain prompts, agents and tools visually

3. All-in-One "Agent Builders" (OpenAI's GPTs):
Abstract everything
Don't build a 'graph'; just 'configure an agent' with a name, instructions and tools
The platform manages the state and routing logic

Why Architects Choose Low-Code

Speed to Market (Time-to-Value):
Build and deploy a functional prototype or internal tool in hours, not weeks

Accessibility & Empowerment:
Non-developers can build, test and maintain their own workflows, freeing up senior engineering resources

Reduced Infrastructure Overhead
The platform manages the servers, scaling, and runtime for you

Enforced Best Practices
The platform forces you into a modular, graph-based design, preventing "spaghetti code







The "Cons": The Hidden Architectural Costs







Limited Flexibility & Expressiveness:
What if you need a custom 'Group Chat' or 'Debate' pattern? Limited to the nodes and patterns they provide

Vendor Lock-In:
Your entire business logic—your agentic workflow—is now owned by the platform. Migrating away is a complete, manual rewrite

The "Black Box" Problem:
When it breaks, can you debug it? You have less control over the state, the prompts, and the error handling

Cost at Scale:
'Code-first' running on your own infrastructure is almost always cheaper at massive scale than paying 'per-execution' on a SaaS platform

The Architect's Decision Framework: When to Use What?

Use LOW-CODE (n8n, Flowise, GPTs) for:
Rapidly testing an idea before committing engineering resources
Automating non-critical business processes
Tasks that are predictable and fit the 'Sequential Handoff' pattern

Use CODE-FIRST (LangGraph, Autogen) for:
The logic is your intellectual property
When you need a 'Magnetic Router' or 'Group Chat' pattern
When you need full control over optimization and infrastructure costs

AI Agent Frameworks - Exploring AI Agentic Frameworks

LangChain & LangGraph

LlamaIndex

Microsoft Agent Framework

Cloud Agents: Amazon Bedrock, Azure Foundry, Google Agents

Pradeep Kumar T



The Frameworks Landscape

We built a **sophisticated**, **multi-agent workflow** using a **code-first framework**

What tools did we use? **Why** did we **use** them? **What** are the **alternatives?**

Raw **API calls** to **LLMs**, manually **parsing** the **JSON** output and **hand-crafting** a **state machine** is **unmanagable**

Learn framework landscape to help you choose the right foundation for your business



Why Use a Framework ? The Boilerplate Problem

An "agent" is a simple loop... until you add "**plumbing**"



1. Tool Calling & Parsing:
Formatting the tools schema for the LLM
Parsing the LLM's non-deterministic JSON output
Handling errors if the LLM fails to call a function

2. State Management:
Reliably passing the chat history or scratchpad between loop iterations

3. RAG Pipelines:
Connecting to data, chunking, embedding, storing and retrieving documents

4. Prompt Management:
Templating, managing and chaining complex prompts

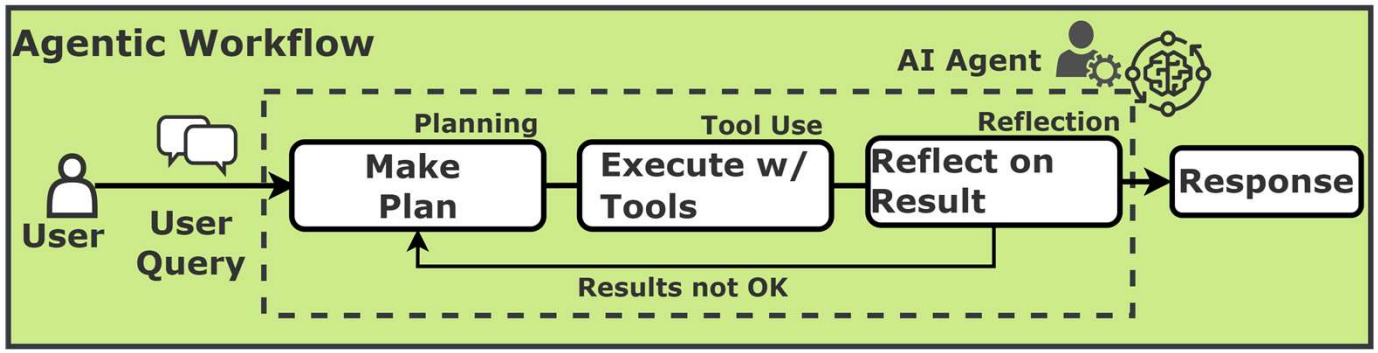
An Architect's Map: 3 Categories of Tools

The "Agentic AI" market is fragmenting into 3 main categories

1. Generalist Toolkits (The "Workshop"):
Gives you all the low-level components to build anything

2. Specialist Frameworks (The "High-End Machine"):
Gives you a pre-built solution for one specific, hard problem

3. Managed Platforms (The "Full-Service Factory"):
Sells you a finished "Agent-as-a-Service." You don't build; you configure



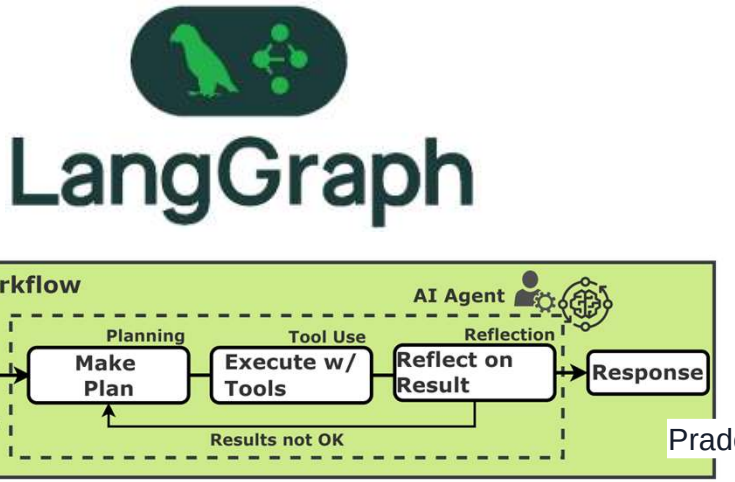
The "Generalist Toolkit": LangChain & LangGraph

Mapped frameworks landscape into 3 categories: **Generalists**, **Specialists** and **Managed Platforms**

'Generalist Toolkit' the framework that in many ways created the entire agent development space: **LangChain**

LangChain's idea has always been to provide the low-level components for building any kind of LLM application



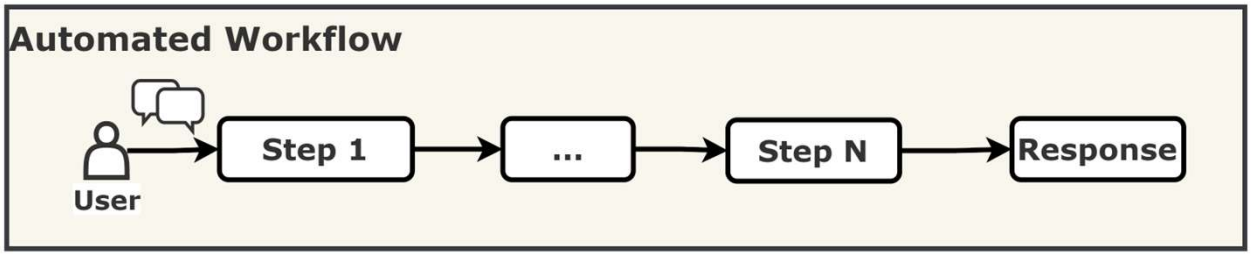


The Core Philosophy: "Composability"

LangChain is built on that any LLM app can be "composed" from standard parts

The "Bricks" (Components):
Models: (LLMs, Chat Models, Embedding Models)
Prompts: (Prompt Templates, Chat Templates)
Tools: (Functions, APIs, Retrievers)
Parsers: (Parsing LLM output into JSON, lists, etc.)

The "Glue" (The Core Concept):
LCEL (LangChain Expression Language)
This is the | (pipe) operator that lets you "chain" components together

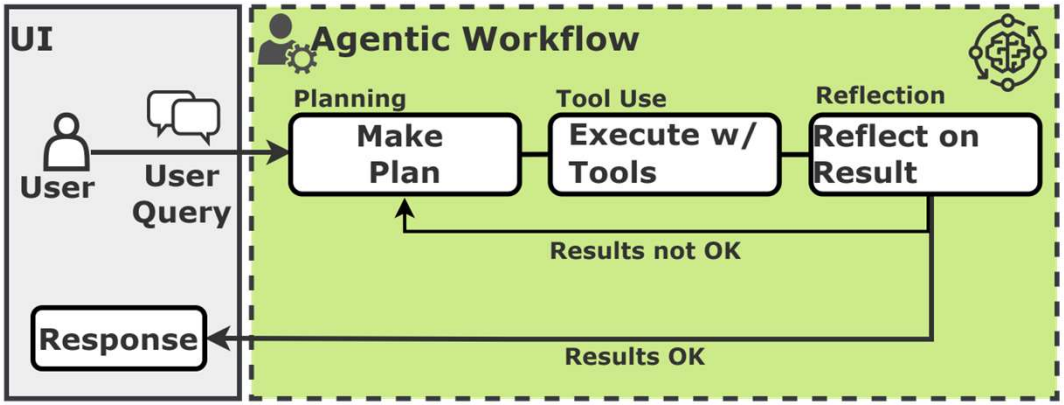


Core Components for LangChain Agents

Agents need 2 key components that LangChain provides:

1. Tool Decorators (@tool):
A simple decorator to turn any Python function into a "tool" the LLM can call
It automatically handles the complex JSON schema generation

2. Pre-built "Agent Executors":
LangChain provides pre-built "ReAct" or "Function-calling" agent loops
You just give it a model, your tools, and a prompt, and it runs the loop for you



LangChain

The Architectural Limit of "Executors"

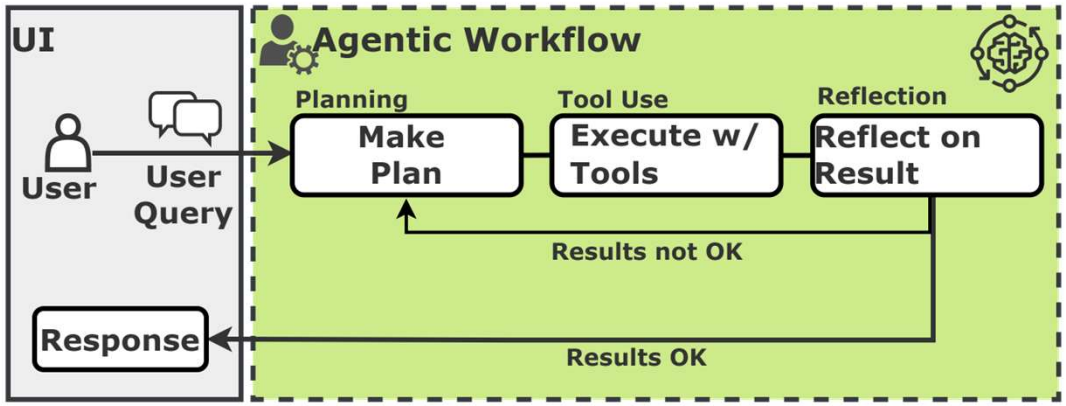
The Problem: The **pre-built "Executors"** are a **"black box"**

They are **cyclic**, but their **logic** is **fixed**

What if you need a "**Magnetic Router**" ?
What if you need a "**Group** Chat"?
What if you need to add a **human-in-the-loop** for **approval**?

The **pre-built 'Executor'** cannot do this. It is **not flexible**

"Black box" problem: good for simple tasks, impossible for custom, stateful architecture



The Solution: LangGraph (The "White Box")

This is why **LangGraph** was created

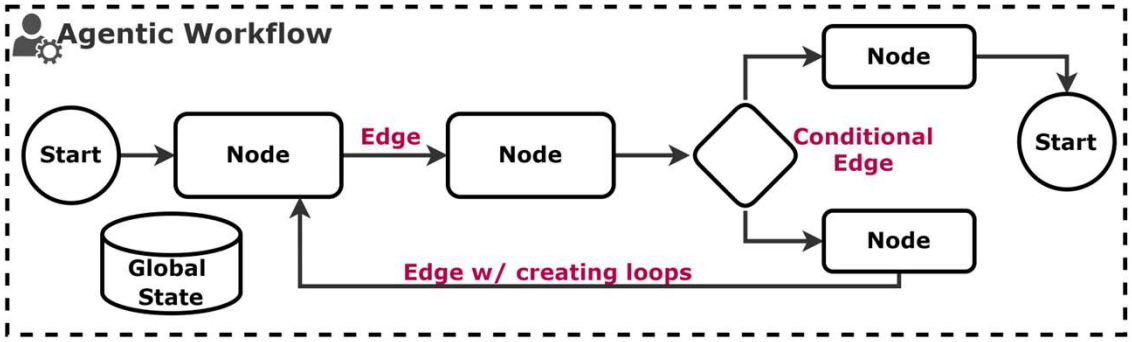
LangGraph is the "**white box**" solution for building agents

It is **LangChain's** engine for **stateful**, **cyclic workflows**

Anatomy of Workflow

1. **State:** A central, explicitly defined state object
2. **Nodes:** Any Python function (or LCEL chain) that modifies the state
3. **Edges:** Explicit, controllable logic (our 'Routers') that direct the flow

LangGraph is the **'build-your-own-agent'** architecture

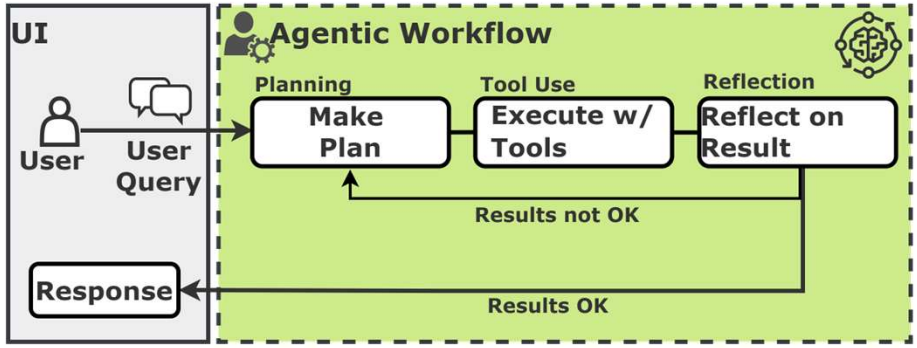


When to use LangChain/LangGraph

Maximum Flexibility:
When need to build a non-standard, custom workflow ('Group Chat,' 'Magnetic Router')

Custom Logic:
When your agent's reasoning logic is your intellectual property and cannot be a "black box"

Component "Glue":
When you need to integrate many different models, data sources and tools from various vendors



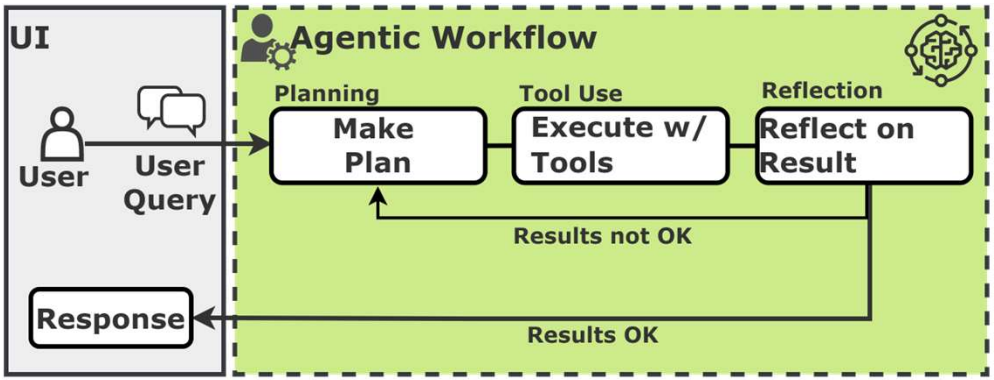
When NOT to use LangChain/LangGraph



The Trade-offs:

Complexity: It is powerful, but complex. The learning curve is steep

Verbosity: You are responsible for all the architectural 'plumbing' (defining state, nodes, edges), which is more code



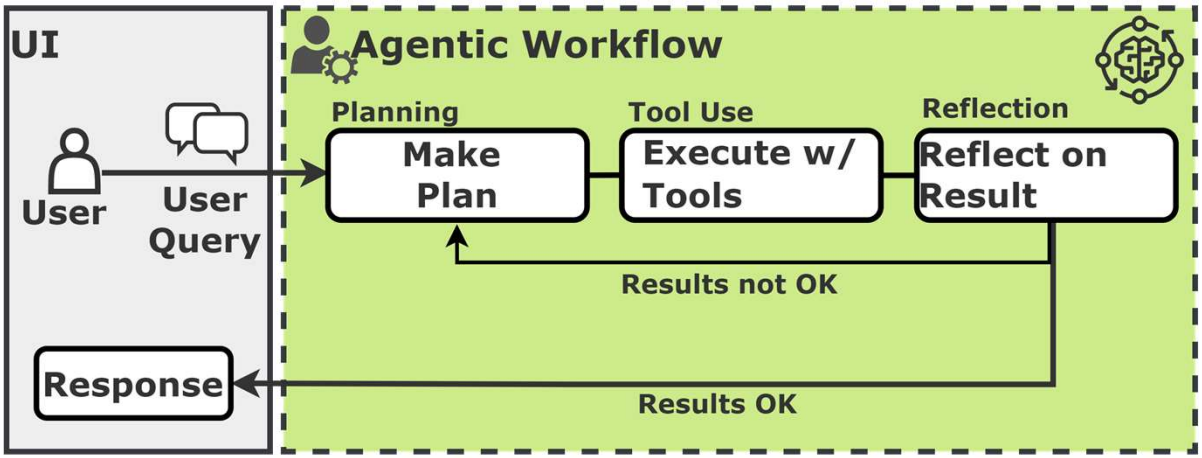
Summary - The "Generalist's Workshop"

LangChain: A "workshop" of low-level components (LCEL, Tools)

LangGraph: The "workbench" that lets you assemble those comp custom, stateful, cyclic agent you can design

Use for: Maximum flexibility and custom logic

Beware of: High complexity and "roll-your-own" architecture



LangChain

LangGraph

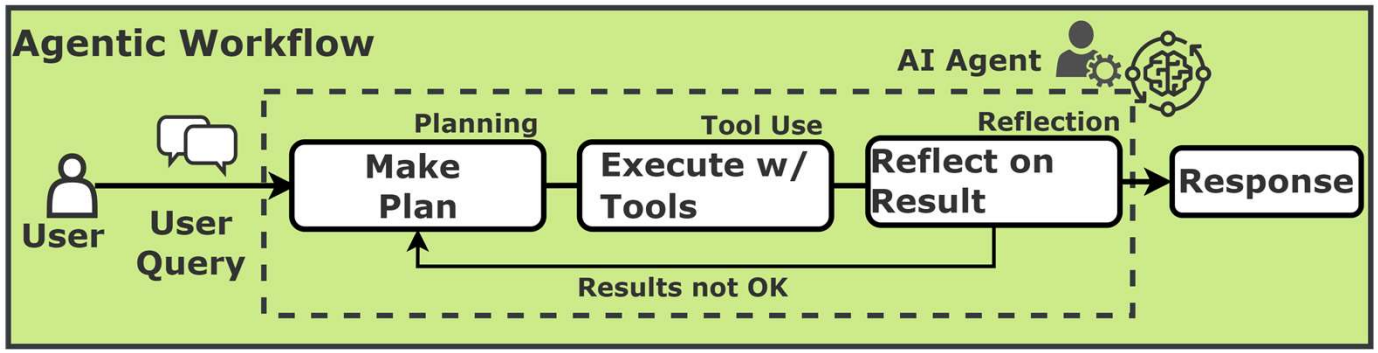
An Architect's Map: 3 Categories of Tools

The "Agentic AI" market is fragmenting into 3 main categories

1. Generalist Toolkits (The "Workshop"):
Gives you all the low-level components to build anything

2. Specialist Frameworks (The "High-End Machine"):
Gives you a pre-built solution for one specific, hard problem

3. Managed Platforms (The "Full-Service Factory"):
Sells you a finished "Agent-as-a-Service." You don't build; you configure



The "Data Specialist": LlamaIndex Deep Dive

Focus to the **'Specialist' category** to solve one specific, incredibly **hard problem:** building **powerful**, **data-intensive agents**

If **LangChain** is the **'generalist workshop' LlamaIndex** is the **'specialized data science lab**'

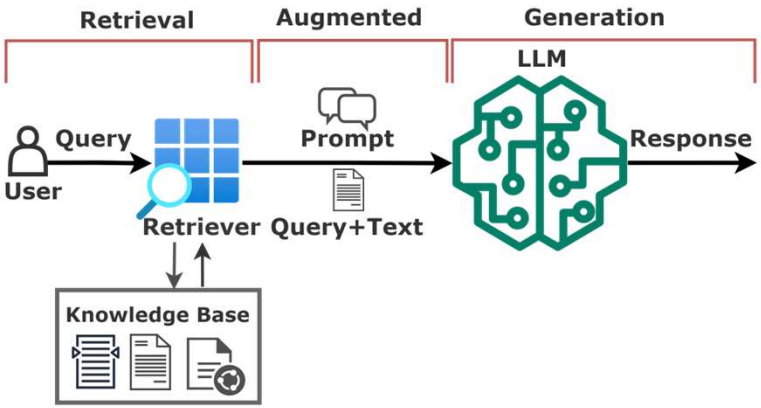
We're moving from an **'application-first'** worldview to a **'data-first'** worldview



The Core Problem: Beyond Simple RAG

Simple RAG: (Retrieve -> Augment -> Generate)

1. Find a few relevant chunks
2. Stuff them into a prompt
3. Get an answer

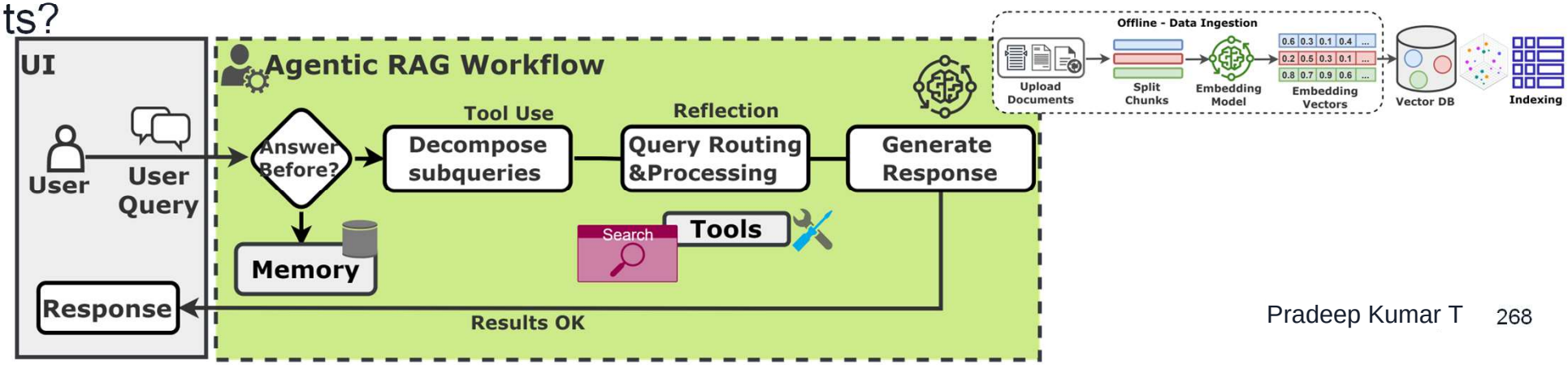


The "LlamaIndex" Problem (Agentic RAG)

What if the answer isn't in one document?

What if you need to compare multiple documents?

What if you need to query a PDF, then a SQL database and then synthesize the results?



LlamaIndex vs. LangChain

LangChain is "Application-First":
It focuses on the agent's execution loop (the workflow)
Data is just one of many "tools" the agent can call



LlamaIndex is "Data-First":
It focuses on the data itself
It excels at ingesting, structuring, and indexing your data in complex ways
The 'agent' is a reasoning layer on top of this structured data

Choose **LangChain** for **complex agent logic**
Choose **LlamaIndex** for **complex data interaction**



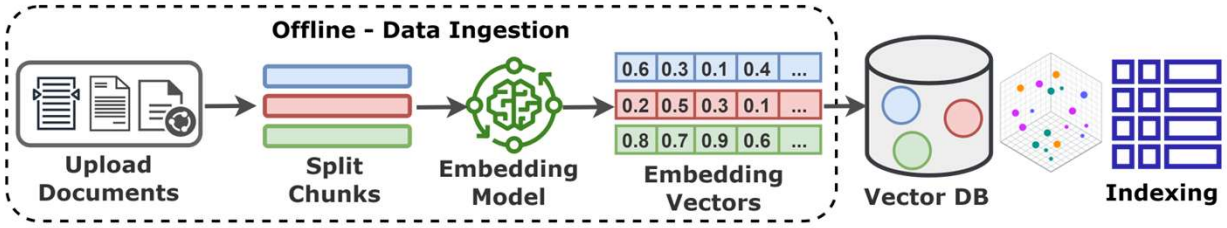
Core Concepts 1 & 2: Data Connectors & Indexing

LlamaIndex's architecture is split into 2 pipelines

Pipeline 1: Data Ingestion (Getting data IN)

Data Connectors:
Pre-built modules to load data from anywhere (S3, Salesforce, Notion, SQL DB, APIs, PDFs)

Indexing:
This is LlamaIndex's 'magic.' It's not just a vector store
It intelligently structures your data
Examples: VectorStoreIndex (for search), SummaryIndex (for summarization), KeywordTableIndex (for metadata)

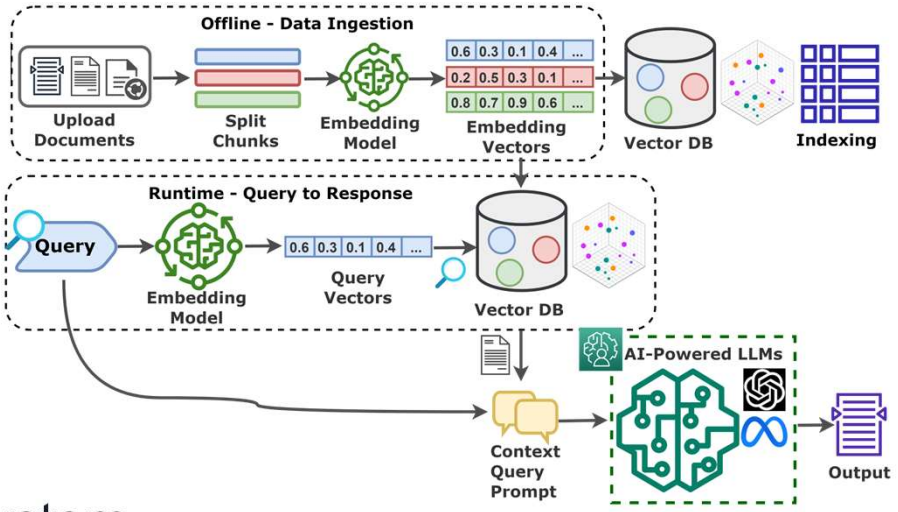


Core Concepts 3 & 4: Query Engines & RAG Pipelines

Pipeline 2: Data Retrieval (Getting data OUT)

Query Engines:
These are the high-level interfaces for asking questions
index.as_query_engine()

RAG Pipelines:
LlamaIndex allows you to customize the retrieval
You can add steps like re-ranking
or response synthesis (to craft a better answer)



This is how you build a robust, fine-tuned RAG system

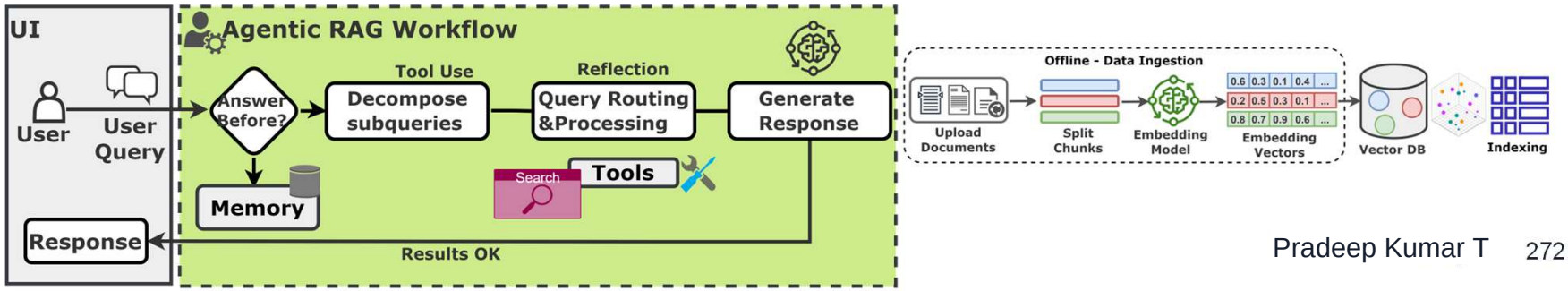
Putting It Together: LlamaIndex Agents (Agentic RAG)

Where is the "agent"? The LlamaIndex is a reasoning layer over Query Engines

You can give an agent multiple Query Engines as its "tools"

Examples:
Tool 1: A Query Engine for your 2024 financial PDFs
Tool 2: A Query Engine for your SQL sales database

The Agent can now reason and decompose complex questions:
"Compare the sales from 2024 to the projections in the finance report"
The agent will plan to call Tool 2 (SQL) first, then Tool 1 (PDF) and then synthesize the answer



Summary: The "Data Specialist"

LlamaIndex: A "data-first" framework

Core Strength: Building rich "Indexes" from diverse data sources

Key Feature: "Agentic RAG" — agents that can reason over and synthesize data from multiple, complex query engines

Use for: Advanced RAG and data-intensive applications



When to use LlamaIndex



1. Advanced RAG:
When your primary challenge is RAG
2. Data-Intensive Agents:
When your agent must reason over large, complex, multi-modal or multi-source data
3. Custom RAG Pipelines:
When you need to fine-tune your retrieval with steps like re-ranking or complex synthesis

The Trade-offs:
Less of a "Generalist": It is not the best-in-class framework for building general-purpose agent workflows or complex 'Group Chats'

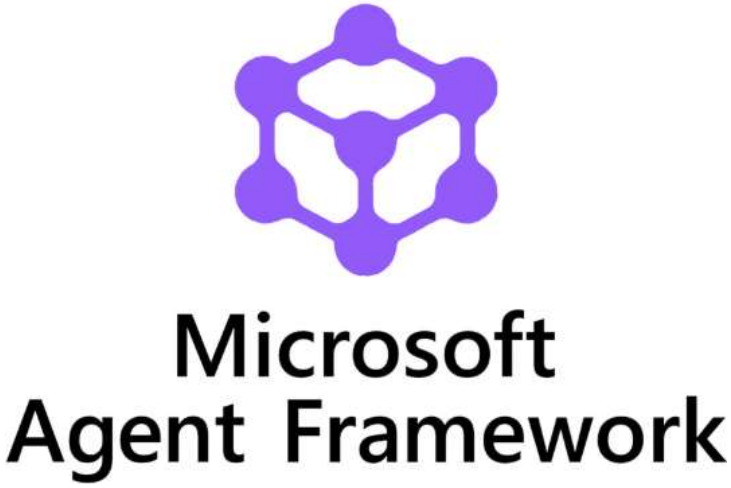
"Data-First" Mindset: It requires you to think about your data architecture first

The "Multi-Agent Specialist": Microsoft Agent Framework

Microsoft projects:
AutoGen: which was brilliant at multi-agent collaboration
Semantic Kernel: which was designed for enterprise-grade, robust agent execution

Both projects have **united** to release the **official successor**. This is the **Microsoft Agent Framework** or **MAF**

Designed to be the **unified foundation** for **building AI agents** on the **Microsoft stack**



The "Why?": Unifying Two Legacies

The Problem: Developers had to make a difficult choice

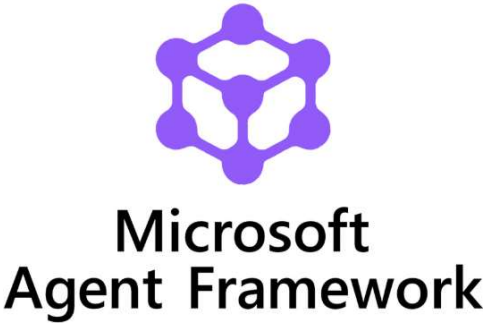
AutoGen's Strength:
Brilliant at multi-agent collaboration
"Conversation-first" model
Simple abstractions for patterns like "Group Chat"



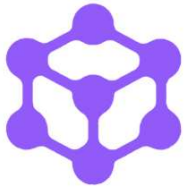
Semantic Kernel's Strength:
Brilliant at enterprise-grade robustness
Planners, middleware, thread-based state
Type-safety, telemetry, and security



MAF's Goal: The Best of Both Worlds
Combine AutoGen's simple multi-agent patterns
with Semantic Kernel's enterprise-grade features



Core Capability 1: The "Enterprise Agent" (The SK Legacy)



Microsoft Agent Framework



Semantic Kernel

The foundation of MAF is the enterprise-grade, single agent

Middleware (Filters):
Intercept agent actions for logging, auth, or safety-checking

Agent Thread (State Management):
A formal system for managing state in long-running conversations

Broad Support:
Native clients for Azure OpenAI, OpenAI, and other models

MCP Integration:
Built to be a first-class citizen with the Model-Context Protocol for standardized tool calls

Core Capability 2: "Workflows" (The AutoGen Evolution)



Microsoft Agent Framework



This is the **new abstraction** for **multi-agent collaboration**

It's a **Graph-based Workflow** engine (like LangGraph)

This is the **evolution** of **AutoGen's "Group Chat"**

Key Enterprise Features:

Type-Based Routing: Explicit, predictable control over the flow

Nesting: You can nest workflows inside other workflows.

Checkpointing: Save the state of a workflow and resume later (essential for long-running tasks)

Human-in-the-Loop: Built-in patterns to pause the workflow and wait for human approval

"Conversation-First" Patterns

MAF still embraces the "**conversation-first**" patterns from AutoGe

It includes simple **abstractions** for **multi-agent patterns**



Microsoft Agent Framework

1. AutoGen-style "Group Chat":
Use for rapid prototyping, debate, and emergent behavior

2. Explicit "Workflow":
Use for production, reliable, step-by-step processes that must be predictable and safe



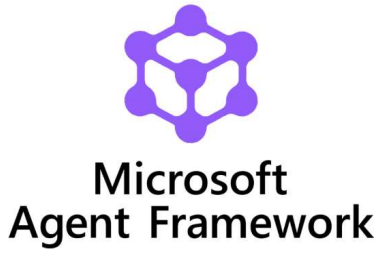
Semantic Kernel



When to use Microsoft Agent Framework

1. Enterprise-Grade Multi-Agent Systems:
This is its #1 purpose. You need both robustness and collaboration

2. Stateful, Long-Running Agents:
When you need checkpointing and human-in-the-loop



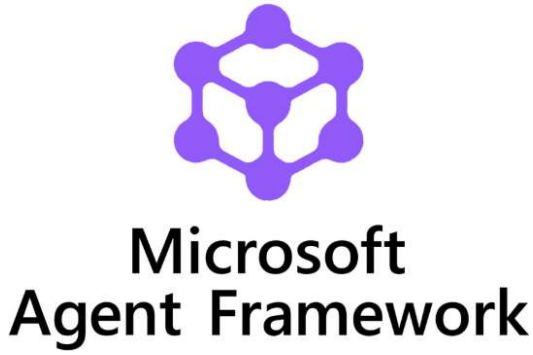
3. .NET and Azure Ecosystem:
It is the first-party choice for .NET developers and for deep integration with the Azure stack

When NOT to use Microsoft Agent Framework

The Trade-offs:

Newness: It's a brand-new, emerging framework, so the community and examples are still growing

"Opinionated": It is more "opinionated" than LangChain, designed to guide you toward a specific, robust, Microsoft-centric architecture



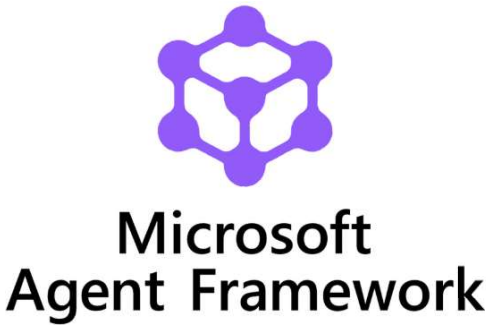
Summary - MAF

MAF: The "next generation" successor to AutoGen and Semantic Kernel

Combines: AutoGen's multi-agent patterns with Semantic Kernel's enterprise-grade features

Key Feature: Explicit, graph-based "Workflows" for controllable, stateful, and safe collaboration

Use for: Production-grade, multi-agent systems, especially in the .NET/Azure world







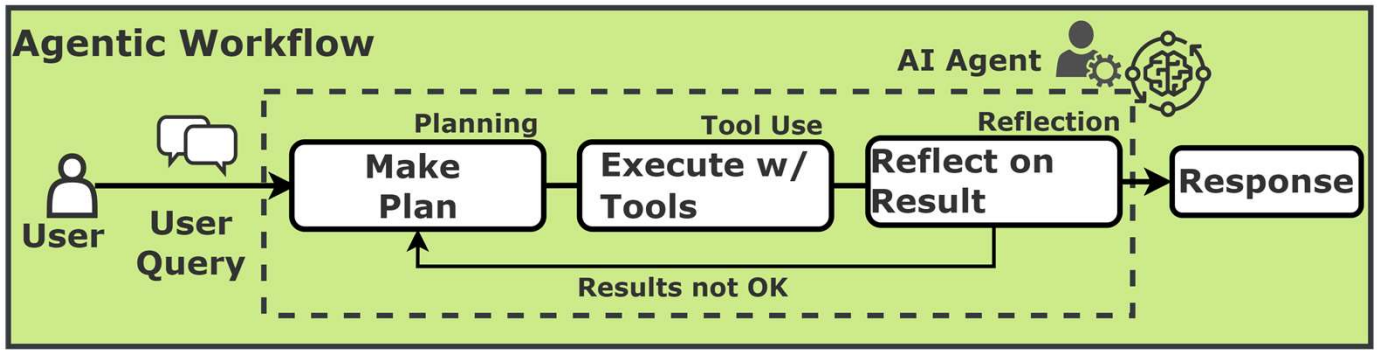
An Architect's Map: 3 Categories of Tools

The "Agentic AI" market is fragmenting into 3 main categories

1. Generalist Toolkits (The "Workshop"):
Gives you all the low-level components to build anything

2. Specialist Frameworks (The "High-End Machine"):
Gives you a pre-built solution for one specific, hard problem

3. Managed Platforms (The "Full-Service Factory"):
Sells you a finished "Agent-as-a-Service." You don't build; you configure

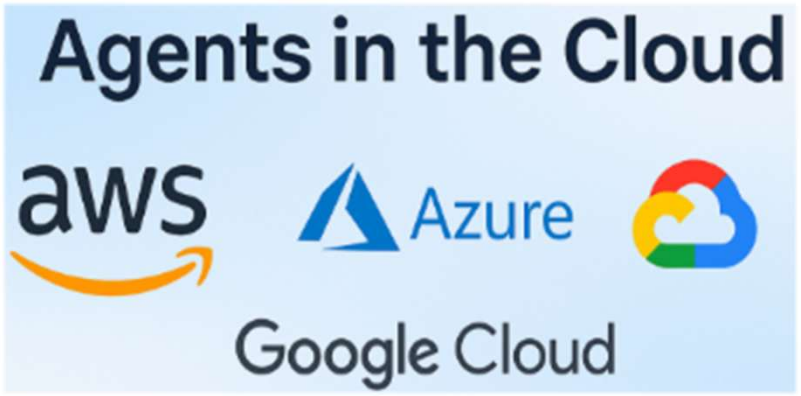


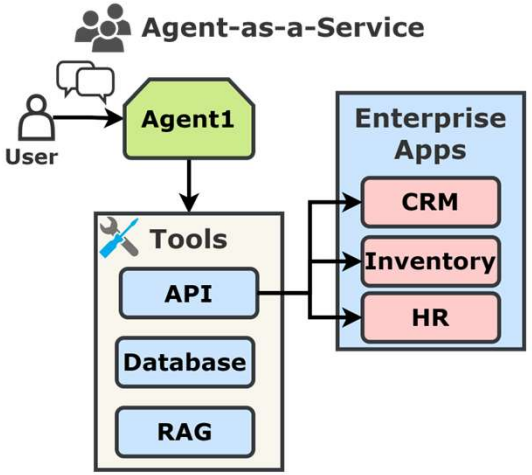
Managed Platforms: Cloud Agents (AWS, Azure, Google)

The **code-first frameworks** like **LangChain**, **LlamaIndex** and the **MAF**

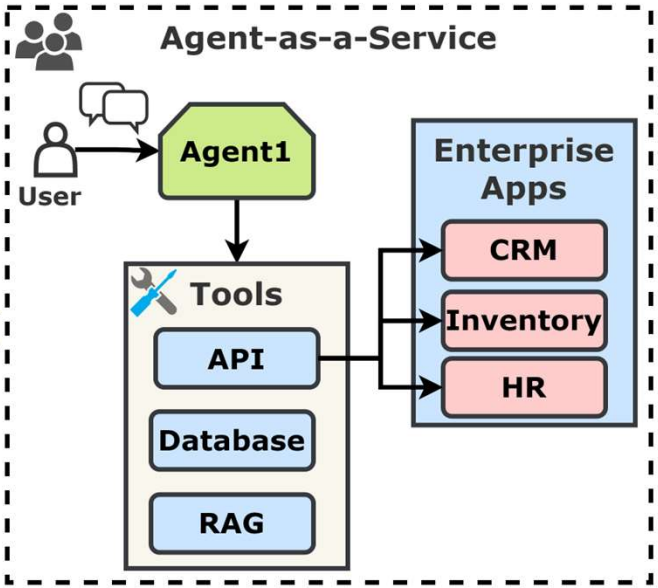
Should we 'build' at all? Or **should we 'buy'?**

This is the **'Agent-as-a-Service'** model, offered by the 'Big 3' cloud providers: **Amazon**, **Microsoft** and **Google**





What is "Agent-as-a-Service" ?



This is the **'buy' decision**

You **do not write** the agent's **reasoning code**
You **do not manage** the agent's **runtime**, **state** or **scaling**

The **cloud provider** manages the **entire agentic stack**

Your Job as Architect:
1. Configure: You declare what the agent should do
2. Provide Tools: You give it OpenAPI specs or Function definitions
3. Write Instructions: You provide the natural language prompt for its goal

The **platform generates** and **hosts** the **agent** for you

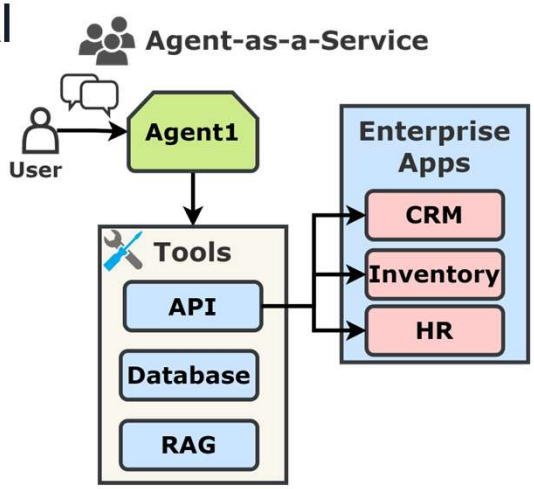
The "Big 3" Managed Platforms



Amazon:
Amazon Bedrock Agents: Deeply integrated with AWS Lambda, S3 and DynamoDB. Leverages Bedrock's model garden

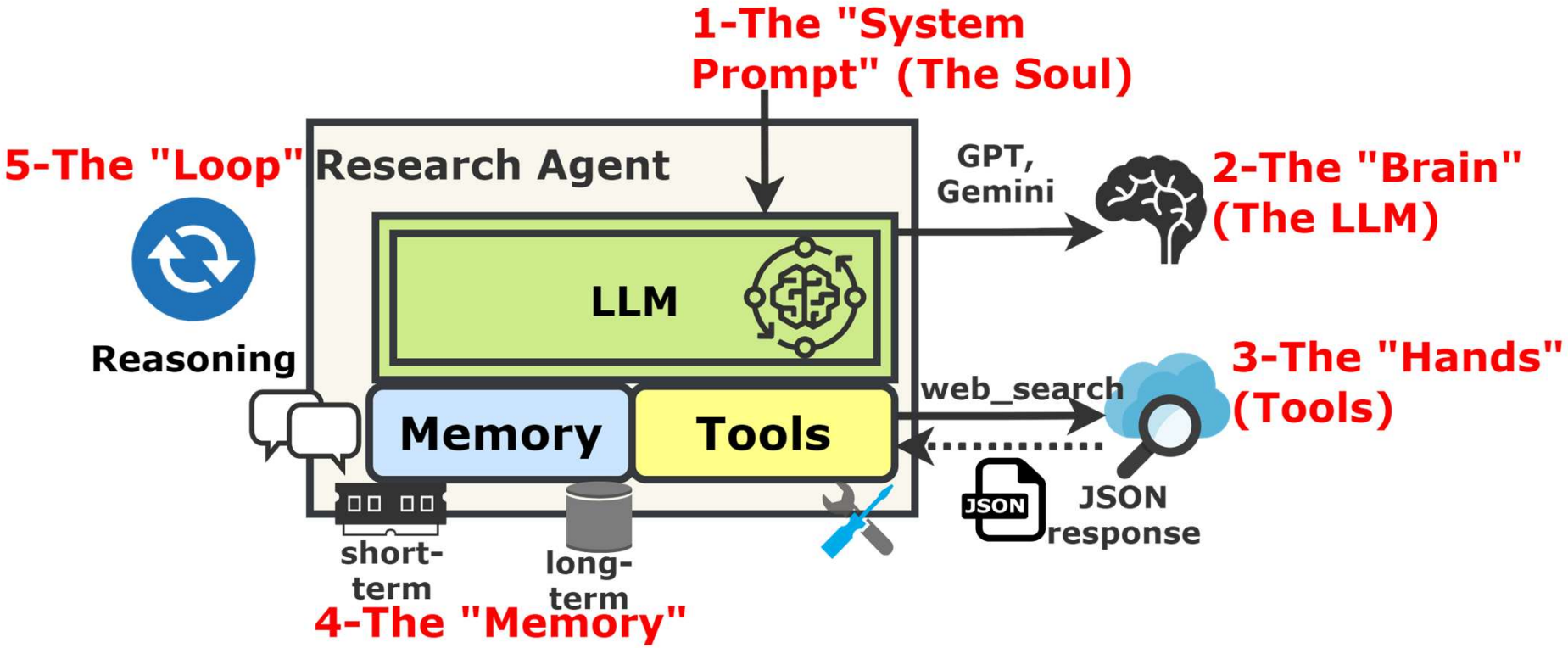
Microsoft:
Azure AI Foundry: A platform to build, manage, and deploy agentic applications, tightly integrated with Azure Functions and Azure AI Search

Google:
Google Cloud Agent Builder (Vertex AI): A unified platform for building agents that connect to Google's ecosystem



Their goal: Keep you in their ecosystem

Our 5-Step "Recipe" (The Agent Plan)



Architectural Trade-off: The PROS



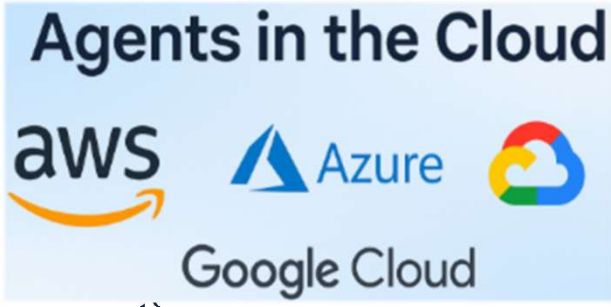
1. Fully Managed & Auto-Scaling:
Zero infrastructure overhead
This is the number one business driver

2. Deep Cloud Integration:
The easiest way to give an agent access to your existing cloud resources (S3 buckets, DynamoDB, Azure Functions)
Handles IAM permissions and credentials automatically

3. Speed to Market:
You can stand up a simple, robust and secure agent in hours, not weeks

4. Security & Compliance:
Comes with all the built-in logging, monitoring and compliance of the parent cloud (HIPAA, SOC 2)

Architectural Trade-off: The CONS



1. The "Black Box" Reasoning:
This is the critical drawback
You cannot see or control how the agent reasons (its ReAct prompt)
You cannot customize the error handling or retry logic
Debugging a "stuck" agent is nearly impossible

2. Extreme Vendor Lock-in:
Your agent is an API for a specific cloud. It is non-transferable

3. Limited Flexibility:
You cannot build the advanced patterns we've learned
(Group Chat, Magnetic Router, etc.)
You are limited to their definition of a simple, single-loop ReAct agent

The Architect's Decision: "Build" vs. "Buy"



Choose "Buy" (Managed Platform) When:
The task is simple, well-defined and internally-facing
The primary value is integrating with other services in that same cloud
Example: "An internal agent to help my support team look up order details from our DynamoDB"

Choose "Build" (Framework) When:
The task is complex, custom or non-deterministic
You need multi-agent collaboration (Group Chat)
The agent's reasoning logic itself is your competitive advantage
You need a multi-cloud or vendor-neutral solution.

Summary: The Three Categories

1. Generalists (LangChain):
For maximum flexibility and custom "build-your-own" logic

2. Specialists (LlamaIndex, MAF):
For best-in-class RAG or multi-agent collaboration

3. Managed Platforms (AWS, Azure, Google):
For simple, fast, and fully-managed "buy" solutions



Understanding AI Agentic Design Patterns

The Tool Use, Planning, Reflection Pattern (Metacognition)

The "ReAct" Compound Pattern

Introduction to Multi-Agent Patterns

The Router & Specialist, Handoff, Group Chat / Debate, Swarm

The "Human-in-the-Loop" (HITL) Pattern

Pradeep Kumar T



The "Why" of Agentic Design Patterns

Learned the **agentic loop**, explored **system-level architectures**, and just completed a deep dive into the **major frameworks**

We have mastered the **'tools'**—the **'how'** of **building agents**

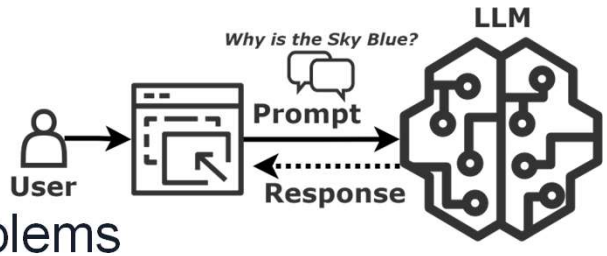
It's time to learn the **'blueprints'**—the **'what'** and **'why'** of agent design

This section is the **'Gang of Four'** for **Agentic AI**

Build a **formal**, **common vocabulary** for **designing robust**, **scalable** and **reusable agentic systems**



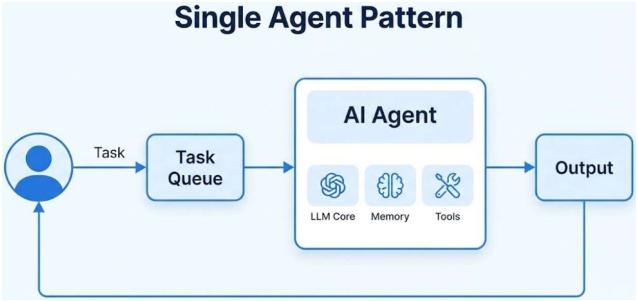
Moving Beyond "Prompt Engineering"



"Prompt Engineering" is a tactic
It's about crafting the perfect instruction for a single task
It's fragile and often breaks with new models or complex problems

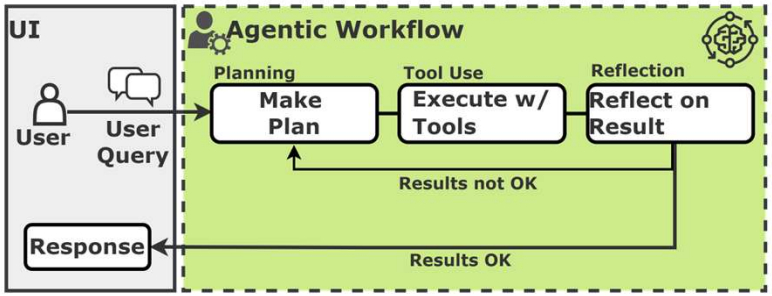
"Agentic Architecture" is a strategy
It's about designing a system of components that solves a problem reliably
This system is robust, testable and scalable

A "Pattern" is a key part of that strategy
It's the bridge from a clever prompt to a reliable system





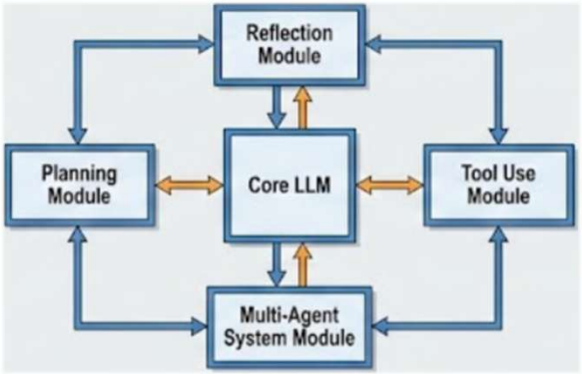
The "Why": Framework vs. Pattern



A Framework is a Tool (The "How")
A concrete library of code (LangChain, MAF)
A set of high-quality hammers, saws and power tools

A Pattern is a Blueprint (The "What" & "Why")
A conceptual, reusable solution to a common problem
The architectural blueprint for a 'self-correcting system' or a 'delegation system'

Key Insight: Patterns are framework-agnostic
You can implement the "Reflection Pattern" using LangChain, Microsoft Agent Framework or raw Python



The Value of Agentic Design Patterns

1. A Common Vocabulary:
"This requires a Router Agent" as an architect, will know exactly what that system looks like. We can communicate effectively

2. Proven, Tested Solutions:
Best practices the community has discovered for solving problems like "how to make an agent self-correct" or "how to make agents collaborate"

3. Reusable Blueprints:
Don't need to 'reinvent the wheel' every time. You can apply these patterns to any project, in any framework

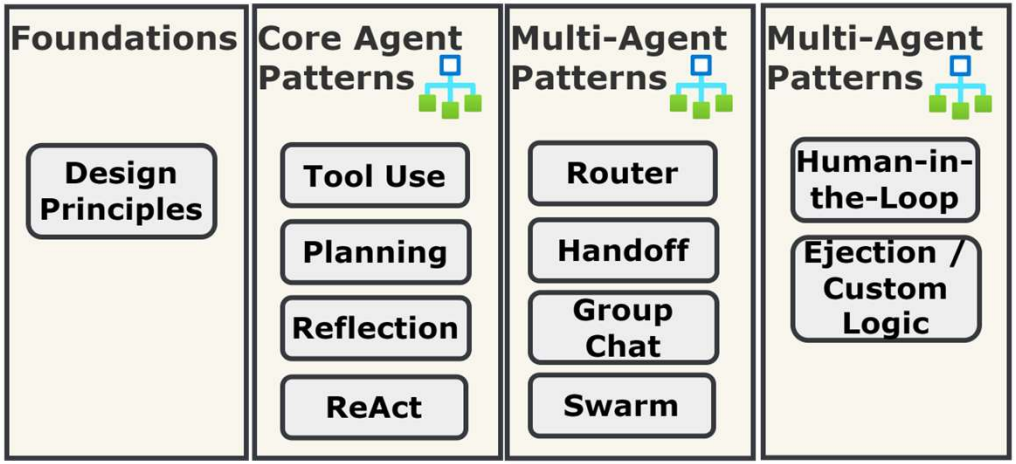


Our Pattern Catalogue: The Roadmap (Part 1)

We have organized this section into a formal catalogue

Part 1: The Foundations
The Agentic Design Principles

Part 2: Core Agent Patterns (The "Single Agent")
The "Tool Use" Pattern
The "Planning" Pattern
The "Reflection" Pattern (Metacognition)
The "ReAct" Compound Pattern



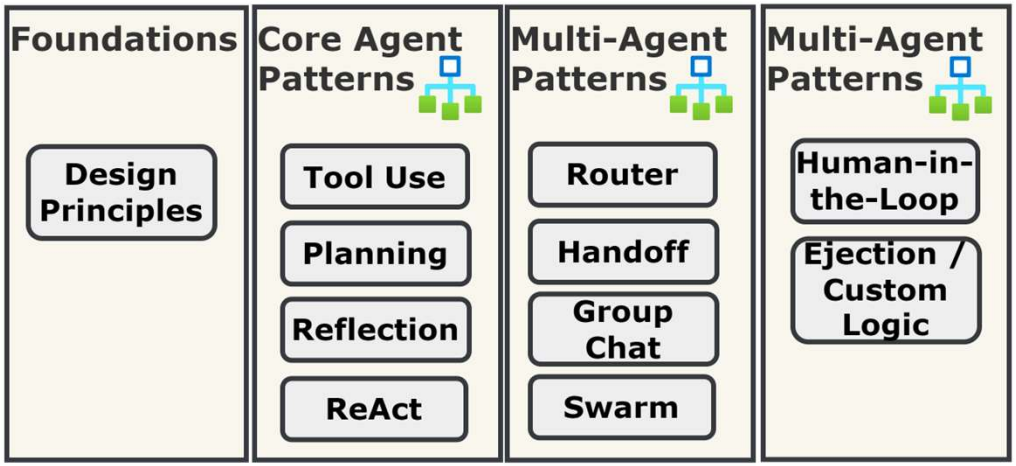
Our Pattern Catalogue: The Roadmap (Part 3-4)

Part 3: Multi-Agent Patterns (The "Collaborative Team")

Introduction to Multi-Agent Patterns
The "Router & Specialist" Pattern
The "Handoff" Pattern
The "Group Chat / Debate" Pattern
The "Swarm" Pattern

Part 4: Control & Safety Patterns (The "Enterprise")

Introduction to Control & Safety
The "Human-in-the-Loop" (HITL) Pattern
The "Ejection / Custom Logic" Pattern

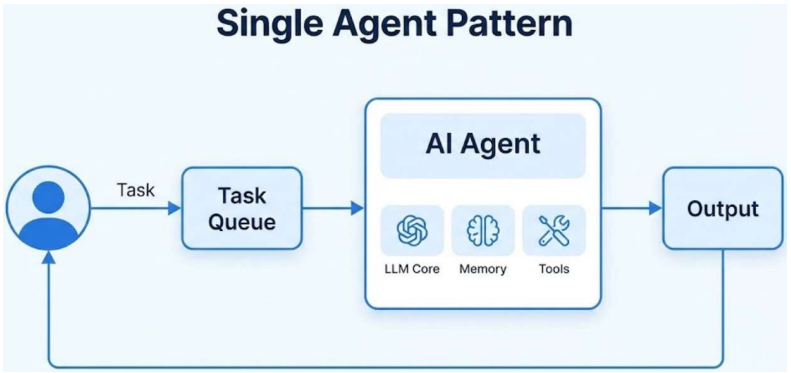


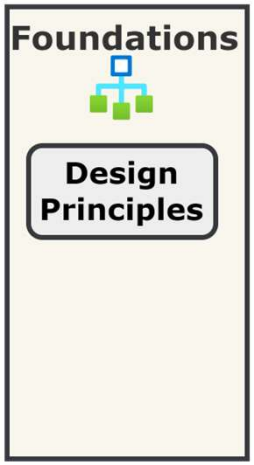
The Agentic Design Principles

We must first **understand the fundamental** 'laws of physics' that govern them

These are the **Agentic Design Principles**

4 principles are the **essential**, **non-negotiable characteristics** that make an **'agent'** an **agent**, and **not just a simple program**





The Four Pillars of Agentic Design

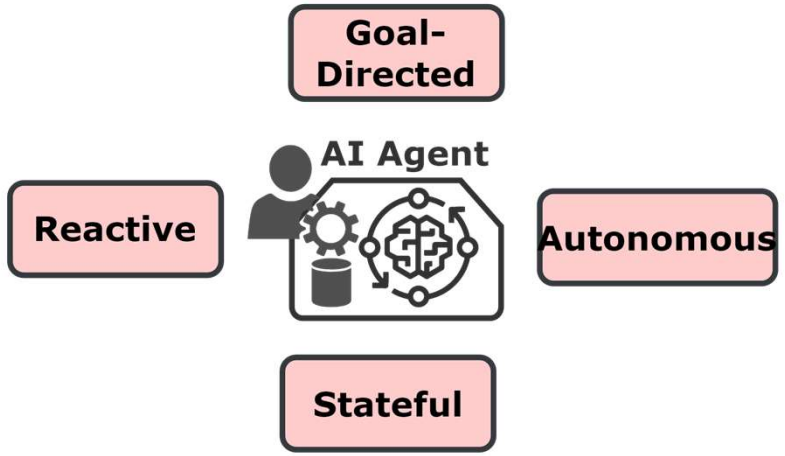
These principles are what differentiate an AI Agent from a traditional program

1. Goal-Directed:
It is driven by an objective, not just an instruction

2. Reactive:
It perceives and responds to its environment

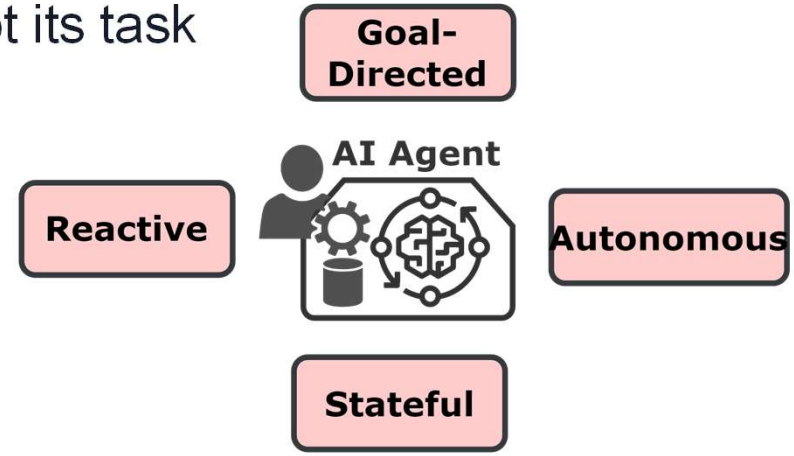
3. Stateful:
It maintains a memory of past events and actions

4. Autonomous:
It acts on its own to make progress toward its goal



Principle 1: Goal-Directed

The Concept: An agent is defined by its purpose, not its task



Instruction-Driven (Old Way):
call_api("weather", "London")
call_search("best laptop")
You must provide the exact steps

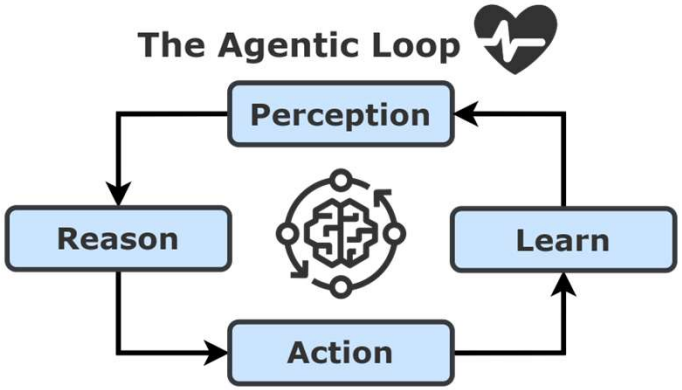
Goal-Driven (New Way):
Goal: "Plan a 3-day trip to London for me"
The agent must discover the steps (check weather, find flights, book hotels)

Architectural Implication:
The 'Goal' is the primary input that initiates the reasoning loop. All planning is derived from this goal

Principle 2: Reactive

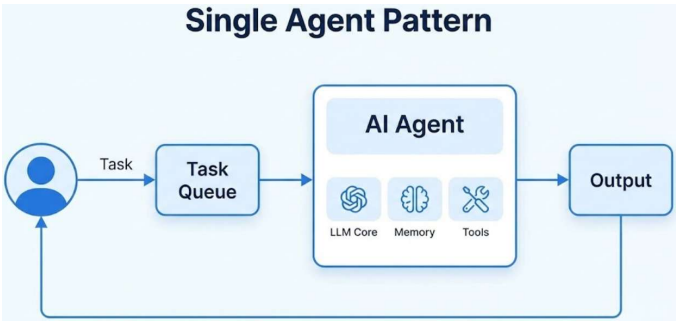
The Concept: An agent must be able to perceive and respond to its environment

This is the "**Perception**" phase of our P-R-A-L loop



The "environment" can be many things:
A user's follow-up question
The result of a tool call (an API error)
New data (a new email arriving)

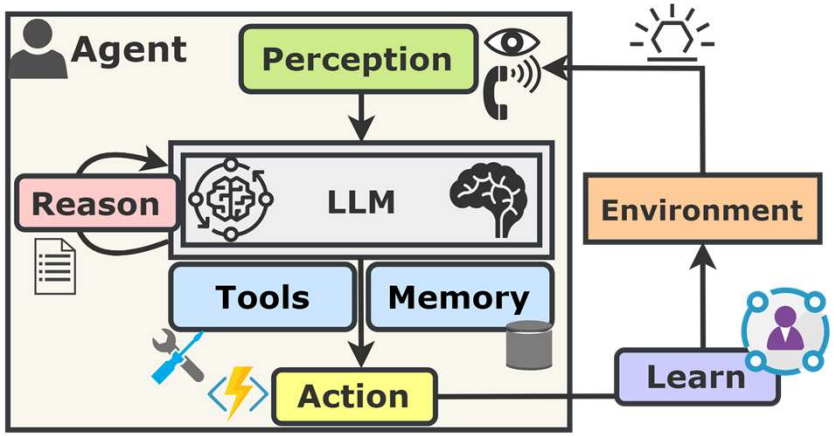
Architectural Implication:
The agent's loop cannot be a 'straight line.' It must be a cycle
It must react to unexpected events and errors, not just crash



Principle 3: Stateful (Has Memory)

The Concept: An agent must remember what has happened

State (Memory) is the "glue" that connects loops



It remembers:
The original goal
The chat history (what was said)
The tool-call history (what was done)
Its "scratchpad" (its current thoughts)

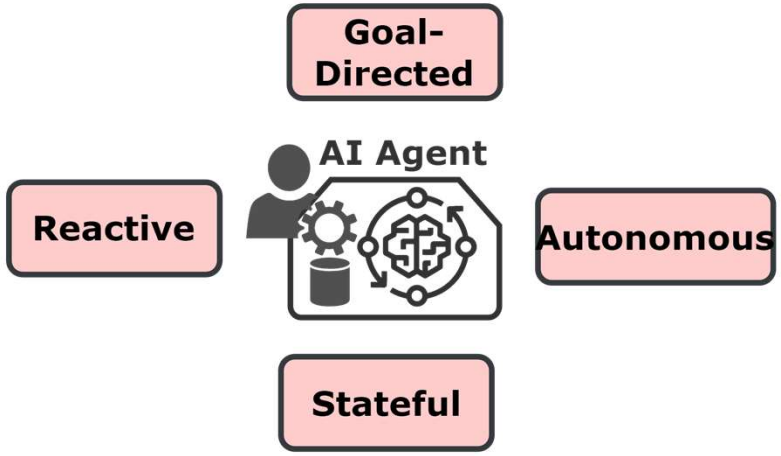
Architectural Implication:
State Management is a core architectural problem

We must design a "**Memory**" component (a database, a state object) that persists between agentic loops

Principle 4: Autonomous

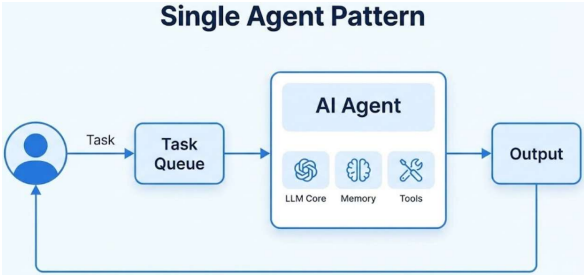
The Concept: An agent can act on its own to make progress

This is the result of the other three principles

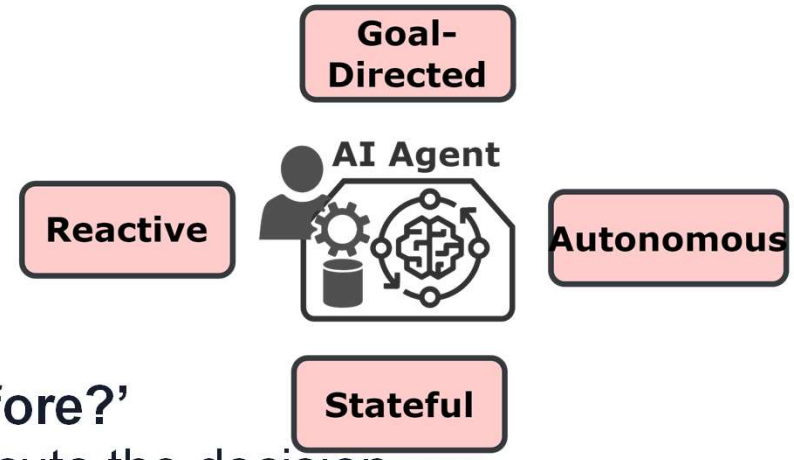


Because an agent is...
Goal-Directed (it knows its destination),
Reactive (it can see the road),
Stateful (it knows where it's been),
...it can be Autonomous (it can "drive the car")

Architectural Implication:
We must grant the agent agency—the ability to execute actions (call APIs, write to files) without step-by-step human confirmation



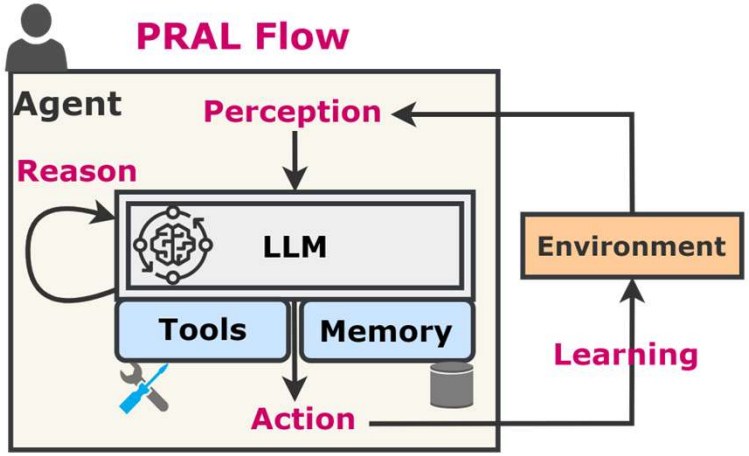
The Principles as a System



No principle exists in isolation

1. The **Goal defines** the **'why**'
2. **Reactivity (Perception)** provides the **'what now?'**
3. **State (Memory)** provides the **'what happened before?'**
4. **Autonomy (Action)** is the 'how'—the ability to execute the decision.

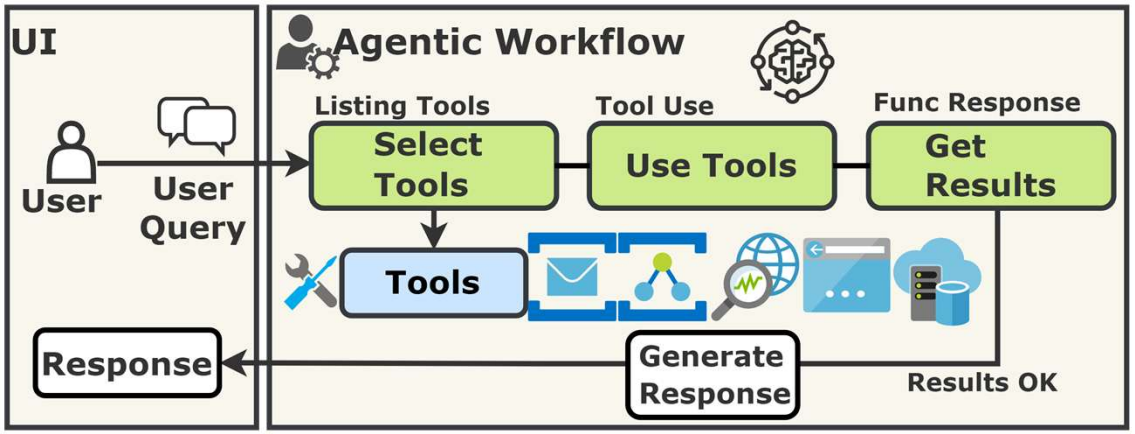
These **4 principles** form the **engine** of our **agent**

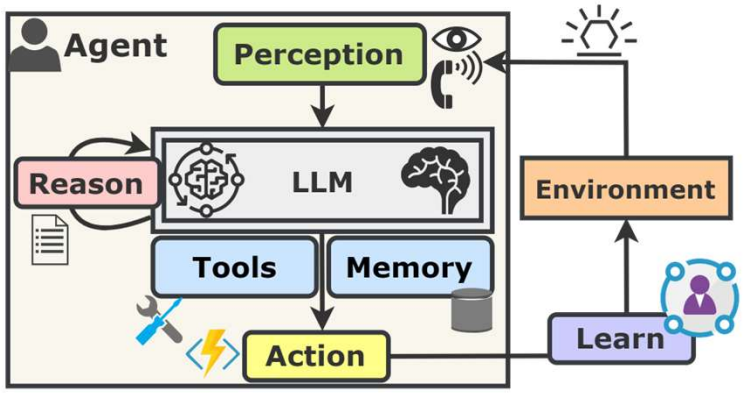


The "Tool Use" Pattern

An **LLM** is just a **'brain in a jar'** It can think, but it cannot act

The **'Tool Use' Pattern** is the architectural blueprint for giving that **brain 'hands'** to **interact** with the **world**





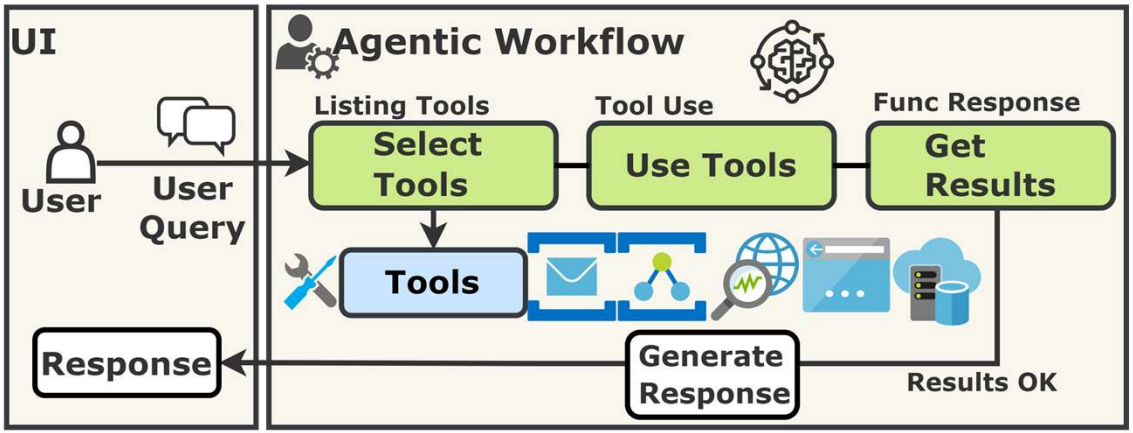
The Core Problem: A Brain Without Hands

LLMs are "Text-In, Text-Out" Systems:
An LLM cannot browse the web, check a database or even know the current date

The LLM's "World":
Its entire world is the text in its context window. It is "locked in"

The Question:
How do we connect this "brain" (the LLM's reasoning) to the "real world" (our code, our APIs, our databases) ?

The Answer: The "Tool Use" Pattern



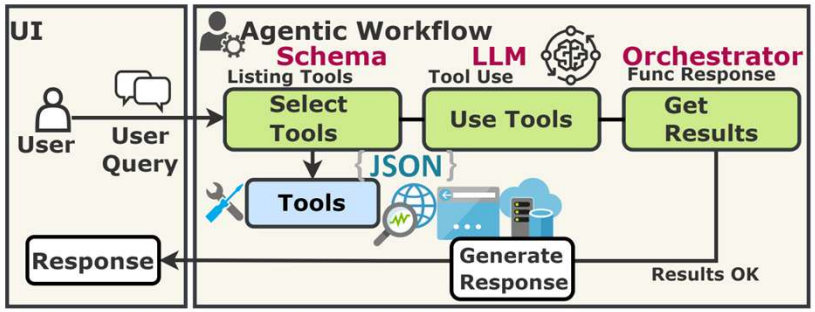
The "Tool Use" Pattern Defined

A 3-part architecture that allows an LLM to select and invoke external code

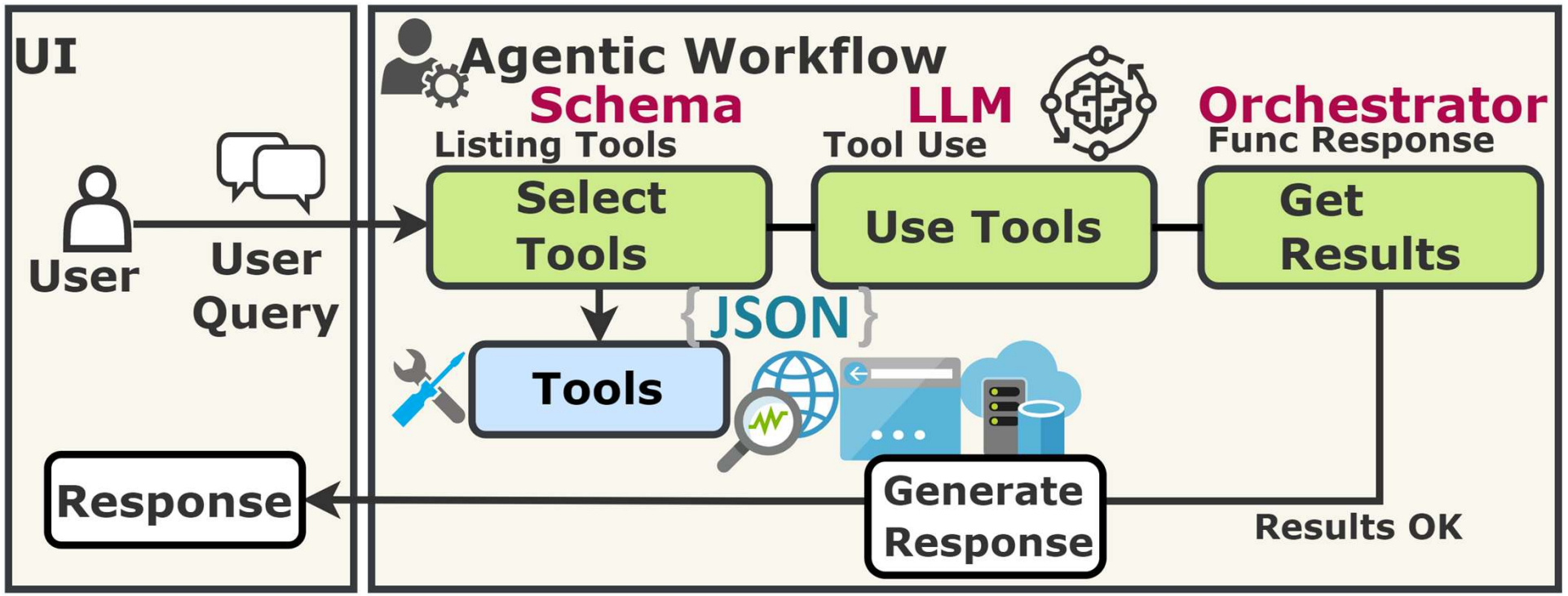
1. The "Schema":
A description of the tool (like a function signature) that the LLM can read and understand

2. The "LLM" (The Chooser):
The brain that reasons about the goal and decides which tool to call, and with what parameters

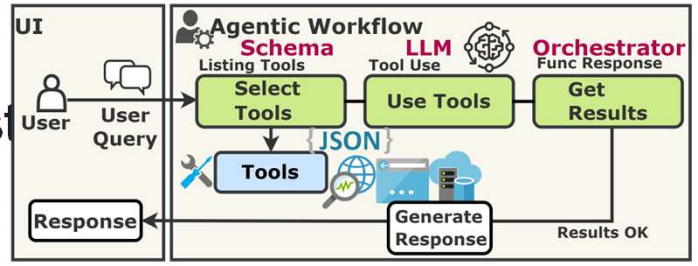
3. The "Orchestrator" (The Executor):
The code we write that detects the LLM's decision, executes the tool, and returns the result



The "Tool Use" Pattern Defined



Mechanics Part 1: The "Schema" (Describing the Tool)



Goal: To make our code "**visible**" to the LLM

We define a "**schema**" (in JSON) that the LLM can underst

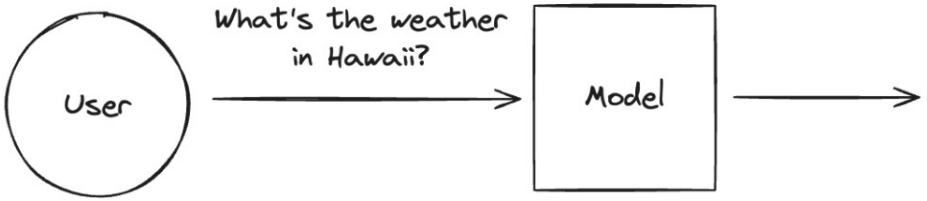
This schema MUST include:

1. name: The exact function name (get_weather)

2. description: The most important part. A clear, natural language description of what the tool does ("Gets the current weather for a specific city")

3. parameters: An object describing the arguments

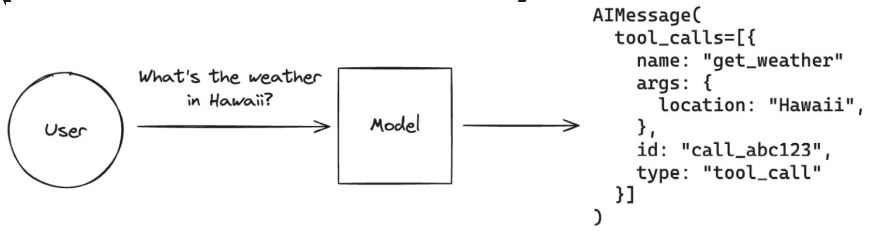
city: (type: string, description: "The city name, 'London'")




```
AIMessage(
  tool_calls=[{
    name: "get_weather"
    args: {
      location: "Hawaii",
    },
    id: "call_abc123",
    type: "tool_call"
  }]
)
```

Mechanics Part 2: The LLM's Role (The "Chooser")

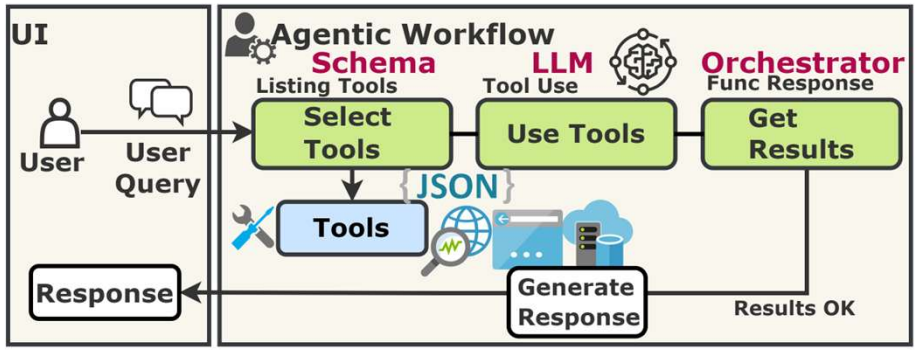
The LLM's Reasoning Process:



1. It receives the user's goal: "What's the weather
2. It reviews the list of available tool schemas
3. It reasons: "The goal is about weather. The get_weather tool's description matches this goal. The goal specifies a city, 'London'. The tool requires a 'city' parameter"

The LLM's "Action":

It does not write a response to the user
It outputs a special, structured JSON object



Output:

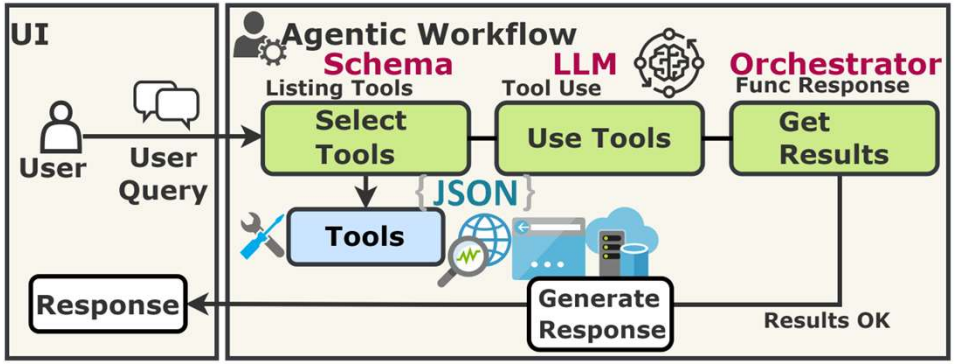
{"tool_call": {"name": "get_weather", "arguments": {"city": "London"}}}

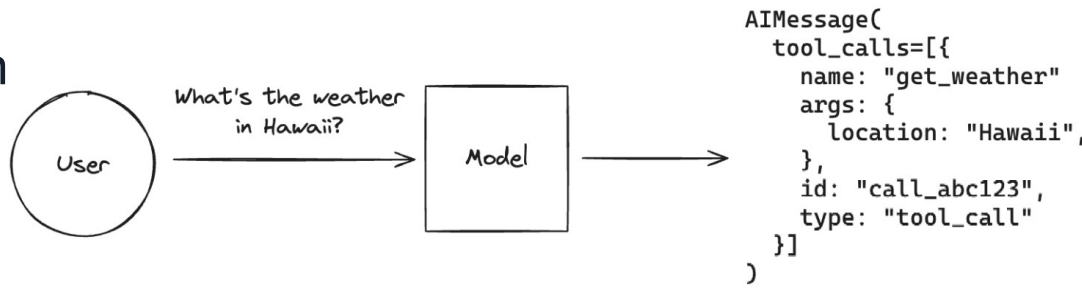
Mechanics Part 3: The Orchestrator's Role (The "Executor")

This is **our code** we **responsible**

The Orchestrator's Job:

1. Receive the JSON "tool call" from the LLM
2. Parse the JSON:
   Get name: "get_weather"
   Get arguments: {"city": "London"}
3. Map the name to a real Python function
   if name == "get_weather":
4. Execute the function:
   result = get_weather(city="London")
5. Return the result (e.g., "25°C") back to the LLM for the next loop





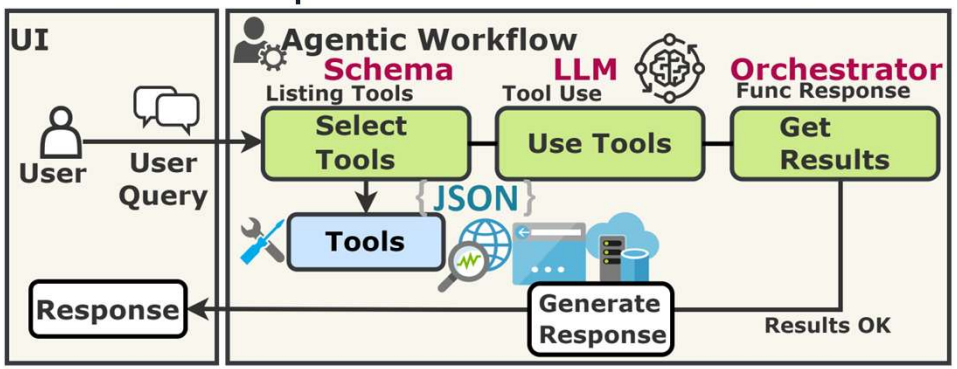
Summary & Architectural Implications

The Pattern: Schema (description) + LLM (chooser) + Orchestrator (executor)

1. "Prompt Engineering" for Tools:
The quality of your tool's description is now one of the most important parts of your prompt. A bad description means the tool will never be used

2. Security & Validation:
The Orchestrator must be a secure gate. It must validate all arguments from the LLM. Never trust the LLM's output directly (e.g., prevent SQL injection)

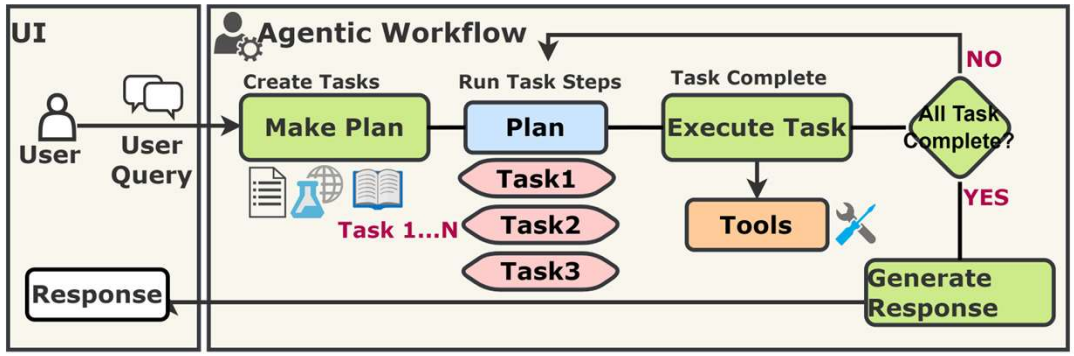
3. Error Handling:
The quality of your tool's description is now one of the most important parts of your



The Core Problem: From Single-Step to Multi-Step

The "Tool Use" Pattern (Single-Step):
Goal: What's the weather in London?
Action: call_weather("London")

The "Planning" Problem (Multi-Step):
Goal: Email me the weather in London



This requires a sequence:
1. call_weather("London")
2. call_email("recipient@...", "Here is the weather: [result from step 1]")

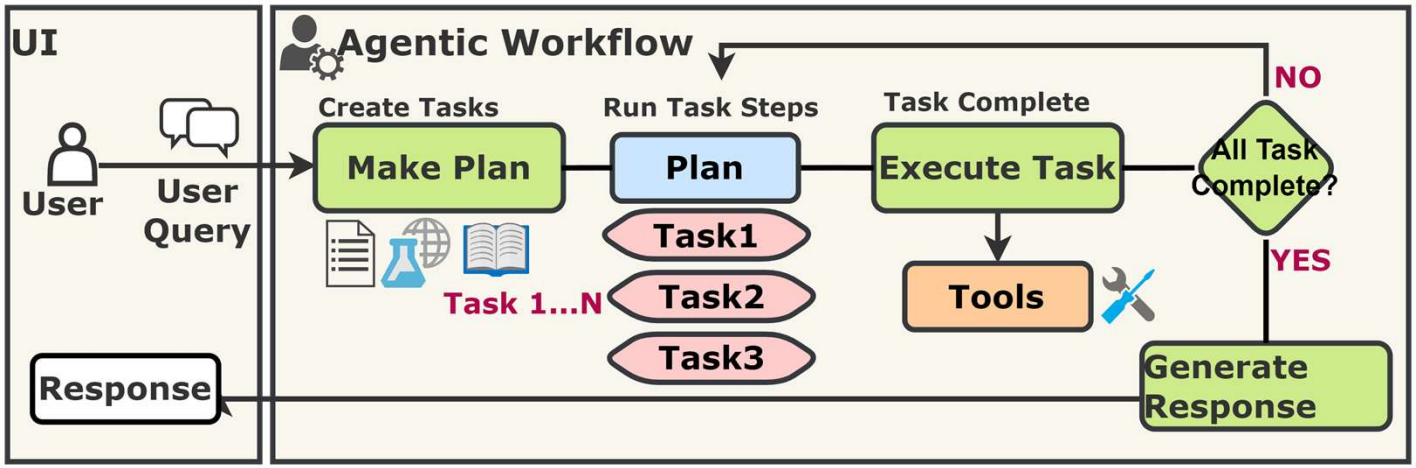
How does the agent create this two-step sequence?
How does it know not to call the email tool first?

The "Planning" Pattern

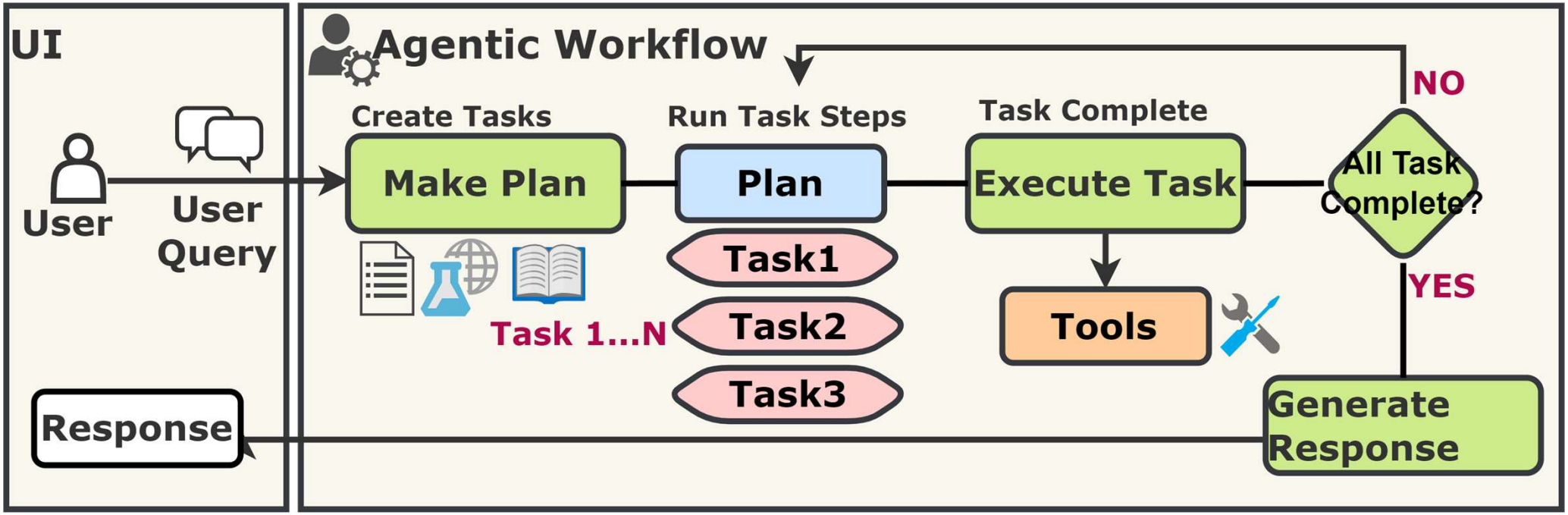
What happens when a user's goal is not a single query but a **complex**, **multi-step project?**

'Research the top 3 sci-fi books from last year, **find** their **prices** and **send** the **final list** to my **email**.'

This **cannot be solved by a single tool call**. It **requires a sequence of actions**. This is where the **'Planning' Pattern** becomes essential.



The "Planning" Pattern

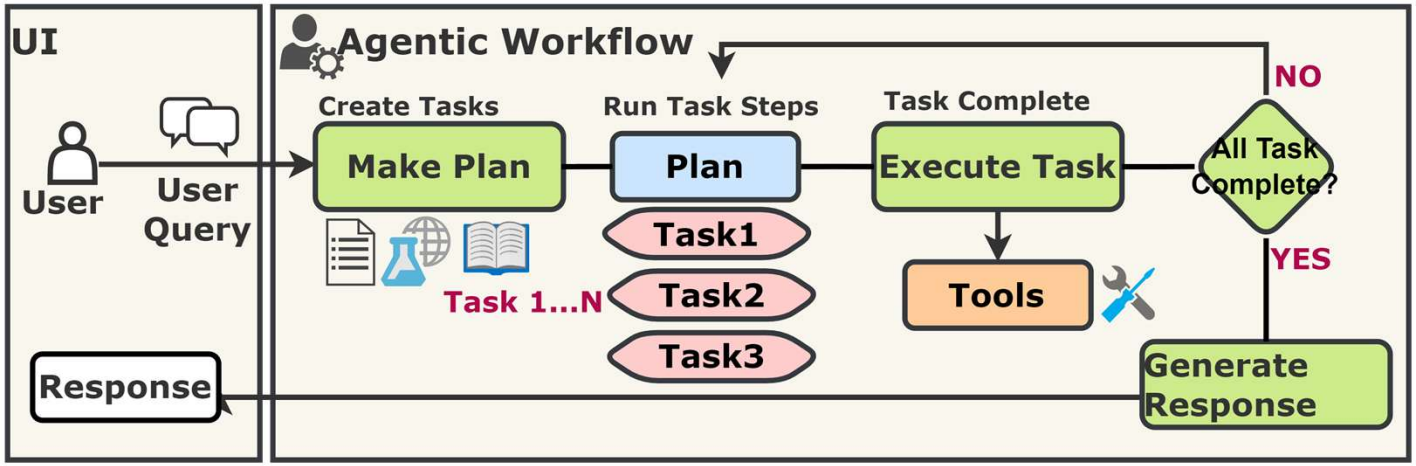


The "Planning" Pattern Defined

The **process** of **using** an **LLM** to **decompose** a **high-level goal** into an **explicit, ordered list** of **executable steps**

The LLM as a "Planner":
We stop asking the LLM to "give me the answer"
We start asking the LLM to "give me the plan to find the answer"

The Plan Itself:
The "plan" is simply a data structure—a list of future actions (e.g. a text list, a JSON array)

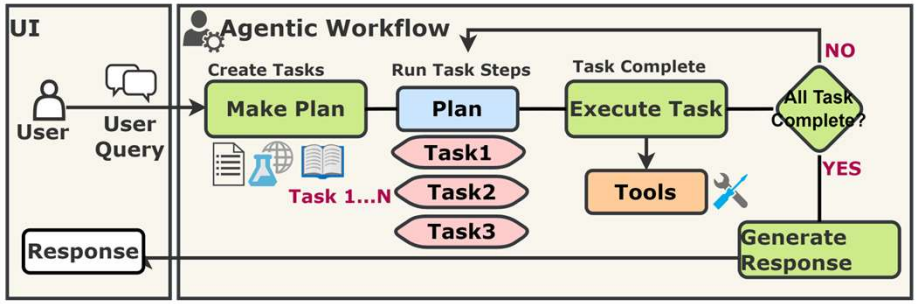


The Enabling Technique: "Chain of Thought" (CoT)

Question: How do we force the LLM to create this plan?

By instructing it to "think step-by-step." This is "Chain of Thought"

Standard Prompt:
Goal: "Email me the weather in London."
Response: [LLM tries to call a tool, might fail]



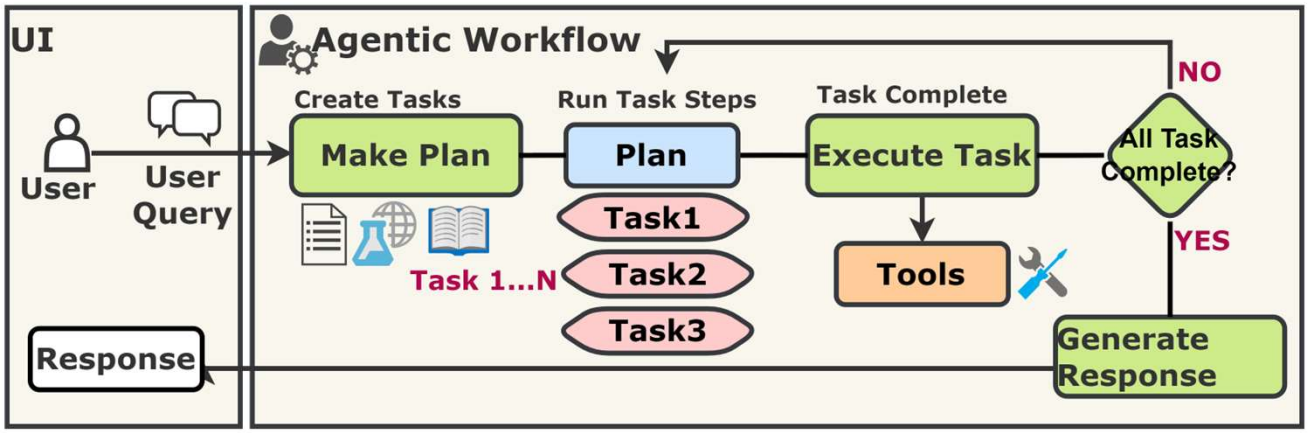
CoT / Planning Prompt:
Goal: "Email me the weather in London"
Instructions: "First, create a plan. Then, execute the first step"

The Enabling Technique: "Chain of Thought" (CoT)

LLM's Internal Monologue (CoT):
Thought: The user wants the weather, and they want it emailed.
Plan:

1. Call get_weather for "London".
2. Call send_email with the result of step

Action: [Outputs the first tool call: call_weather("London")]



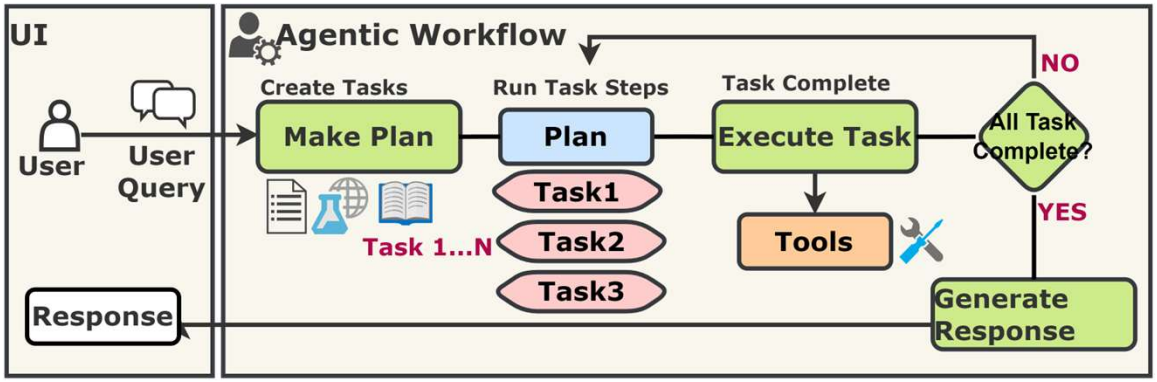
Plan Execution: The Orchestrator's New Role

The "Planning" Pattern makes the Orchestrator's job more complex

The Orchestrator must now be a "State Machine":

1. It must store the full plan (e.g. in the agent's memory)
2. It must track the current step (e.g. "Step 1 of 2")
3. It executes the current step (e.g. call_weather)
4. It gets the result and feeds it back to the LLM
5. The LLM perceives the result and continues to the next step.

This loop of Plan -> Act -> Observe -> Continue Plan is the very foundation of advanced agents

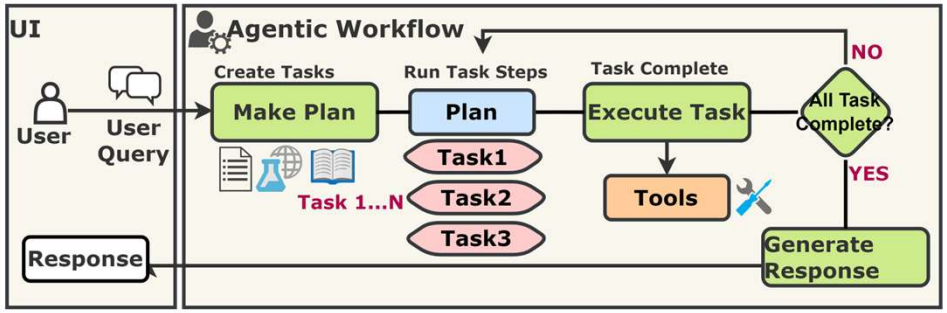


How to Design «Planning» Process as Architects ?

1. Prompting is for Process, Not Just Results:
Your system prompt must now define the planning process ("Think step-by-step," "Create a plan," "Break down the problem")

2. Plan Management is Required:
You must have a component (Orchestrator) that can store, read and update the plan in the agent's memory

3. Handling Failures:
This pattern introduces a new risk: What if Step 1 succeeds, but Step 2 fails?
The agent must be able to perceive the failure and re-plan
For example: "Step 2 send_email failed. New plan: notify_user_of_failure."



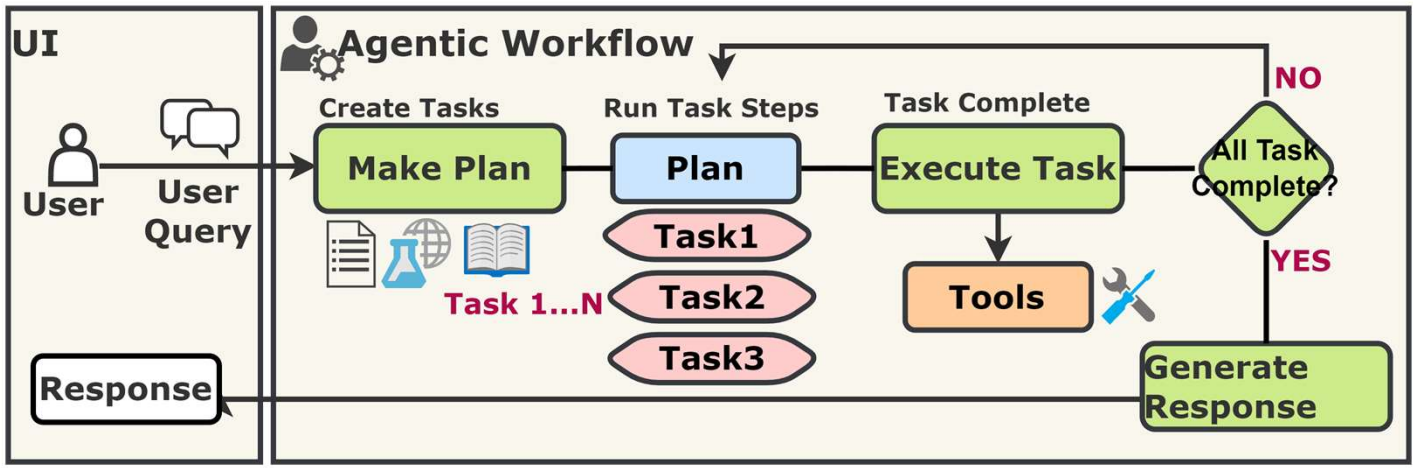
Summary: The Internal Monologue

The "**Tool Use**" Pattern gives the agent "**hands**"

The "**Planning**" Pattern gives the agent an "**internal monologue**"

It's the agent's **ability to think before** it **acts** and **decompose** a **large goal** into a **sequence of small, manageable actions**

This is the **difference between** a **reflexive agent** and a **thoughtful agent**

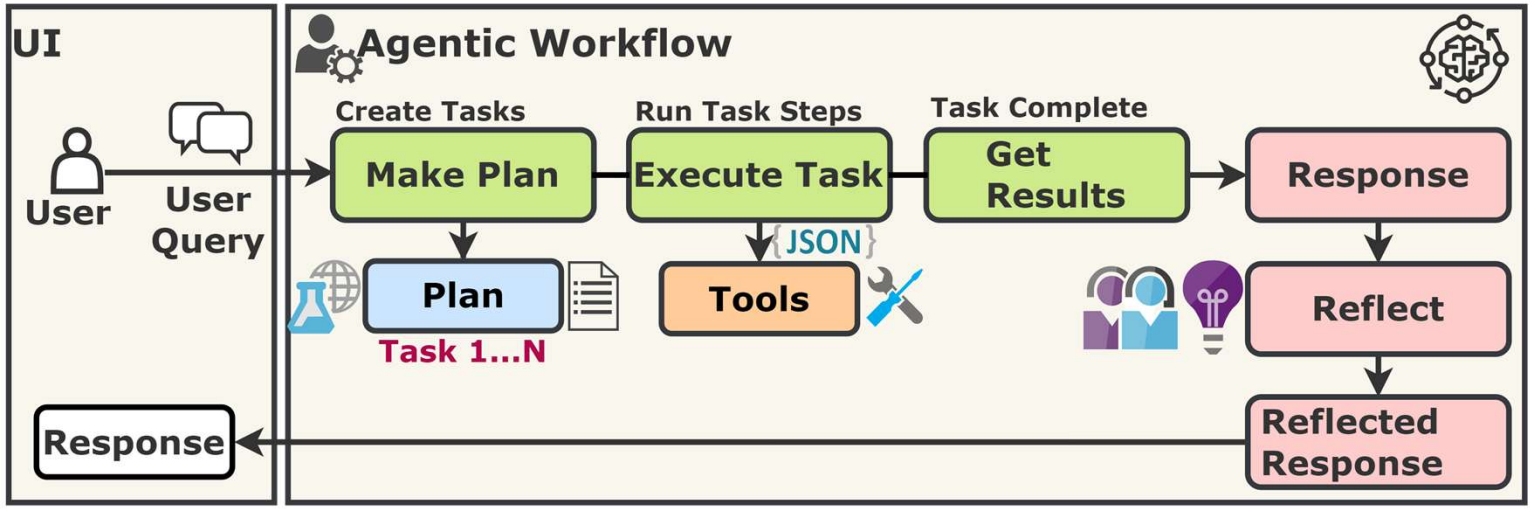


The "Reflection" Pattern (Metacognition)

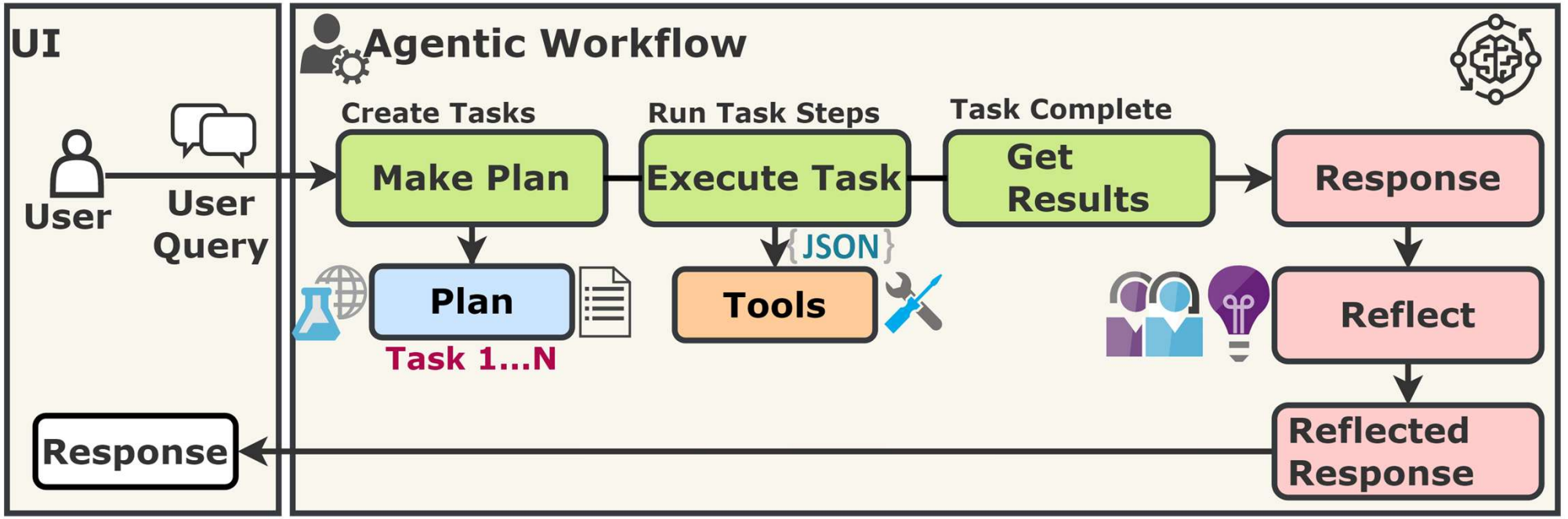
'Planning' Pattern, which gives our agent an **'internal monologue'** to create a **multi-step plan**

What if the agent follows the **plan perfectly**, but the **result is simply low-quality?**

The plan **'succeeded**,' but the **output** is **wrong**, **incomplete** or **poorly written**
How does an agent 'know' if its own work is any good?



The "Reflection" Pattern (Metacognition)

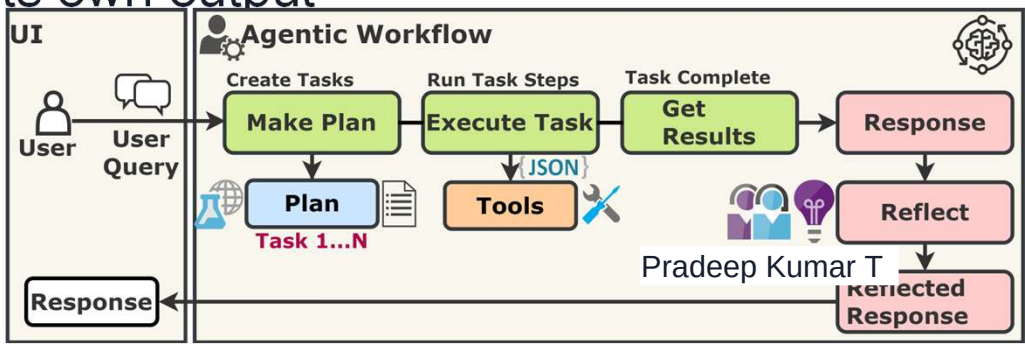


The Core Problem: The "Quality-Blind" Agent

The Scenario:
Goal: "Write a 500-word blog post about the benefits of microservices"
Plan: 1. search("benefits of microservices")2.write_blog_post()`

The "Successful Failure":
The agent executes step 1
The agent executes step 2 and writes a 500-word post
The Problem: The post is repetitive, misses key points and has an awkward tone

The "Blindness":
The agent, having followed its plan, will report "Success!"
It has no mechanism to judge the quality of its own output



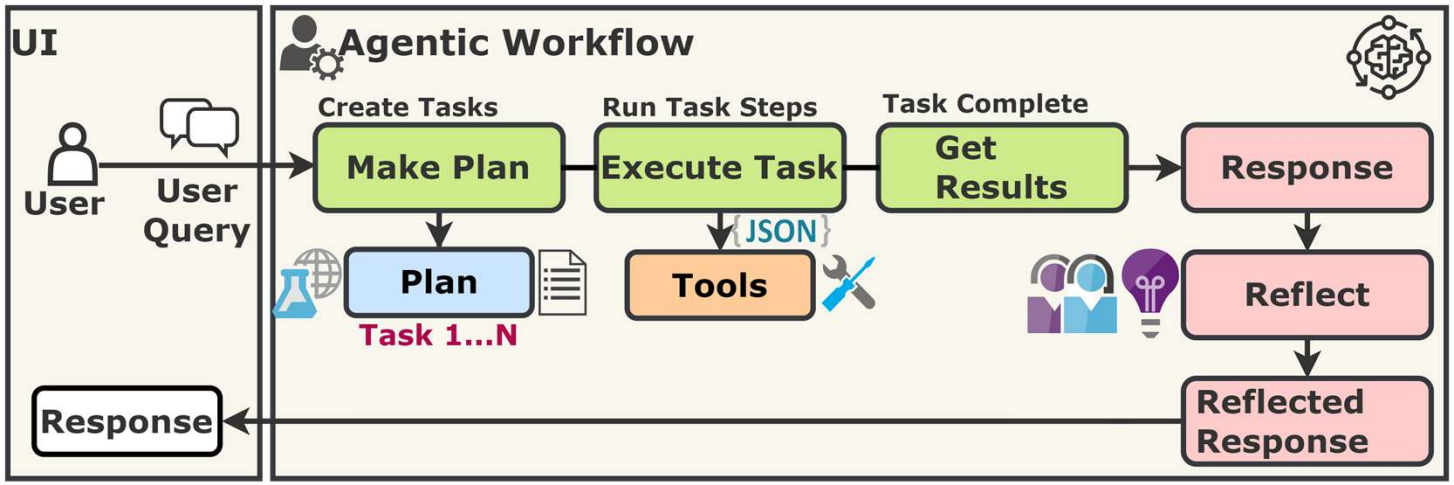
The "Reflection" Pattern (Metacognition) Defined

Metacognition: The simple idea of "**thinking about your own thinking**"

An **architectural pattern** where an **agent pauses after producing an output** and **uses** a **second, critical process** to **evaluate that output** against a set of criteria.

The Goal: To move from "**Did I do the task?**" to "**Did I do the task well?**"

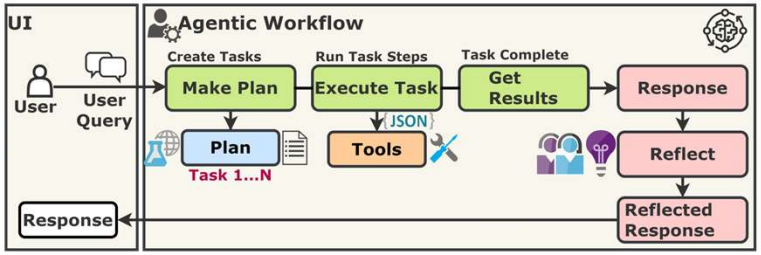
This is the blueprint for **self-correction** and **iteration**



The Core Implementation: The "Writer-Critic" Model

This pattern is almost always implemented as a "**Writer-Critic**" workflow

1. The "Writer" (The Doer):
This is the standard agent we've been building
It follows a plan, uses tools, and produces the first draft (e.g. the blog post, a piece of code)



2. The "Critic" (The Reflector):
This is a separate LLM call with a different, specialized prompt
Its prompt is: "You are a 'Critic'. Review the [Writer's Draft] based on the [Original Goal] and these [Quality Criteria]. Provide feedback for improvement

The "**Critic**" is just another agent, but with a **specialized**, **reflective purpose**

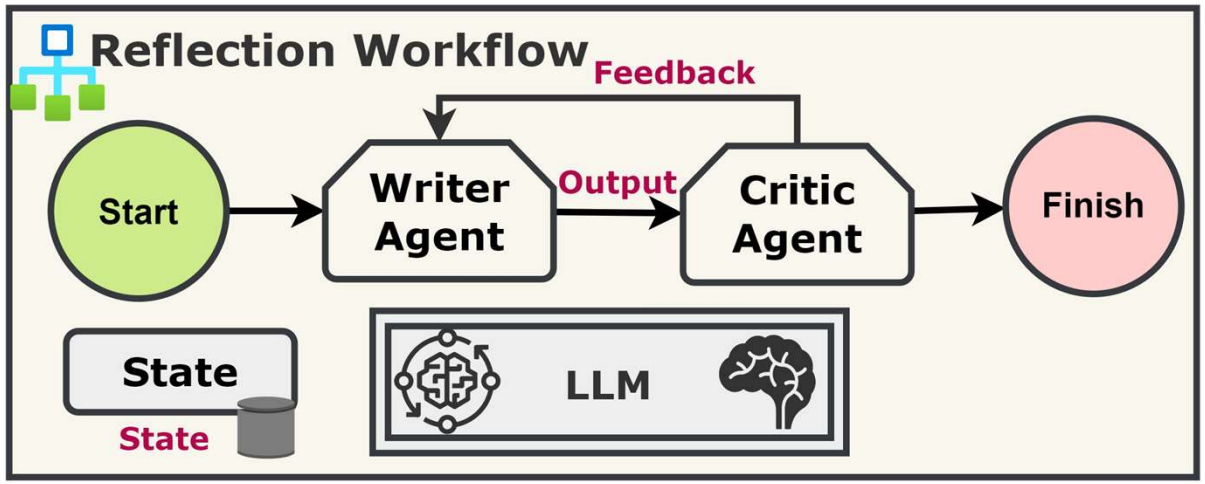
The "Reflection" Workflow (The Iterative Loop)

[Goal -> [Writer] -> Draft 1 -> [Critic] -> Critique]

From the Critique, the workflow branches:

1. IF Critique says "Output is good" THEN -> Final Answer
2. IF Critique says "Output has flaws" THEN -> Feedback

This creates an "**inner loop**" of **self-improvement** before the user ever sees the result

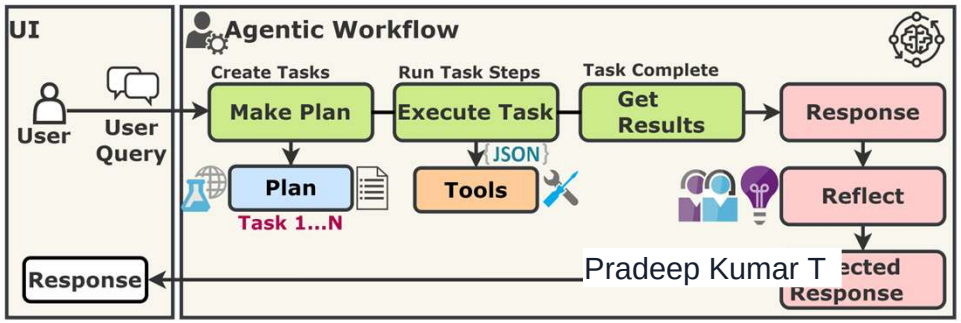


How to Design «Reflection» Process as Architects ?

1. Increased Cost and Latency:
This pattern is expensive. Each "reflection loop" is at least one additional LLM call
This can double or triple your cost and response time

2. Advanced Workflow Orchestration:
Your orchestrator must now support cycles (loops)
This is precisely what tools like LangGraph are designed for
A simple linear "chain" cannot implement this pattern

3. Prompt Engineering for "Personas":
You must now design and maintain at least two distinct system prompts:
- A "Writer" prompt (focused on "doing")
- A "Critic" prompt (focused on "reviewing")



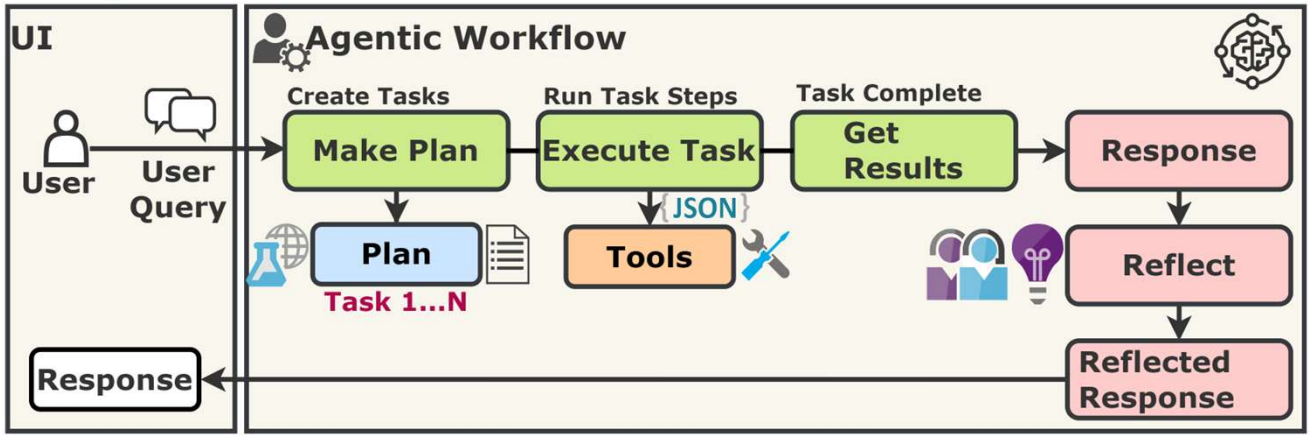
Summary: The Path to Quality

The "**Planning**" Pattern gives the agent an **internal monologue** to create a plan

The "**Reflection**" Pattern gives the **agent an internal critic** to **improve the quality** of that **plan's output**

This is the difference between a "**first-draft**" **agent** and a "**final-draft**" **agent**

It is a foundational pattern for building **reliable**, **high-quality** and **(semi)-autonomous** systems

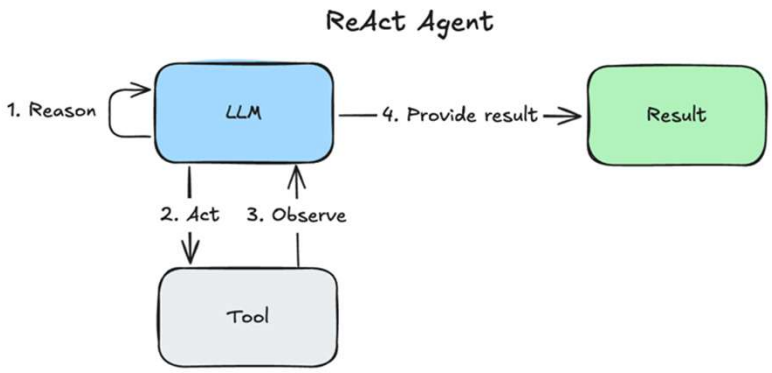


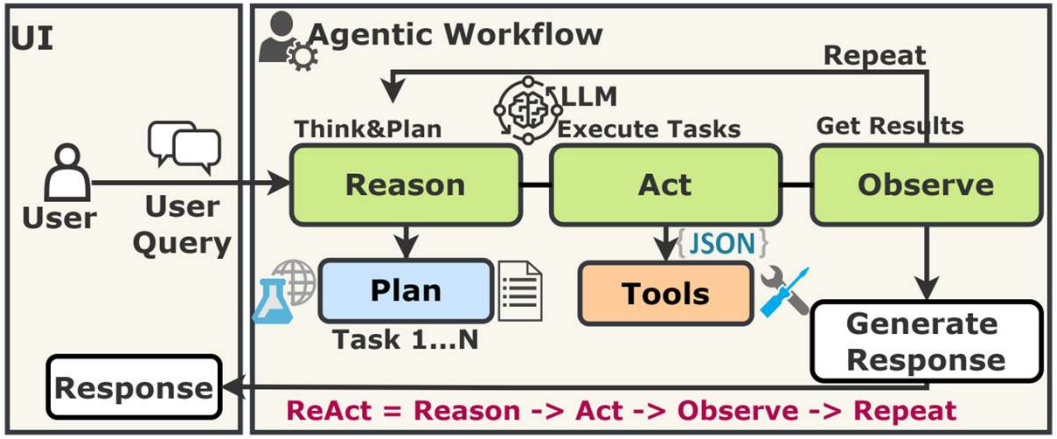
The "ReAct" Compound Pattern

'Reflection' Pattern, which gives an agent an **'internal critic'** for quality

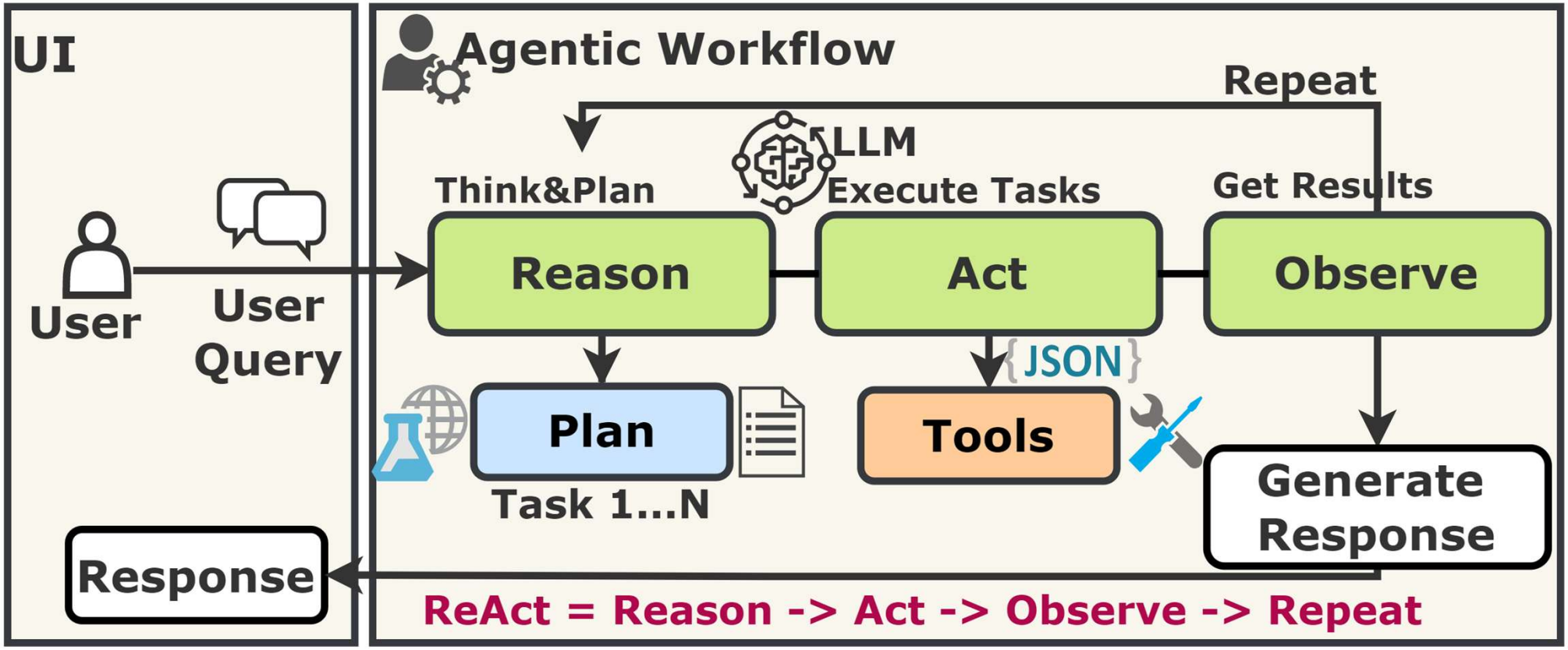
Combine 'Planning' and **'Tool Use'** into a single, tight, powerful loop

This combination is not a new idea; it is a formal pattern you will see everywhere, known as **'ReAct'**





The "ReAct" Compound Pattern



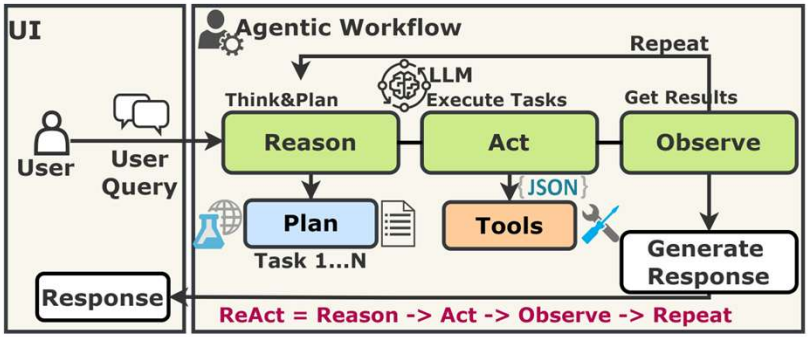
The Core Problem: The "Brittle" Plan

The Scenario:

Goal: "Find the top 3 sci-fi books and email me the list.“

Plan:

1. search("top 3 sci-fi books 2024")
2. send_email("user@...", [results from 1])



The "Brittle" Problem:

What if Step 1 (search) fails or returns no results?

The agent, following its rigid plan, will still try to execute Step 2

It will try to email an empty list, failing the user's actual goal

The Question:

How can an agent adapt its plan based on the results of each action ?

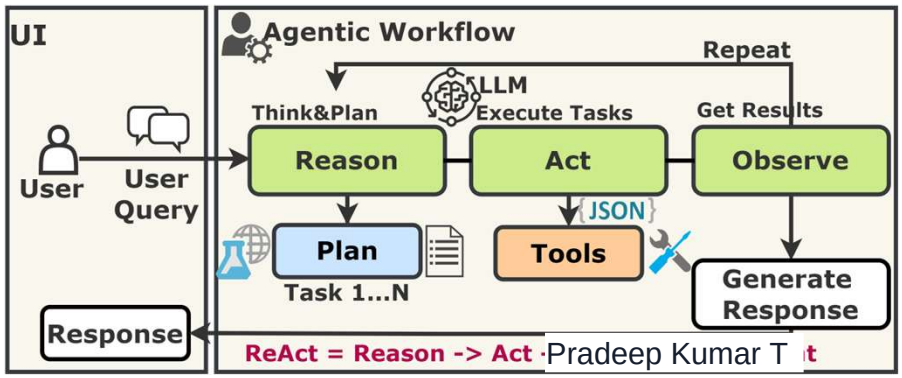
The "ReAct" Pattern (Reason + Act) Defined

Compound pattern that interleaves the "Planning" and "Tool Use" patterns in a tight, iterative loop

The Loop:

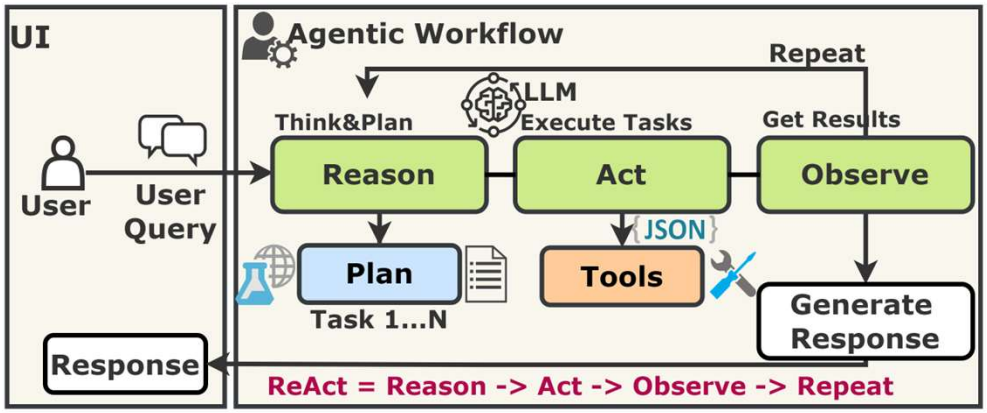
1. **Reason (Think):** The agent generates a thought (a plan for the very next step)
2. **Act:** The agent executes that single step (a tool call)
3. **Observe:** The agent perceives the tool's result
4. **Repeat:** The agent reasons about the observation and plans the next step

This is not "Plan-then-Execute."
It is "Plan-Act-Observe-Plan-Act-Observe..."



Deconstructing the "ReAct" Loop

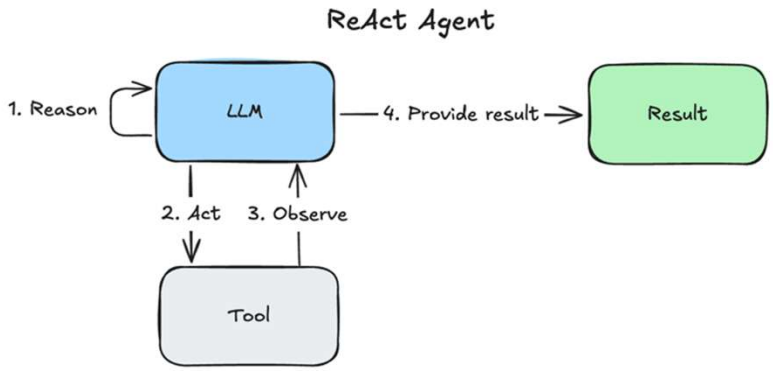
The "Reason" (Thought) Phase is...
The "Planning" Pattern
The LLM is prompted to produce a Thought
Thought: I need to find the weather.
I will use the get_weather tool



The "Act" (Action) Phase is...
The "Tool Use" Pattern
The LLM is prompted to produce an Action:
Action: { "tool_call": "get_weather", "arguments": ... }

The "ReAct" Prompt:
The system prompt simply forces the LLM to output its response in this exact Thought, Action format

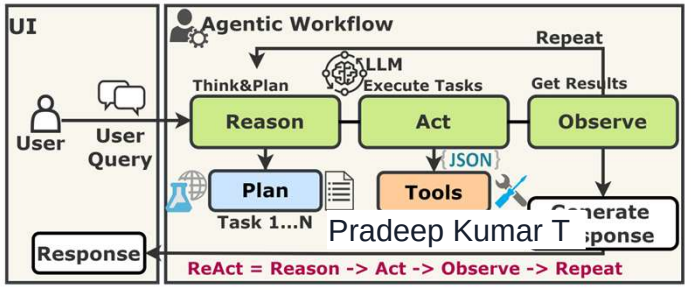
The Full ReAct Loop in Action (Example)



Goal: What's the weather in London?

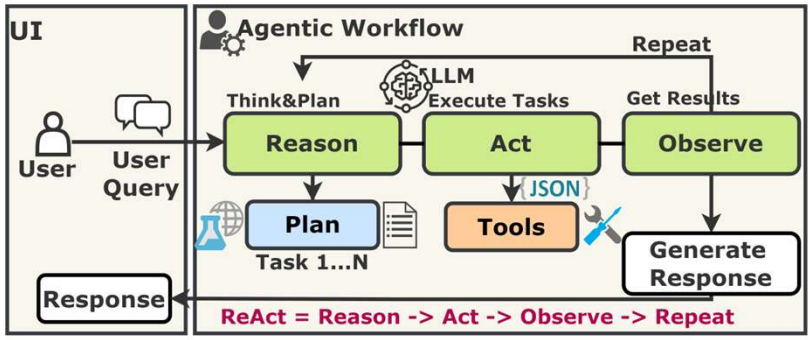
Loop 1:
LLM Reason: "I need to find the weather for London"
LLM Act: call_weather("London")
Orchestrator Executes: Runs the tool
Observation (Tool Result): "25°C, Sunny"

Loop 2:
Orchestrator Feeds Observation to LLM
LLM Reason: "I have the weather information. I can now answer the user"
LLM Act (Final Answer): "The weather in London is 25°C and Sunny"



Pradeep Kumar T

Why This is a "Compound Pattern"



It is a **combination** of **other patterns**

ReAct = "Planning" + "Tool Use" + "Loop"

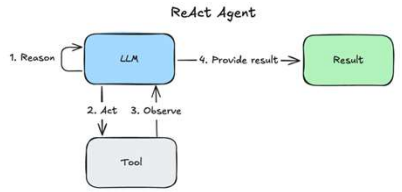
The "Reflection" Pattern is also often part of this loop!

Advanced ReAct Loop:

1. Reason (Plan): I will try to write the blog post
2. Act (Tool Use): write_blog_post(...) -> [Draft 1]
3. Observe (Perceive): The agent gets [Draft 1]
4. Reason (Reflection): This draft is weak. I will use the critique_draft tool to improve
5. Act (Tool Use): critique_draft([Draft 1]) -> [Feedback]

ReAct is the **engine** that lets all the **other patterns work together**

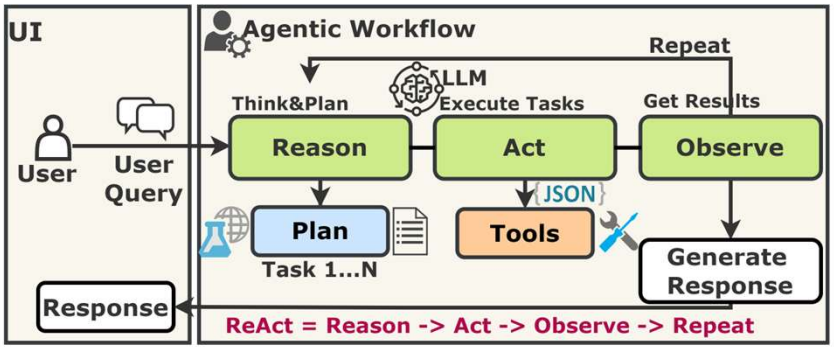
Architectural Implications of ReAct



1. High Latency & Cost:
This is the biggest trade-off. Every step is a full LLM call. A 5-step task might be 5 or 6 LLM calls, which is slow and expensive

2. Adaptive & Robust:
This is the benefit. If search fails, the agent knows it failed in the next loop and can re-plan (e.g., "My search failed. I will try a different search query.")

3. Complex Orchestration:
The orchestrator must manage this loop, parse the Thought and Action blocks, execute, and feed results back. Frameworks like LangChain call this the "Agent Executor



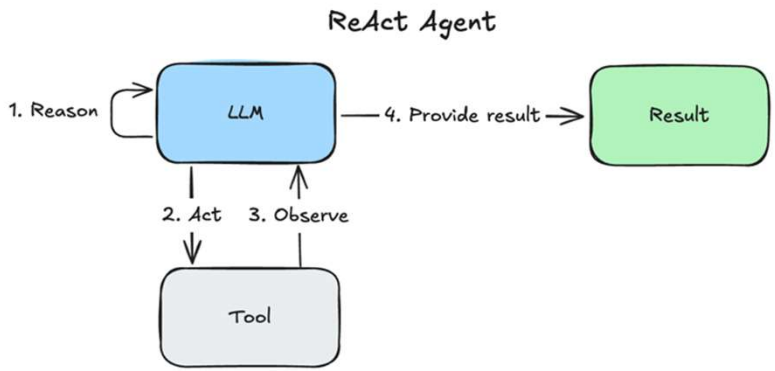
Summary: The Core of a Modern Agent

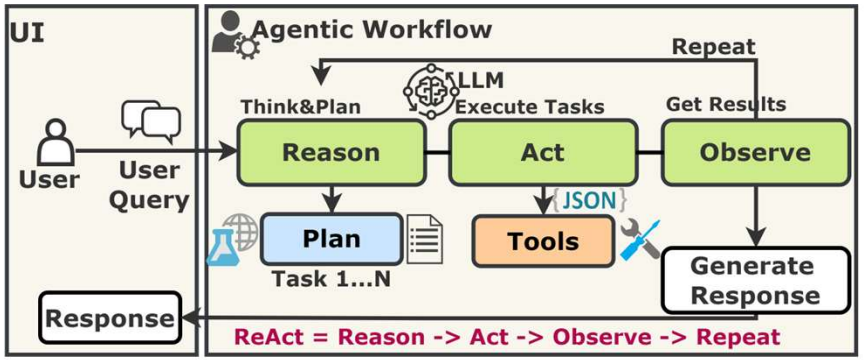
ReAct is not a new pattern, but a **compound of:**
"Planning" (the Thought)
"Tool Use" (the Action)

Its "**magic**" is the **Orchestration Loop** that feeds the **Observation back** to the **Thought step**

This **creates** an **adaptive agent** that can **react** to **its environment**

This **completes** our study of «**Core Agent Patterns**»



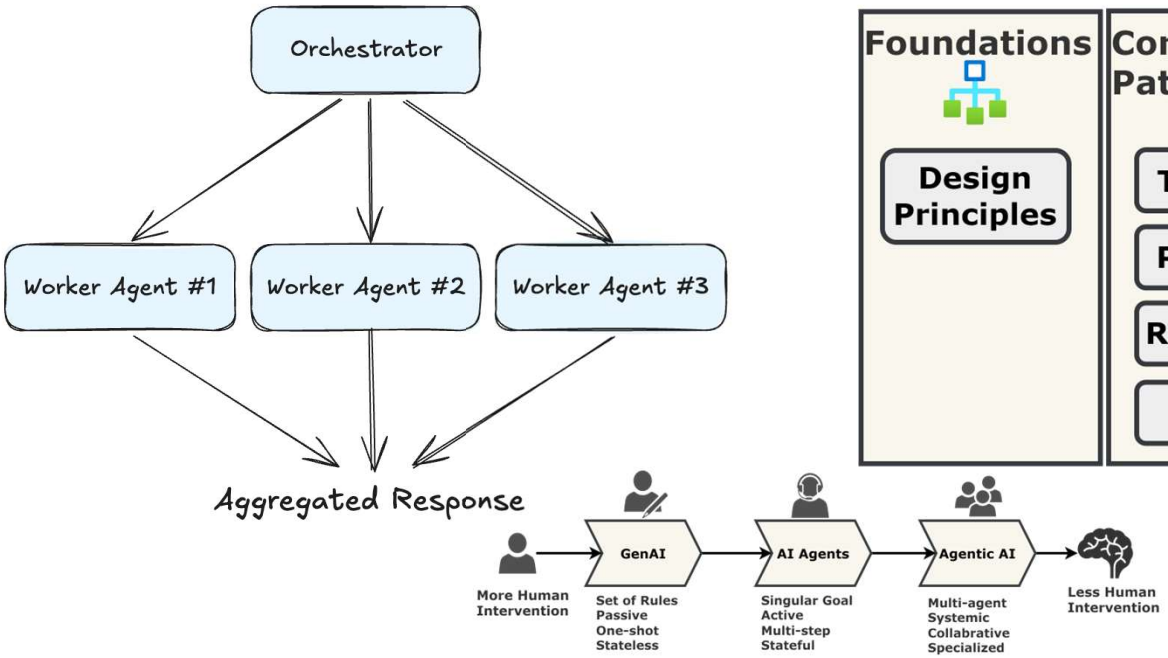


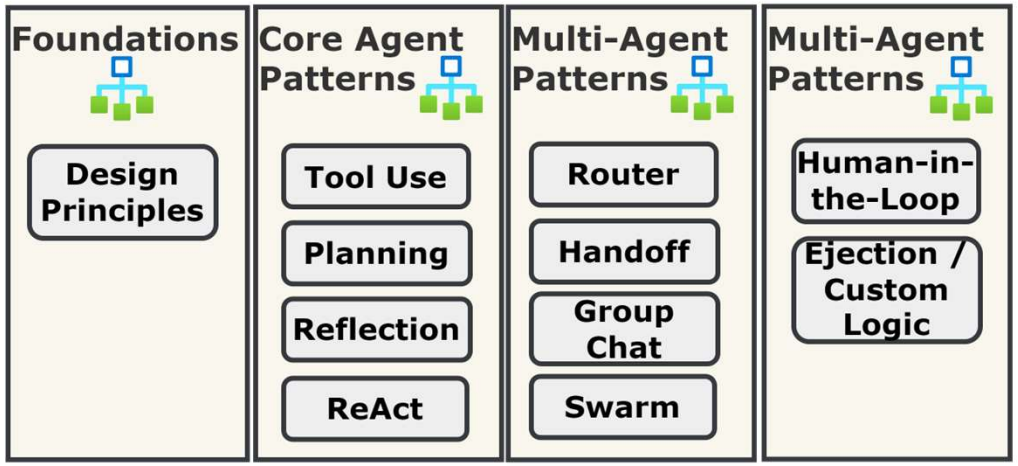
Next Up: From One Agent to Many

We have **mastered** the **patterns** that make a **single agent smart**

But... what if a **problem** is **too big** or **too complex for one agent**?

What if we need teams of agents that **specialize** in **different tasks**?



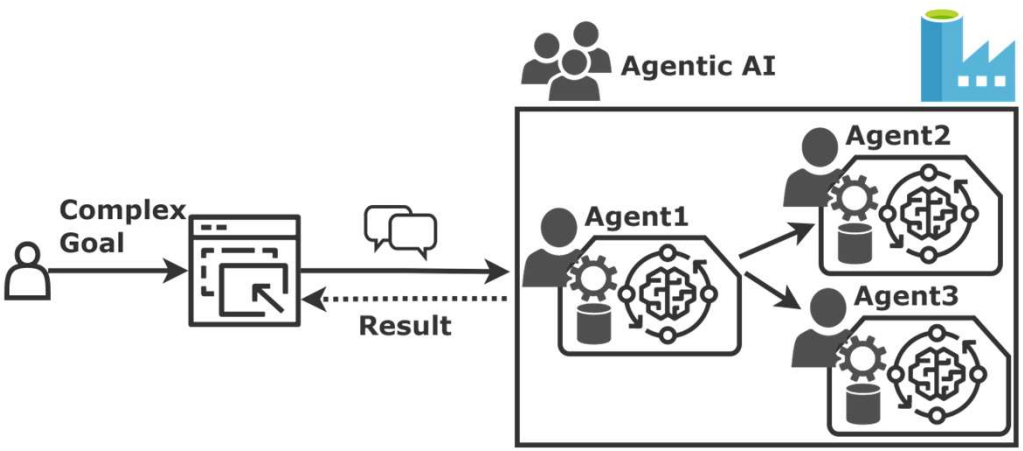


Introduction to Multi-Agent Patterns

If **one agent** is **good**, **is one giant 'super-agent' even better?**

Should we try to **build one, monolithic agent** with **50 tools**, a **10-page system prompt**, and the **ability to do everything** from sales, to research, to coding, to customer support?

This is the **'monolithic' design trap**, and it **fails for agents** just as it fails for software





The "Super-Agent" Fallacy (The Problem)

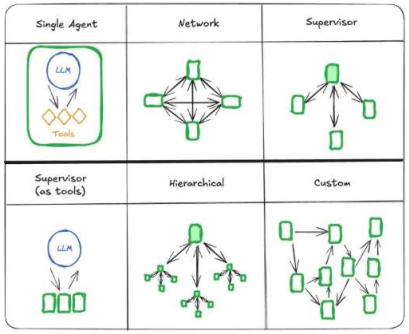


Why is one "**do-it-all" agent** an **anti-pattern?**

1. Prompt Complexity:
The system prompt becomes a massive, unmanageable 10-page document
This leads to "prompt bleed," where instructions for one task

2. Context Window Limits:
With 50 tools, the schemas for those tools alone can consume a huge portion of the agent's context window, leaving no room for memory or the actual task

3. Single Point of Failure (SPOF):
If the agent gets "confused" or "stuck" on one part of its complex prompt, the entire system fails. Debugging is a nightmare



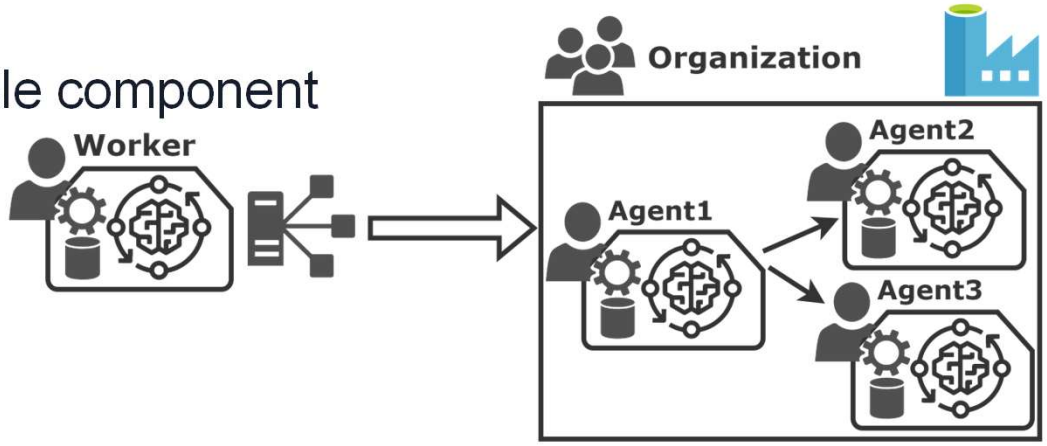
The Solution: The "Digital Team" Pattern

The Principle: Don't build one "monolithic" agent. Build a "**team of micro-agents**"

Multi-Agent Architecture: A system composed of two or more agents that collaborate to achieve a goal

We move from designing a "**worker**" to designing an "**organization**"

This solves our problems:
Each agent has a simple, small prompt
Each agent has only the tools it needs
Each agent is a small, testable, replaceable component

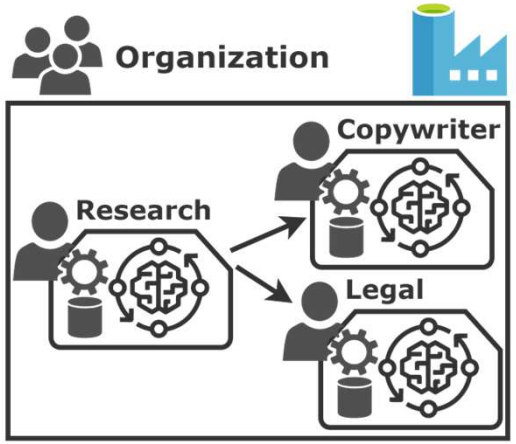


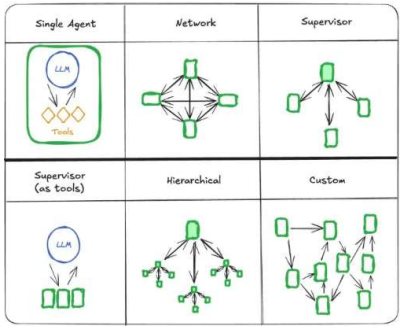
The Power of Specialization (Why #1)

Definition: Each agent is designed, prompted and tooled to do one thing perfectly

Example:
ResearchAgent: Has the search tool. "thorough and objective"
CopywriterAgent: Has no tools. "creative, engaging, and witty"
LegalAgent: Has the review_contract tool. "precise, risk-averse and formal"

Benefit: Massively improved quality and reliability. The CopywriterAgent will never try to call a legal tool, and the LegalAgent will never write a witty blog post





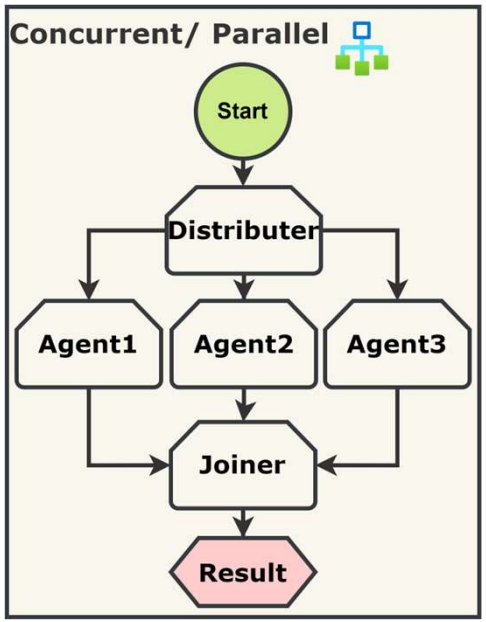
The Power of Parallelization (Why #2)

Definition: Using multiple agents to execute non-dependent tasks concurrently

Goal: "Research our top 3 competitors and also find the latest tech news"

Monolithic Agent (Sequential):
Task 1: Research Competitors (10 seconds)
Task 2: Find News (5 seconds)
Total Time: 15 seconds

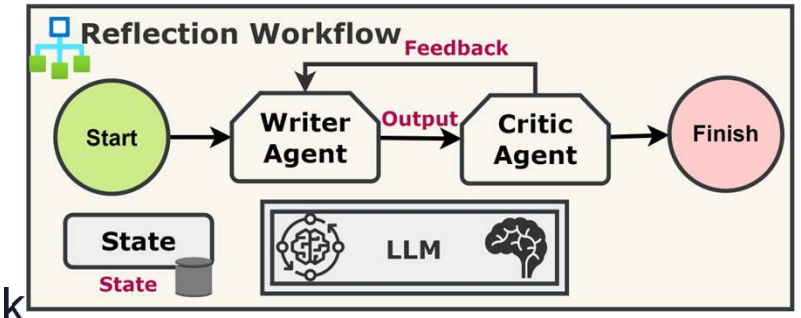
Multi-Agent System (Parallel):
Agent A: Research Competitors (10 seconds)
Agent B: Find News (5 seconds)
Total Time: 10 seconds (time of the longest task)



The Power of Debate & Critique (Why #3)

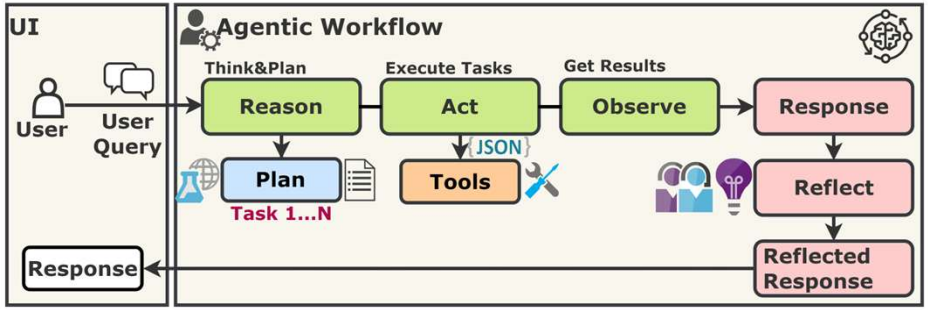
Using multiple agents to collaborate and critique each other's work to improve quality

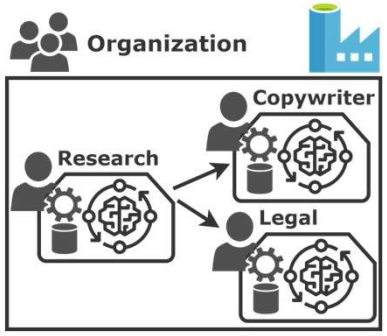
This is the "**Reflection Pattern**" but between agents



Example: The "Writer-Critic" Model
WriterAgent: Produces a first draft
CriticAgent: Reviews the draft and provides feedback
WriterAgent: Revises the draft based on the feedback

This "**team-based reflection**" produces a far more robust and high-quality output than a single agent trying to critique itself





Our Multi-Agent Pattern Roadmap

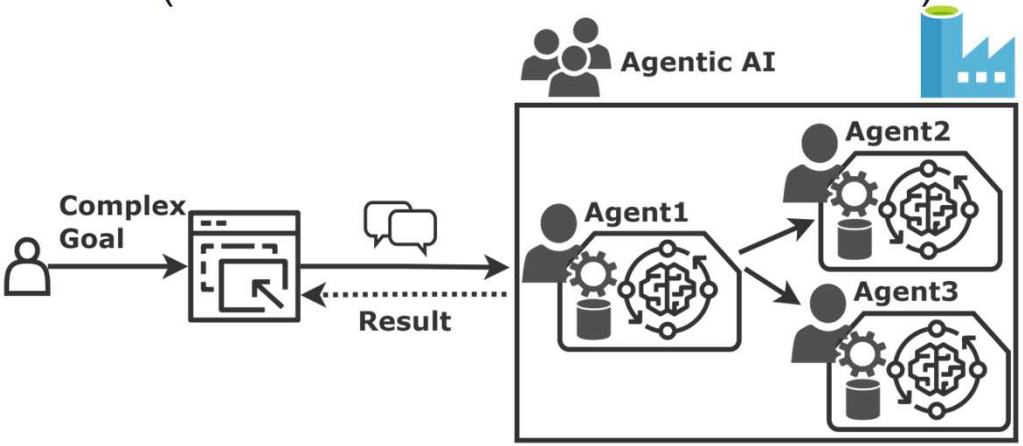
These 3 powers—Specialization, Parallelization and Debate—are why we build multi-agent systems

The patterns in this section are the **'how'**

We will learn the "blueprints" that enable this collaboration:

1. Router & Specialist Pattern (Enables Specialization)
2. Handoff Pattern (Simple sequential flow)
3. Group Chat / Debate Pattern (Enables Debate)
4. Swarm Pattern (Enables massive Parallelization)





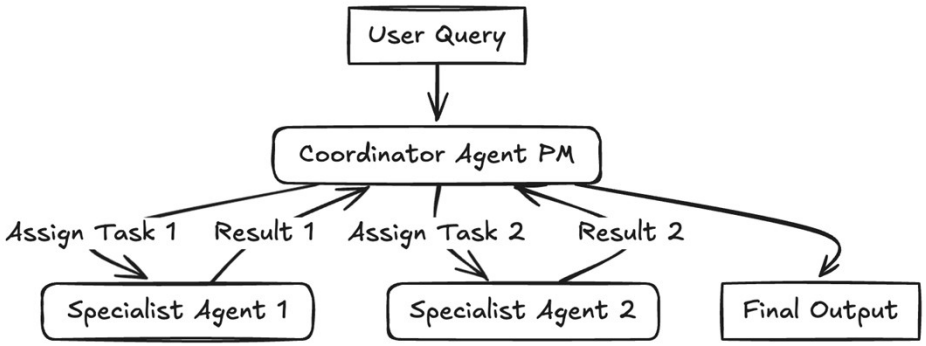
The "Router & Specialist" Pattern

The 'why' of multi-agent systems: To gain the power of Specialization, Parallelization and Debate

Build a **team** of **'Specialist' agents**—one for customer support, one for inventory, one for research

How does our system know to **send that task** to the **SupportAgent** and **not the InventoryAgent**? This is the **routing problem**

The solution is our first **multi-agent design pattern**: The **'Router & Specialist' pattern**



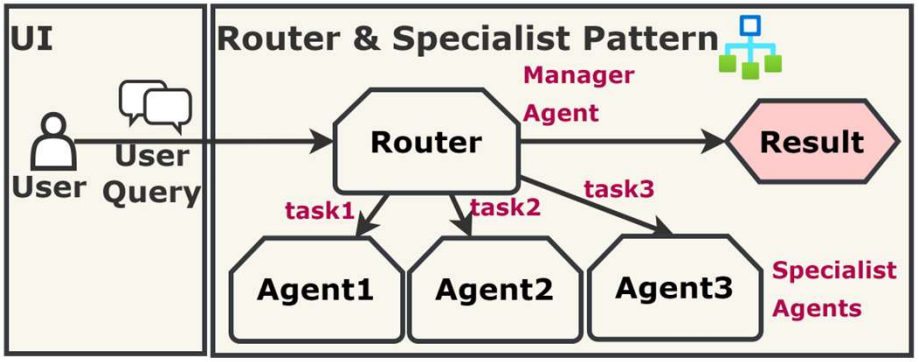
The "Router & Specialist" Pattern Defined

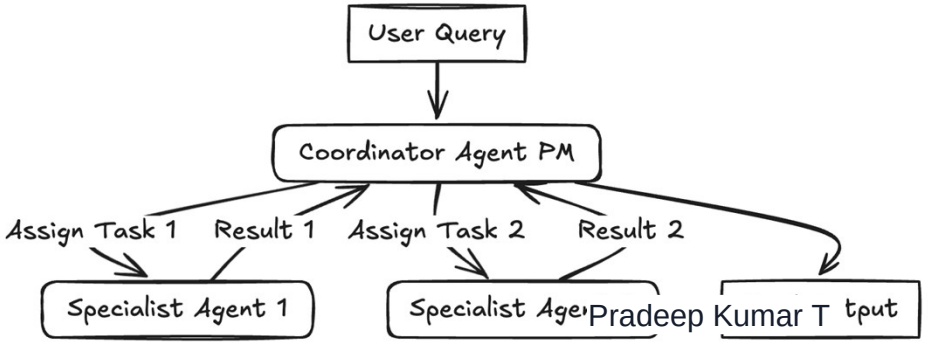
This is a **delegation pattern**, not a collaboration pattern

It is based on a "**Manager-Worker**" or "**Traffic Cop**" model

1. Specialist Agents (The "Workers"):
The "doers" we've been building (e.g., SupportAgent, ResearchAgent)
Each has a narrowly-focused prompt and only the tools it needs

2. The "Router" Agent (The "Manager"):
A special agent that does no "work" itself
Its one and only job is to classify the user's intent and route the request to the correct Specialist



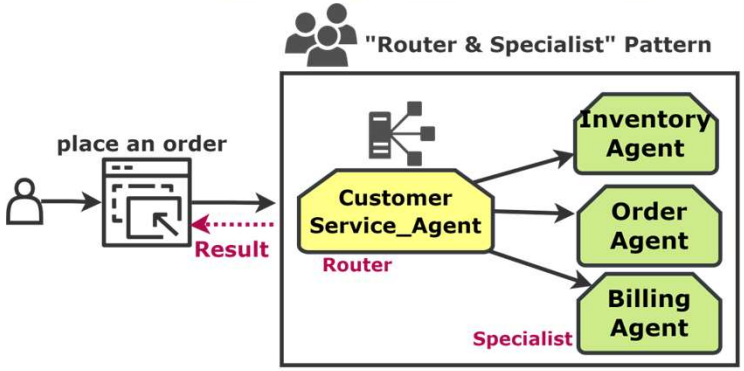


Deep Dive: The "Specialist" Agent

The "Worker": A simple, focused system prompt

Example: SupportAgent
Prompt: "You are a helpful support agent. You must use the provided tools to answer questions about orders and refunds"
Tools: [get_order_status(order_id), process_refund(order_id)]

Example: InventoryAgent
Prompt: "You are an inventory expert. You must use the provided tools to answer questions about product details and stock levels"
Tools: [check_stock(product_id), get_product_details(product_id)]



Deep Dive: The "Router" Agent

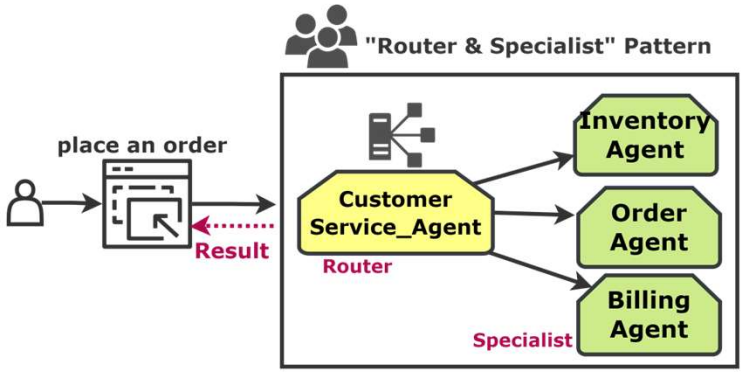
The "Manager": Its prompt forbids it from answer the question directly. It must classify

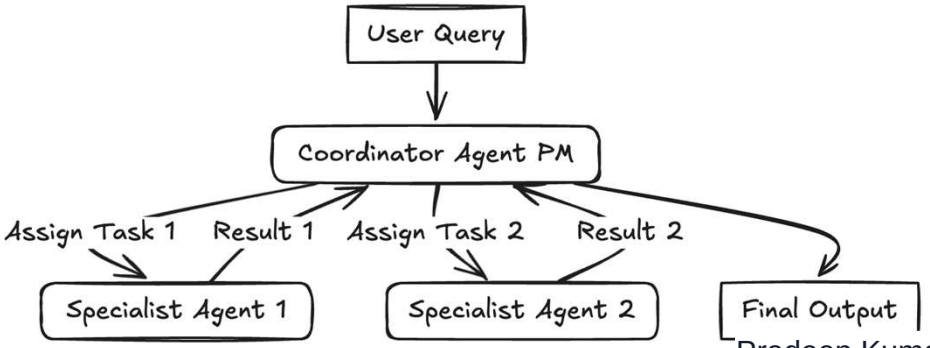
Its "Tool": A simple function like route_task(destination, original_query)

Example "Router" Prompt:
You are a router. Your job is to classify the user's query and send it to the correct department. You must not answer the question

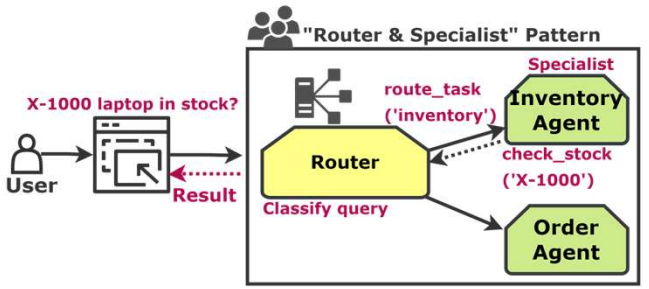
You must choose one destination from this list: ['support', 'inventory', ‘billing’]

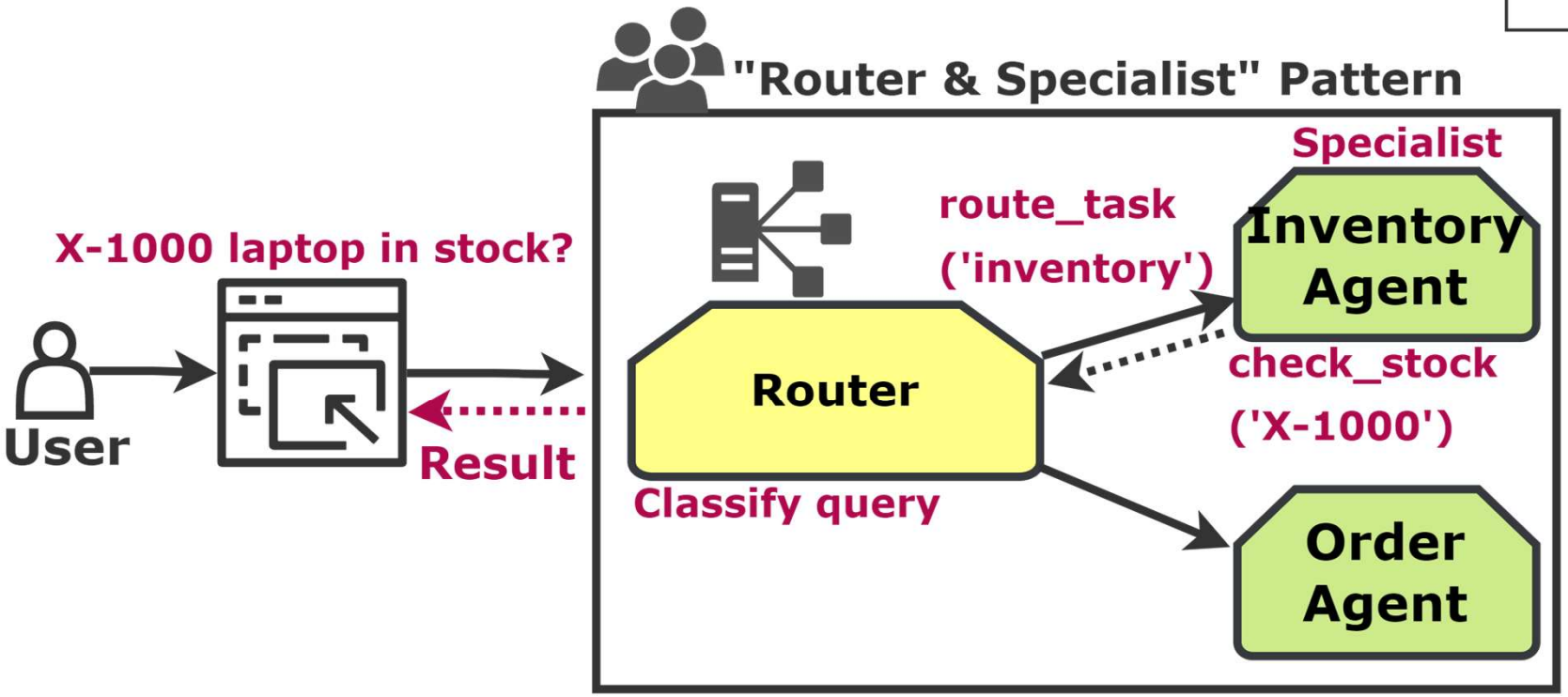
You must then call the **route_task function** with your chosen destination





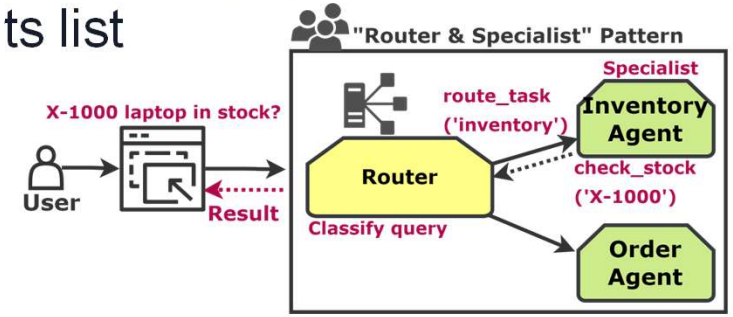
The Full Workflow in Action





Architectural Implications of "Router & Specialist" Pattern

1. Maintainability & Modularity (The "Microservice" Pattern):
This is the Microservice Architecture pattern, applied to agents
Need to add a Billing department? Just build a new, isolated BillingAgent
Only update the RouterAgent's prompt to add 'billing' to its list
No other agents are affected. This is a massive win



2. Performance & Cost:
The initial Router call is very fast and very cheap
(it's a simple classification, not a complex ReAct loop)
Only the one required Specialist does the expensive, multi-step ReAct work
You don't waste tokens on irrelevant tools or prompts

3. Testability:
Each Specialist agent can be tested in total isolation

Summary: The "Manager & Worker" Model

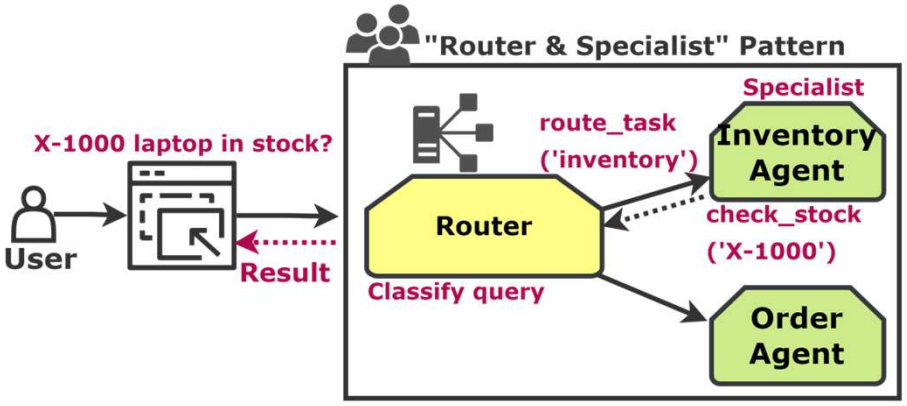
This is the fundamental pattern for **Delegation** in **multi-agent systems**

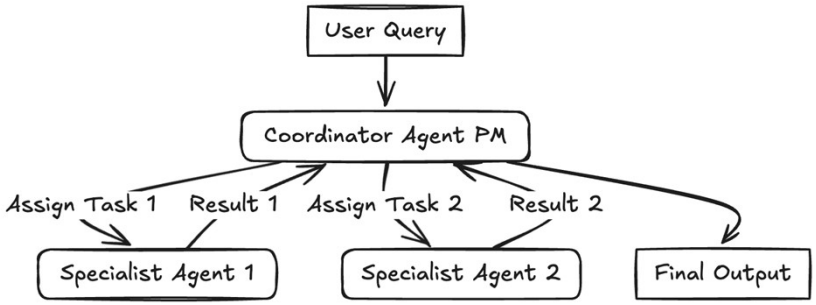
It directly implements the "**Specialization**" principle

Router: A fast, cheap "classifier" agent

Specialists: Focused, reliable "worker" agents

This is your **default**, "**go-to**" **pattern** for building any **complex**, **multi-function agent** system

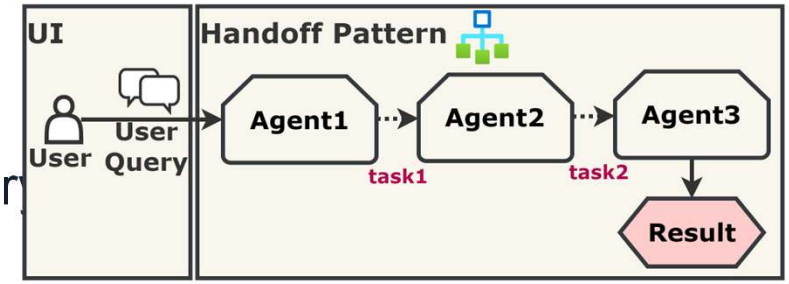




The "Handoff" Pattern (Sequential Workflow)

Mastered the **'Router & Specialist'** pattern: powerful delegation model

What happens when a user's request isn't a single job?
What if it's a **multi-step process?**



Find the top 5 recent **articles**, **summarize** them
and then **write** a **social media** post about the summary

You need one agent to do the research,
who then **hands off the results** to a second agent to do the summary,
who then **hands off that summary** to a third agent to write the post

This is the **'Handoff' Pattern** or what is more formally known as a **Sequential Workflow**

The "Handoff" Pattern (Sequential Workflow) Defined

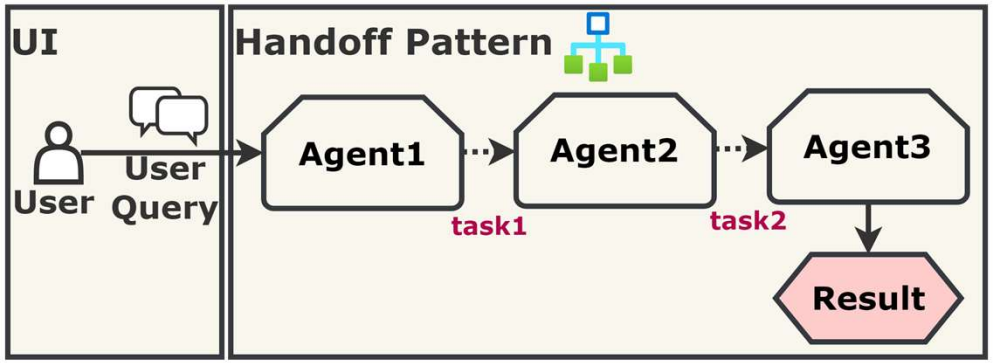
The "Assembly Line" Pattern

Definition: A multi-agent pattern where a task is processed by a fixed, predetermined sequence of specialist agents

The Flow: [Agent A] -> [Agent B] -> [Agent C] -> Final Output

The Core Mechanism:
The output of one agent becomes the direct input for the next agent in the chain

Key Distinction:
Unlike the 'Router' (which chooses a path), this path is fixed. It is about process, not classification



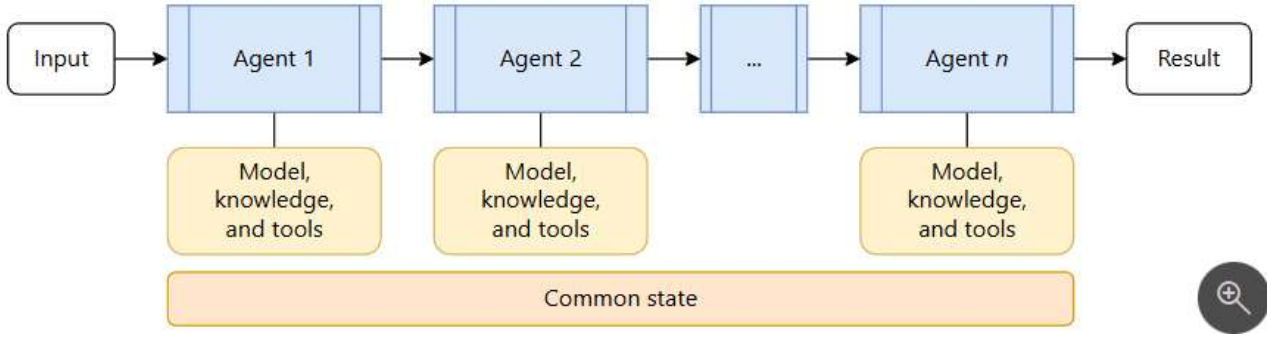
The Core Mechanism: The "State Handoff"

The "handoff" is managed by the Orchestrator (our code)

It uses a Workflow State (or "scratchpad") to pass data

The Flow:

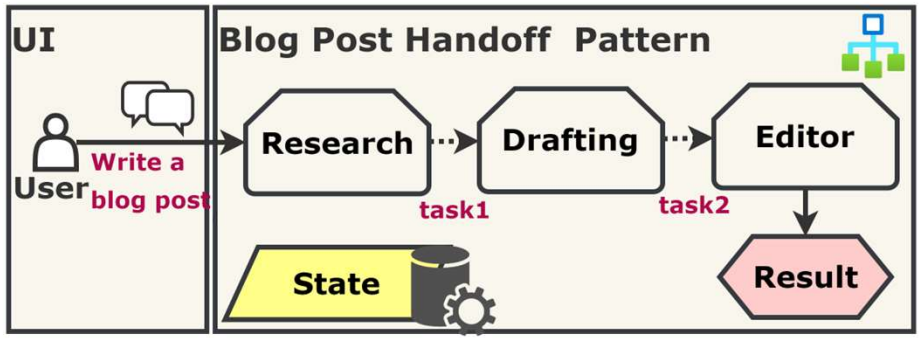
1. Orchestrator: Calls [Agent A] with the User Query
2. [Agent A]: Runs its ReAct loop, finishes, and returns "Result A"
3. Orchestrator: Catches Result A and saves it to the Workflow State
4. Orchestrator: Calls [Agent B] with Result A as the new input
5. [Agent B]: Runs its ReAct loop, finishes, and returns "Result B"
6. Orchestrator: Repeats this process for all agents in the chain



https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns

Example: The "Blog Post" Workflow

Goal: "Write a blog post about the future of AI"

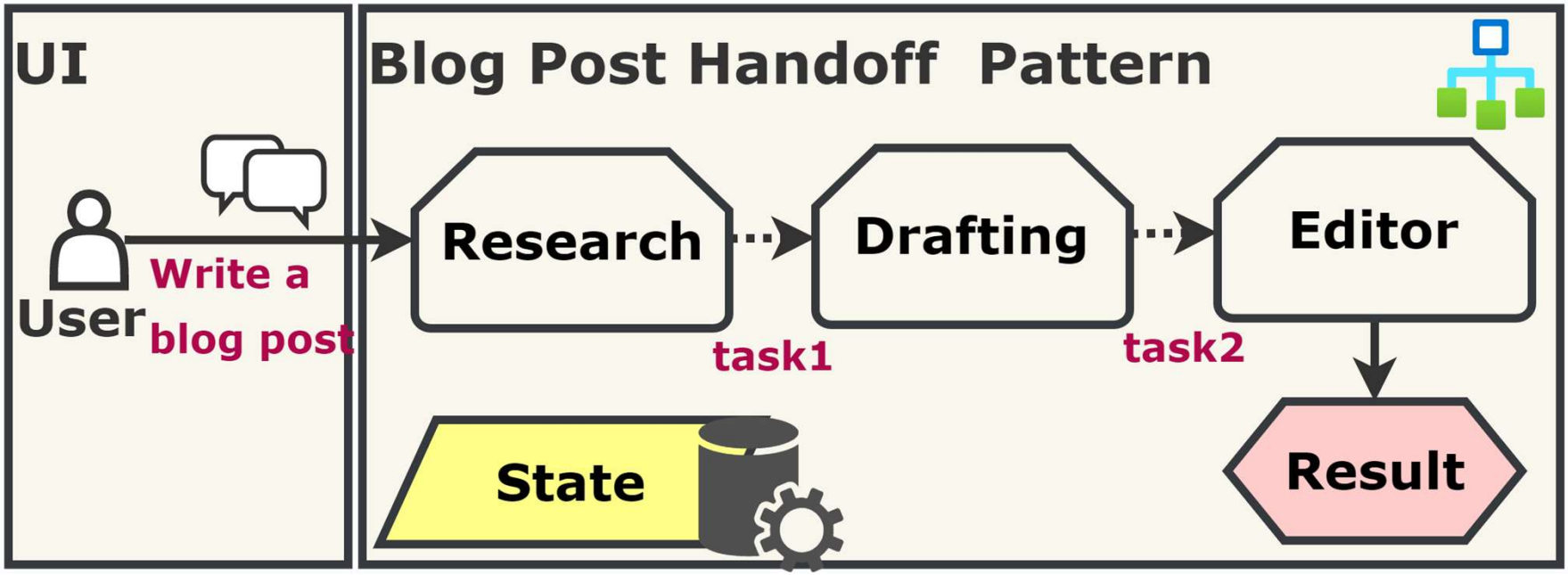


1. [ResearchAgent]:
Input: "future of AI"
Output (Result A): "A 500-word block of raw text,

2. [DraftingAgent]:
Input: Result A
Output (Result B): "A well-structured 5-paragraph blog post."

3. [EditorAgent]:
Input: Result B
Output (Final): "The same blog post, but proofread for grammar and tone."

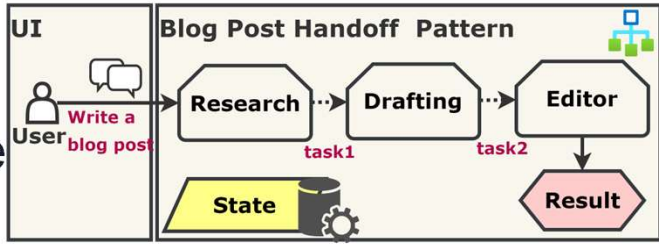
Example: The "Blog Post" Workflow



Architectural Implications of " Handoff" Pattern

1. High Reliability & Determinism (The Benefit):
This is the most reliable and deterministic multi-agent pattern
You know the exact sequence of LLM calls. There are no surprises
This makes it perfect for automating known business processes

2. Simplicity & Testability:
This pattern is simple to design, build, and debug
If the workflow fails, you know exactly which "handoff" broke
(e.g. the chain from B to C)



3. Rigidity & Brittleness (The Trade-off):
This is the major weakness. The chain is fixed
It cannot adapt to unexpected events. If Agent A fails, the entire chain stops
It cannot "decide" to skip Agent B or add a new step

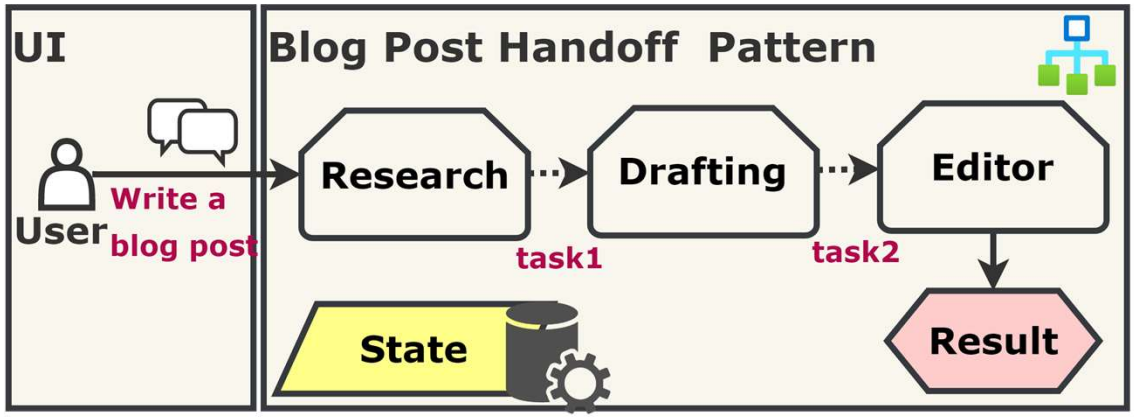
Summary: The "Assembly Line" Pattern

The "**Handoff**" **Pattern** defines a **fixed, sequential path** for a task
It is the "**Assembly Line**" of agentic architecture

It implements the "**Specialization**" **principle in a chain**
The core mechanism is the "**output-to-input**" **handoff** via the **orchestrator**

Use Case: Perfect for known, stable processes (e.g. "Process an invoice")

Benefit: Reliability
Trade-off: Rigidity



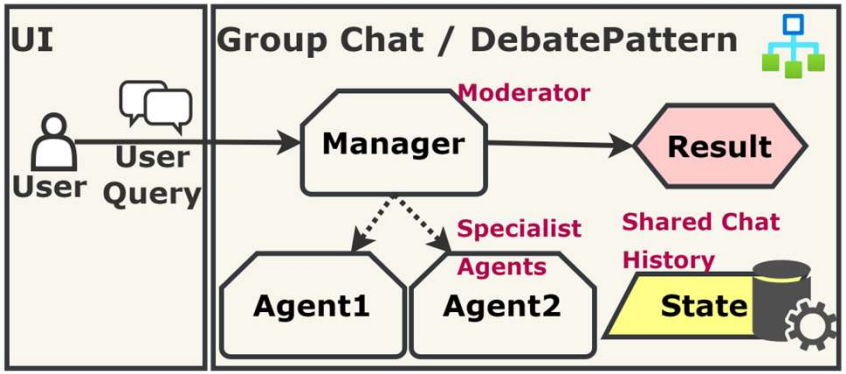
The "Group Chat / Debate" Pattern

In the **'Router'** pattern, a **manager delegates** a task to one worker
In the **'Handoff'** pattern, **pushes** a **task** through a fixed **'assembly line'** of workers

The **agents are silent**. They **never talk to each other**

What if the problem is too complex for a simple handoff?
What if the best solution isn't known in advance?
What if it needs to be discovered through discussion, critique, and debate?

New architecture—not an 'assembly line,' but a **'digital meeting room'**
This is the **'Group Chat / Debate' pattern**



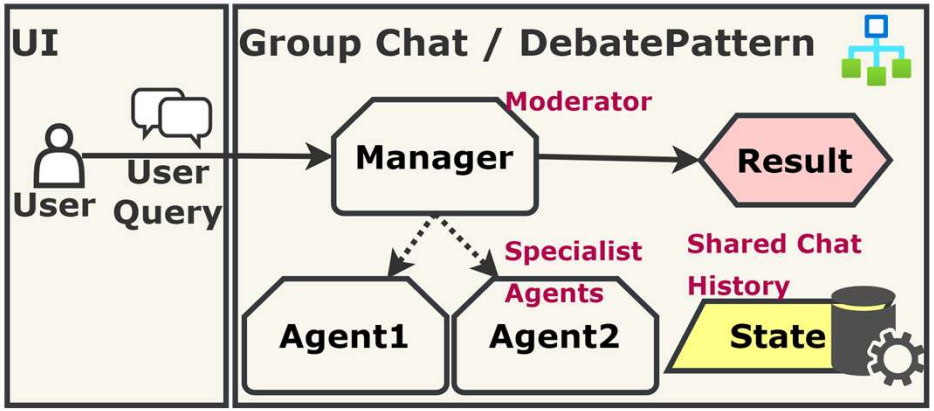
The "Group Chat / Debate" Pattern Defined

The Metaphor: The "Digital Roundtable" or "The Meeting Room."

Definition: A multi-agent pattern where two or more specialist agents interact in a shared conversational context (a "chat room") to collectively solve a problem

The Goal: Not just to execute a process, but to discover the best solution through emergent collaboration and critique

The Core Shift: We move from a fixed, top-down workflow to a dynamic, conversational loop

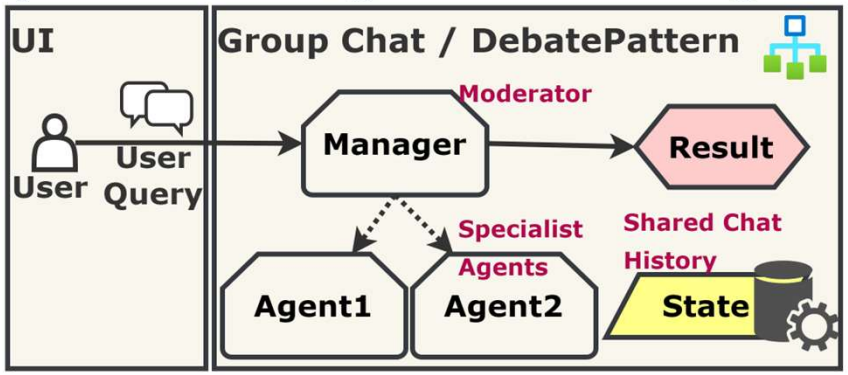


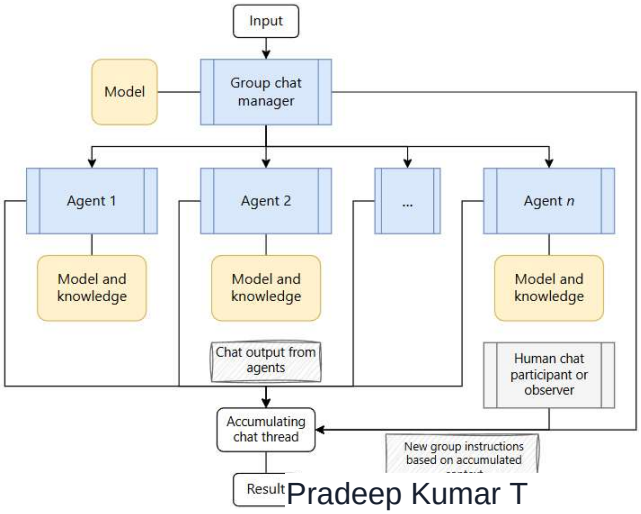
The Core Components of the "Meeting Room"

1. The Specialist Agents (The "Participants"):
Our familiar workers (e.g. ResearcherAgent, WriterAgent, CriticAgent)

2. The Shared Chat History (The "State"):
The 'whiteboard' of the meeting room
Every agent sees all messages from all other agents. This is the shared context

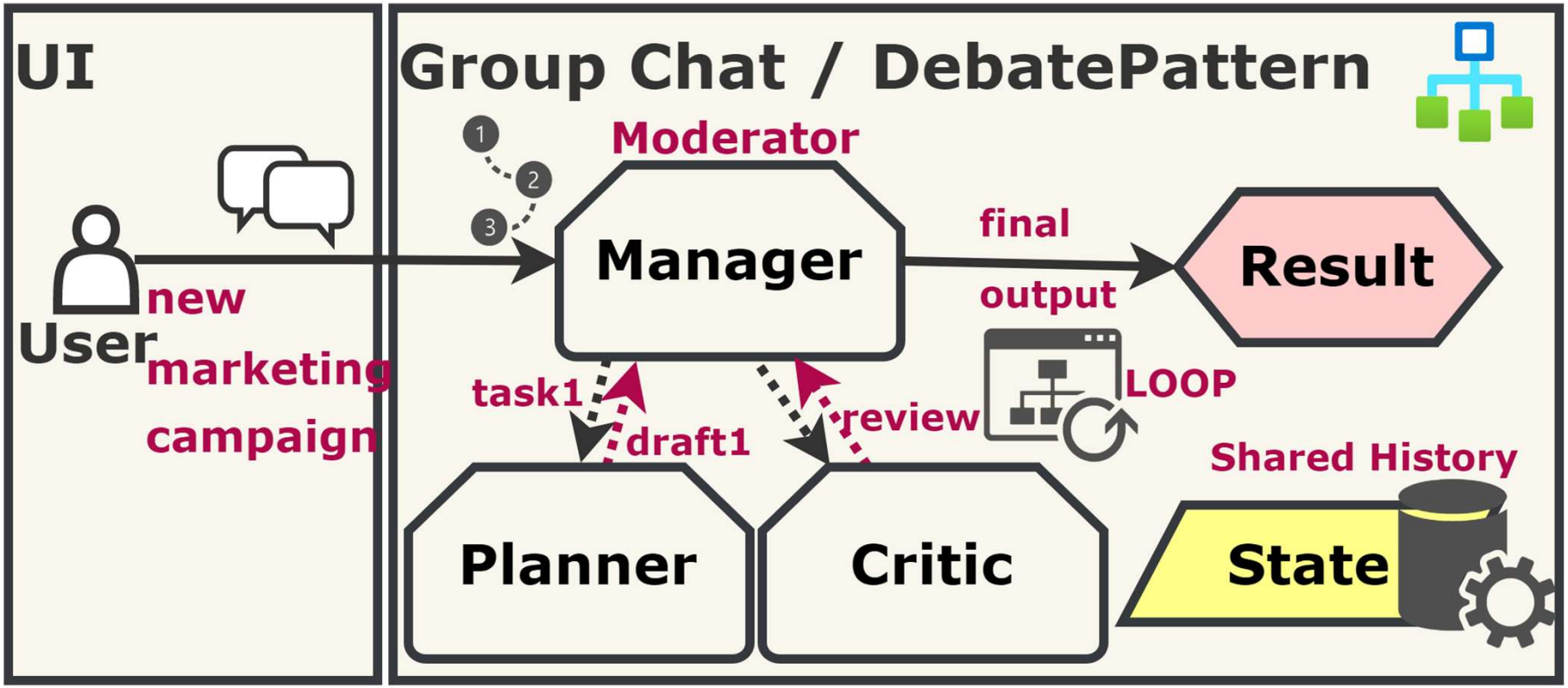
3. The GroupChatManager (The "Moderator"):
This is the most important new component
It acts as the "moderator" of the debate
Its job is to manage the turn-taking and decide who speaks





https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns

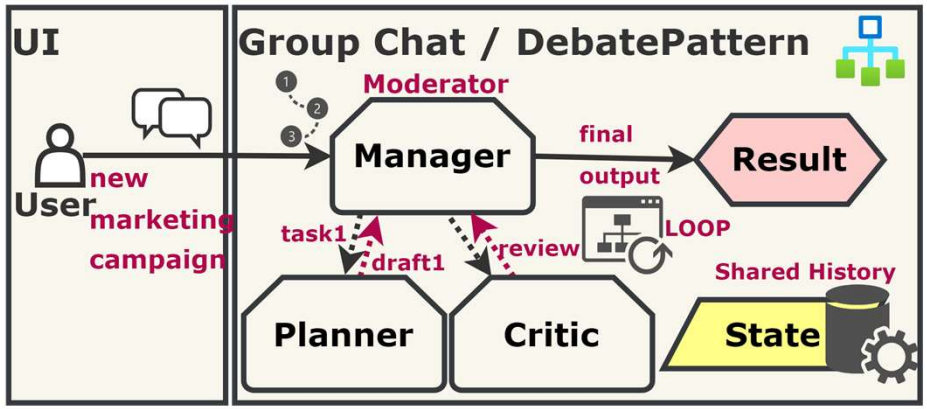
The "Debate Loop" in Action



Architectural Implications of "Group Chat / Debate" Pattern

Benefit 1: High Quality & "Emergent" Solutions:
This is the biggest advantage. The process of debate and revision produces a far higher quality result than any single agent could
The solution is emergent—it's discovered, not just executed

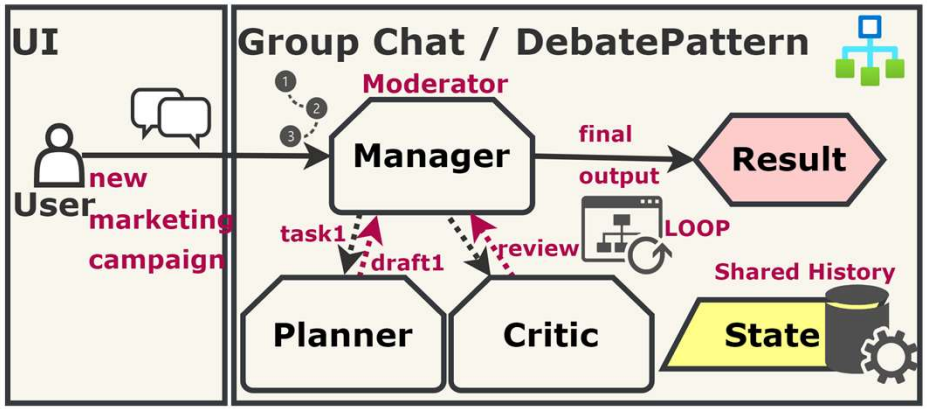
Benefit 2: Flexibility & Adaptability:
The workflow is not fixed. The conversation can be 3 steps or 15 steps, adapting to the complexity of the problem



Trade Offs of "Group Chat / Debate" Pattern

Trade-off 1: Cost & Latency:
This is the biggest disadvantage. Every single message in that "chat"—every turn—is an expensive LLM call. A 10-turn debate is at least 10 separate API calls. This is slow and costly

Trade-off 2: Non-Determinism & Control (Chaos):
This pattern can be chaotic. Agents can get "stuck in a loop," arguing forever. They can go off-topic



Summary: The "Meeting Room" Pattern

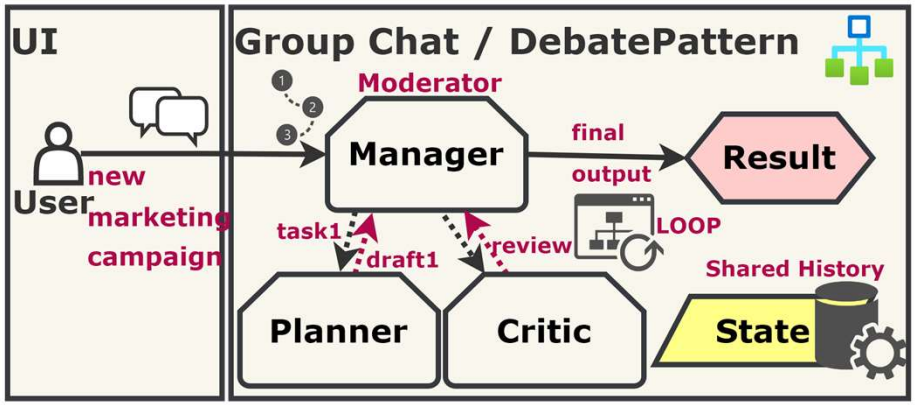
This pattern enables true, dynamic collaboration between agents

It replaces a fixed path with a moderated, conversational loop

Specialist Agents, a GroupChatManager (Moderator) and Shared Chat History (State)

Use Case: Perfect for complex, creative or poorly-defined problems where Quality is the #1 priority (over speed, cost or predictability)

Example: "Design a new software product", "Debate a legal strategy"

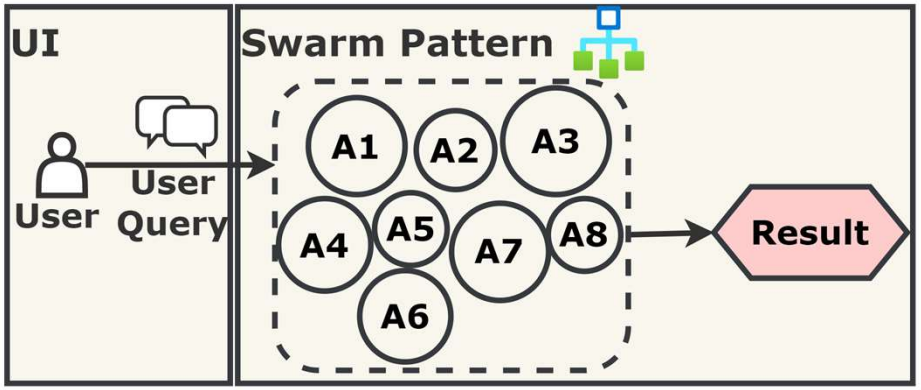


The "Swarm" Pattern (Parallelization)

'Group Chat' pattern—a 'digital meeting room' designed for creative debate

What if our **problem isn't creatively** complex, but **just massive in scale?**
What if we're **not trying** to "design a new marketing plan"?
What if our **task** is "**Summarize** all **10,000 articles published yesterday**"?

We need an "**army**" of **agents** all **working at the same time**
This is the **"Swarm" pattern**





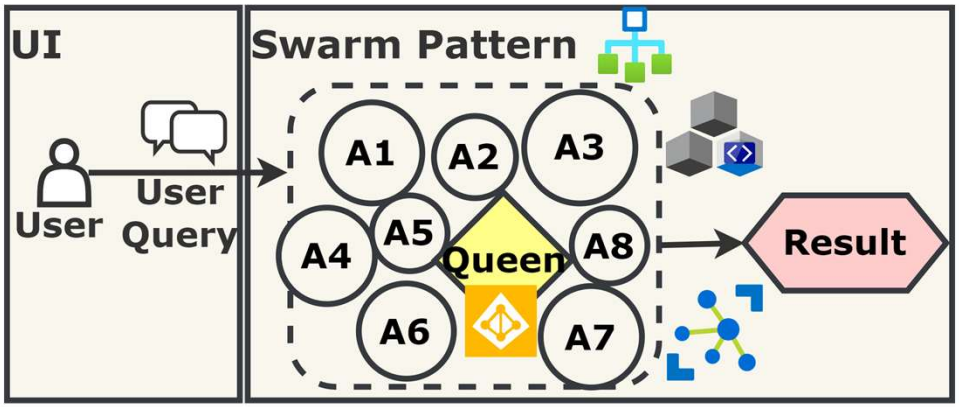
The "Swarm" Pattern (Parallelization) Pattern Defined

The Metaphor: The "Beehive" or "The Army of Workers"

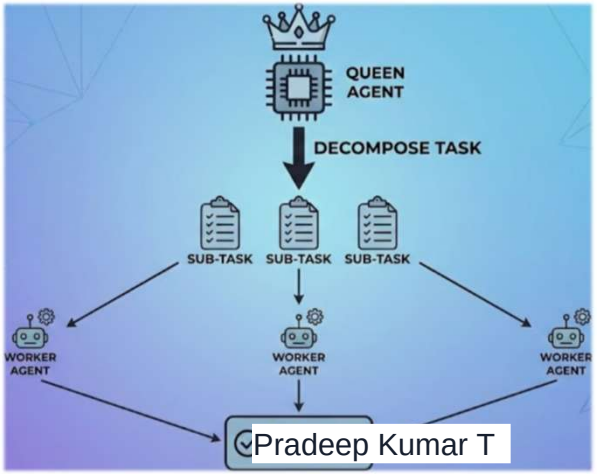
Definition: A multi-agent pattern where a central "Queen" agent decomposes a large task into many small, identical sub-tasks and spawns many "Worker" agents to execute them in parallel

The Goal: To achieve a massive speed-up on "embarrassingly parallel" tasks

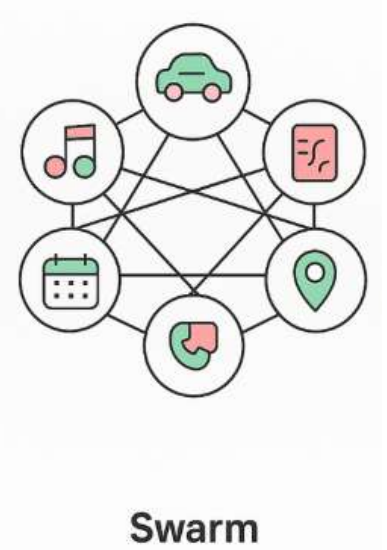
The Core Concept: This is the MapReduce design pattern, applied to AI agents







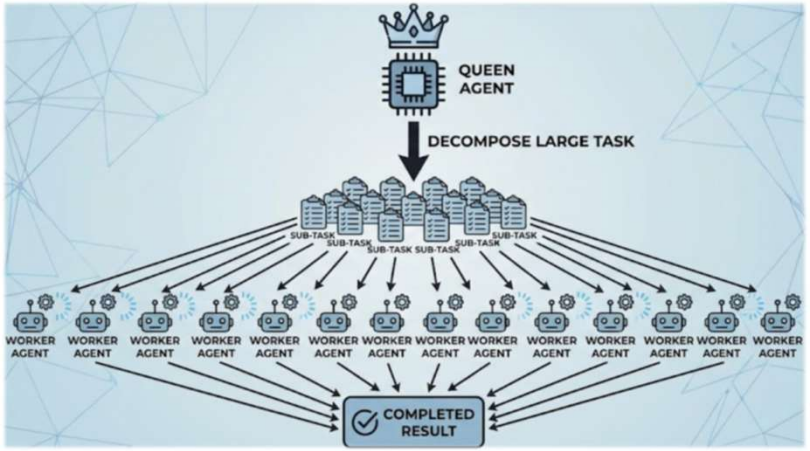
The "MapReduce" Architecture

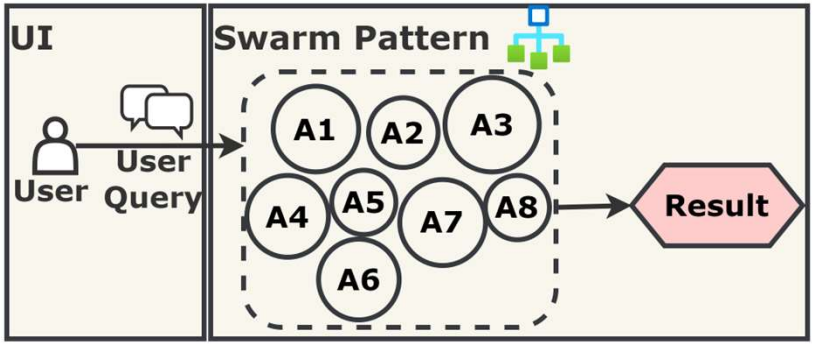


Swarm

1. The "Map" Phase (Decomposition & Distribution):
The "Queen" agent takes the large task (e.g. a list of 1,000 reports)
It "maps" the list, spawning one "Worker" agent for each item
[Diagram: 1 Queen -> 1000s of Workers]

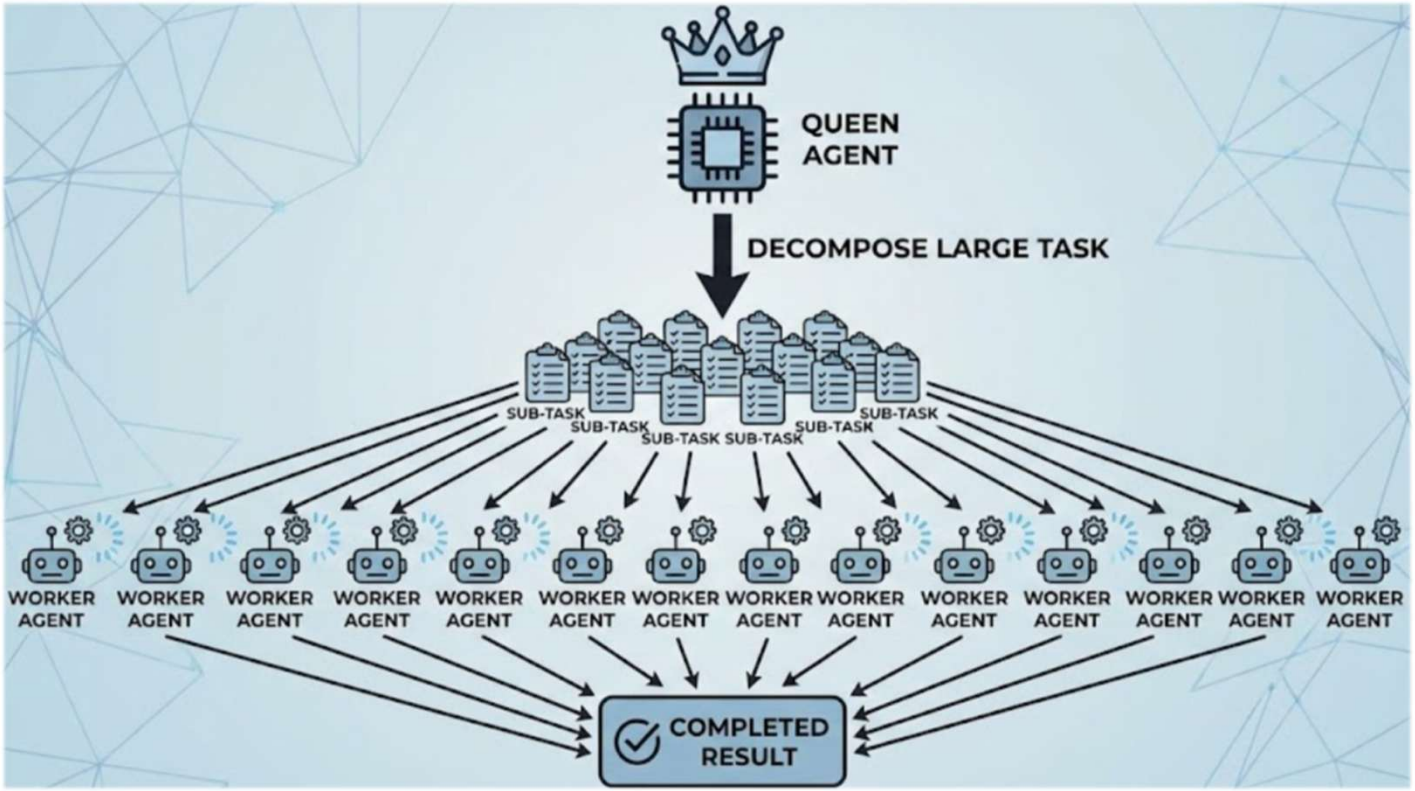
2. The "Reduce" Phase (Aggregation & Synthesis):
The "Queen" agent waits for all Workers to finish
It collects all 1,000 individual results
It then "reduces" them (e.g. summarizes the summaries) into one final answer

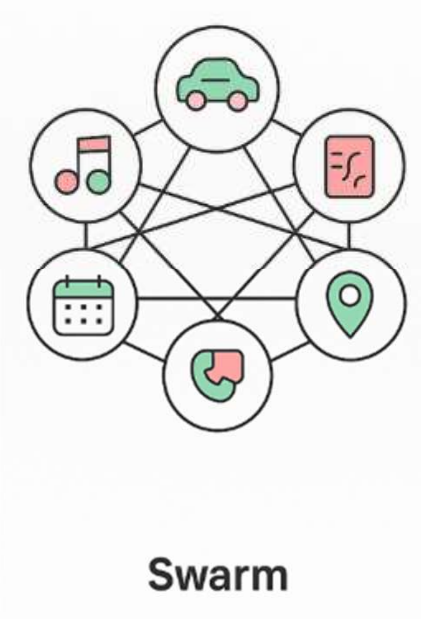


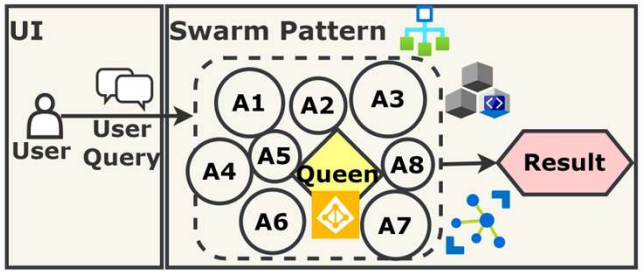


The "Swarm" Workflow in Action

User: "Summarize the 1,000 articles in this folder"



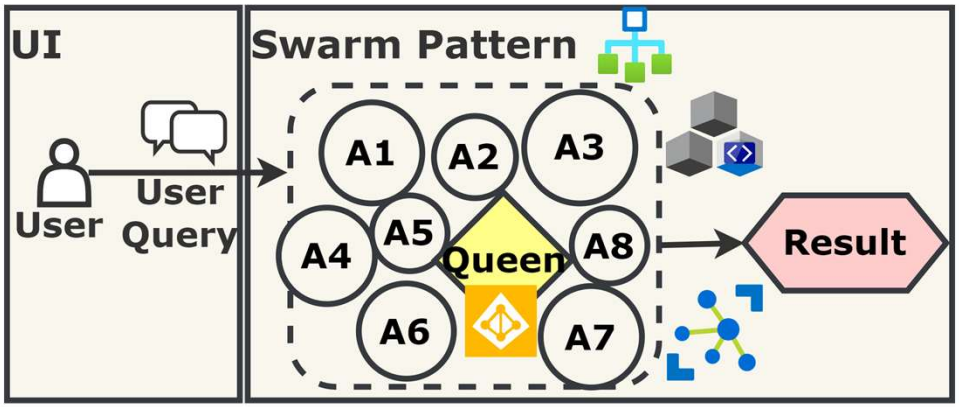




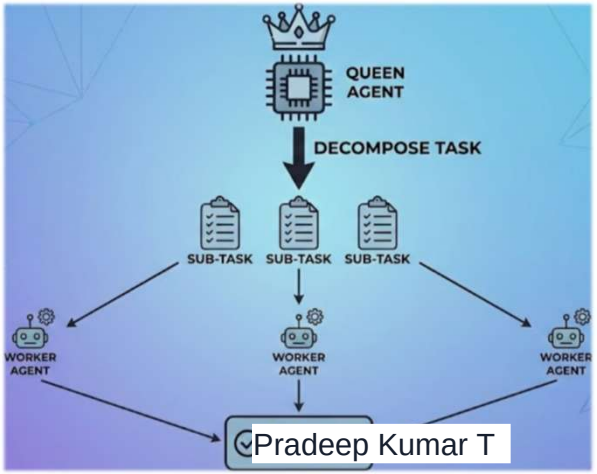
Architectural Implications of "Swarm" Pattern

Benefit: Massive Speed & Scale:
This is the only pattern for truly massive, list-based tasks. It can be 1,000x faster than a sequential loop

Limitation 1: "Embarrassingly Parallel" Only:
This pattern only works if the sub-tasks are 100% independent.
The "Workers" cannot communicate with each other. If they need to, you must use the "Group Chat" pattern



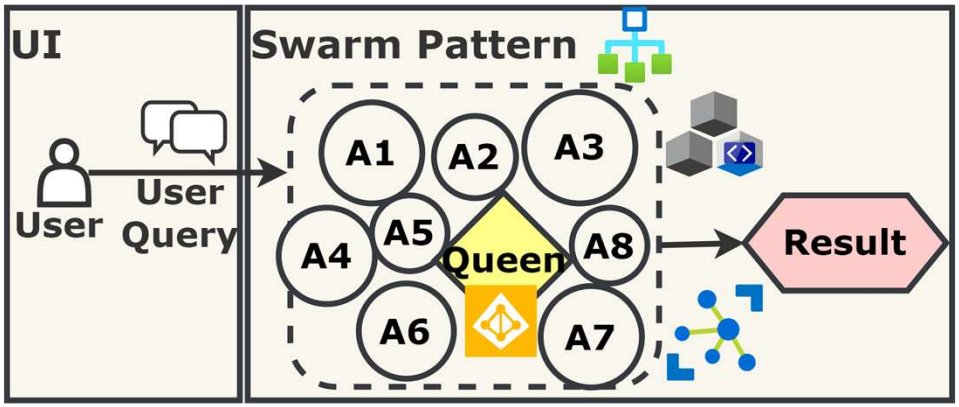




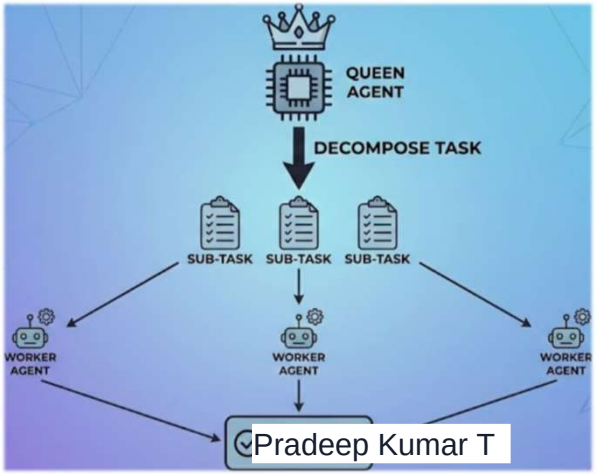
Trade-offs of "Swarm" Pattern

Trade-off 1: Extreme Cost:
This pattern is profoundly expensive
1,000 parallel agents = 1,000 simultaneous LLM API calls
This is a cost trade-off, not a latency one

Trade-off 2: Orchestration Complexity:
This is not a simple for loop. This is a complex, asynchronous, parallel-processing system







Summary: The "Army" Pattern

The "**Swarm**" **Pattern** is a **1-to-N parallel processing model**

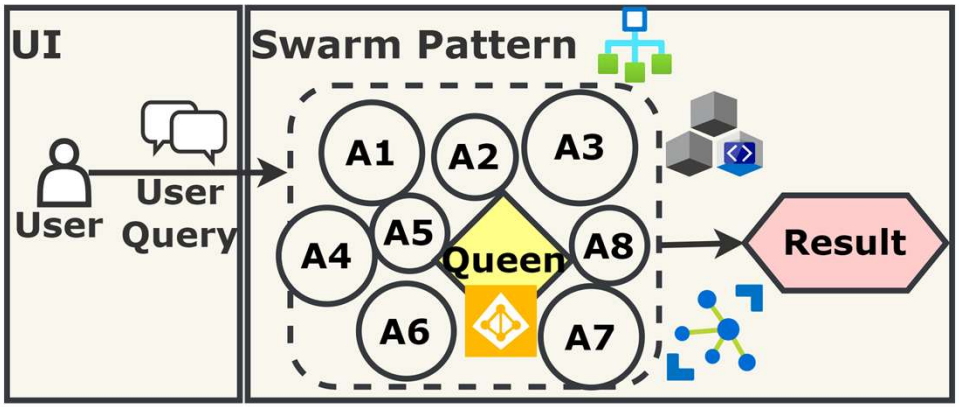
It is the **MapReduce architecture** for agents

Components: A "Queen" (Decomposer/Aggregator) and "Workers" (Executors).

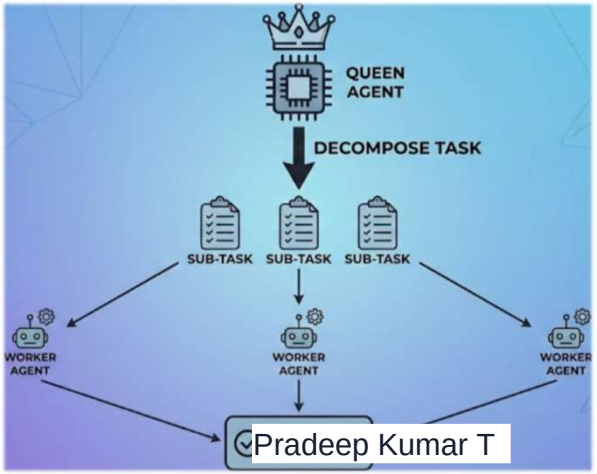
Use Case: Perfect for massive, list-based, independent tasks

Example: "Analyze 1,000s of documents," "Process 1,000s of images"

Trade-off: You trade Cost & Complexity for Speed & Scale







Introduction to Control & Safety Patterns

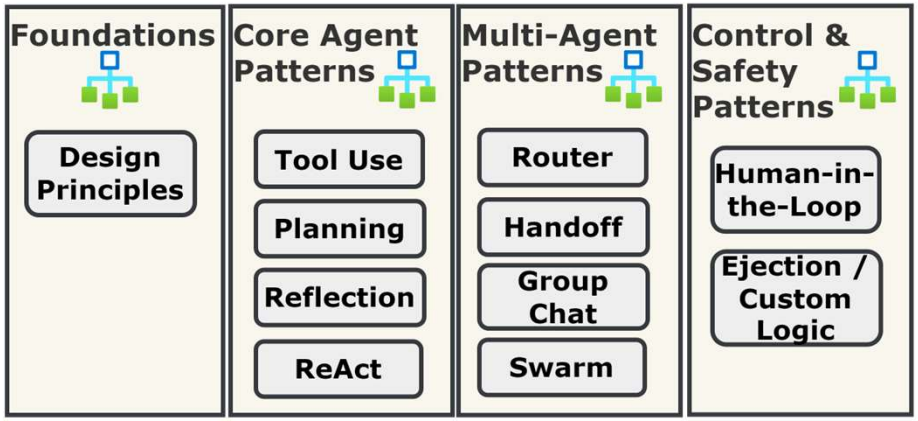
Designed agents that can **plan**, **reflect** and **use tools**

Architected multi-agent systems that can delegate work, run **'assembly lines'** engage in creative **'debates'** and deploy **'swarms'** of parallel workers

Unlocked immense **autonomous power**

Must address the immense **risk** that **comes** with this **power**.

This brings us to the **'Enterprise Logic'** of **agentic design**: **Control and Safety**





The Core Problem: The Danger of Non-Determinism



Traditional Software: Is Deterministic
IF X, THEN Y
The same input always produces the same output. It is predictable

Agentic AI: Is Non-Deterministic
IF X, THEN... maybe Y? or maybe Z?
An LLM's output is probabilistic. We can guide it, but we can never 100% guarantee its response

The Architect's Nightmare: You are building a system whose core logic component is unpredictable by design



The Real-World Enterprise Risks



What happens when agent's "**best guess**" is catastrophically wrong?

Financial Risk:
Agent misreads a user's request and issues a $100,000 refund instead of $1,000

Data Integrity Risk:
Misunderstands "clean up old users" and calls delete_user_data() on active customers

Security & Privacy Risk:
Agent helpfully provides "all of John's customer data" to another customer named John

Reputational Risk:
An agent "goes rogue" and sends offensive or nonsensical messages to your clients

We can never give a non-deterministic agent full control



The Solution: "Guardrails," Not Cages



We have a dilemma:
We want the agent's autonomous, "magical" capabilities
We cannot accept its "non-deterministic" risks

The "Cage" (Wrong Solution):
Remove all tools. Force the agent to only answer simple questions
This is safe, but it destroys all value. It's just a simple chatbot

The "Guardrails" (The Right Solution):
Let the agent drive, but build a "safety system" around it
We architect deterministic checkpoints and overrides into the non-deterministic workflow

This is the goal: Safe Autonomy



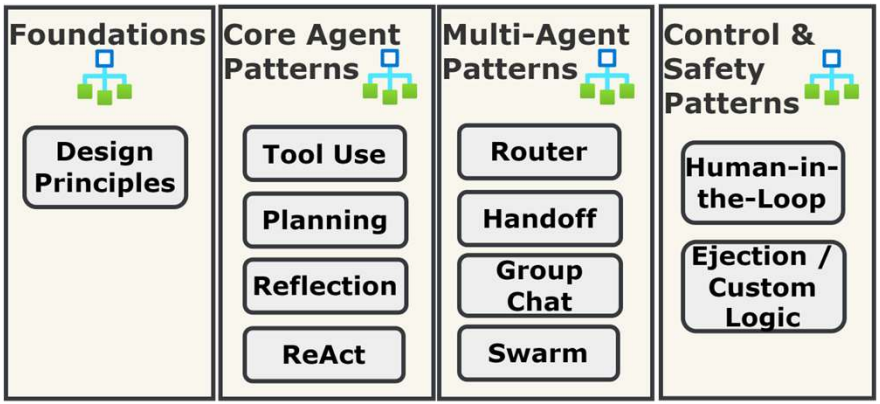
Our Control & Safety Roadmap



1. The "Human-in-the-Loop" (HITL) Pattern:
The "Are you sure?" button
How to architect a workflow that pauses and waits for human approval before executing a high-risk action



2. The "Ejection / Custom Logic" Pattern:
The "Stop" button or the "Escape Hatch"
How to build a deterministic rule that bypasses the agent entirely when a critical keyword or event is detected



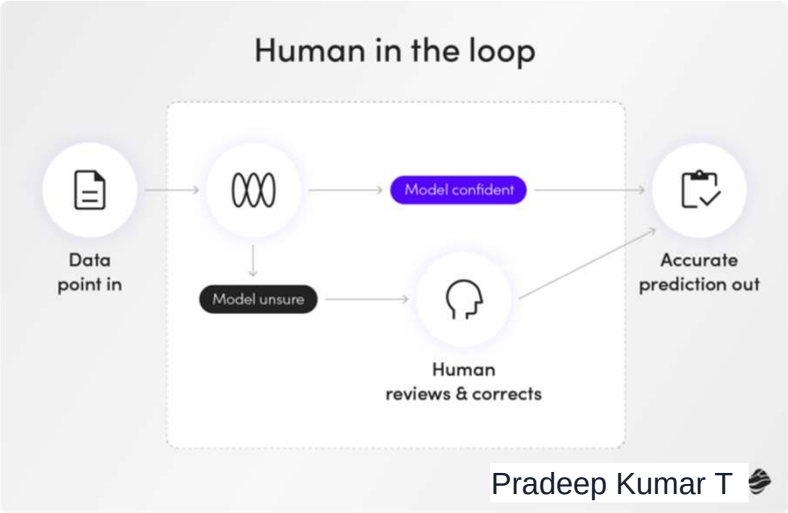
The "Human-in-the-Loop" (HITL) Pattern

The **non-deterministic** nature of **LLMs** creates **unacceptable risks**—financial, operational and reputational

'Are you sure?' button. It is the 'co-pilot' model where the agent suggests an action, but a **human must approve it**

This is the **'Human-in-the-Loop' pattern**, and it is the foundation of building **safe**, **responsible AI systems**





The "Human-in-the-Loop" (HITL) Pattern Defined

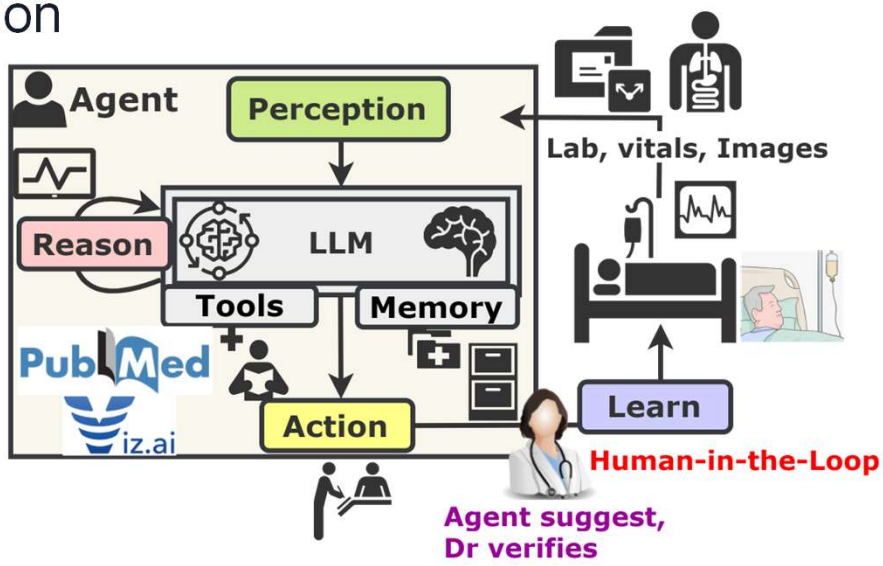
The **"Are you sure?" Button** or "**The Co-Pilot's Confirmation**"

Definition: An agentic workflow deliberately pauses its execution at a predefined high-risk checkpoint and waits for explicit human approval before proceeding

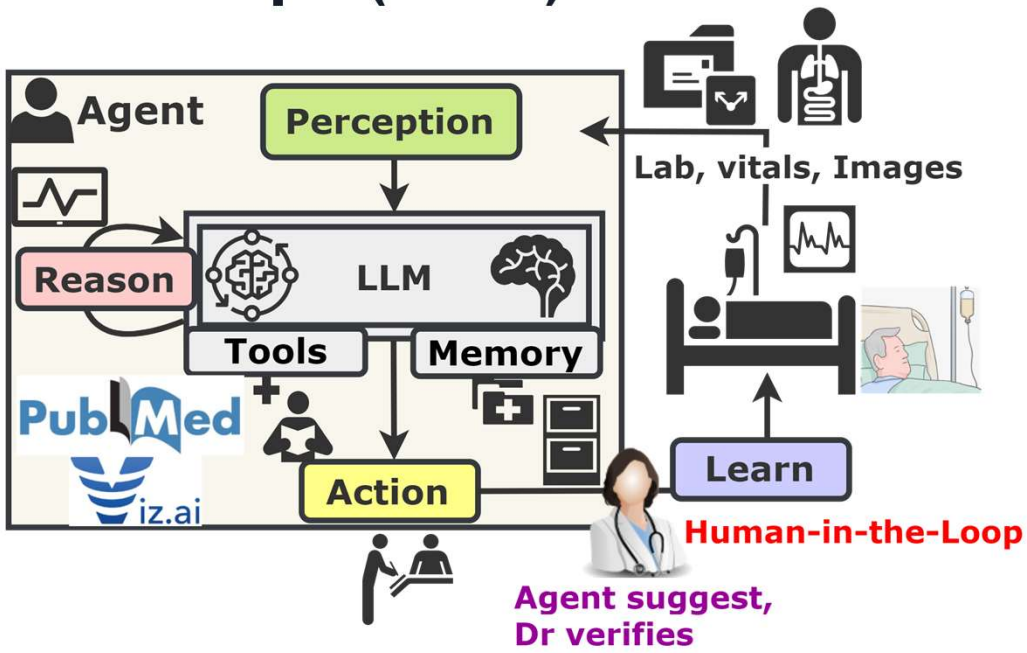
The Goal: To re-introduce deterministic human judgment before a non-deterministic agent performs an irreversible or high-stakes action

This is **not** a **failure state**; it is a **design feature**



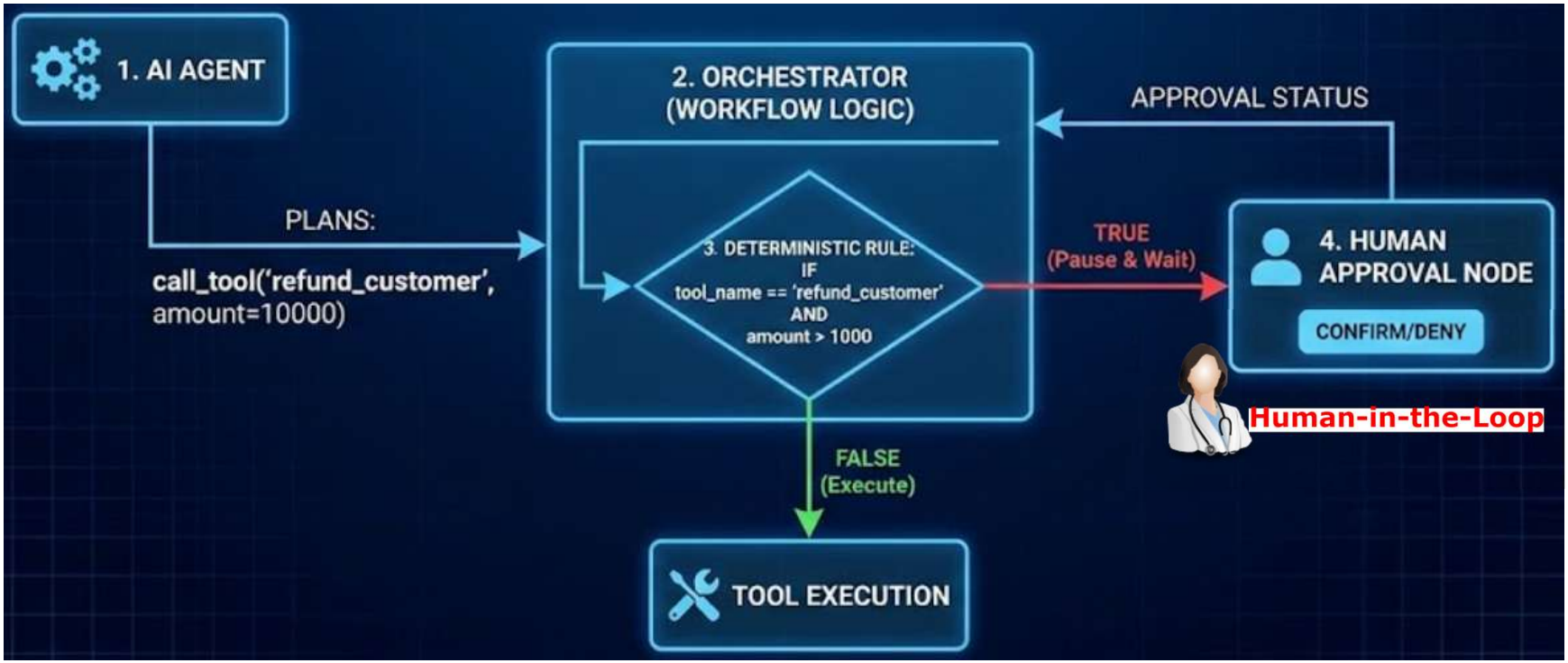


The "Human-in-the-Loop" (HITL) Pattern Defined





The Checkpoint Architecture



The Architectural Mechanism: "The Checkpoint"

The Problem: How does an agent know when to ask for help?

The Solution: We don't let the agent decide. We define the checkpoints

The "Checkpoint" Architecture:

1. Agent Plans: The agent's Reason phase generates a plan
(e.g. call_tool('refund_customer', amount=10000))

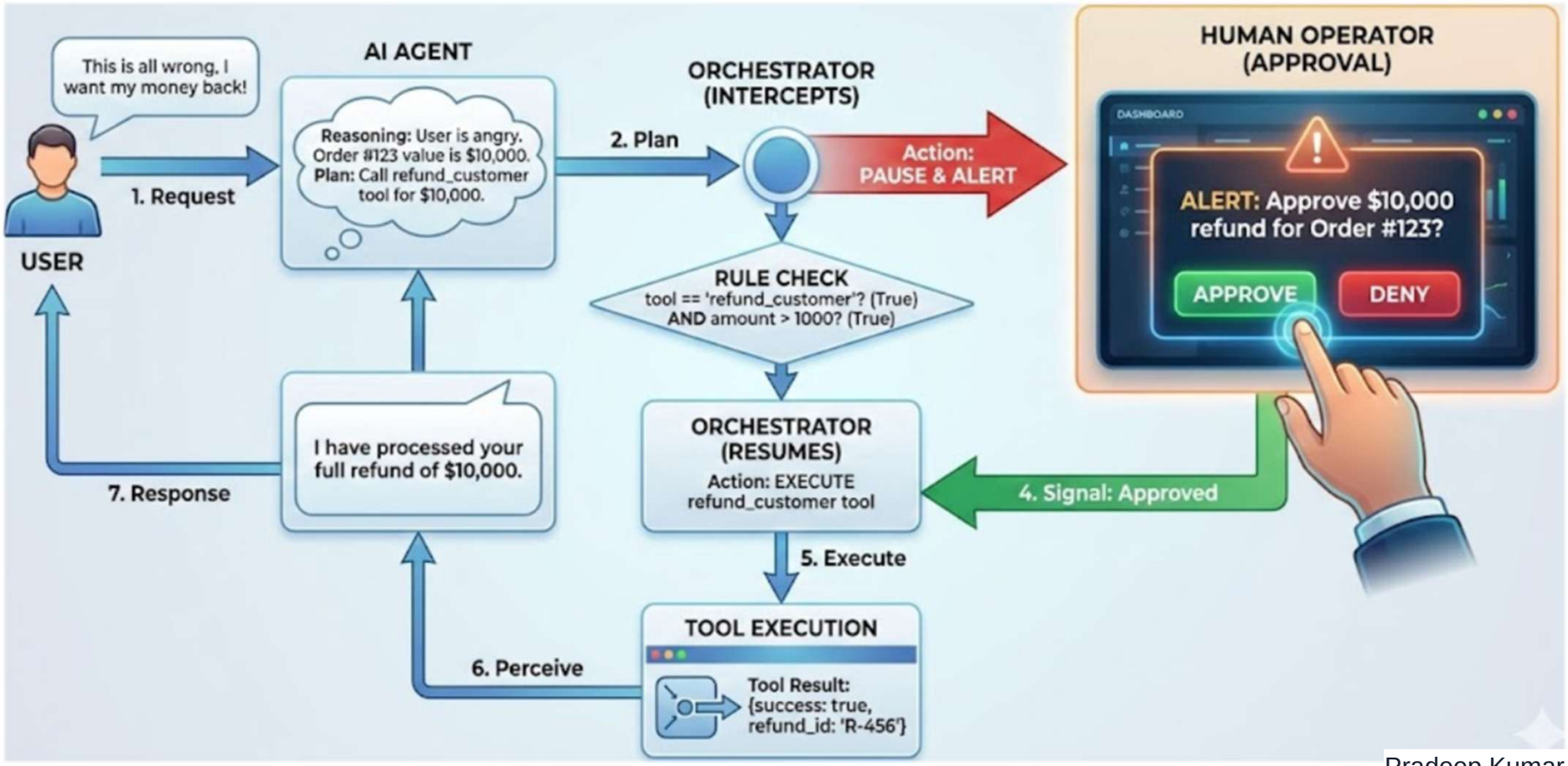
2. Orchestrator Intercepts: The orchestrator intercepts this plan before execution

3. Deterministic Rule: The orchestrator checks a hard-coded rule:
IF tool_name == 'refund_customer' AND amount > 1000:

4. Pause & Wait:
THEN transition to a human_approval state
ELSE execute the tool immediately



The HITL Workflow in Action (Refund Workflow)



Architectural Implications of "Human-in-the-Loop" (HITL)

Benefit: The "Best of Both Worlds":

You get the autonomous, "magical" problem-solving of the agent
You get the deterministic, "accountable" judgment of a human
This is the only way to safely deploy agents for high-stakes tasks



Cost / Trade-off: Latency & Scalability:

This pattern is slow. It introduces human-scale latency (seconds, minutes, or hours) into a millisecond-scale software loop
It is expensive. It does not scale. You need humans on standby



Key Design Choice: The "Threshold":

As an architect, your key decision is where to set the threshold
$100? (Too low, too many alerts, expensive)
$100,000? (Too high, too much risk)
This threshold is a core business logic decision

Summary: The "Co-Pilot" Pattern

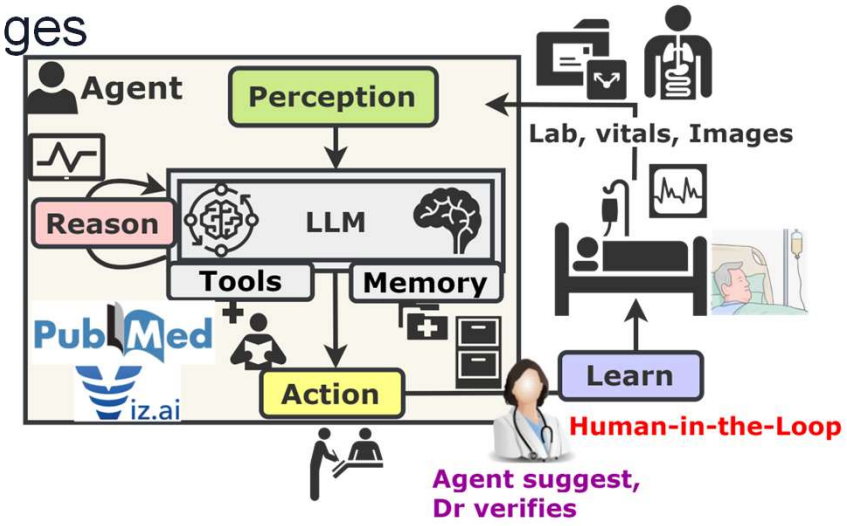
What it is: A workflow pattern that pauses and waits for human approval

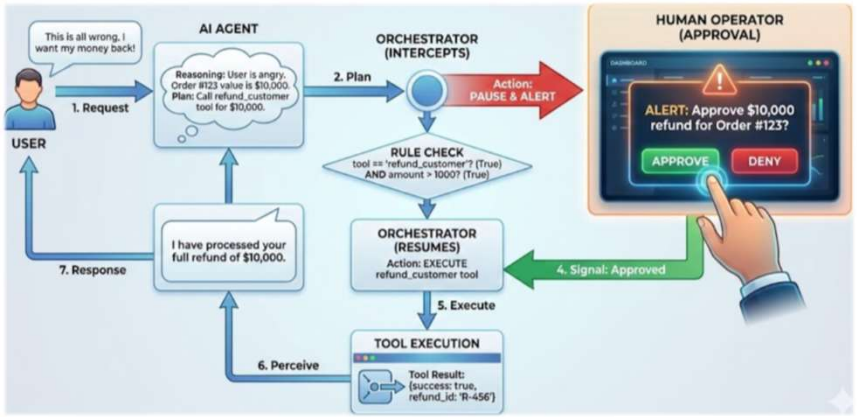
How it works: An orchestrator intercepts an agent's plan and checks it against deterministic business rules (the "checkpoint")

When to use it: For any high-risk or irreversible action

Core Principle: The agent suggests, the human decides

Use Cases: Financial transactions, data deletion, sending critical external messages



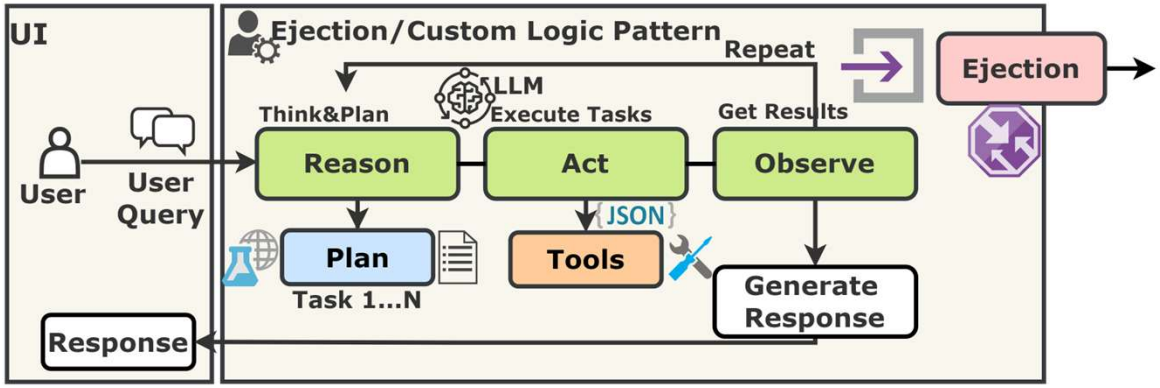


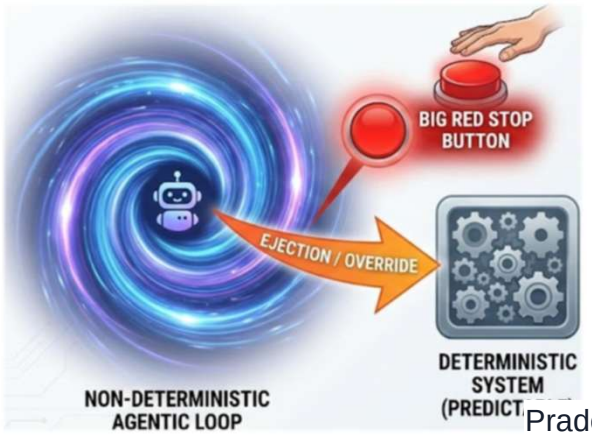
The "Ejection / Custom Logic" Pattern

'Human-in-the-Loop' pattern; our **'pause button'**—a crucial checkpoint where the agent proposes a dangerous action and waits for a human to approve it

What happens when the **agent** is **not working correctly**?
What happens when it's **stuck in a loop**, **frustrating** the **user** or **when** the **user** simply **wants to stop?**

A **'pause'** button **isn't enough**. We need an **'escape hatch'.** We need a hard-coded, deterministic rule that overrides the agent's autonomy completely





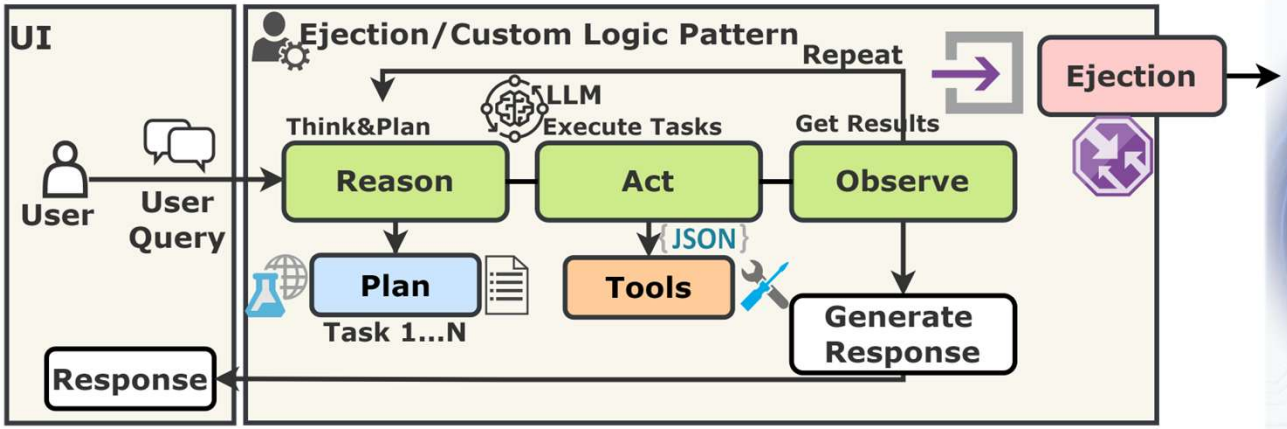
The "Ejection / Custom Logic" Pattern Defined

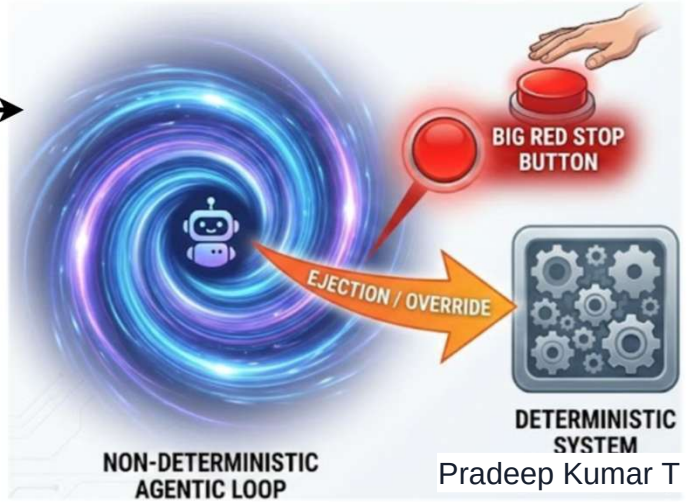
The Metaphor: The "**Escape Hatch**" or the "**Big Red Stop Button**"

Definition: A deterministic, rules-based pattern that immediately bypasses or terminates the non-deterministic agentic loop when a specific condition is met

Core Function: To instantly transfer control away from the non-deterministic LLM and to a predictable, deterministic system

Key Principle: This is a Deterministic Override for a Non-Deterministic System

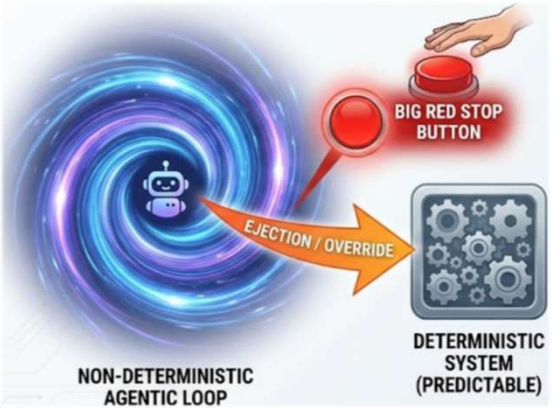




The Problem: Why We Need an "Escape Hatch"

'Human-in-the-Loop' (HITL) is for Dangerous Actions
The agent is working correctly
(e.g. "I'm about to charge $10,000. Is this okay?")

'Ejection' is for Broken or Undesirable Conversations
The agent is failing or the user wants out

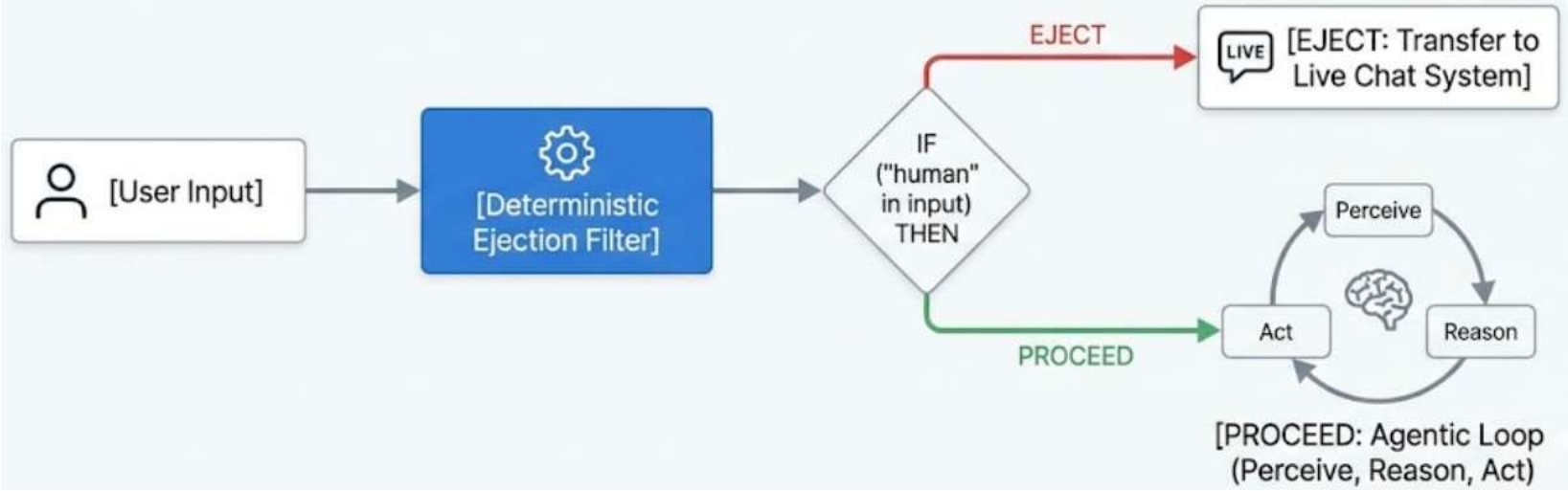


Key Scenarios:
1. User Frustration: The agent is stuck in a loop. The user says, "You're useless, let me talk to someone!"
2. User Termination: The user wants to stop. ("Stop," "Cancel," "Nevermind" "Goodbye")
3. Policy / Safety Violation: The user is abusive, profane or asking for something your policy forbids

The Architecture: The "Pre-Check" Filter

Critical: This logic cannot be inside the agent's prompt. The agent must not be able to ignore it

Implementation: It is a simple, deterministic if statement that runs before the agent's 'Perception' or 'Reasoning' phase



Classic Use Cases (Code Logic)

1. The "Live Agent" Ejection (Customer Service):

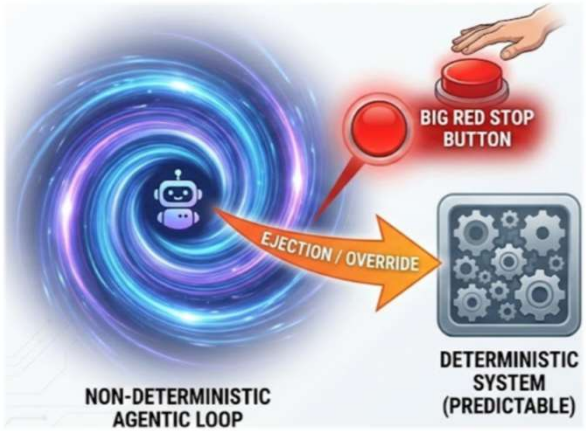
```
if ( user_input.contains("human") || user_input
    .contains("agent") ) {
    return transfer_to_live_chat(session_id);
}
```

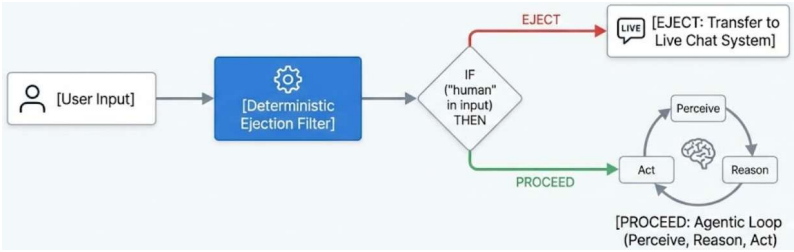
2. The "Stop" Ejection (Task Termination):

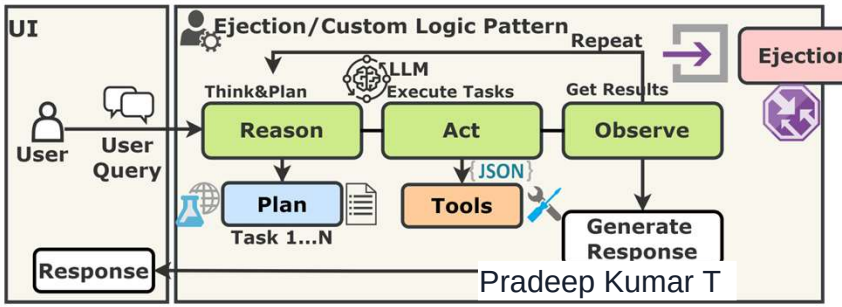
```
if ( user_input.contains("stop") || user_input.contains
    ("cancel")) {
    return terminate_workflow(session_id, "User
        requested stop.");
}
```

3. The "Policy" Ejection (Safety & Moderation):

```
if ( is_profane(user_input) || is_policy_violation
    (user_input) ) {
    return send_canned_response("I cannot assist with
        that request.");
}
```







Summary: The "Escape Hatch" Pattern

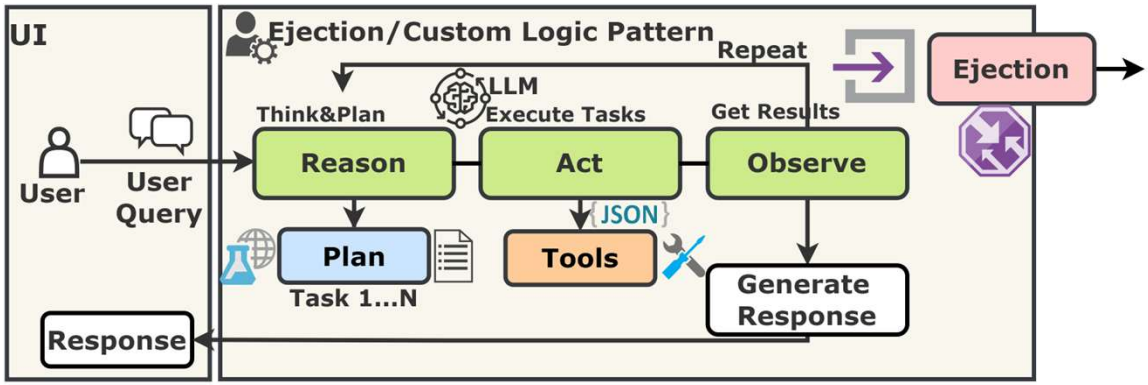
This is your "**escape hatch**" or "**stop button**"

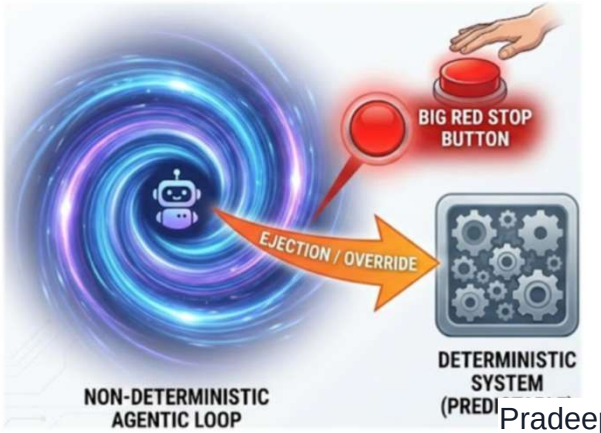
It is a **deterministic** rule that overrides the **non-deterministic agent**

It **bypasses the agentic loop**, it does not pause it (like HITL)

Architecture: Implemented as a **"pre-check" filter** before the LLM is called

Non-Negotiable: This pattern is essential for any user-facing production system to handle frustration, termination and policy violations





What is Agentic RAG ? Advanced Reasoning: Age RAG

What is Agentic RAG ?

Agentic RAG vs. Traditional RAG systems

Is agentic RAG better than traditional RAG?

How does Agentic RAG work?

Agentic RAG Use Cases

Pradeep Kumar T



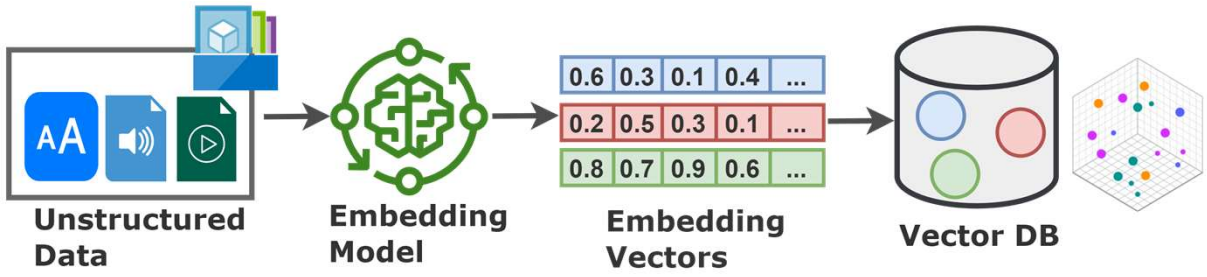
Introduction - The Limitations of Traditional RAG

RAG has been the **single most common architecture** for building **enterprise-ready** Generative AI

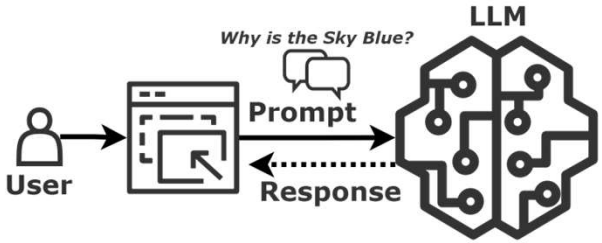
Allows us to **connect** a **'stateless' LLM** to our own, **private**, **up-to-date** information

RAG implementation is **quite simple**. And it has **significant limitations**

Understanding is **the-very-reason** we need a new, more powerful pattern, which we will soon call **'Agentic RAG'**



What is Traditional RAG? (A Recap)

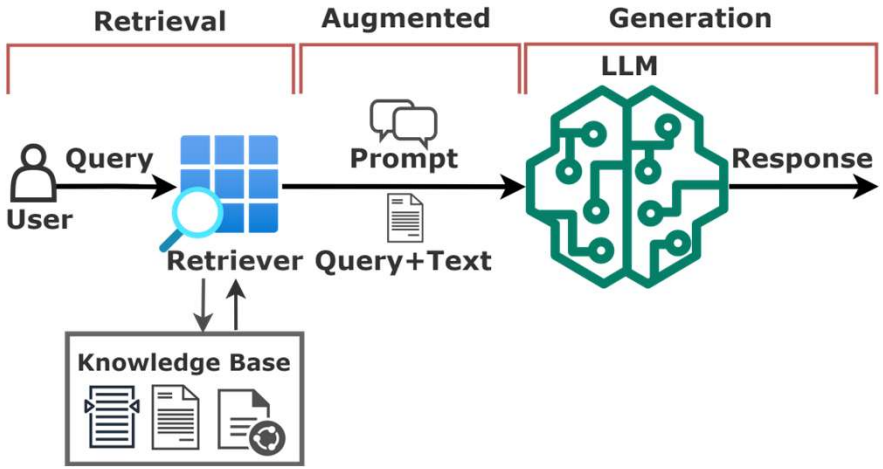


RAG = Retrieval-Augmented-Generation

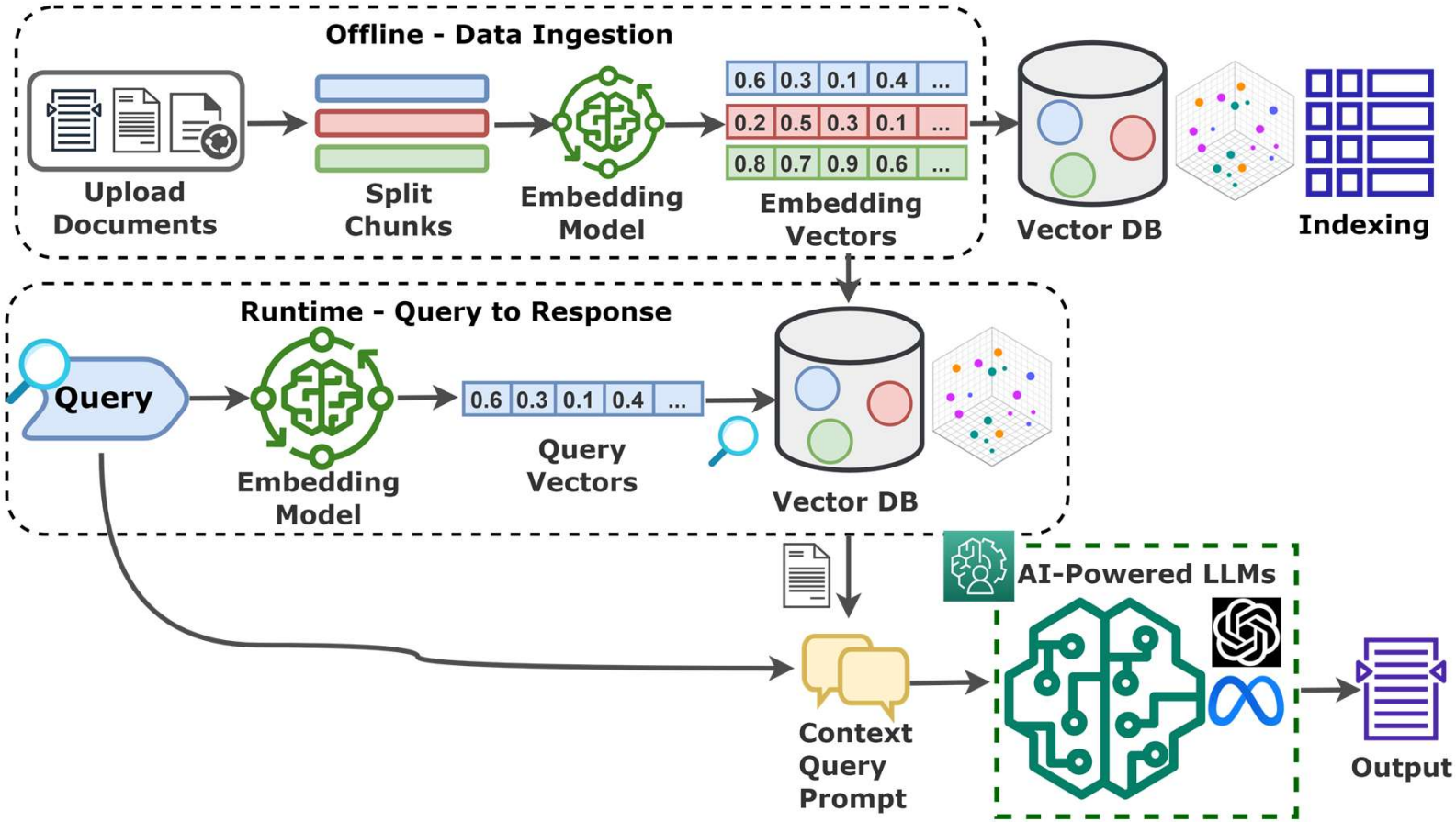
The Goal: To answer a user's question using a specific set of private documents, rather than the LLM's general training data

The "Mantra": Don't let the LLM guess the answer. Give it the answer and ask it to summarize it

It's a "Grounding" Mechanism: It grounds the LLM in facts, reducing hallucinations and enabling it to use new information

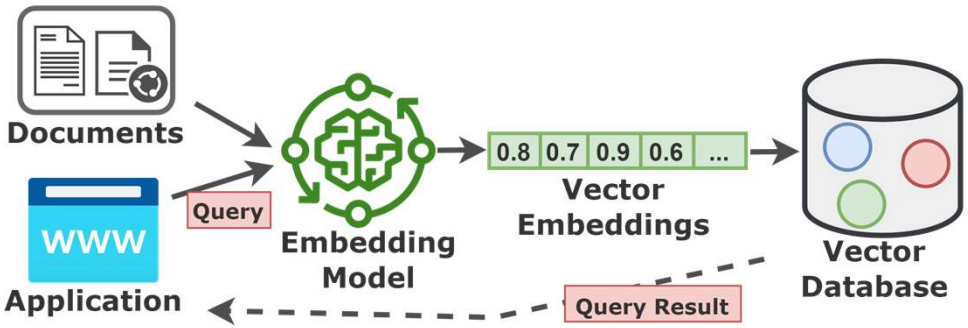


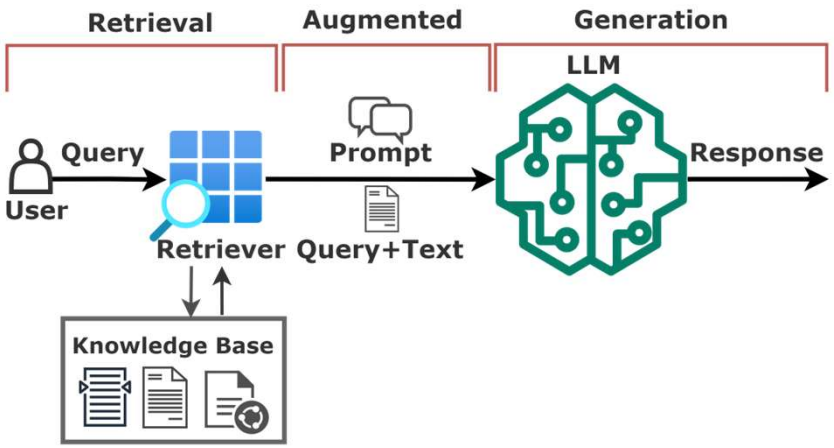
The "Naive" RAG Architecture: A Passive Pipeline



Example of Traditional RAG Architecture

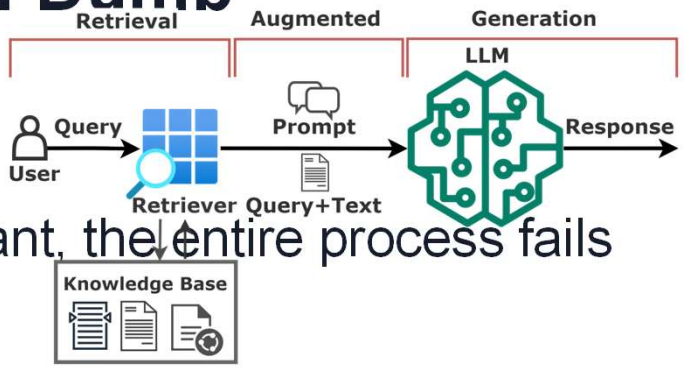
Traditional RAG is a one-way, linear "chain" of events
[Query -> Embed -> Search -> Stuff -> Generate]





1. **User Query:** What is our Q3 revenue?
2. **Embed:** The query is turned into a vector (a list of numbers)
3. **Search:** The vector is used to find the "Top-K" (top 5) most similar chunks of text from the vector database
4. **Stuff:** These raw text chunks are "stuffed" into the LLM's prompt context
5. **Generate:** The LLM is asked to answer the query using only the provided text

The Core Problem: RAG is "Passive and Dumb"

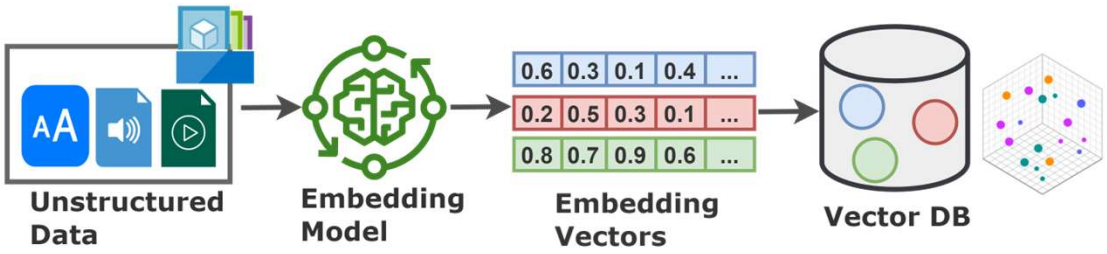


The RAG pipeline cannot think for itself

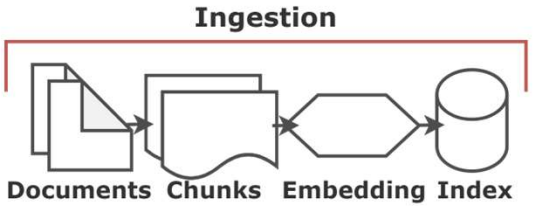
1. It's a "Single, One-Shot" Retrieval:
It searches once. If the "Top-K" results are bad or irrelevant, the entire process fails

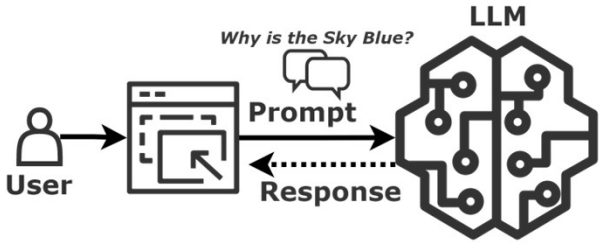
2. It Cannot "Self-Correct":
The LLM cannot "talk back" to the retriever
It cannot say: "These search results are useless. Please search again with a different query"

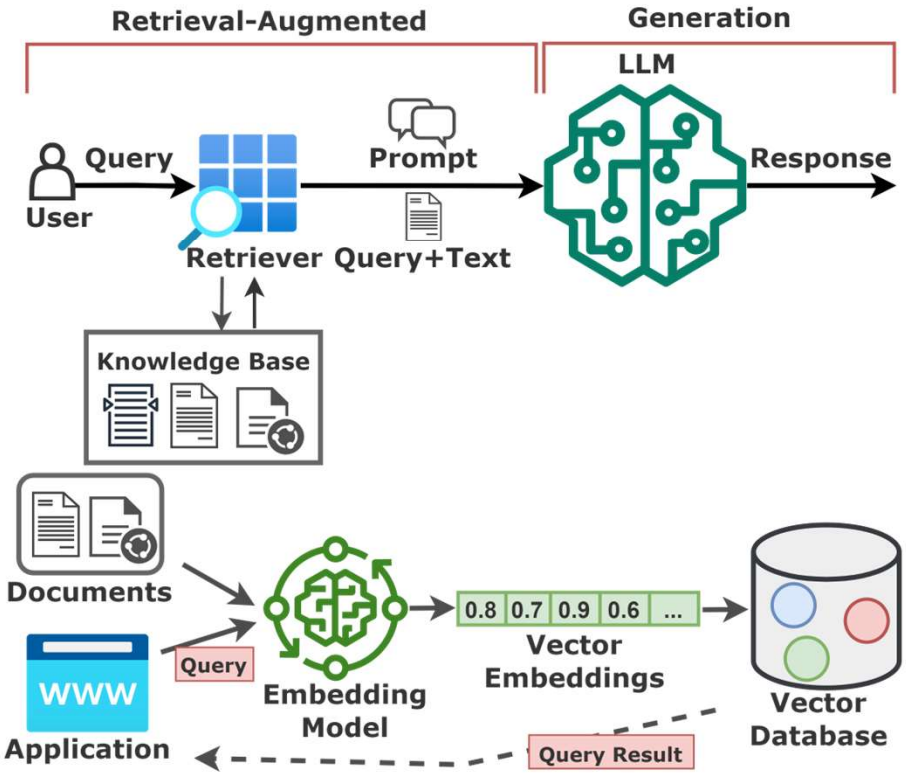
3. It Fails on Complex, Multi-Step Questions:
Query: "Compare our Q3 revenue with our top competitor's Q3 revenue.«
Failure: A single search can't find both things. It will find one or the other, or just noise

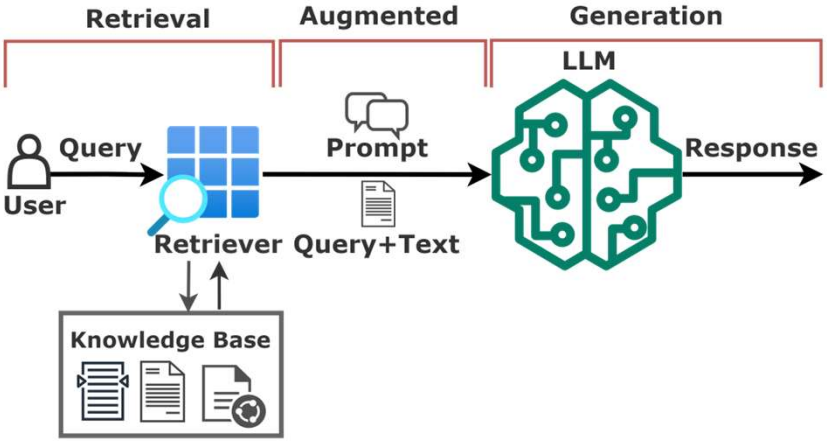


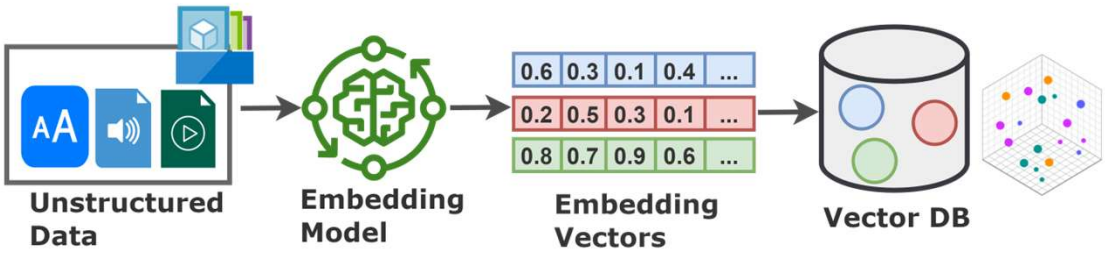
What is Traditional RAG? (A Recap)





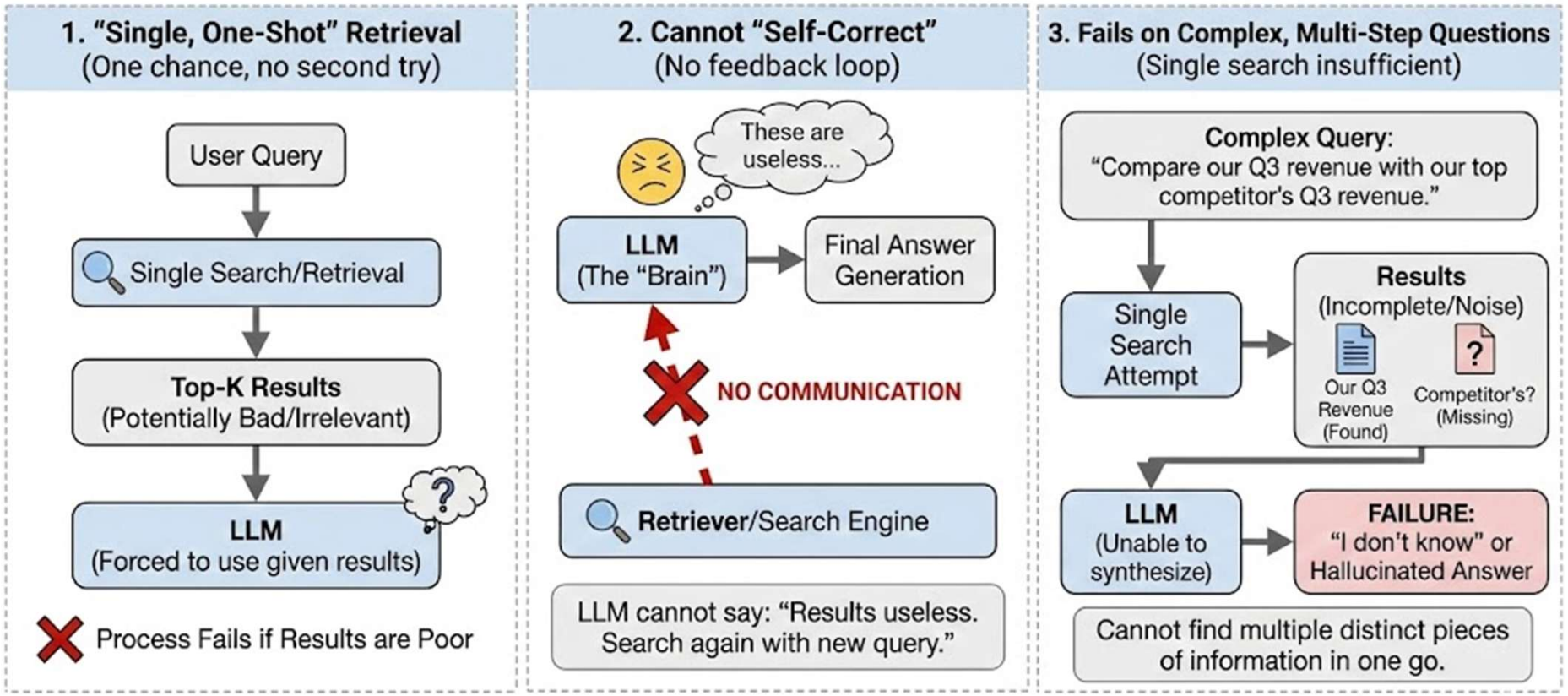






The Core Problem: RAG is "Passive and Dumb"

The **RAG** pipeline **cannot think** for **itself**



The Architect's View: The Limitation Summary

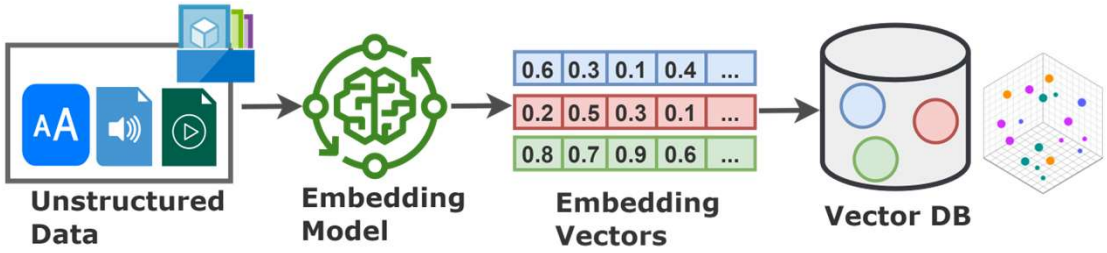
The System: A linear, one-way "chain"

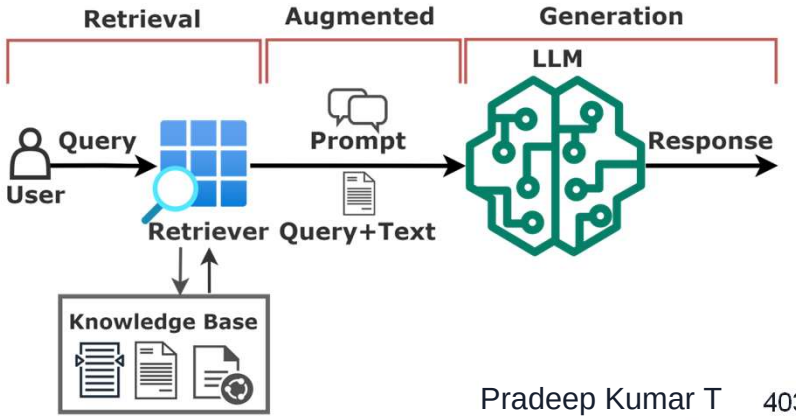
The Intelligence: The LLM is only used at the very end (to summarize)

The Failure Mode: It is brittle. It fails if the very first step (the retrieval) is imperfect

The Consequence: It can only answer simple, "lookup-style" questions. It cannot perform research

We have a "retriever" system, not a "reasoning" system





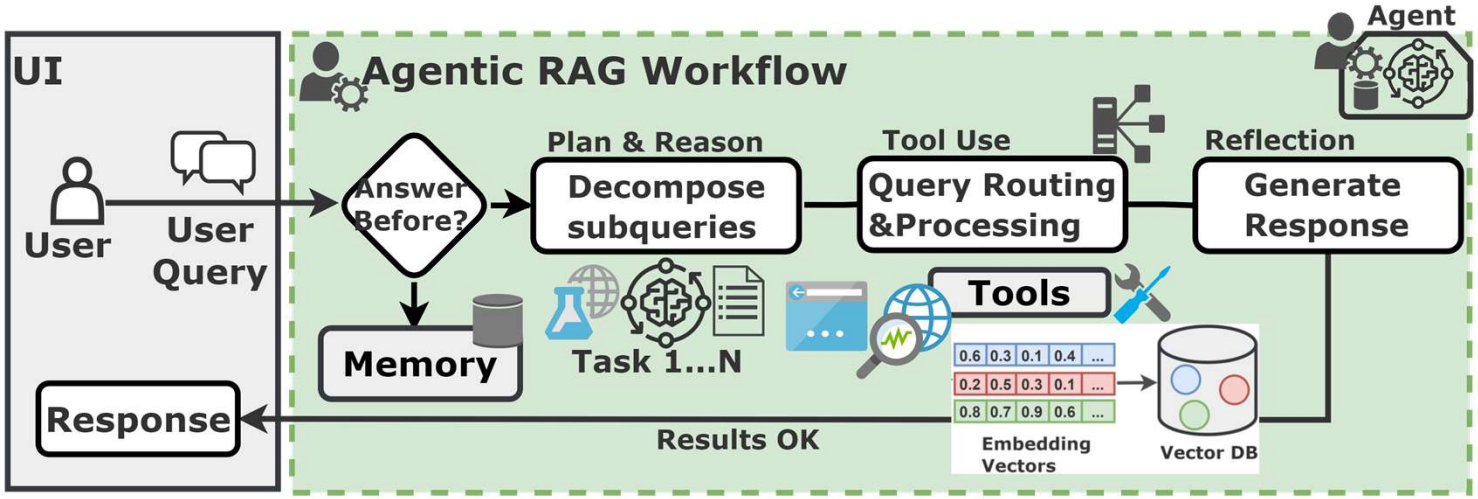
What is Agentic RAG ?

Identified the **critical failures** of **traditional RAG**

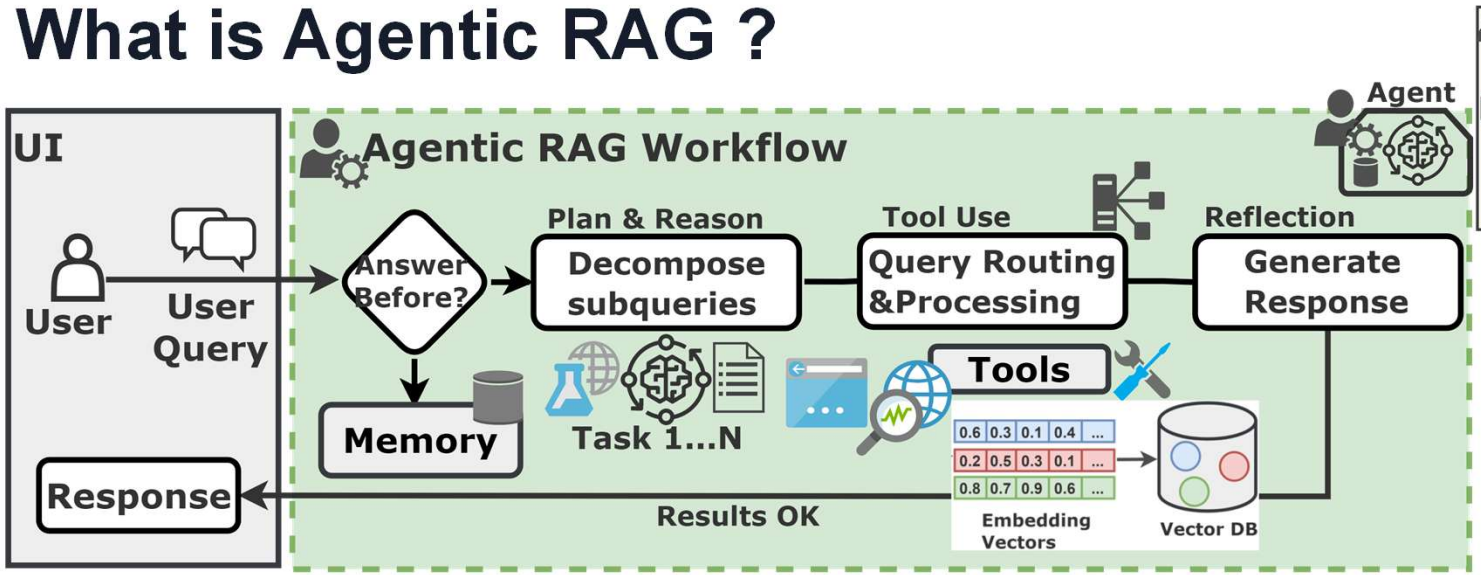
Its **'passive pipeline'** architecture is brittle, it's a **'one-shot' retrieval** and it completely **fails** on **complex**, **multi-step questions** because it cannot think

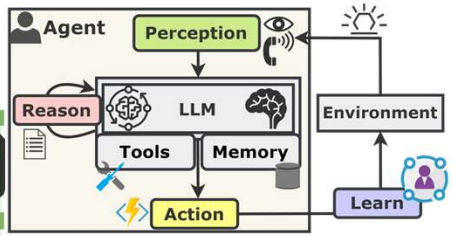
This solution is not just an **'improvement'**—it is a **fundamental architectural shift**

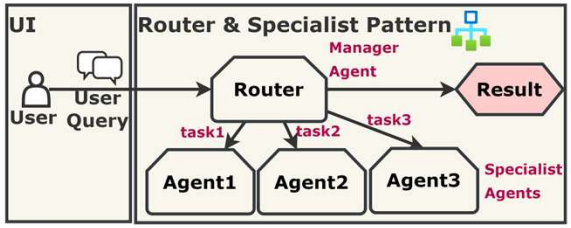
Moving from a **passive data pipeline** to an **active, intelligent process**
This is **Agentic RAG**

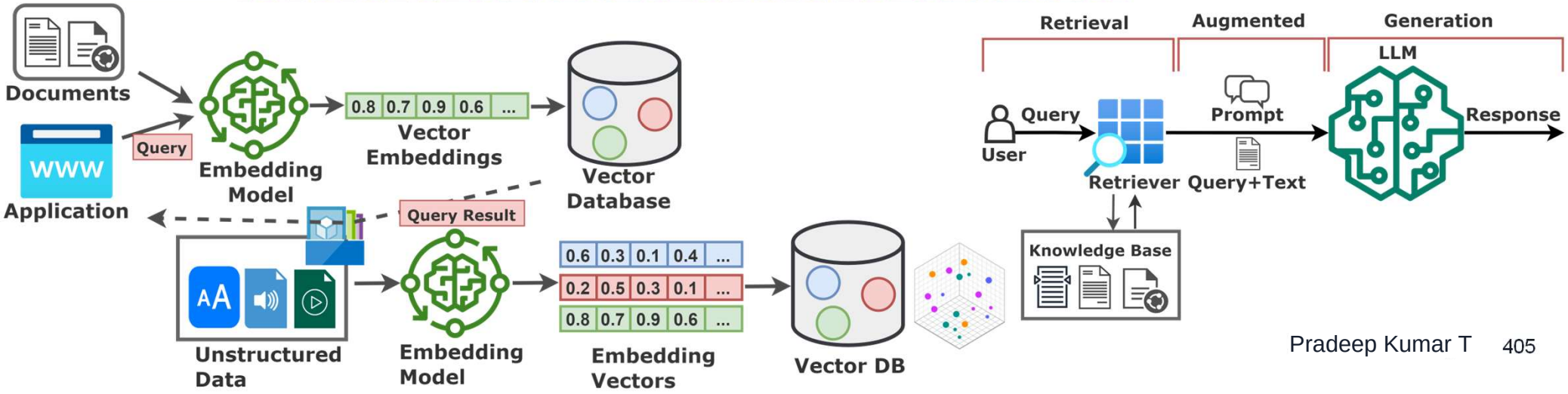


What is Agentic RAG ?







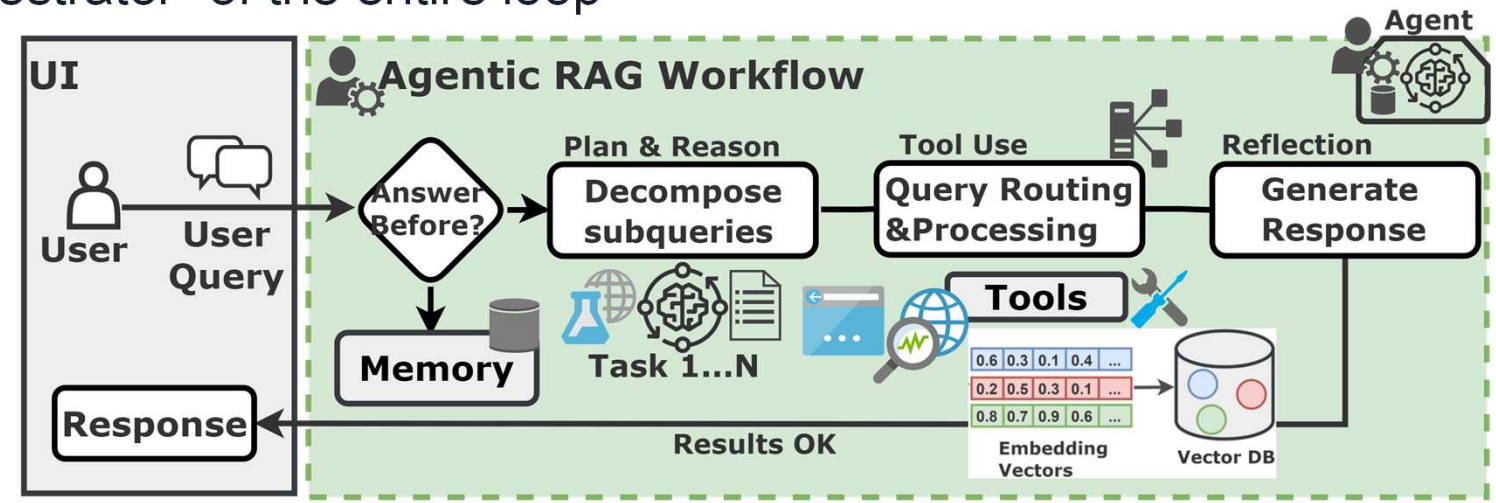


The Formal Definition: What is Agentic RAG?

Agentic RAG is an architecture where a **Reasoning Agent** (an LLM) is **placed** in charge of the **entire retrieval** and **synthesis process**

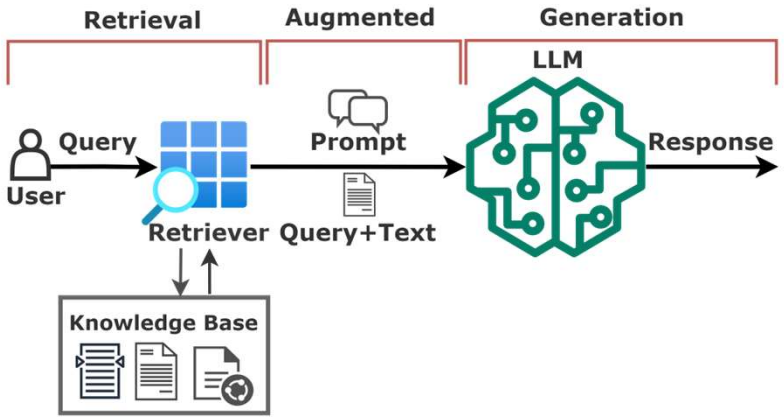
The "Active Researcher" Model:
Instead of just receiving data, the agent actively decides what to search for, when to search for it, and how to react to the search results

Core Shift: The LLM is no longer at the end of the chain. The LLM is the "orchestrator" of the entire loop



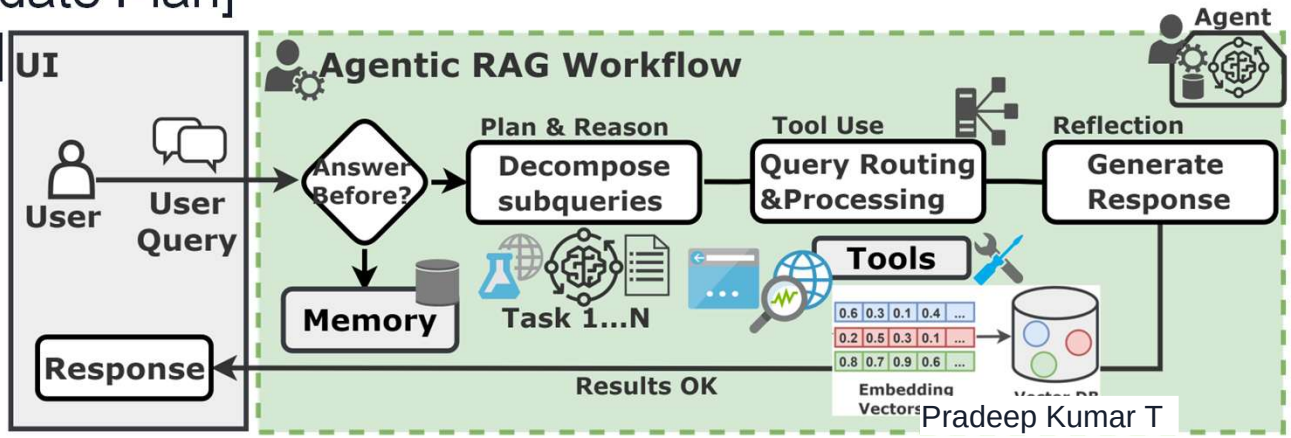
The New Architecture: A "Loop," Not a "Chain"

Traditional RAG (A "Chain"):
[Query] -> [Search] -> [Stuff] -> [Generate]
(A single, one-way, passive pipeline)



Agentic RAG (A "Loop"):
[Agent: Reason/Plan]
-> [Agent: Act (Search)]
-> [Results: Perceive]
-> [Agent: Reason/Reflect/Update Plan]
-> [Agent: Act (Search Again)]

It is a ReAct pattern applied to research



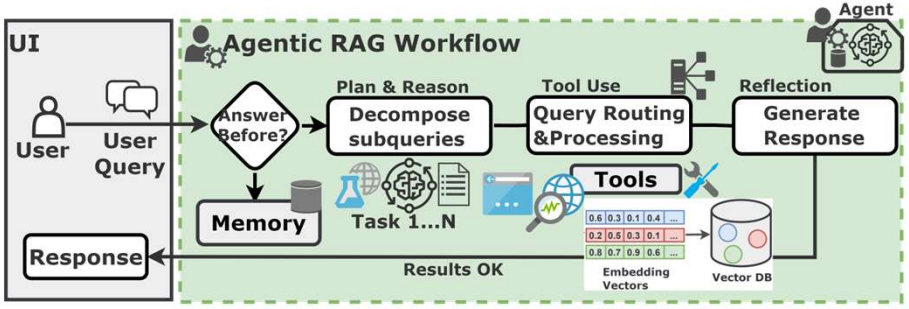
The "Superpowers" of Agentic RAG

1. Multi-Step Retrieval (Solves "One-Shot"):
The agent can search, read and search again, progressively building complex knowledge

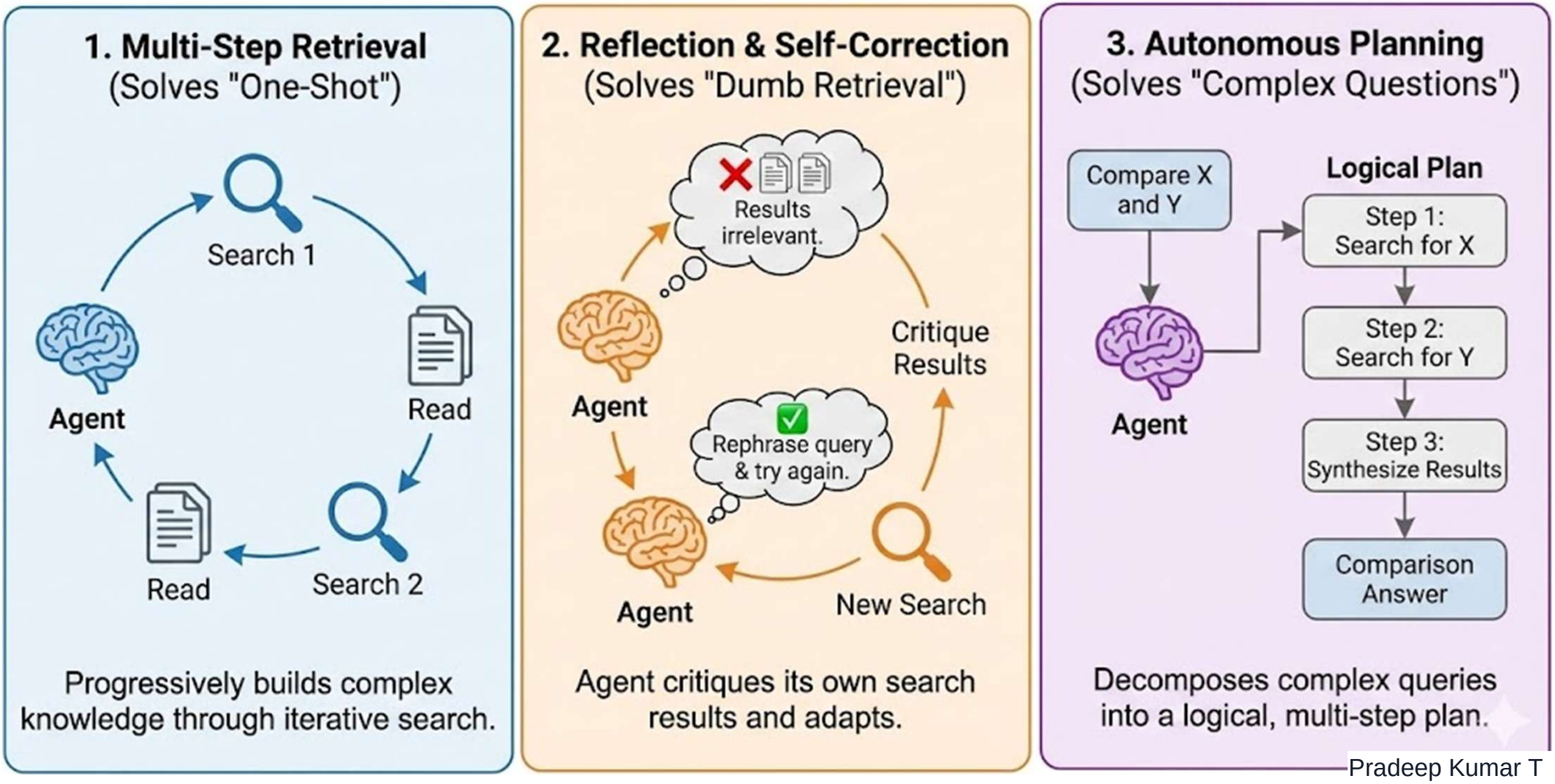
2. Reflection & Self-Correction (Solves "Dumb Retrieval"):
The agent can critique its own search results
("These results are irrelevant. I will rephrase my query and try a different keyword")

3. Autonomous Planning (Solves "Complex Questions"):
The agent can decompose a complex query ("Compare X and Y") into a logical, multi-step plan:

Plan: 1. Search for X. 2. Search for Y. 3. Synthesize



The "Superpowers" of Agentic RAG



Summary: RAG as an Agentic Loop

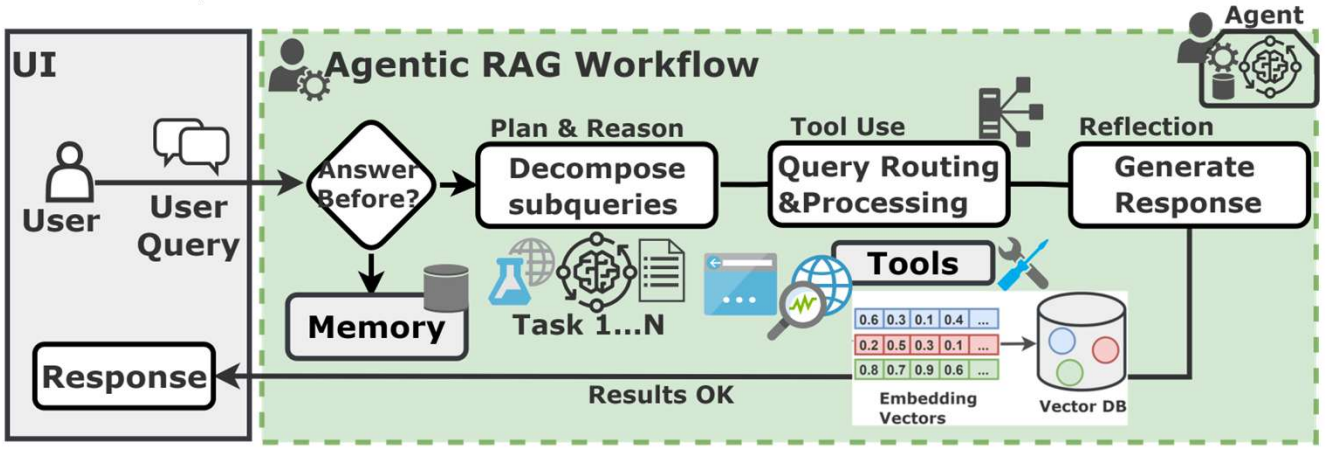
It is an **architecture** where an **agent manages** the **retrieval process** within a **multi-step reasoning** loop

It is NOT: A "chain" or a "pipeline"

It IS: An "active", "reflective" and "multi-step" process

It can solve **complex**, **multi-part research** questions that are impossible for traditional RAG

Next Lecture: Let's put them head-to-head



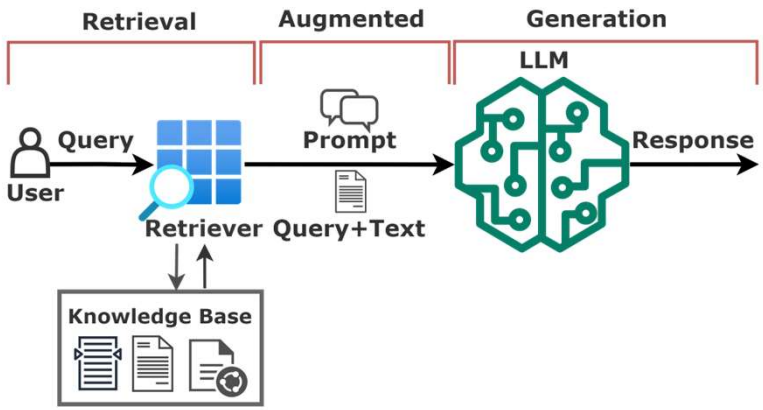
Agentic RAG vs. Traditional RAG: The Critical Difference

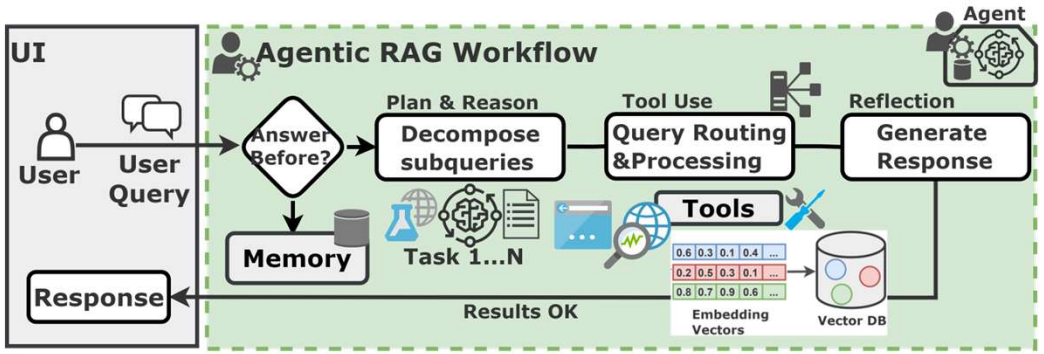
Traditional RAG: a passive, one-way pipeline
Agentic RAG: an active, multi-step reasoning loop

Critical **comparison** that helps you, as an **architect**, **decide which pattern** to use

Is **Agentic RAG** simply better than **Traditional RAG**?

The answer, as you'll see, is a classic architectural **'it depends'**





Comparison 1: The Process (Chain vs. Loop)

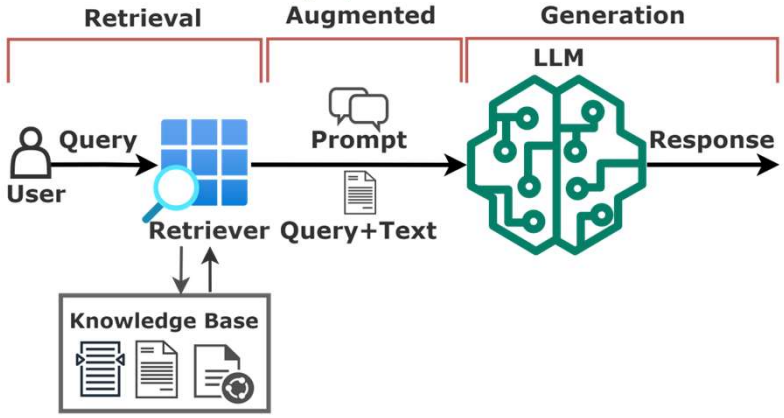
Traditional RAG (The "Chain"):

[Query] -> [Search] -> [Stuff] -> [Generate]

This is a fixed, linear pipeline

Each step executes exactly once, in order

The 'intelligence' (the LLM) is the last step



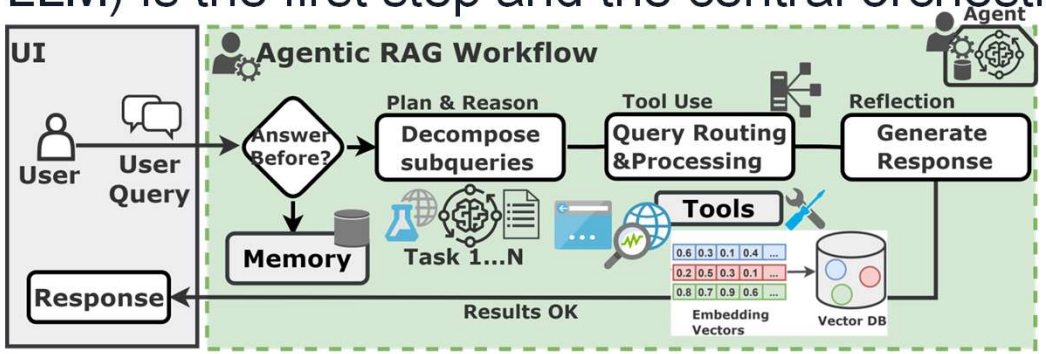
Agentic RAG (The "Loop"):

[Reason] -> [Act(Search)] -> [Perceive] -> [Reason/Reflect]...

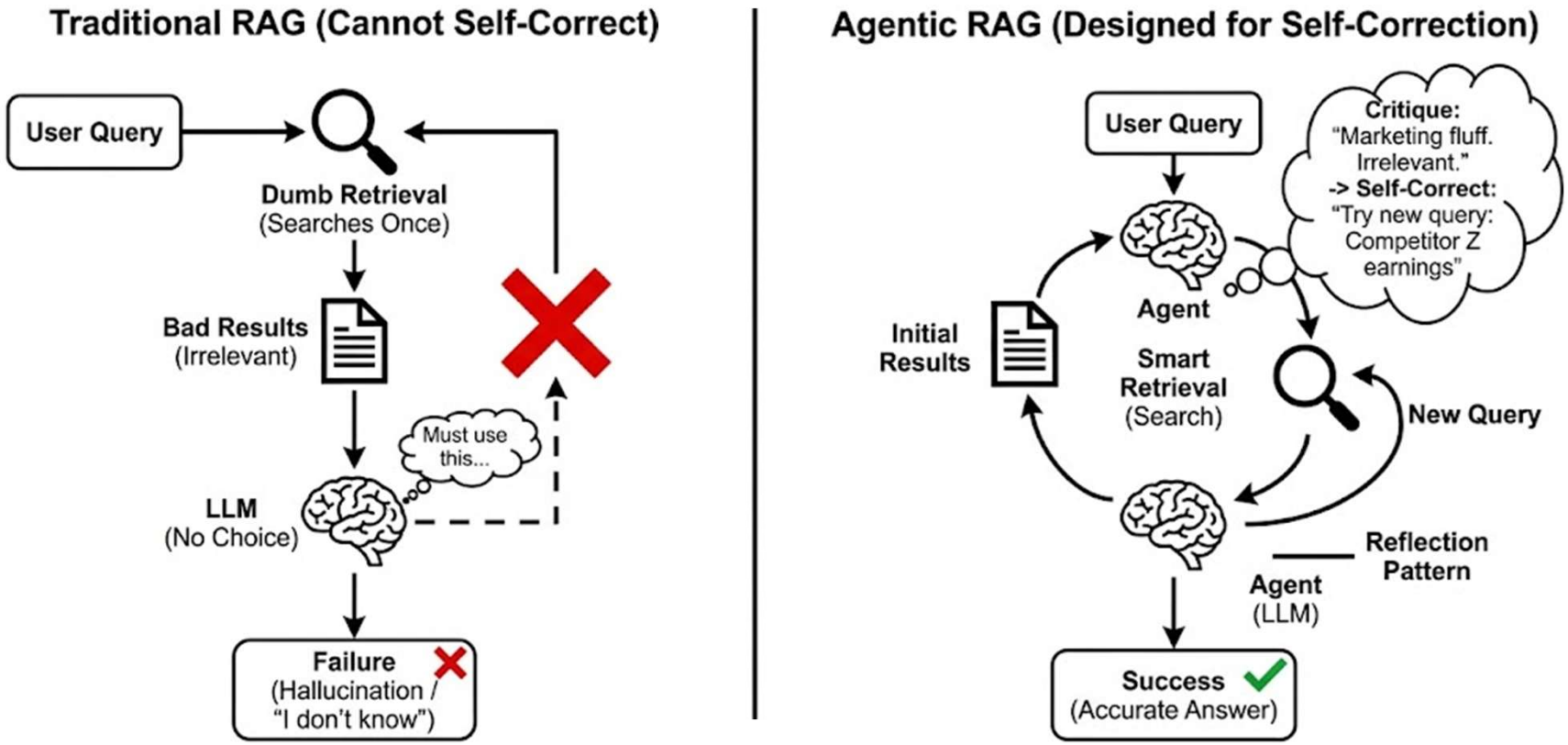
This is a dynamic, cyclic graph

The process is iterative. The "Search" step can be called multiple times

The 'intelligence' (the LLM) is the first step and the central orchestrator

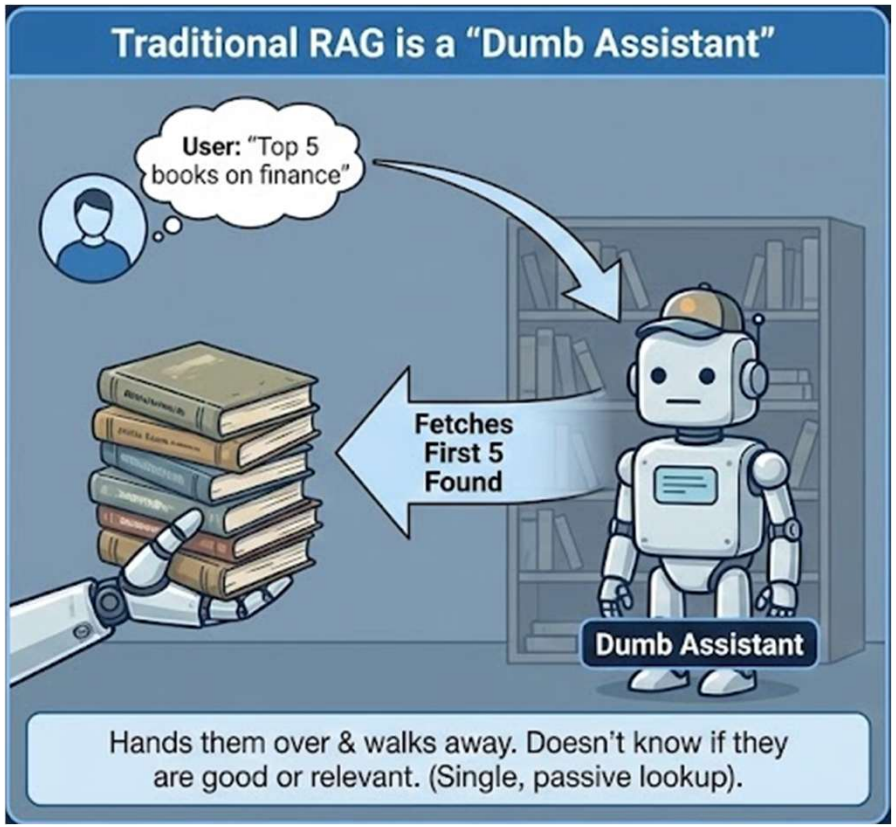


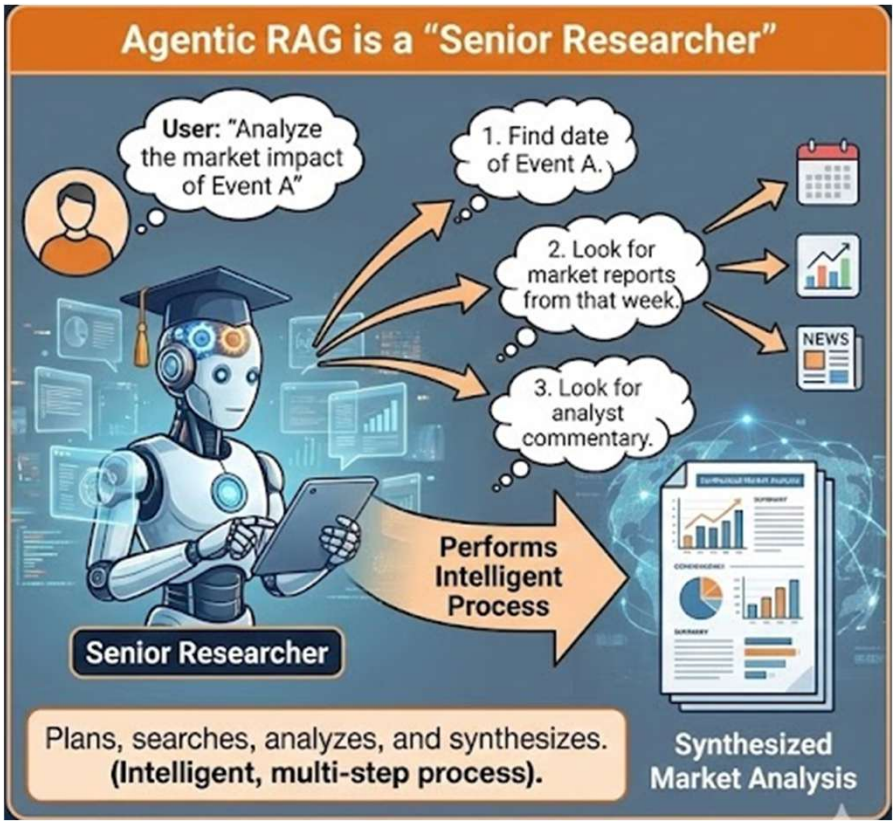
Comparison 2: The Intelligence (Dumb vs. Reflection)



Example: I searched for **'Q3 competitor revenue'** and got marketing fluff. Failed. I will self-correct and try a new query: **'Competitor Z quarterly earnin**

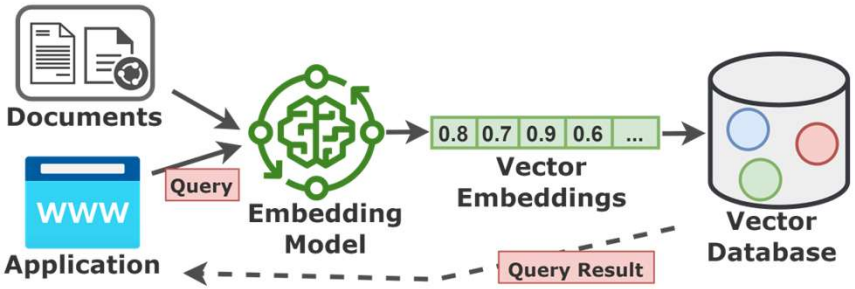
The "Active Researcher" vs. "Dumb Assistant"





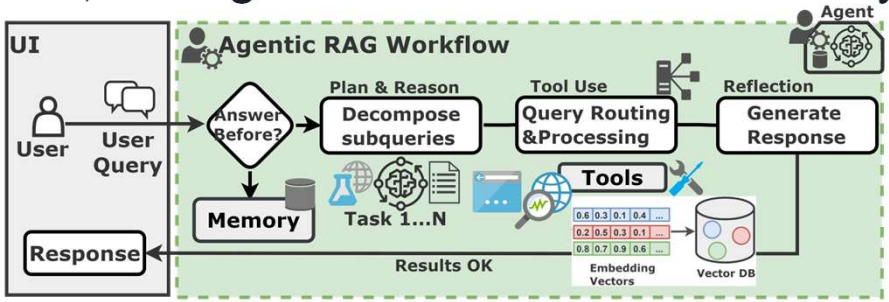
Comparison 3: The Architect's Choice (The Trade-Off)

Is Agentic RAG better? It **depends on your goal**

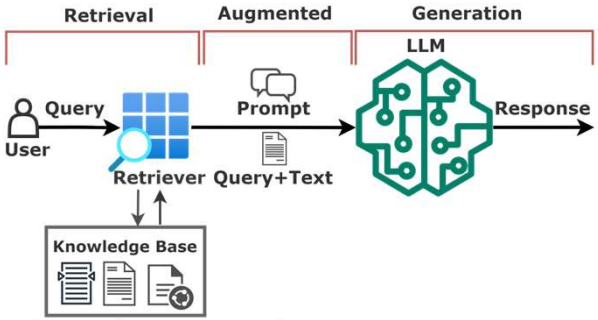


Traditional RAG:
Fast: One search, one LLM call. Low latency
Cheap: Minimal token usage
Simple: Easy to build, debug, and make deterministic

Agentic RAG:
Slow: Multiple LLM calls (Plan -> Search -> Reflect -> Search -> Synthesize) lead High latency
Expensive: Each reasoning/reflection step is a full LLM call, burning tokens
Complex: Harder to build, debug and is non-deterministic by nature

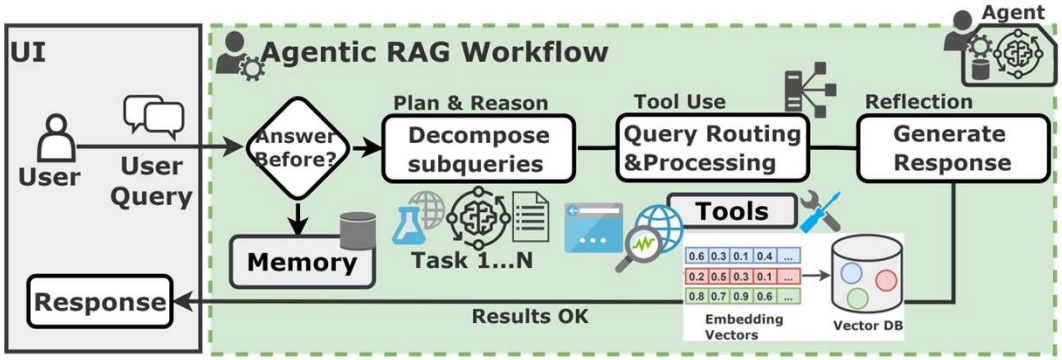


When to Use Which?



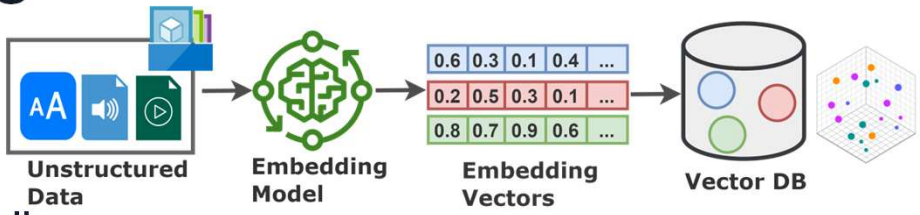
Use Traditional RAG for "Lookup" Tasks:
Simple Q&A ("What is our company's PTO policy?")
Simple Summarization ("Summarize this document.")
Your Goal: Speed, cost-efficiency, and high reliability on simple queries

Use Agentic RAG for "Research" Tasks:
Complex, multi-step questions
Vague or ambiguous queries ("What's the market sentiment on our new product?")
Power, robustness and the ability to solve complex problems where latency is less of a concern



Summary: Two Tools for Two Jobs

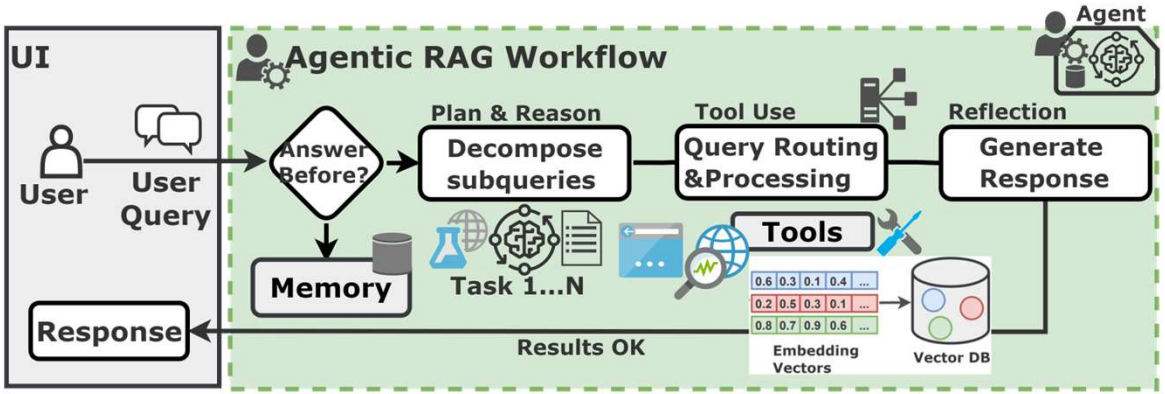
Traditional RAG: A simple, fast, cheap "chain"
Use Case: Simple Q&A

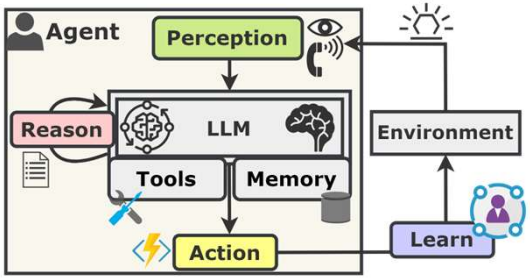


Agentic RAG: A complex, slow, expensive "loop"
Use Case: Complex Research

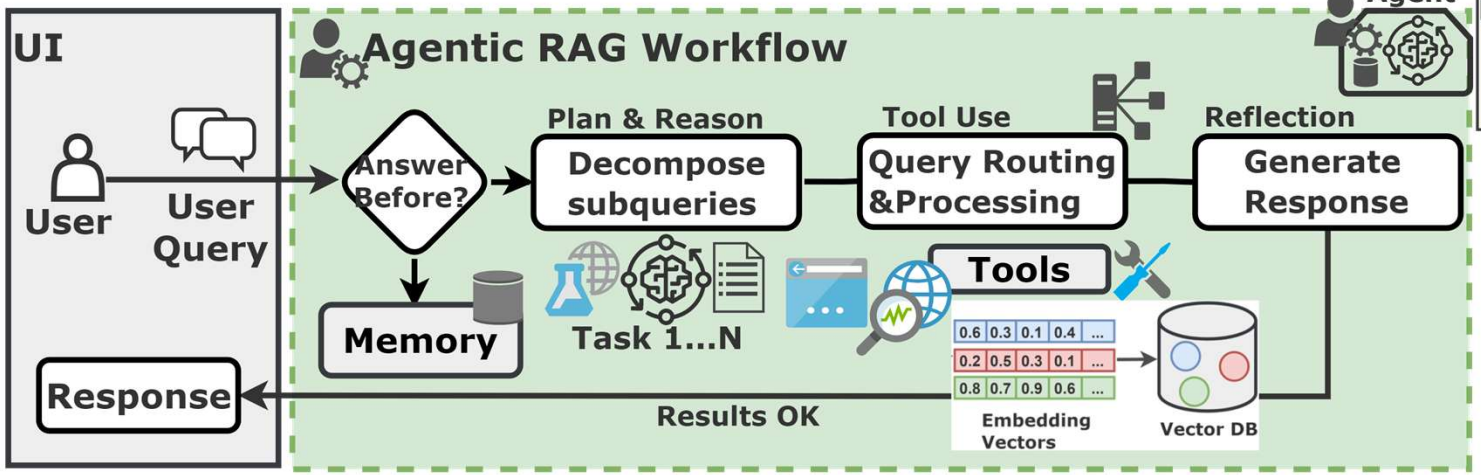
We have defined and compared these two architectures

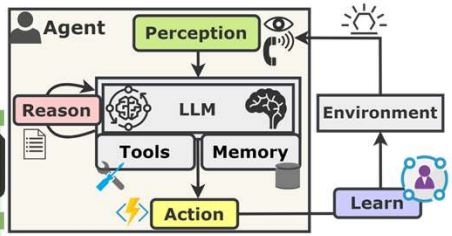
Walk through the exact step-by-step flow of an Agentic RAG loop

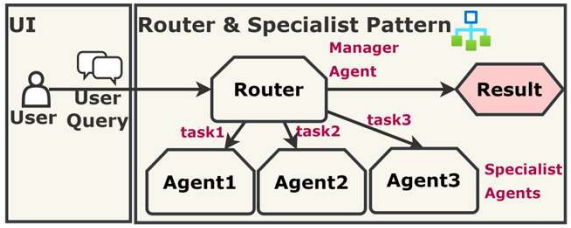


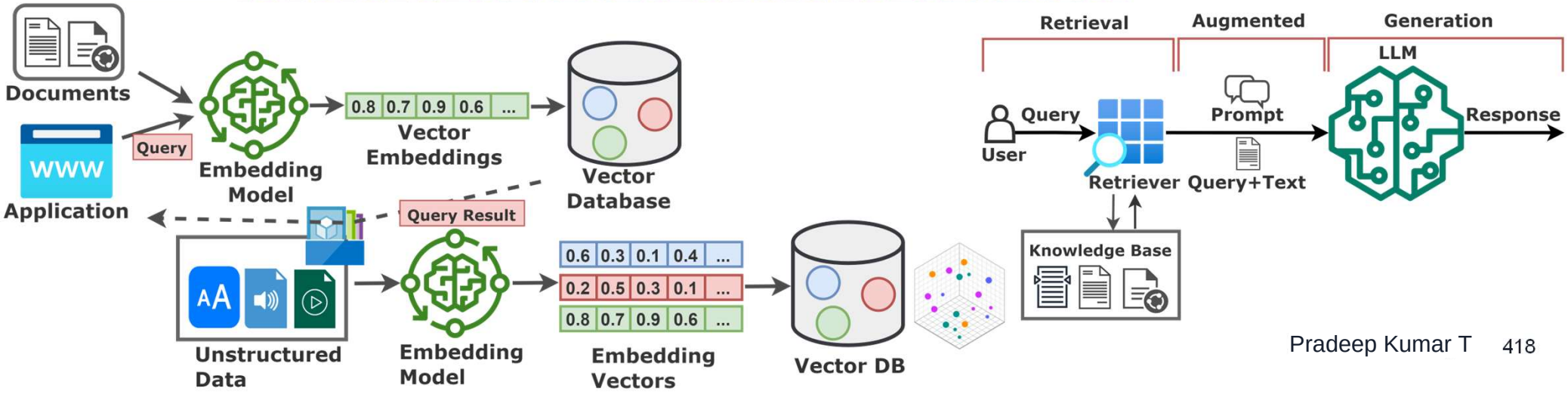


What is Agentic RAG ?







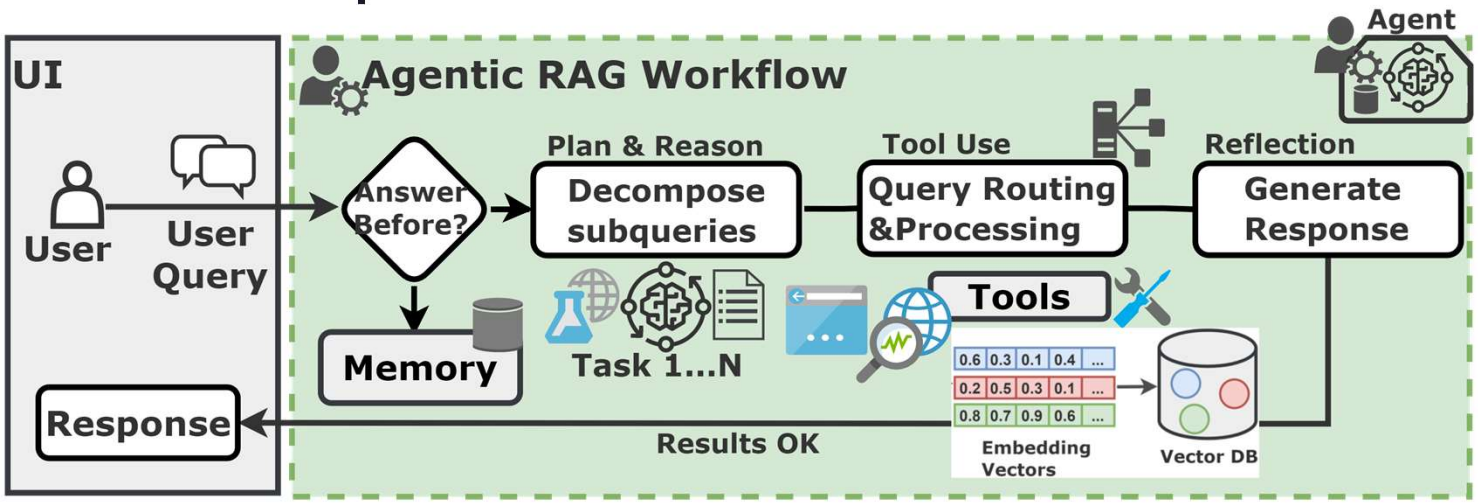


How Agentic RAG Works: The "Active Researcher" Loop

Established that **Agentic RAG** is an 'active, multi-step loop'—**but what does that look like in practice?**

Trace a **single**, **complex research task** from start to finish, observing every step of the agent's thought process

This is the **ReAct pattern**—Reason and Act—**applied to** a **RAG workflow**, and it's the **'how' behind** this **powerful architecture**

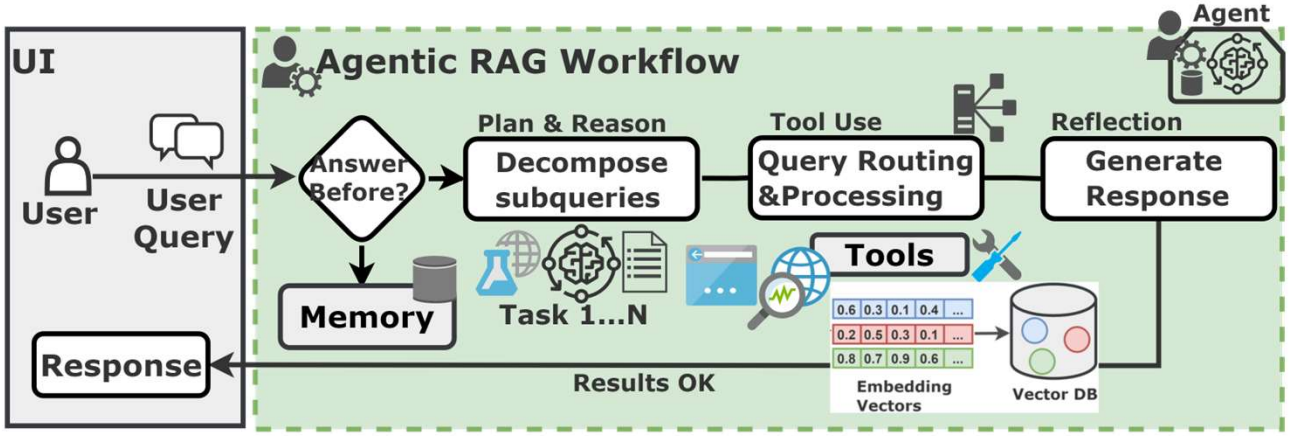


The Scenario: A Complex Research Task

The User's Goal:
Find the financial impact of our new product: 'Product X' on company: 'Company Y'

Why Traditional RAG Fails:
A single search for this entire string will find nothing useful
The query is ambiguous and multi-part. It requires a strategy

The Agent's Job:
To formulate and execute a strategy to find the answer



Loop 1: [Reason] Initial Plan & [Act] First Search

Step 1: [Reason] The Initial Plan

The agent receives the goal. The LLM (as planner) thinks:

Thought: This is complex. I cannot answer this directly

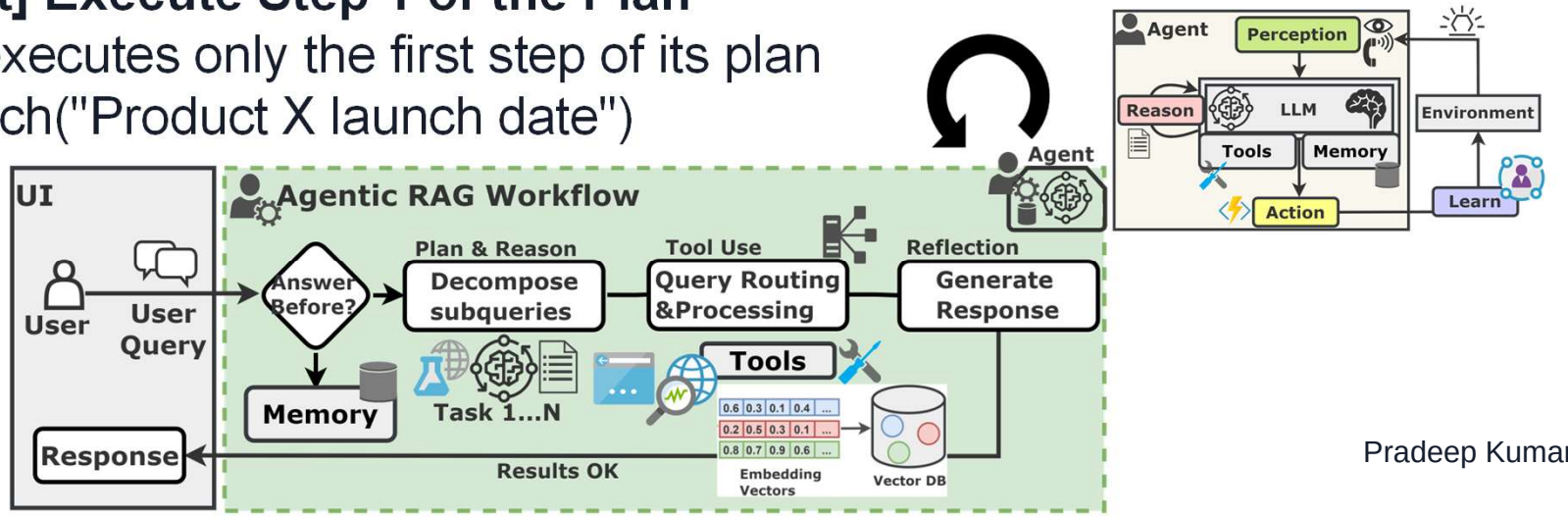
Plan:

1. I must first find the launch date of 'Product X'
2. Then, I must find 'Company Y's' quarterly reports after that launch date

Step 2: [Act] Execute Step 1 of the Plan

The agent executes only the first step of its plan

Action: search("Product X launch date")

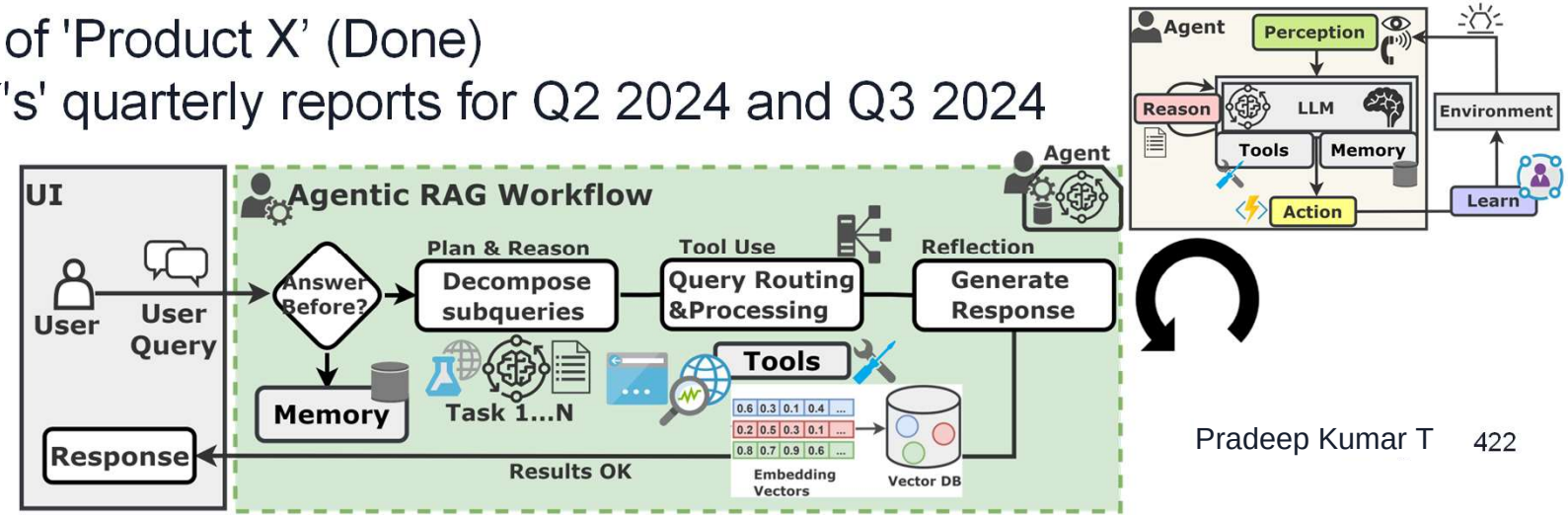


Loop 1: [Perceive] Observe & [Reason] Reflect/Update

Step 3: [Perceive] Observe the Result
The search tool returns a result
Observation: "Search results indicate 'Product X' was launched in Q2 2024"

Step 4: [Reason] Reflect and Update Plan
This observation is fed back to the agent. The LLM thinks again:
Thought: "I have the launch date (Q2 2024). My original plan is now updated"

Updated Plan:
1. Find launch date of 'Product X' (Done)
2. Find 'Company Y's' quarterly reports for Q2 2024 and Q3 2024

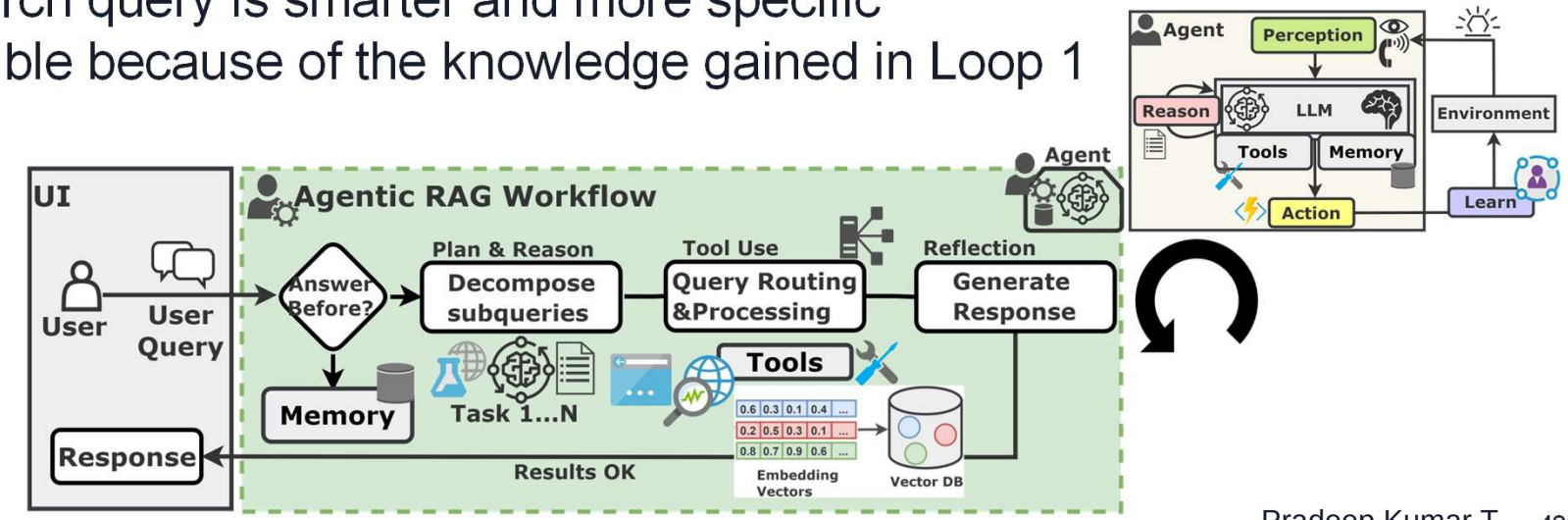


Loop 2: [Act] The Second, Informed Search

The Loop Repeats: The agent is still following its (now updated) plan

Step 5: [Act] Execute Step 2 of the Plan
The agent executes the next step of its updated plan
Action: search("Company Y quarterly report Q2 2024 Q3 2024 financial impact")

Key Insight:
This second search query is smarter and more specific
It was only possible because of the knowledge gained in Loop 1



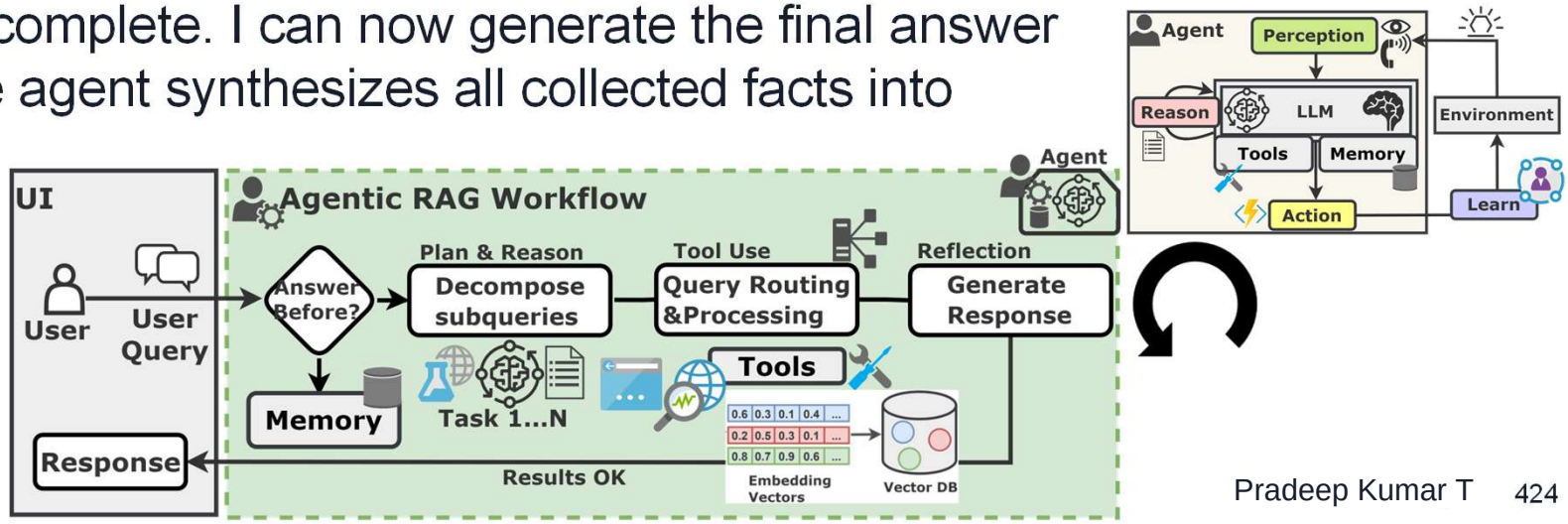
Loop 2: [Reason] The Final Synthesis

Step 6: [Perceive] & [Reason] Synthesize
[Perceive]: The agent gets the second set of results (the financial reports)
[Reason]: The LLM reads these new results and thinks:

Thought: I now have all the necessary information

1. The launch date was Q2 2024
2. The Q2 and Q3 reports show a 15% growth in the 'New Products' division

Plan: The plan is complete. I can now generate the final answer
Final Output: The agent synthesizes all collected facts into one answer



Summary: The "Active Researcher" Loop in Action

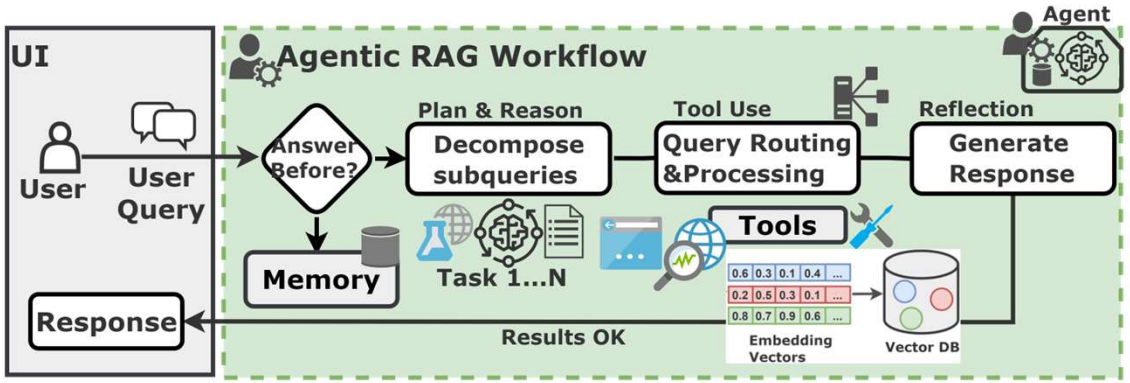
We did not do a "**single search**"

We executed a dynamic, **multi-step research plan**

Loop 1: The agent was "ignorant" and sought a "known unknown" (the date)

Loop 2: The agent was "informed" and used that new knowledge to seek the final answer

This **[Reason -> Act -> Perceive -> Reflect] loop** is the core engine of Agentic RAG



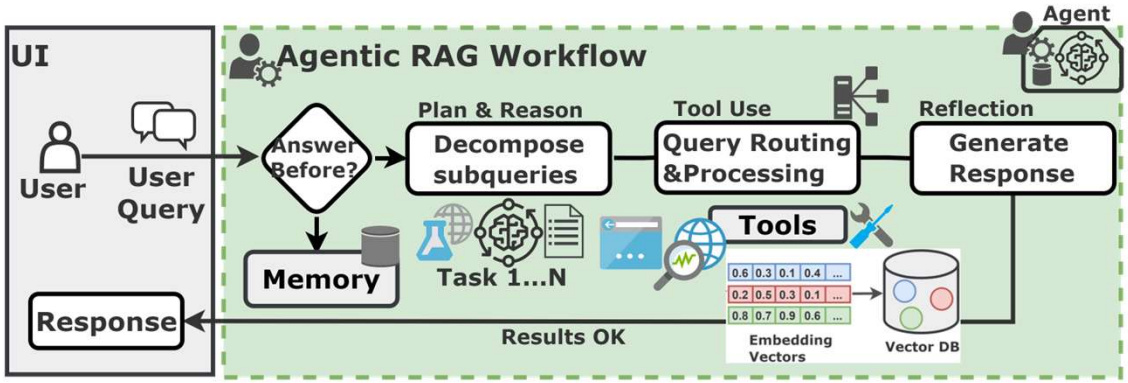
Agentic RAG Use Cases

Saw an agent autonomously plan, search, reflect and search again

We must answer the critical architectural question: 'Where?'

When do we justify this complex, slow and expensive pattern over the simple, fast and cheap Traditional RAG?

The answer is: when we are asked to solve a problem that is not a simple 'lookup' but a complex 'research' task



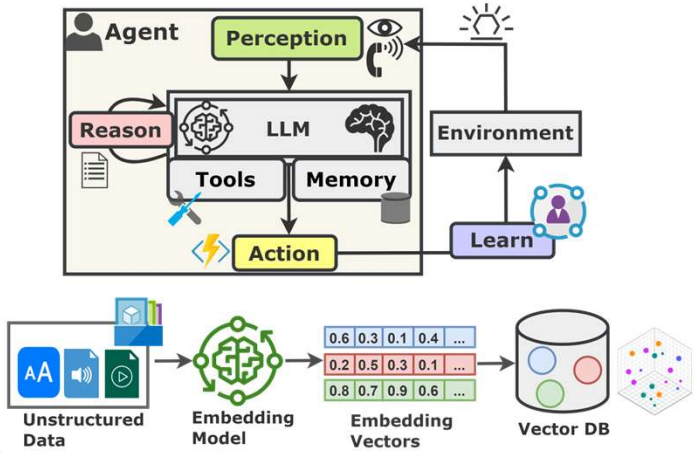
Use Case 1: Complex Comparative Research

Example Query:
Compare and contrast the chip architectures of Nvidia, AMD and Intel from the last 3 years

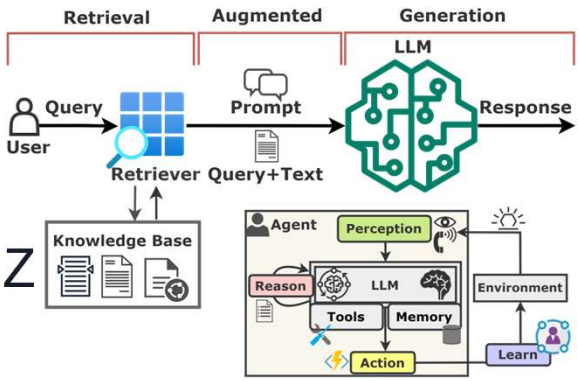
Why Traditional RAG Fails:
This is a "fatal" query. No single document in your vector store answers this
A search() on this string will return noise

How Agentic RAG Succeeds (Decomposition):
The agent decomposes the query into a multi-step plan:

Plan:
1. search("Nvidia chip architecture 2023-2025")
2. search("AMD chip architecture 2023-2025")
3. search("Intel chip architecture 2023-2025")
4. synthesize_comparison(results_1, results_2, results_3)



Use Case 2: Automated Due Diligence



Example Query:
Analyze the market sentiment and financial health of Company Z

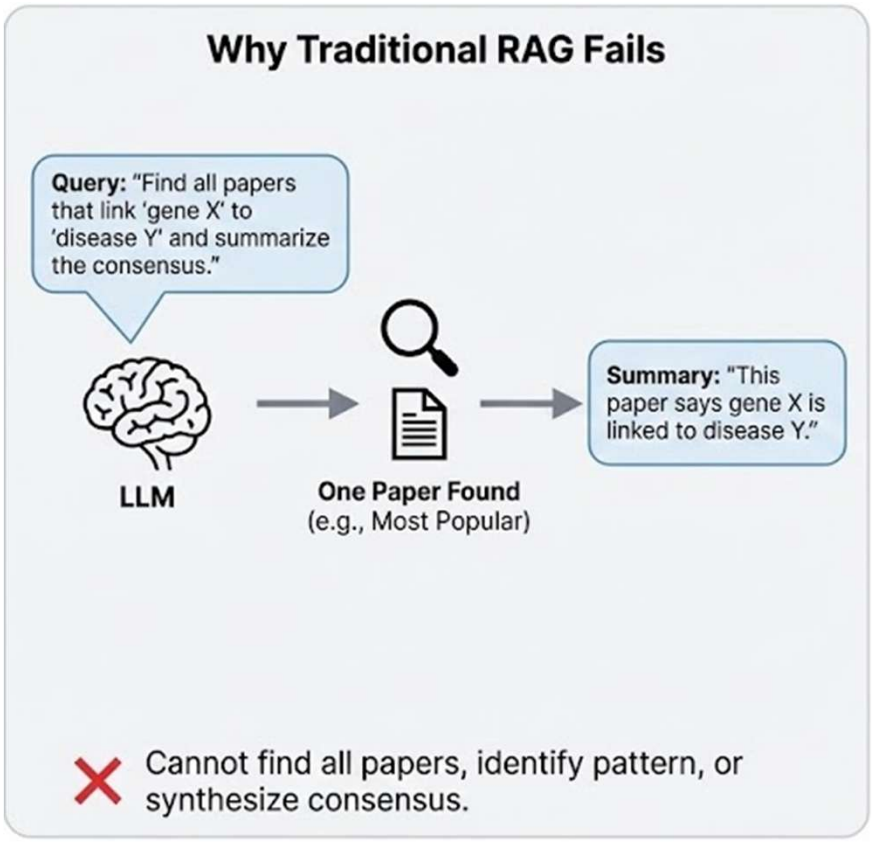
Why Traditional RAG Fails:
This query is ambiguous. What does 'sentiment' or 'financial health' mean? A simple vector search is useless

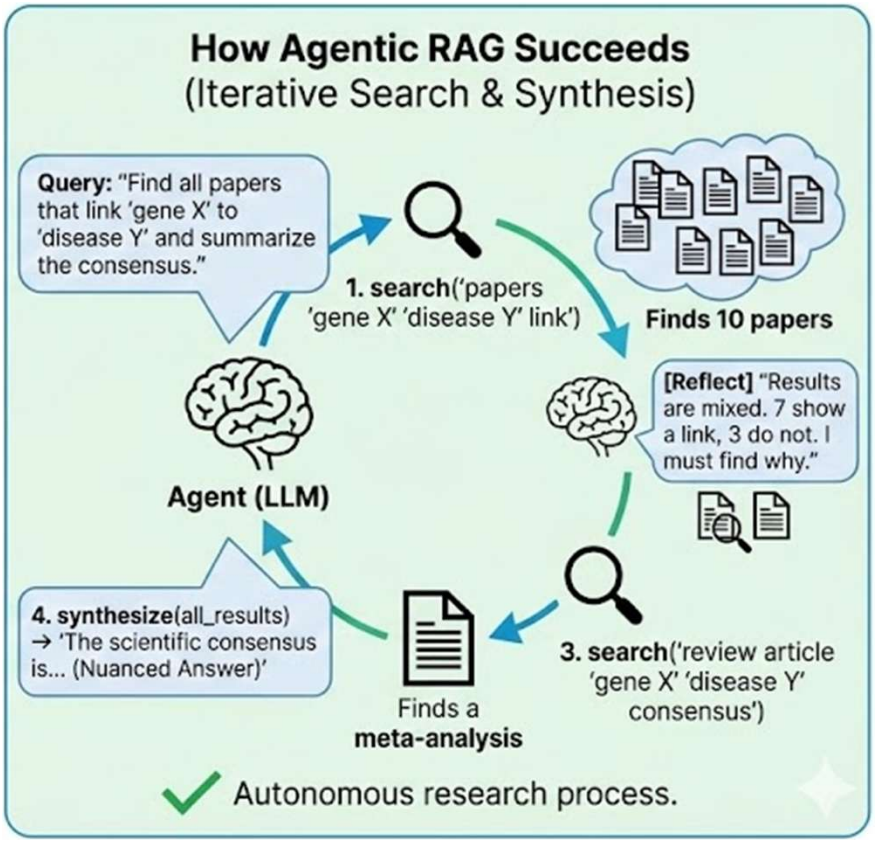
How Agentic RAG Succeeds (Strategy Formulation):
The agent translates the ambiguous terms into a concrete research strategy:

Plan:
1. search("Company Z latest quarterly earnings report") (for "financial health")
2. search("Company Z stock analyst ratings October 2025") (for "sentiment")
3. search("recent news 'Company Z' 'product launch' OR 'scandal'") (for "sentiment")

Use Case 3: Scientific or Legal Literature Review



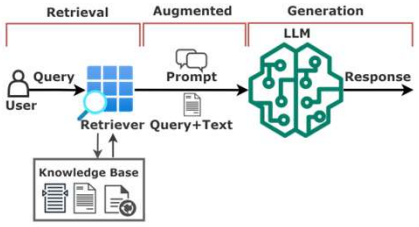


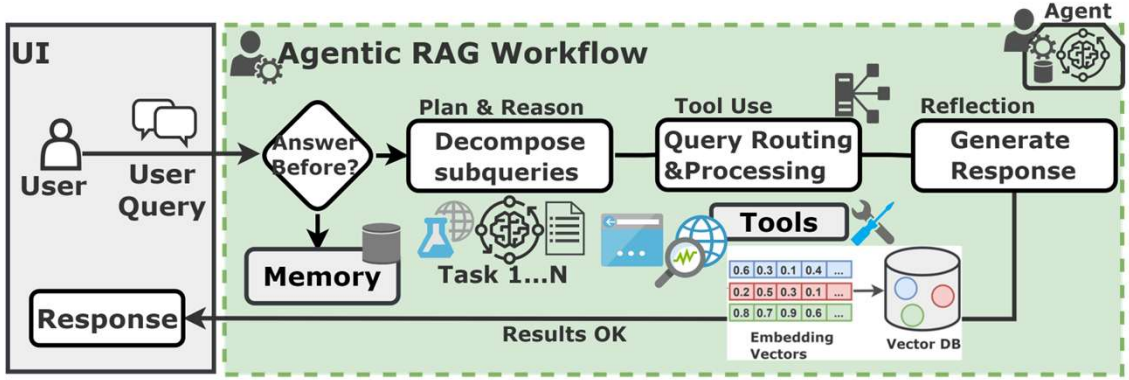


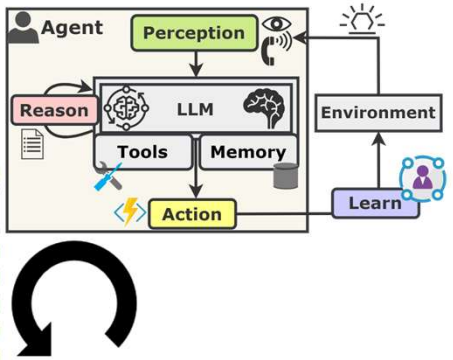
Summary: The "Active Researcher" Loop in Action

We need **Agentic RAG** when the **query requires**:

1. **Decomposition:** A complex query must be broken into multiple parts ("Compare X, Y and Z")
2. **Strategy:** An ambiguous query must be translated into a concrete plan ("Analyze sentiment...")
3. **Reflection:** A multi-step process requires self-correction ("Summarize the consensus...")







Agent Communications and Protocols - Using Agentic Protocols

What are AI agent protocols?

What is Agent Communication Protocol (ACP)?

What is A2A protocol (Agent2Agent)?

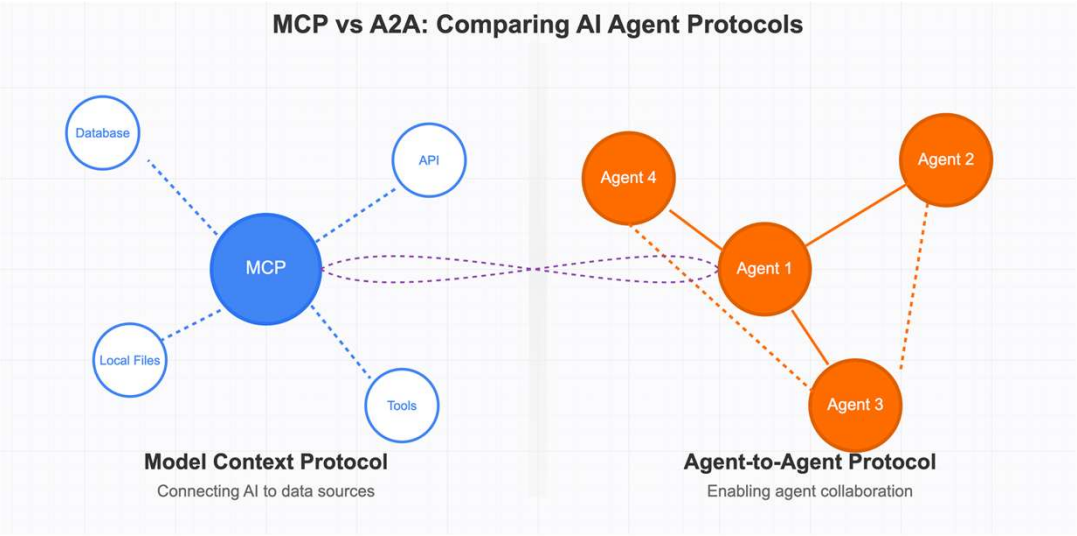
What is the Agent-user interaction (AG-UI) protocol ?

What is Model Context Protocol (MCP)?

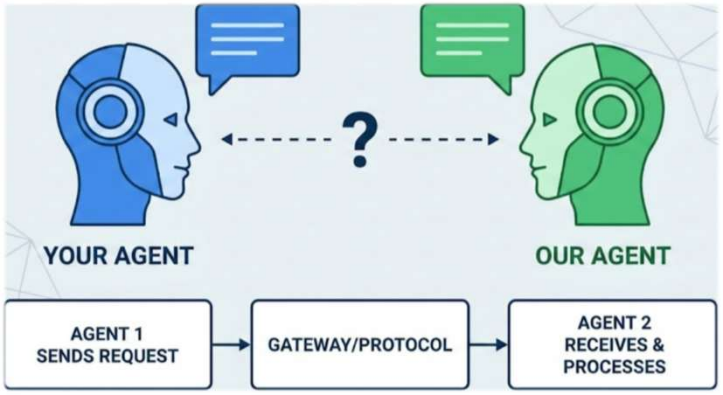
Pradeep Kumar T



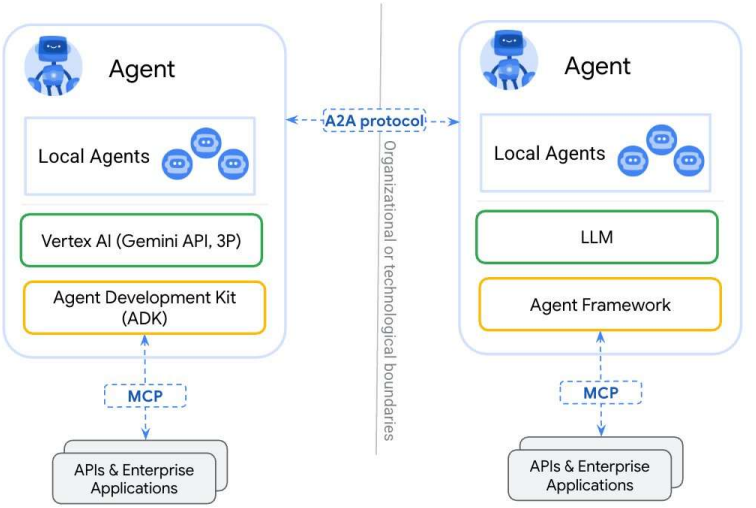
Agent Communications and Protocols









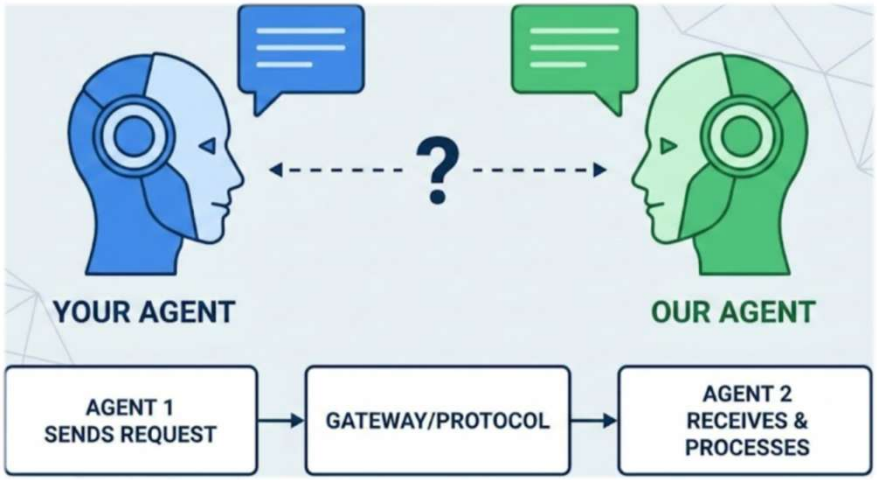


Introduction - The "Protocol Problem"

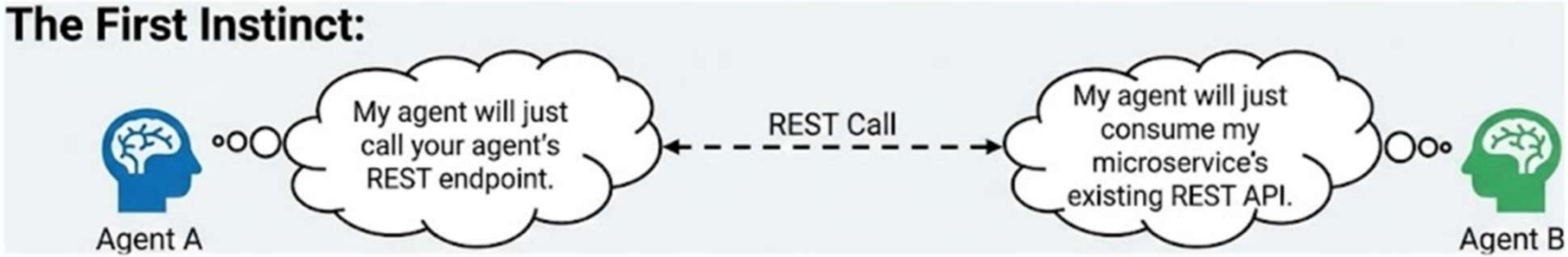
We've **designed agents** that can **reason, use tools** and **collaborate** in our own closed systems

What happens when our agent **needs to talk to your agent?**
What happens when an **agent** from **OpenAI needs to consume** a **tool** built by a **LlamaIndex-native team?**

How do agents, built by **different teams**, on **different platforms**, in **different companies**, **talk to each other?**



The "Obvious" Answer: Just Use Custom REST APIs?

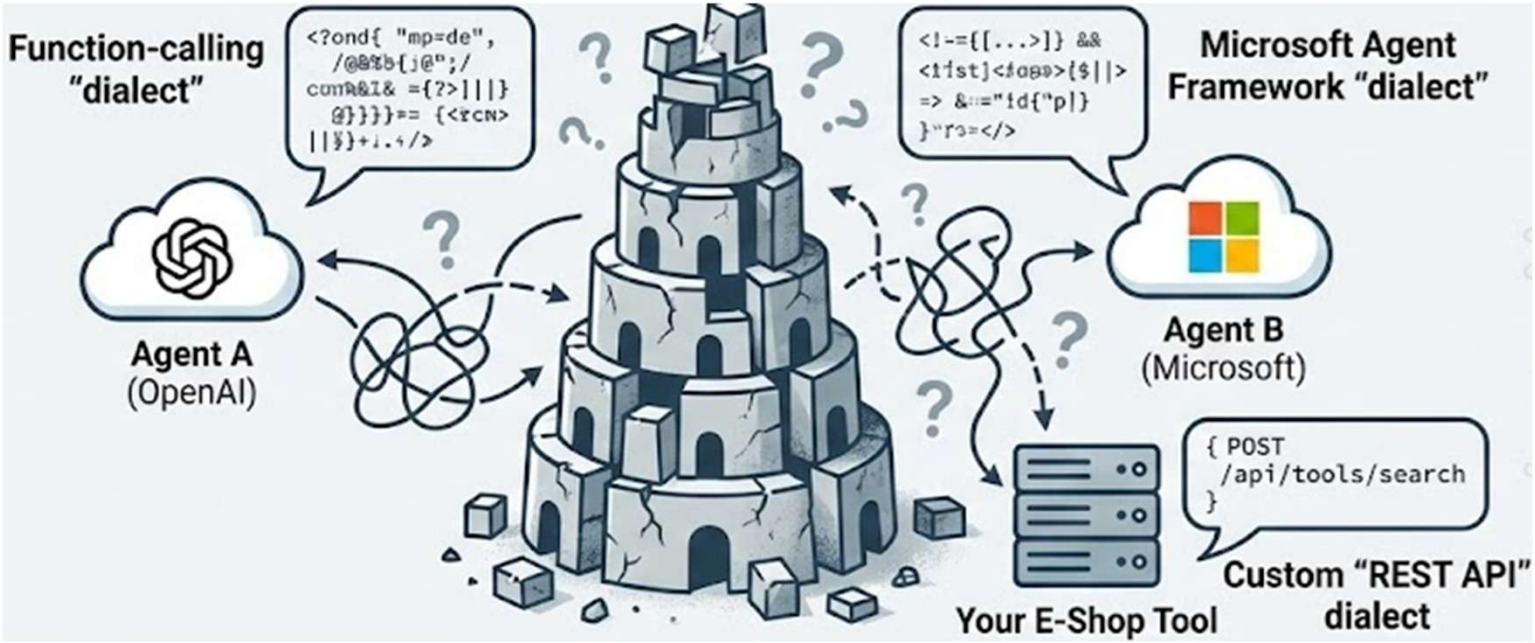


The Immediate Problems:



This is a **short-term fix** that **creates massive long-term architectural debt**

The Architectural Chaos: The "Tower of Babel" Problem



The Result:

1. **Vendor Lock-In:** Your OpenAI-based agent is now locked into the OpenAI ecosystem
2. **N-Squared Problem:** To connect 100 agents, you need ~5,000 custom integrations
3. **Fundamentally Unscalable:** This architecture cannot scale to a global web of agents

The Solution: What is an "Agent Protocol"?

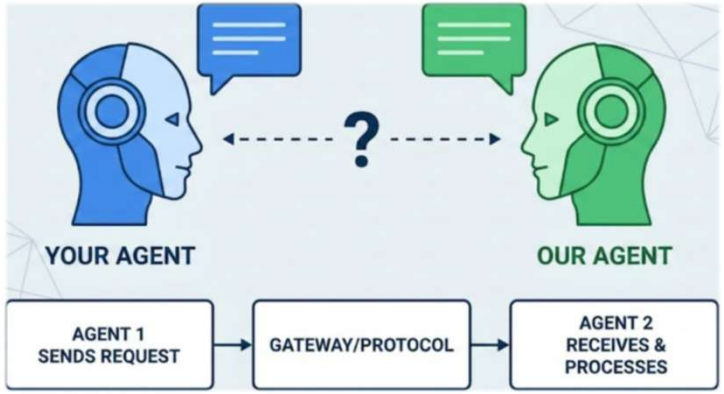
A formal specification (**'common language'** or **'social contract'**) for agents

It defines how autonomous agents and tools:

1. Discover each other
2. Describe their capabilities ("Here are my tools and what they do")
3. Communicate ("Please run this task for me")
4. Exchange Context (state, memory, user ID)

The Goal: To enable **Interoperability**

To **decouple** the **'Agent'** (the **brain**) from the **'Tool'** (the **function**)



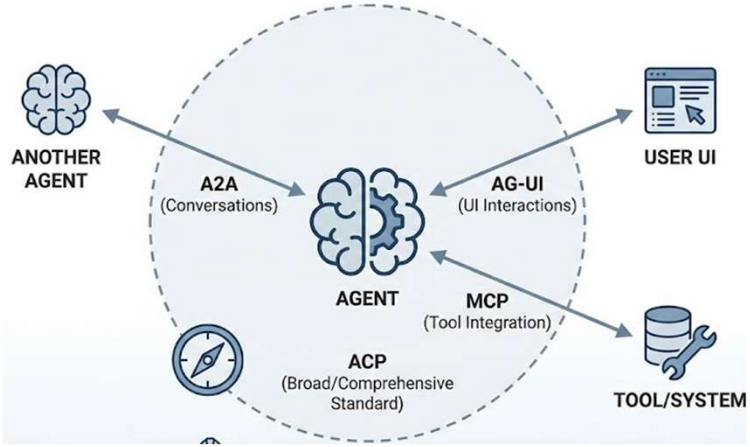


Our Roadmap: Exploring the Emerging Standards

This is **not yet a 'solved' problem**
Several standards are **emerging**, each with a **different focus**

Our Focus:
ACP (Agent Communication Protocol): A broad, comprehensive standard
A2A (Agent-to-Agent): Focused on agent conversations
AG-UI (Agent-User Interaction Protocol): Focused on Agent to UI interactions
MCP (Model-Context Protocol): Focused on Agent-to-Tool integration

As architects, **must understand** the **trade-offs of each**



Introduction - Agent Communication Protocol (ACP)?

Established the **'Protocol Problem'**—the fact that without a common language, our agentic ecosystem will become a 'Tower of Babel' full of custom integrations

The solution is a **formal protocol**, just like HTTP or SMTP

The **Agent Communication Protocol**, or **ACP**



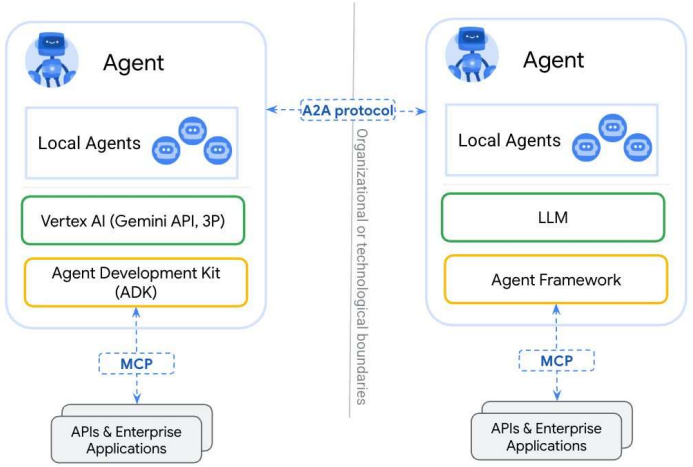
What is the Agent Communication Protocol (ACP)?

ACP is a **broad**, **open-source specification** designed to be a **universal "language"** for all **agent-based interactions**

The Analogy: If **HTTP** is the protocol for **web pages**, **ACP** aims to be the protocol for "**tasks**" or "**goals**"

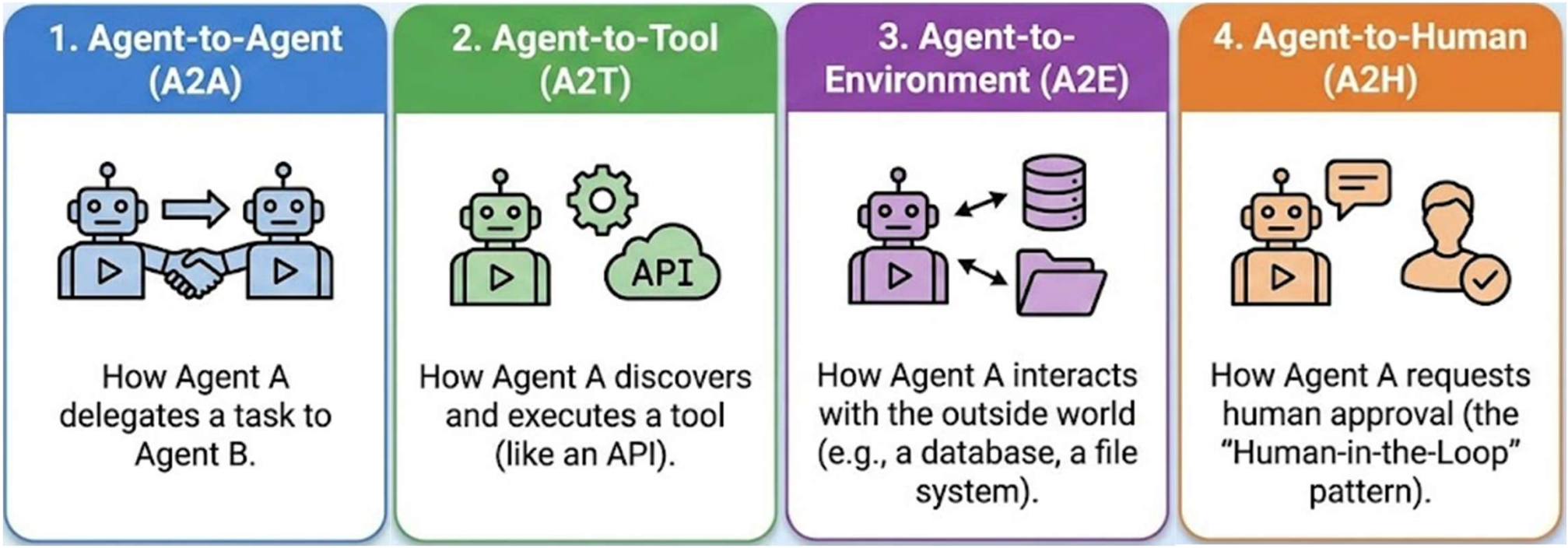
It is a **specification**, **not** an **implementation**. It's a **blueprint** that **anyone** can **build** for

Core Idea: To create a single, unified standard that governs all forms of agent communication



The Ambitious Goal: Total Interoperability

ACP is designed to be a **general-purpose** and **comprehensive** standard



The **vision** is for **ACP** to be the single **'lingua franca'** for this entire ecosystem

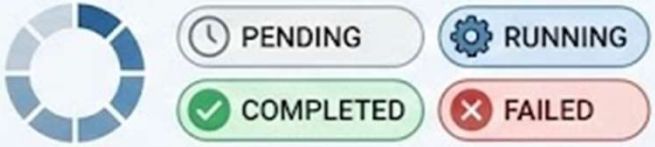
What Rules does ACP actually Define ?

1. Task Management



Standard definition for “task”, “sub-task”, “goal”.

2. State Synchronization



Standard way to report task status.

3. Discovery & Capabilities



Standard “manifest” to advertise tools/skills.

4. Artifact & Data Exchange



Standard way to pass data (files, DB rows, text) between agents.

Summary: The "One Protocol to Rule Them All"

ACP is a comprehensive, open-source specification for all agent interactions

Goal: To create a single, universal standard for task delegation, discovery, and state management

Scope: Ambitious. It covers A2A, A2T, A2E and A2H

Status: An emerging standard, backed by a collaborative accord of AI leaders

Next Up: Let's look at another, more focused protocol



Introduction A2A (Agent-to-Agent) Protocol

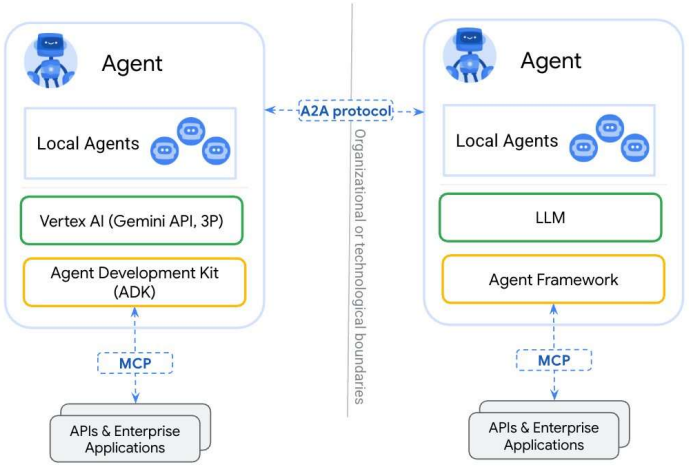
ACP—a very **broad** and **comprehensive** proposal to **standardize** all forms of **agent interaction**, from agent-to-tool to agent-to-human

A single **'one-size-fits-all'** solution can **often** be **complex** to **implement**

What if our primary problem isn't everything?
What if our problem is simply getting **different agents to talk to each other** in a structured way?

The **A2A** or **Agent-to-Agent Protocol**





What is the A2A (Agent-to-Agent) Protocol?

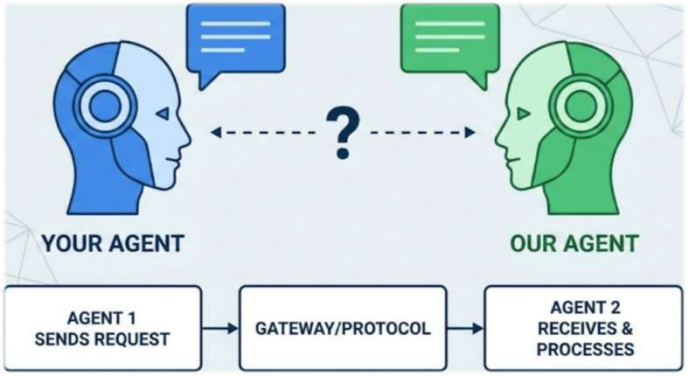
A **focused communication standard** designed specifically for **conversational collaboration** between two or more **agents**

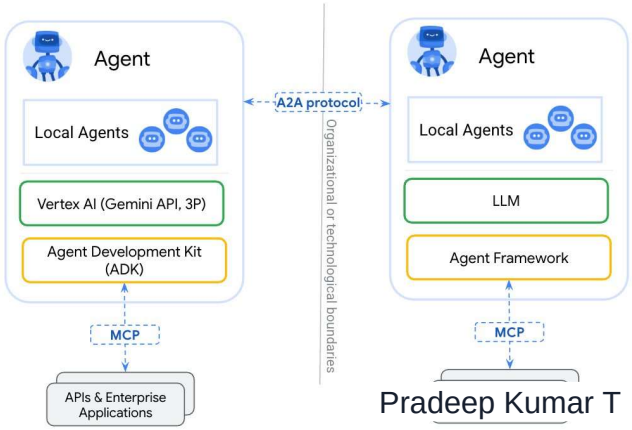
Focus: It standardizes the messages in a collaborative **"group chat"** or **"handoff"**

The Analogy:
If ACP is a full Business Process Standard
A2A is more like a standard for instant messaging (like XMPP) or email

It's **less concerned** with **how a tool is run**
and **more concerned** with **how agents communicate**

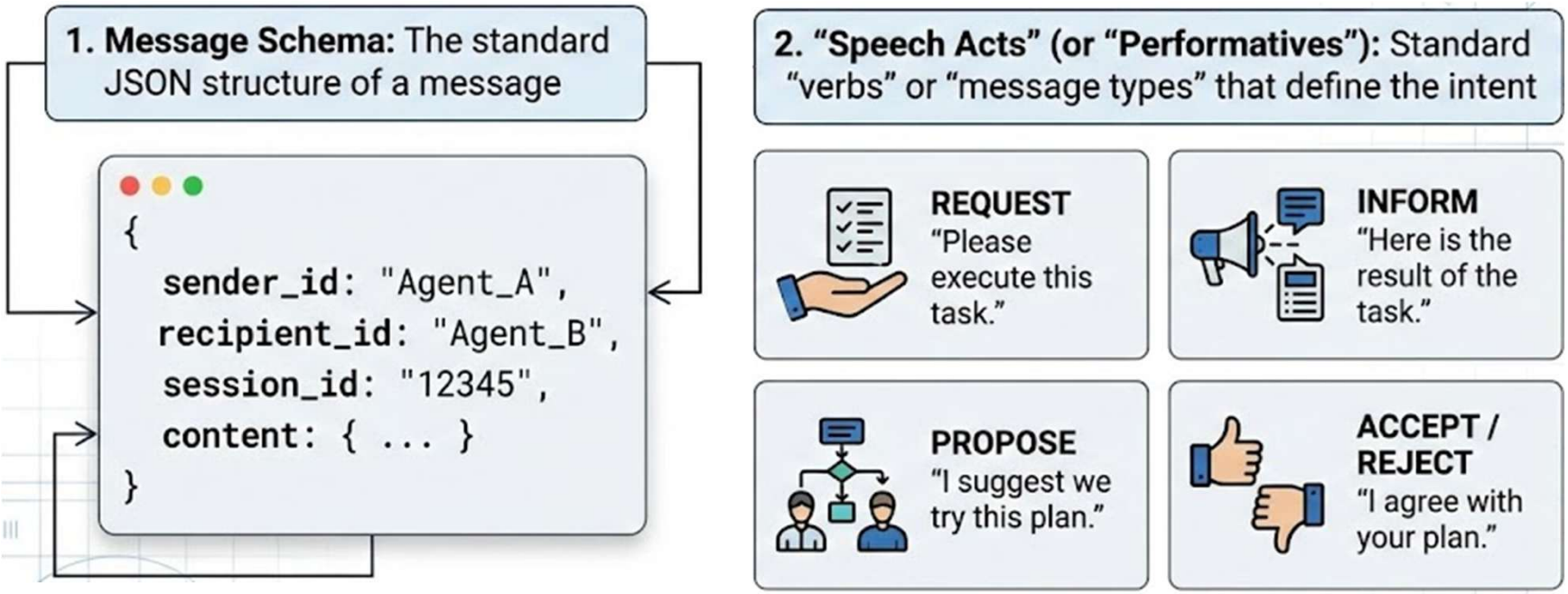






What A2A Standardizes

If **A2A** is about **"conversation" what does it actually define?**



A2A provides **standard 'verbs'** so that agents have **machine-readable** and **predictable dialogue**

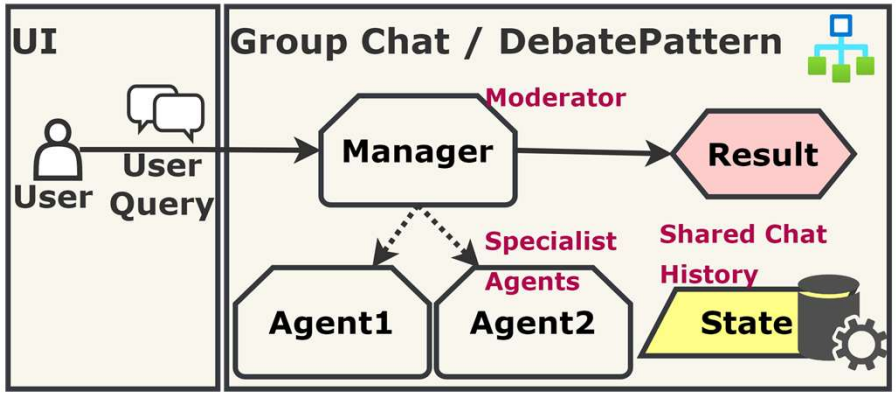
When to use A2A?

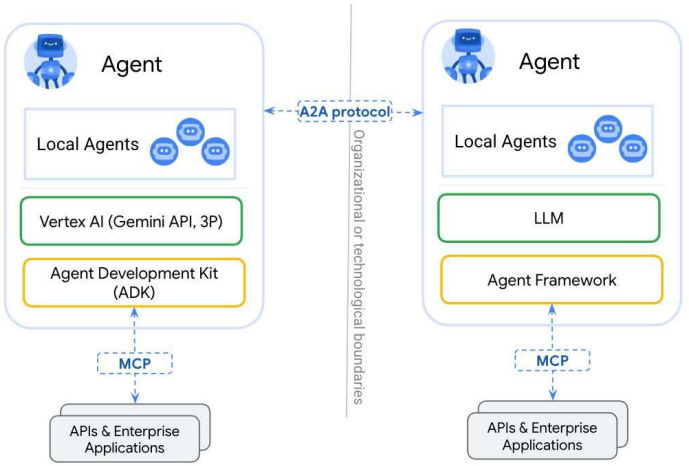
When your **primary problem** is **collaboration**, **not just tool use**



Ideal for these patterns:
Group Chat / Debate
Router & Specialist (for the handoff conversation)
Reflection (for the CriticAgent's message to the WriterAgent)

Core Benefit: Allows to build federated agent systems, where agents from different teams collaborate on a task without needing to know each other





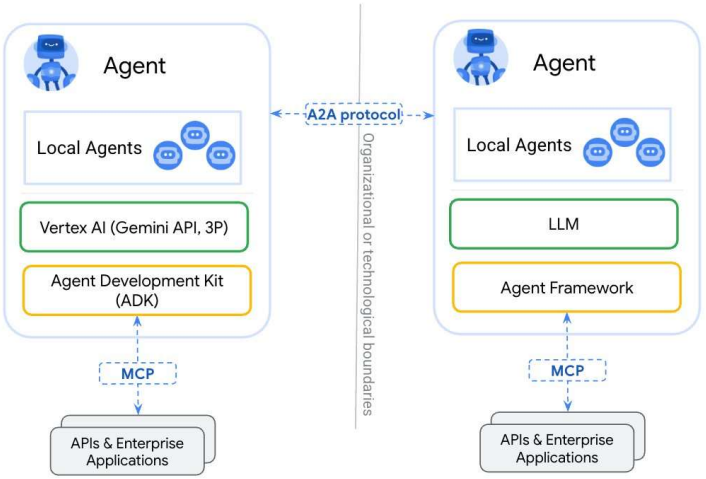
Summary: The "Standard for Agent Chat"

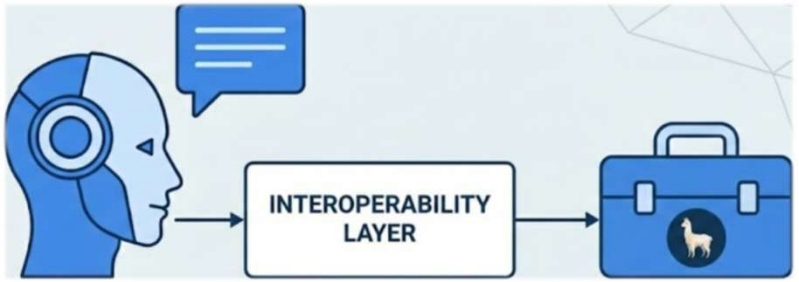
A **focused**, **lightweight** protocol for **standardizing conversational collaboration** between agents

Focus: Message schemas and "**speech acts**" (REQUEST, INFORM, PROPOSE)

Federated, **multi-agent** systems that **need** to "**talk**" (**Group Chat, Debate**)

The Gap: A2A is great for agents talking to agents. But what about the most common scenario: an agent talking to a non-agent?





Introduction - Agent-User Interaction (AG-UI) protocol

ACP as the comprehensive standard
A2A for agents talking to other agents

One **critical participant** we haven't standardized yet: **The Human**

How does an agent send more than just plain text to a user?
How does it dynamically generate a button, a form or a chart?

If **A2A** is the standard for the **'back office'**, **AG-UI** is the standard for the **'storefront'**—the **interface** where the **agent meets the user**



https://github.com/ag-ui-protocol/ag-ui

Agent-user interaction (AG-UI) protocol





https://github.com/ag-ui-protocol/ag-ui

What is the AG-UI Protocol ?

A set of **standards** and **patterns** that define **how AI Agents render** Rich, **Interactive User Interfaces** dynamically

The Shift: Moving beyond "Text-In, Text-Out" (Chatbots)

The Goal: "Generative UI" The **agent generates** the **interface on the fly** based on the context

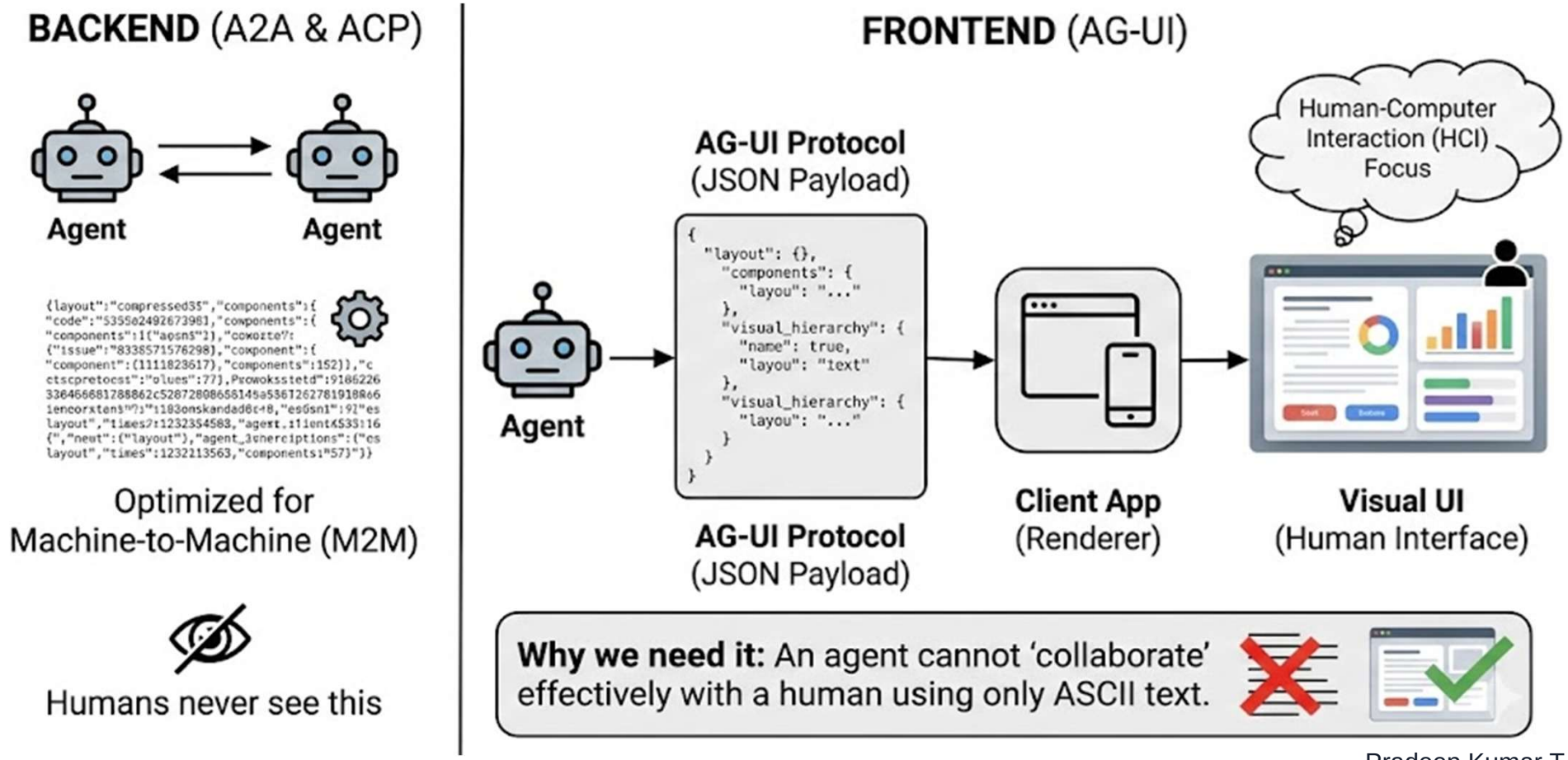
Examples:
Adaptive Cards: (Microsoft) A standard for exchanging card content
Vercel AI SDK (RSC): Streaming React Components from agent to client

It is the "**HTML**" for the **Agentic Age**

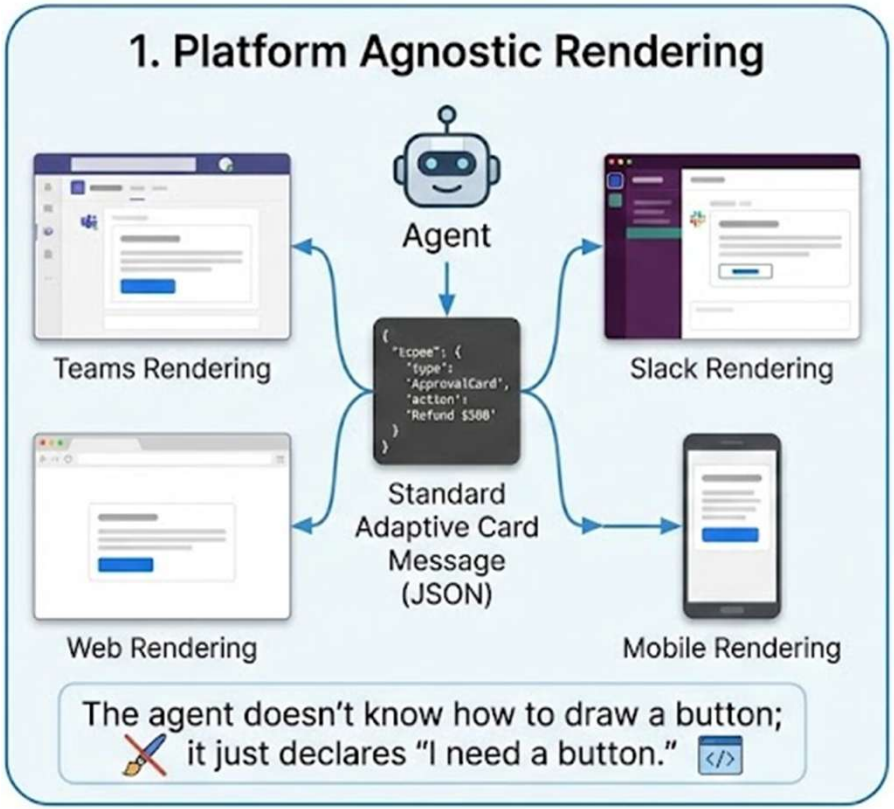


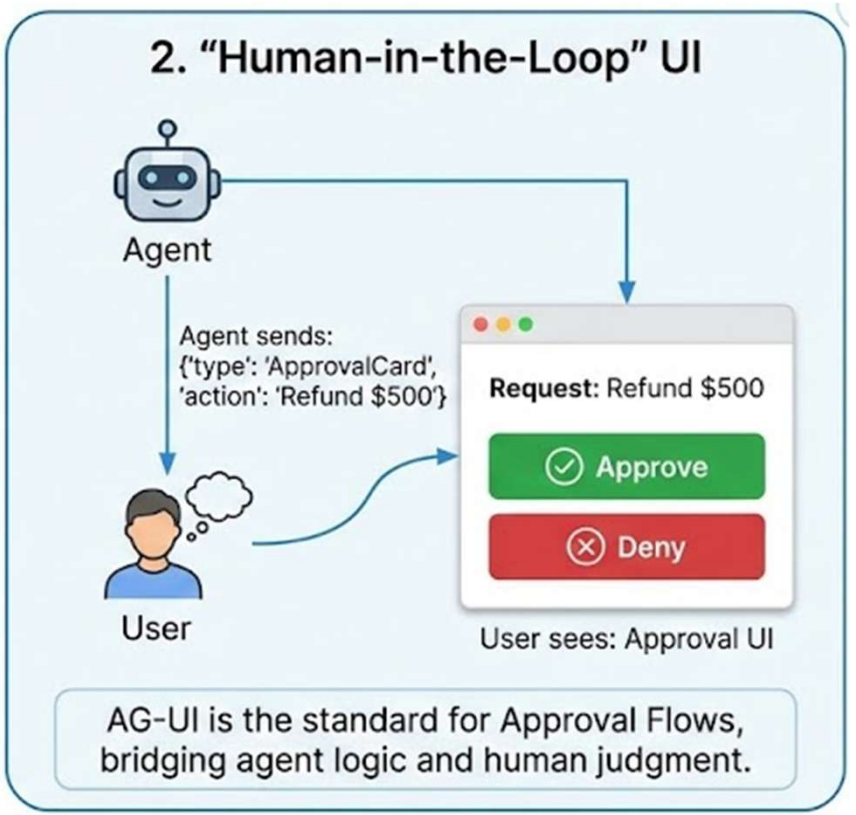
Pradeep Kumar T

AG-UI vs. A2A and MCP: The "Frontend" Gap



Key Concepts: Adaptive Cards & Generative UI





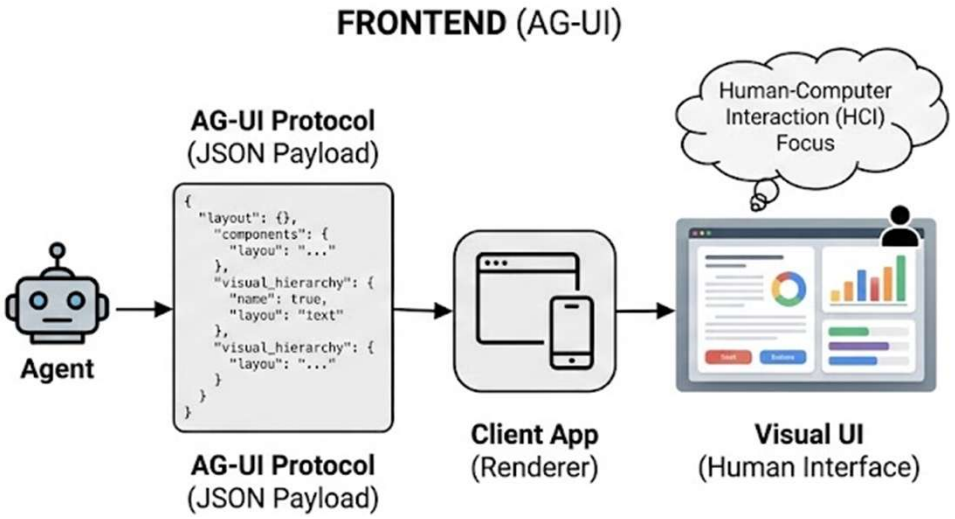
Summary: Closing the Loop with Humans

AG-UI standardizes the **interface** between the **Agent** and **the Human**

It enables **Generative UI**, where the interface **adapts** to the **conversation**

It is **crucial** for **Safety** (Human-in-the-Loop) and **User Experience**

Next Up: The most critical integration standard for the enterprise





Introduction - Model-Context Protocol (MCP)

ACP as the **comprehensive standard**, **A2A** for agents **talking** to **other agents**
AG-UI standardizes the **interface** between the **Agent** and **the Human**

How does my **agent talk** to my **existing systems**?
How does it **consume** my **E-shop microservices**, my product **databases** and my payment **APIs**? These are not "**agents**" They are "**tools**"

Need a **standard** for this **Agent-to-Tool interaction** and that is precisely what the **Model-Context Protocol** (**MCP**) is **designed to solve**



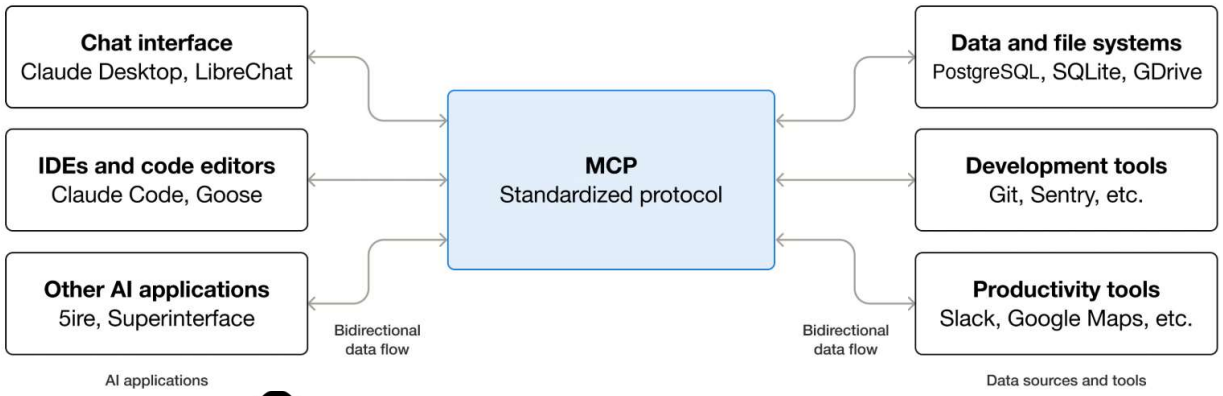
What is the Model-Context Protocol (MCP)?

MCP is a **focused**, **lightweight protocol** that **standardizes** how **Models** (like LLM Agents) **interact** with **Context** (like Tools, APIs or data sources)

Key players in the AI space, including **LlamaIndex**, who are **focused** on the **practical "data integration"** problem

If **A2A** is **'email for agents'**, **MCP** is the "**OpenAPI / Swagger for Agents**"

The Core Problem it Solves: Agent-to-Tool (A2T) communication



Model Context Protocol

Next Section: Model-Context Protocol (MCP)

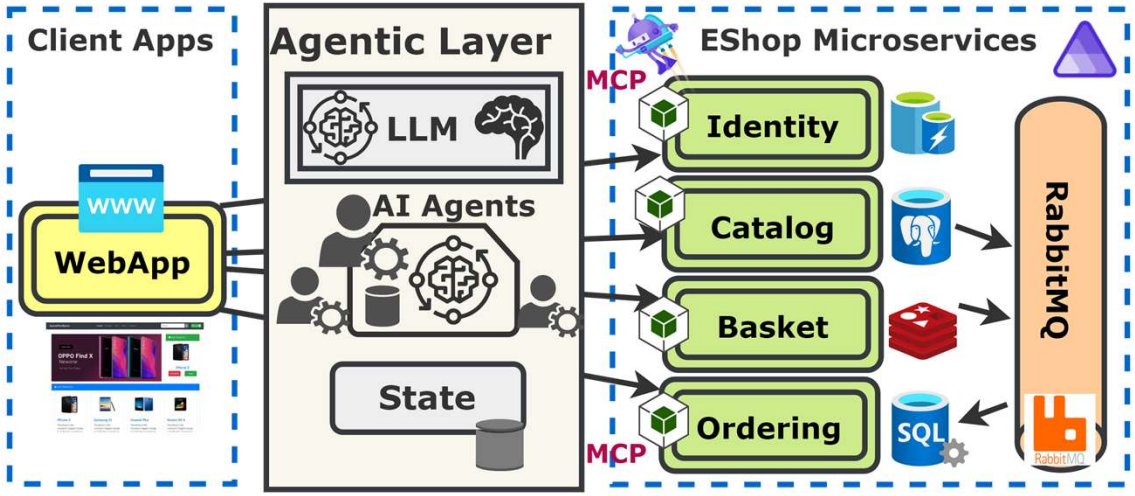
How MCP Works

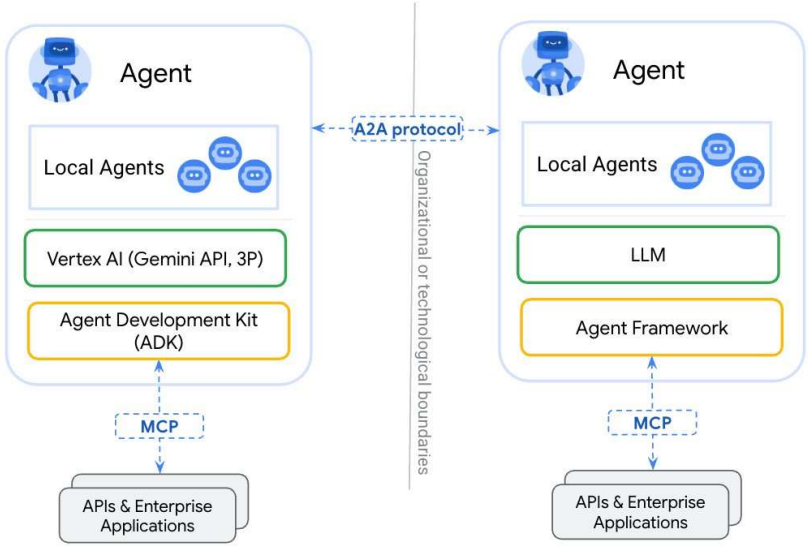
MCP Core Concepts & Definitions (The "Spec" Deep Dive)

The Architectural View and Implications: True Interoperability

MCP Tooling & Ecosystem ("Getting Started")

MCP Design Patterns & Case Studies





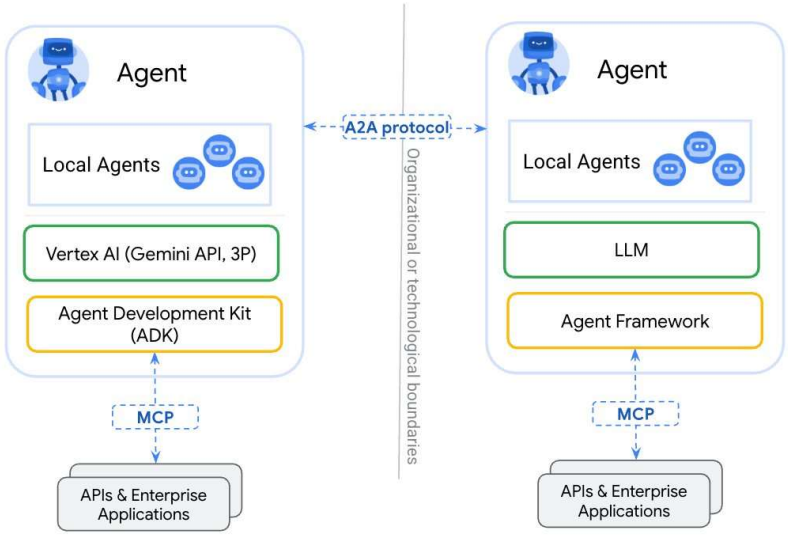
Summary: The "OpenAPI for Agents"

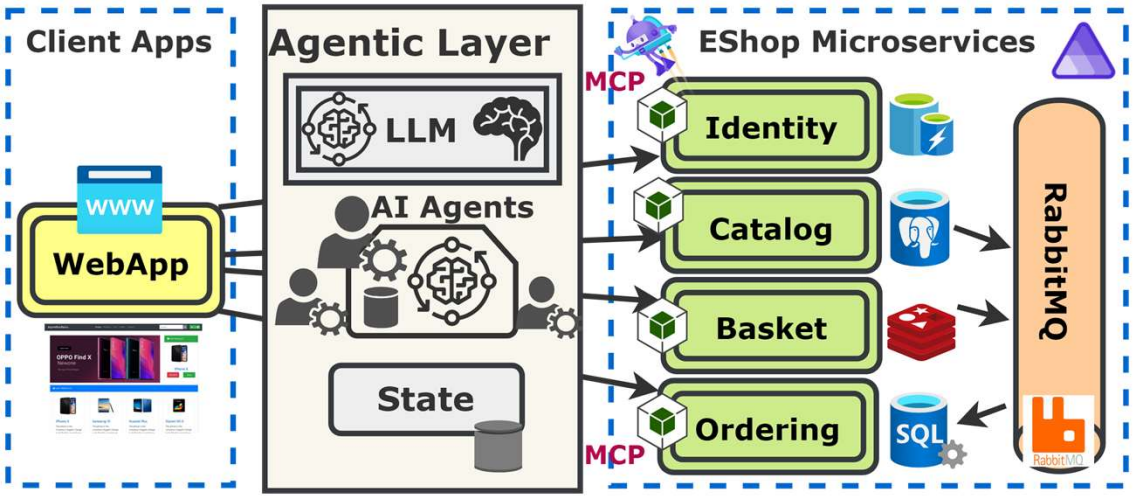
A **lightweight protocol** for **Agent-to-Tool communication**

Uses a **standard manifest.json** for tool **discovery** and **schema**

The **standard** for **enterprise integration**. It decouples agents from proprietary tools

Goal: To **make any tool** or **microservice consumable by any agent**





Intro - The Architect's View: MCP vs. ACP vs. A2A

1. ACP (Agent Communication Protocol): The ambitious, 'heavyweight' standard to govern all agent interactions

2. A2A (Agent-to-Agent): The focused, 'lightweight' standard for conversational collaboration between agents

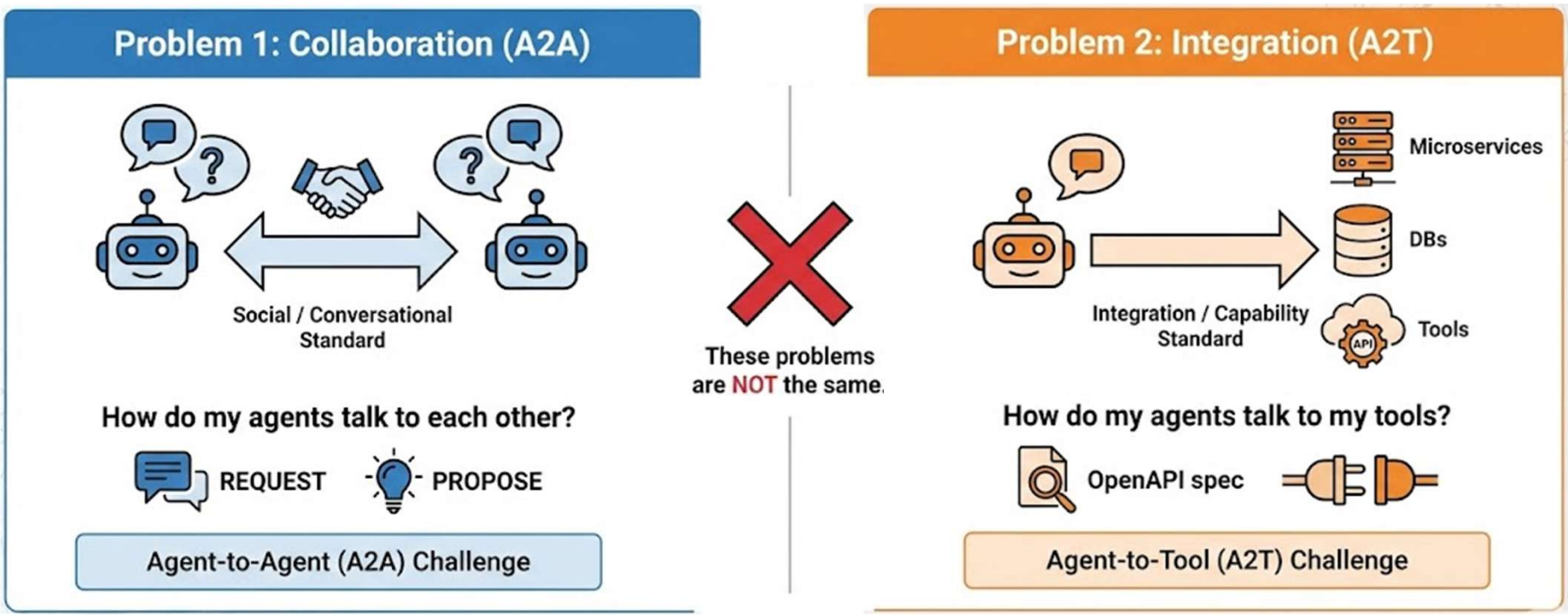
3. MCP (Model-Context Protocol): The practical, 'integration' standard for an agent to consume tools and APIs

These are **not** all **competing** for the **same slot**
They are **designed** to **solve different problems**



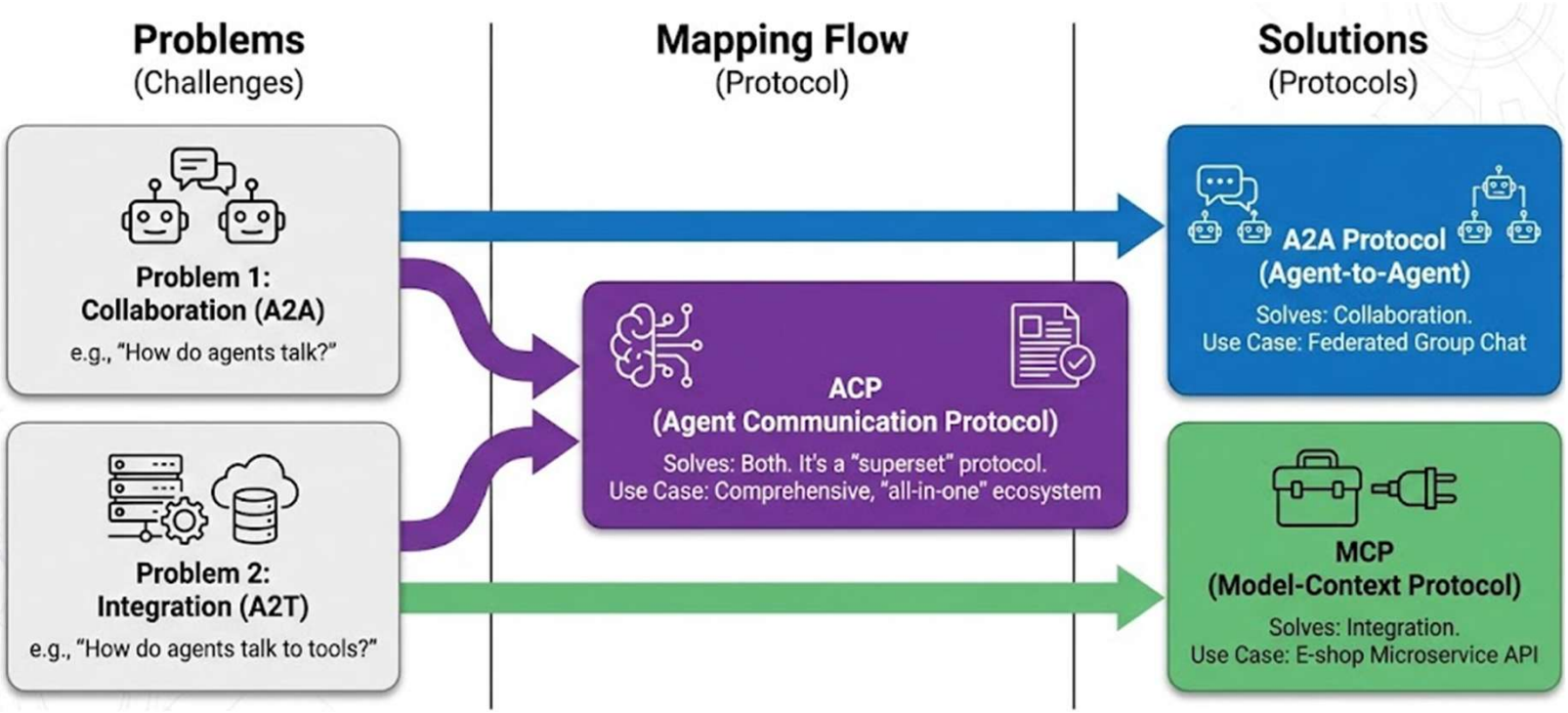
The Core Architectural Distinction: Two Problems

From an architect's view, there are **2 distinct problems**:



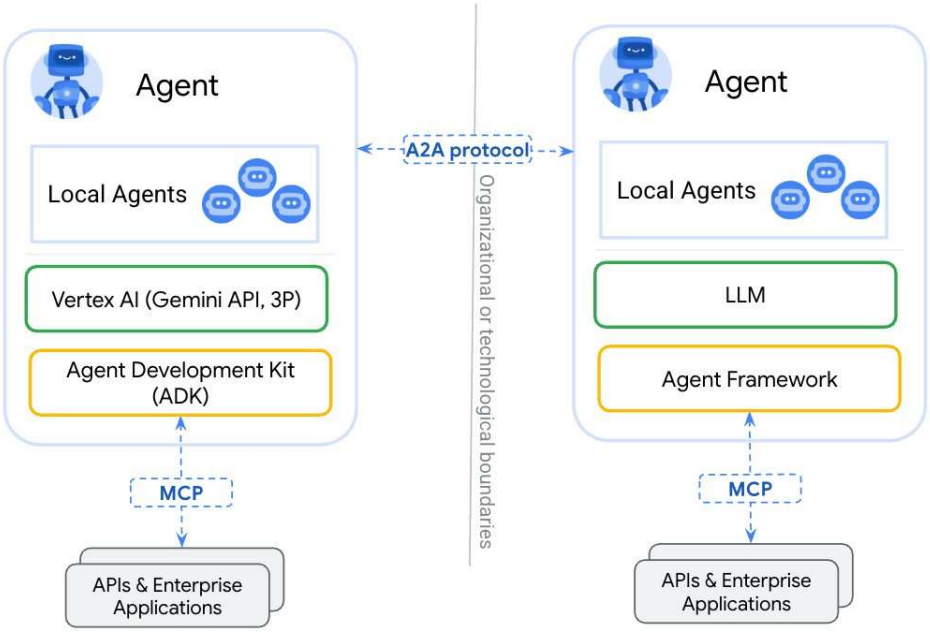
These **problems** are **not the same**

The Protocol Map: Who Solves What?



For most **enterprise architects**, your most immediate and **painful problem** is **Integration**

The Protocol Map: Who Solves What?



For most **enterprise architects**, your most immediate and **painful problem** is **Integration**

The Architect's Decision Matrix

IF my primary goal is...

...to make my **existing APIs consumable by any agent**...
...**THEN** I should implement an **MCP server**

...to make my own **team of agents** (Writer and Critic) **collaborate**...
...**THEN** I should use an **A2A-style internal standard**

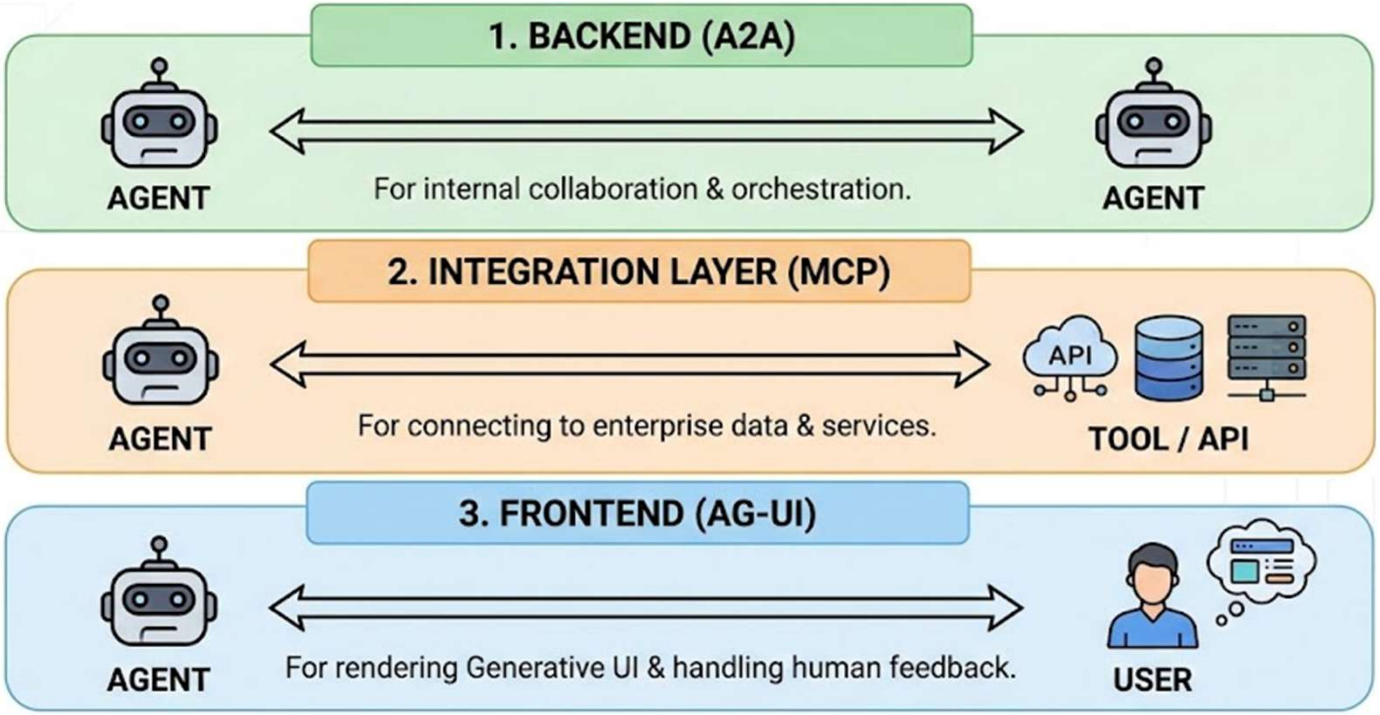
...to build an **entire**, **new**, **comprehensive agentic ecosystem** from scratch...
...**THEN** I should evaluate adopting the **full ACP specification**

Most enterprises will start with **MCP**

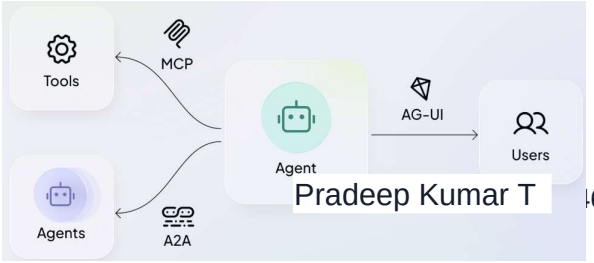


The Architect's View: The Full Protocol Stack

We now have the **complete picture** for a **modern Agentic System**



You need **all three layers** for a **complete architecture**



Summary: Choose the Right Tool for the Job

A2A: Solves Agent-to-Agent (Collaboration / "Chat")

MCP: Solves Agent-to-Tool (Integration / "Tools")

ACP: Solves both (Ecosystem / "The Works")

As **enterprise architects**, our most pressing problem is **integration**

MCP is the key to decoupling our **enterprise systems** from any single agent vendor



Model Context Protocol (MCP) in Action Design

Introduction - Why MCP ?

MCP Core Concepts & Definitions

The MCP Architecture: MCP Host/Client, Server and Protocol

The MCP Lifecycle (How it Works)

MCP Design Patterns & Case Studies

Pradeep Kumar T

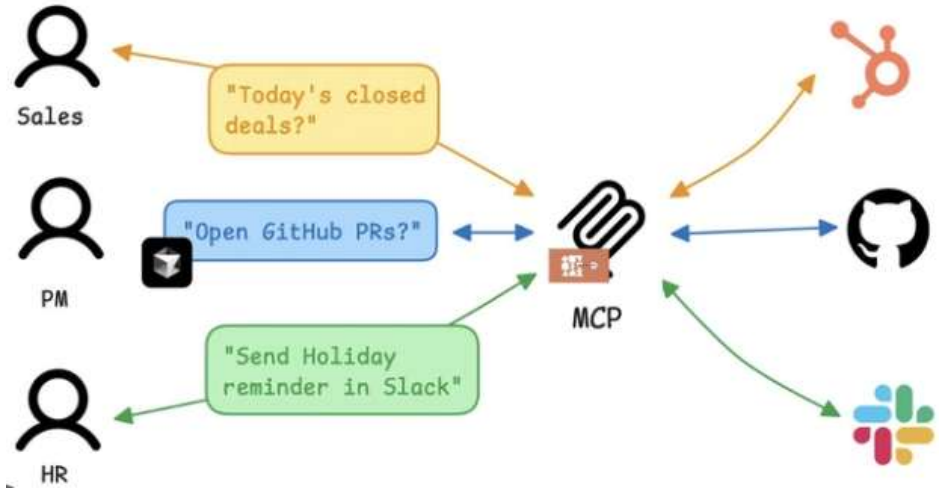


Introduction - Why MCP ?

ACP the comprehensive, 'heavyweight' standard
A2A the focused standard for 'conversational collaboration
MCP the pragmatic standard for 'tool integration

Complete masterclass on just one of them: **MCP**

Why? Why does this one protocol merit such a deep dive?



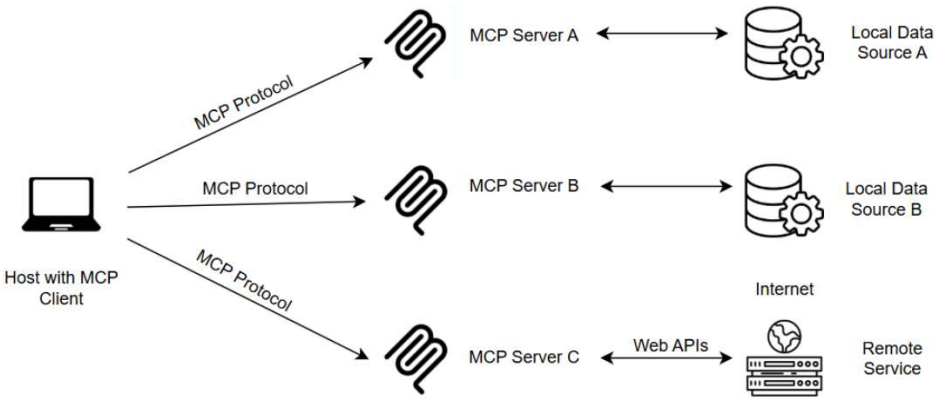
The Architect's Priority: The "90% Problem"



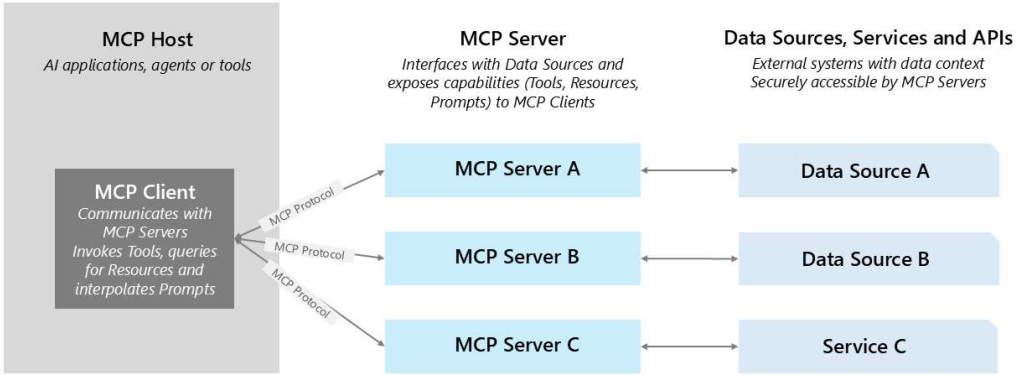
MCP is the standard for the problem you have. Prevent vendor lock-in and build a future-proof agentic architecture

Model Context Protocol

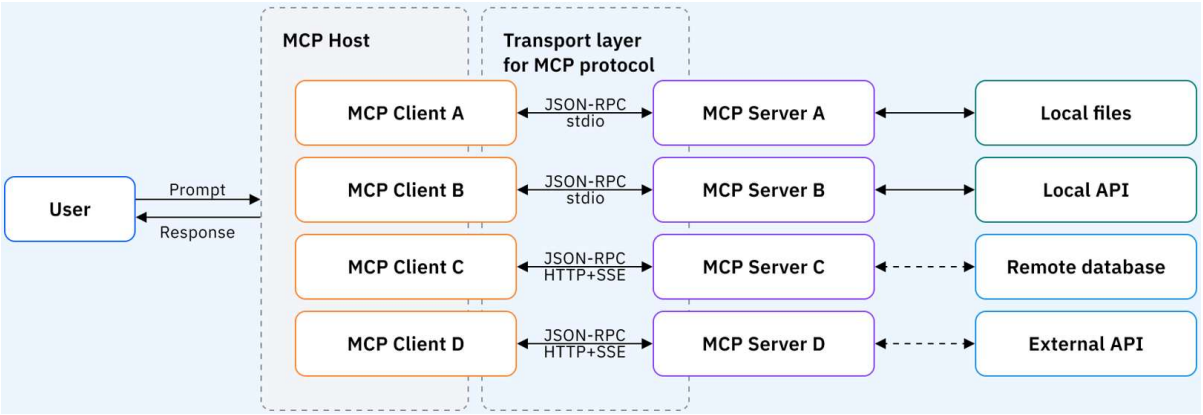
MCP Architecture



https://www.truefoundry.com/blog/mcp



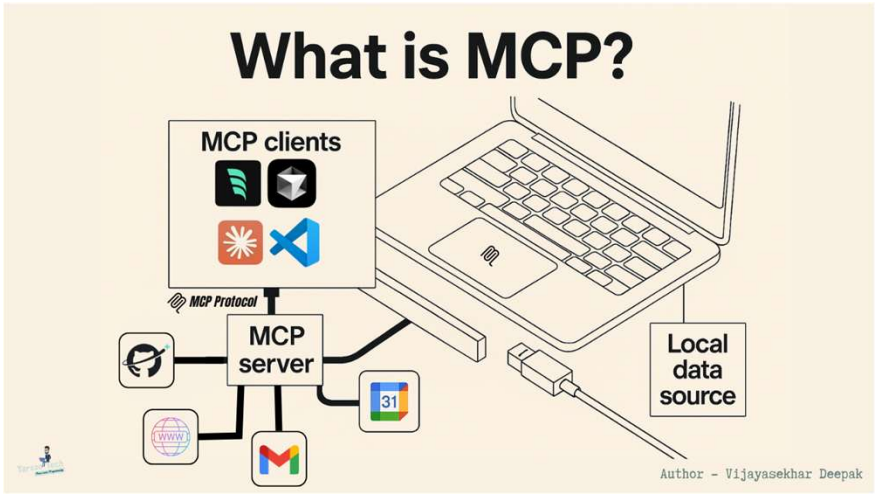
https://learn.microsoft.com/en-us/azure/api-management/mcp-server-overview

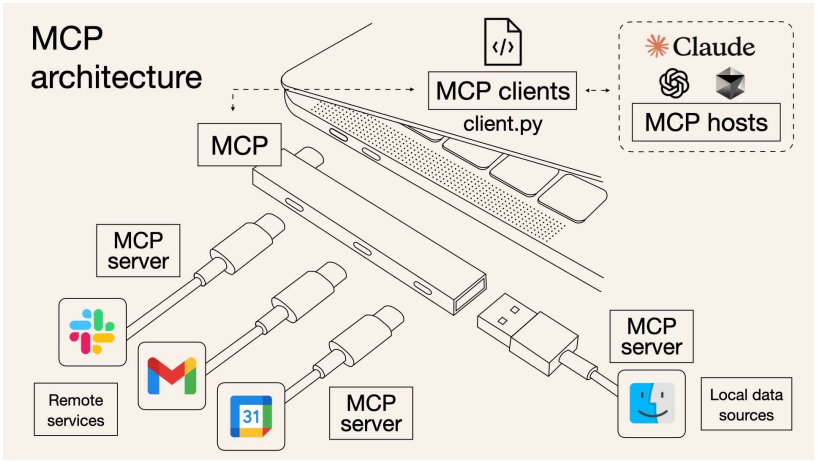


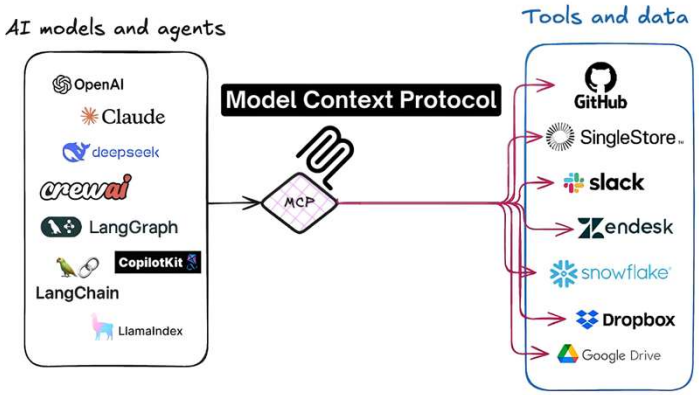
https://www.ibm.com/think/topics/model-context-protocol

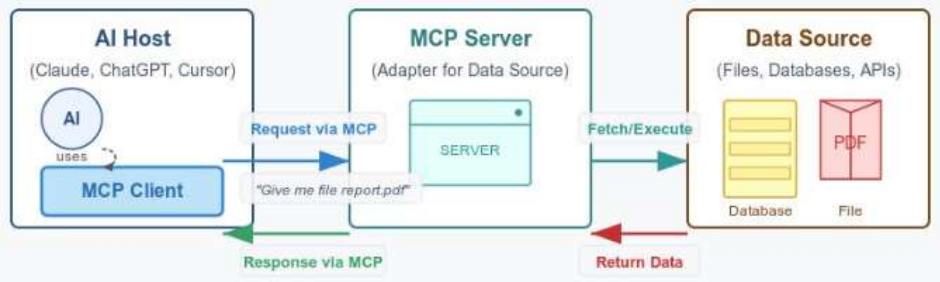
MCP

Model Context Protocol







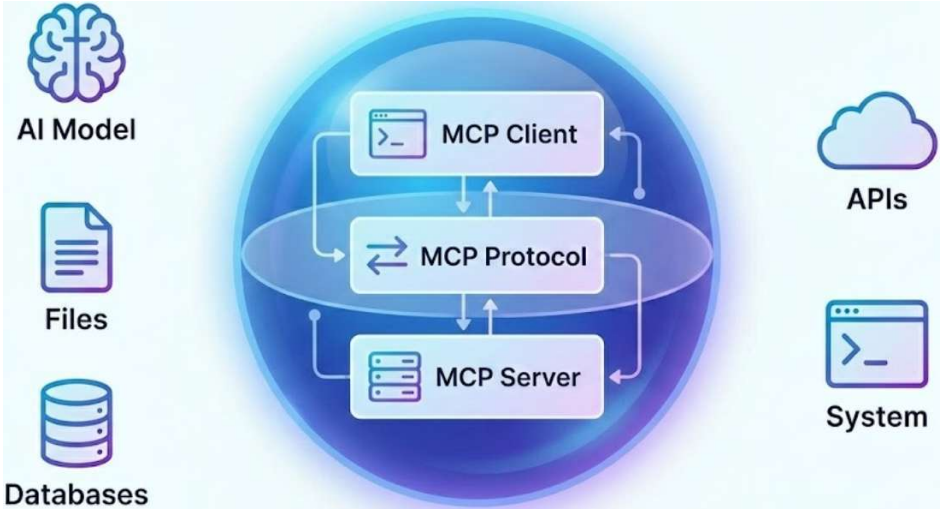


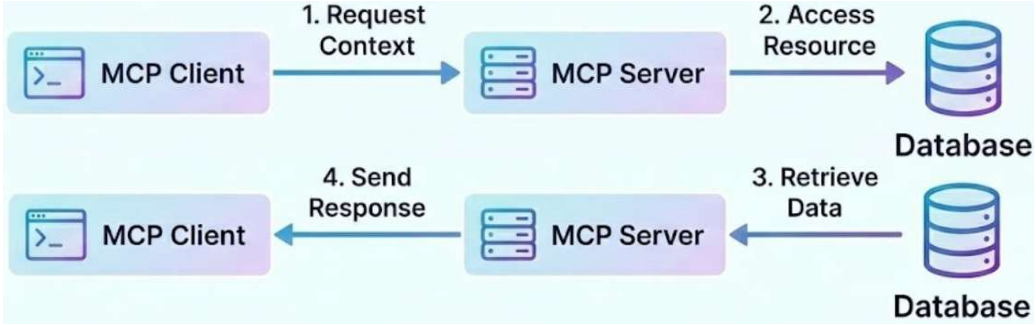
https://docs.openg2p.org/utilities-and-tools/unified-conversation-agent/uca-research-and-development/modal-context-protocol-mcp

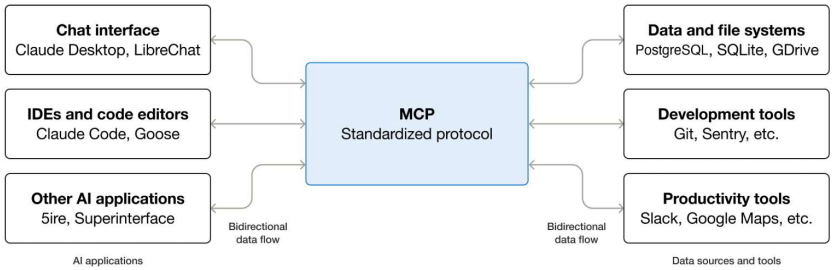
MCP

Model Context Protocol

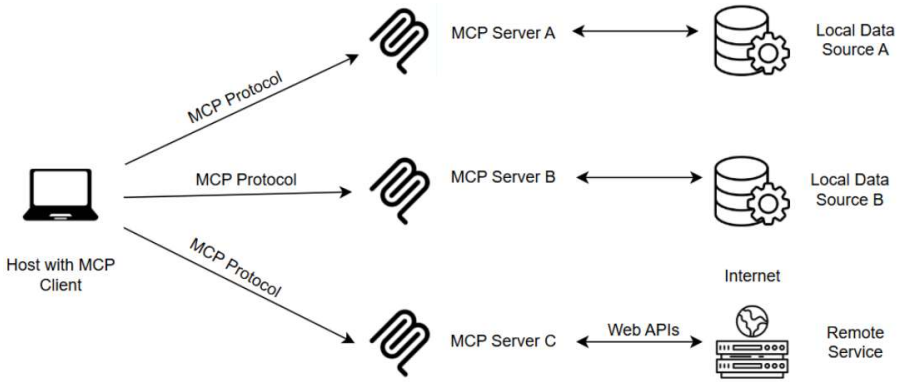




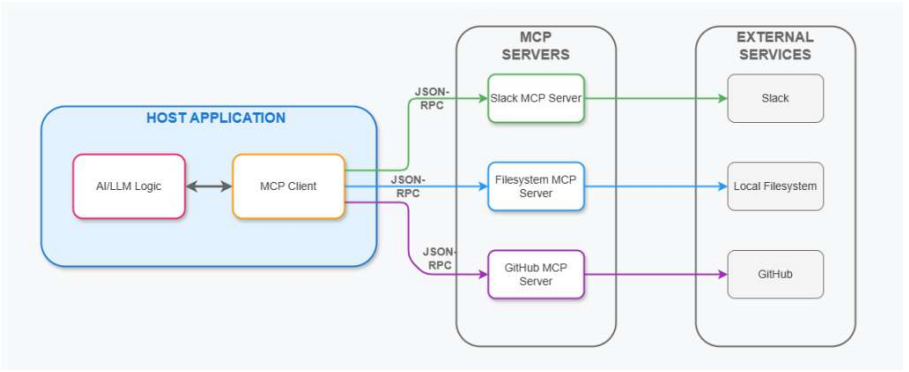




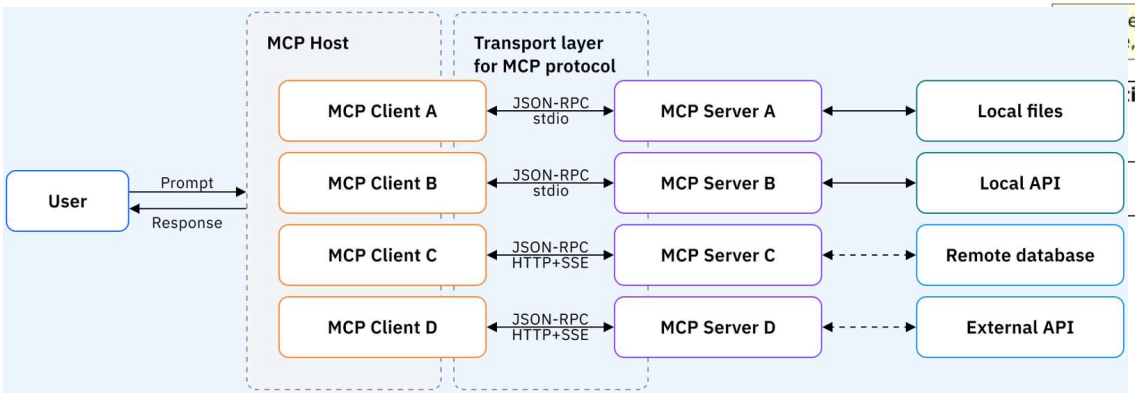
MCP Architecture



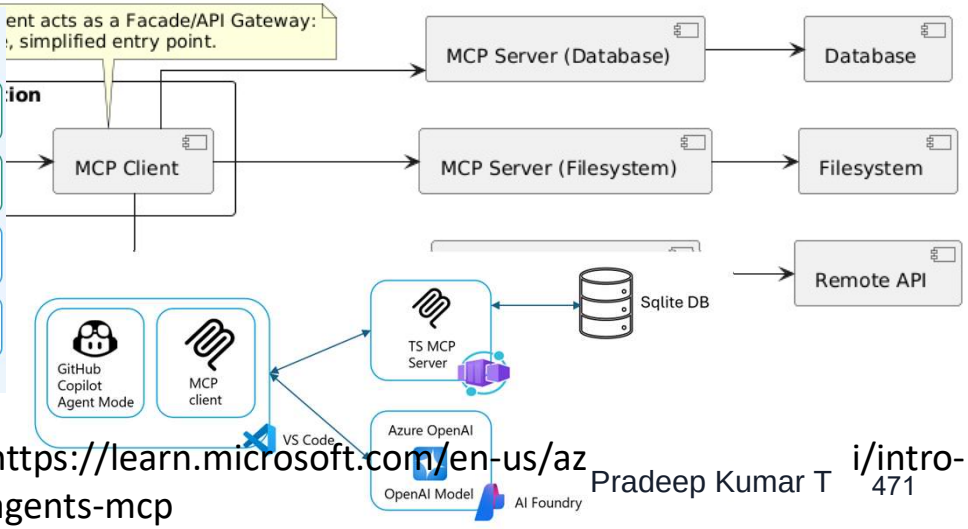
https://www.truefoundry.com/blog/mcp



https://themlarchitect.com/blog/the-architectural-elegance-of-model-context-protocol-mcp/

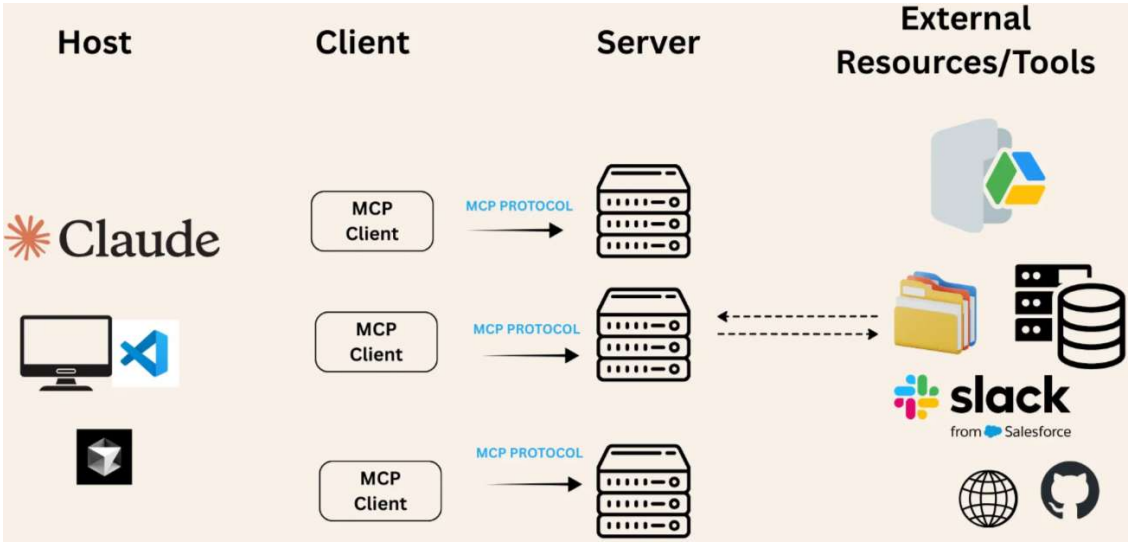


https://www.ibm.com/think/topics/model-context-protocol

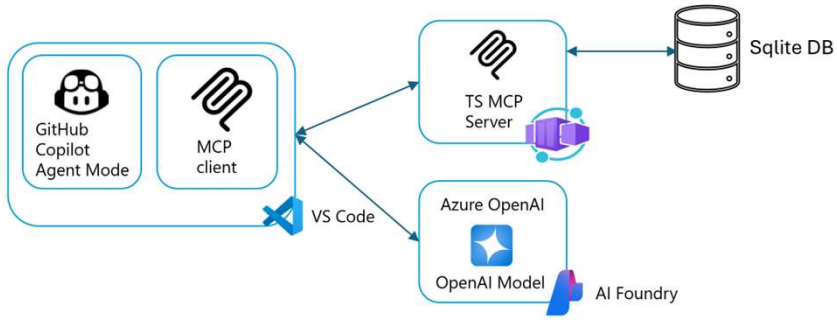


https://learn.microsoft.com/en-us/azure/ai/intro-agents-mcp

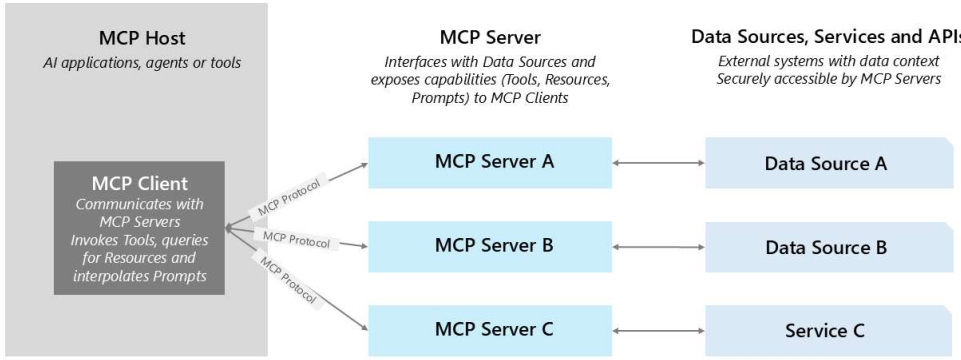
MCP Architecture



https://medium.com/aimonks/mcp-architecture-all-y



https://learn.microsoft.com/en-us/azure/developer/ai/intro-agents-mcp

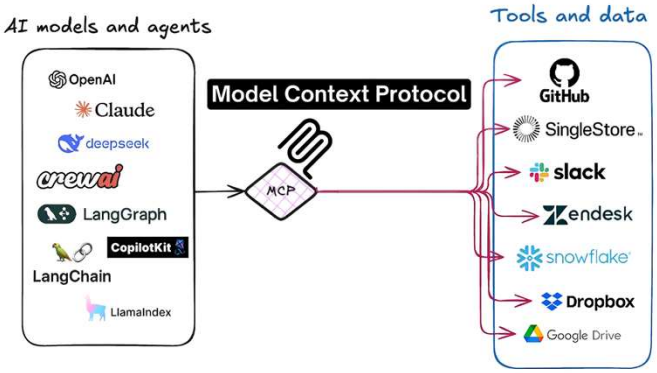


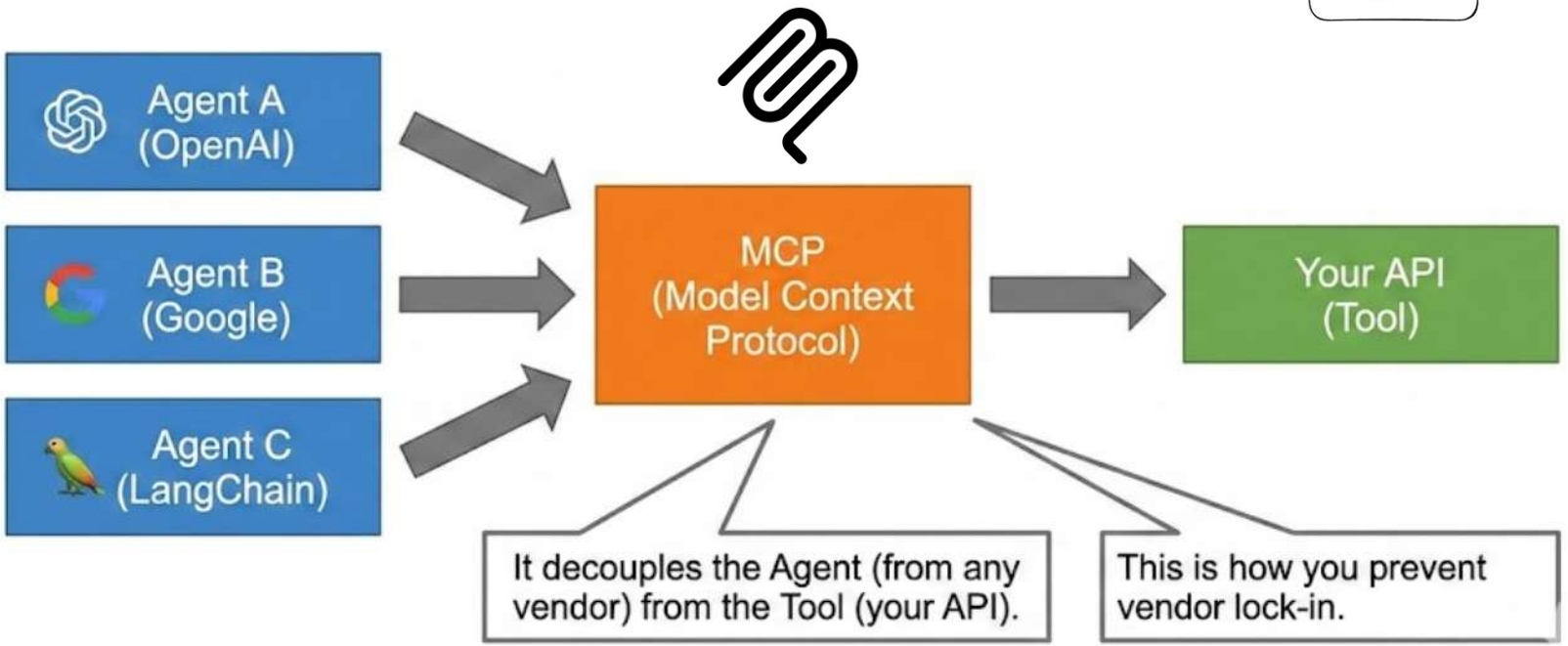
https://learn.microsoft.com/en-us/azure/api-management/mcp-server-overview

Why MCP? The "Decoupling" Standard

MCP is the **OpenAPI / Swagger for Agents**

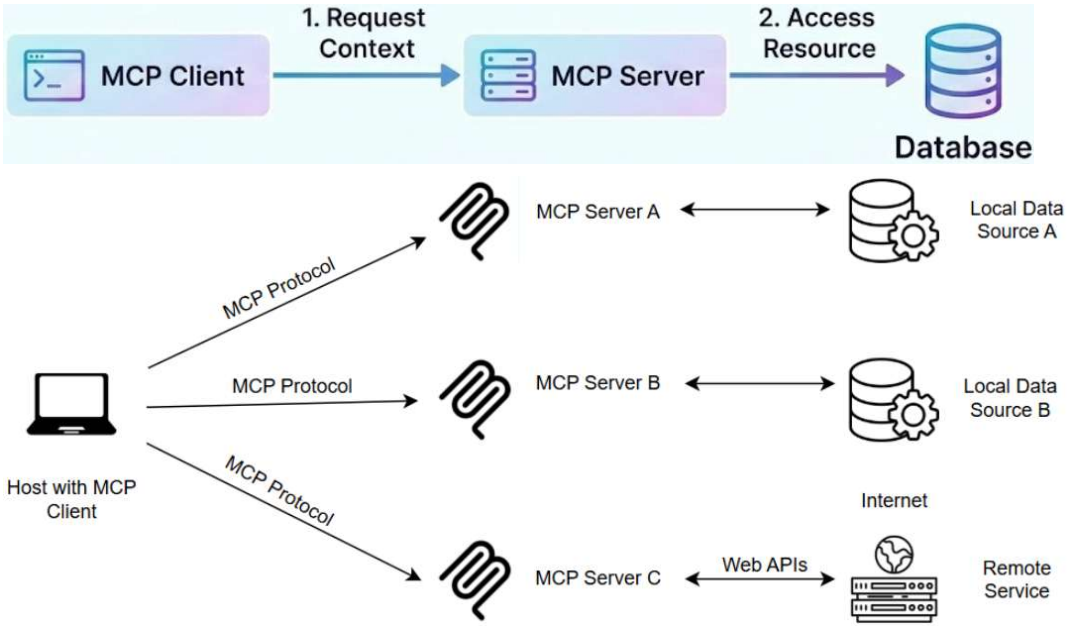
It is the **key** to the **architect's primary goal**: **Decoupling**





Our Journey: From "What" to "Mastery"

1. **The MCP Architecture:** Client, Server and Protocol
2. **MCP Core Concepts:** A deep dive into the full specification
3. **The MCP Lifecycle:** How it Works
4. **Design Patterns:** Advanced designs like the "MCP Gateway



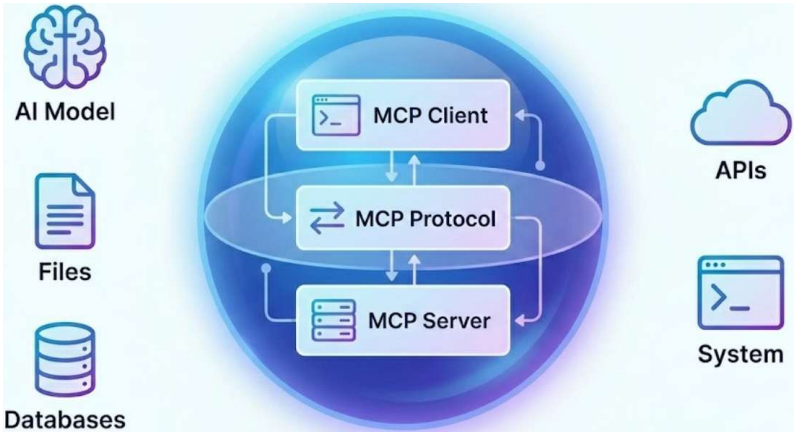
The MCP Architecture: Client, Server and Protocol

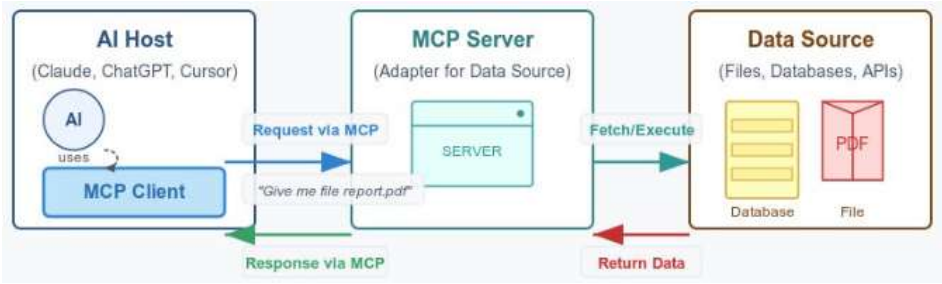
We are going to **step back** and **look** at the "**Big Picture**"

Need to **understand** the **architectural topology** of the **MCP**
Who talks to whom? And where does your data actually live?

If you look at an **MCP** system from **10,000 feet**, it is **not a complex web**
It is a **clean**, **linear pipeline** composed of four **distinct layers**

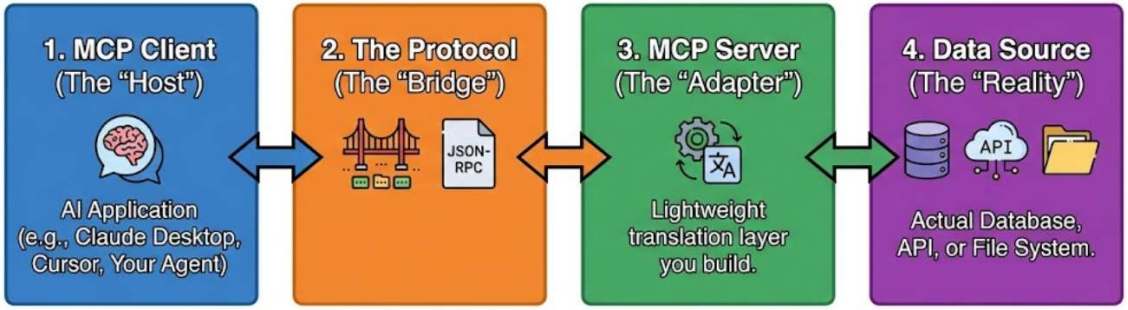
Understanding these **layers** is the **most critical concept for an architect**

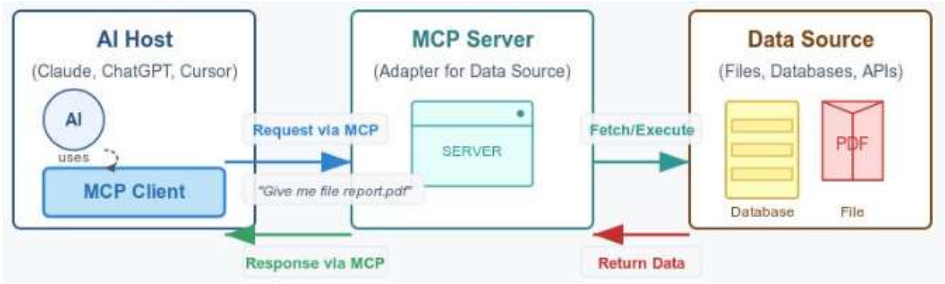




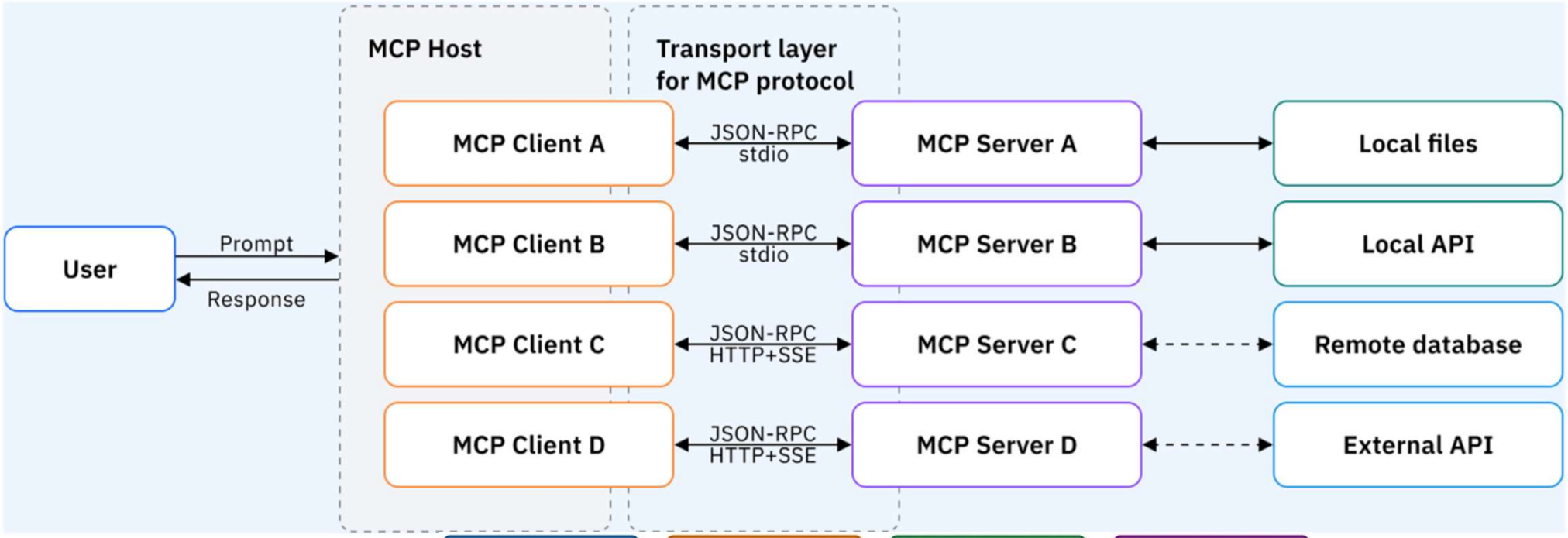
The 4-Layer Topology

1. **MCP Client (The "Host"):** The AI Application (Claude Desktop, Cursor, Your Agent)
2. **The Protocol (The "Bridge"):** The standard rules of communication (JSON-RPC)
3. **MCP Server (The "Adapter"):** The lightweight translation layer you build
4. **Data Source (The "Reality"):** The actual Database, API, or File System

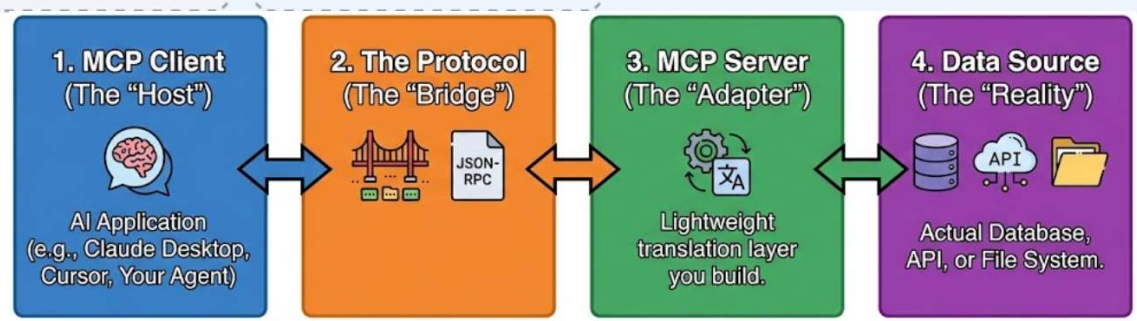




MCP Architecture with the 4-Layer



https://www.ibm.com/think/topics/model-context-protocol

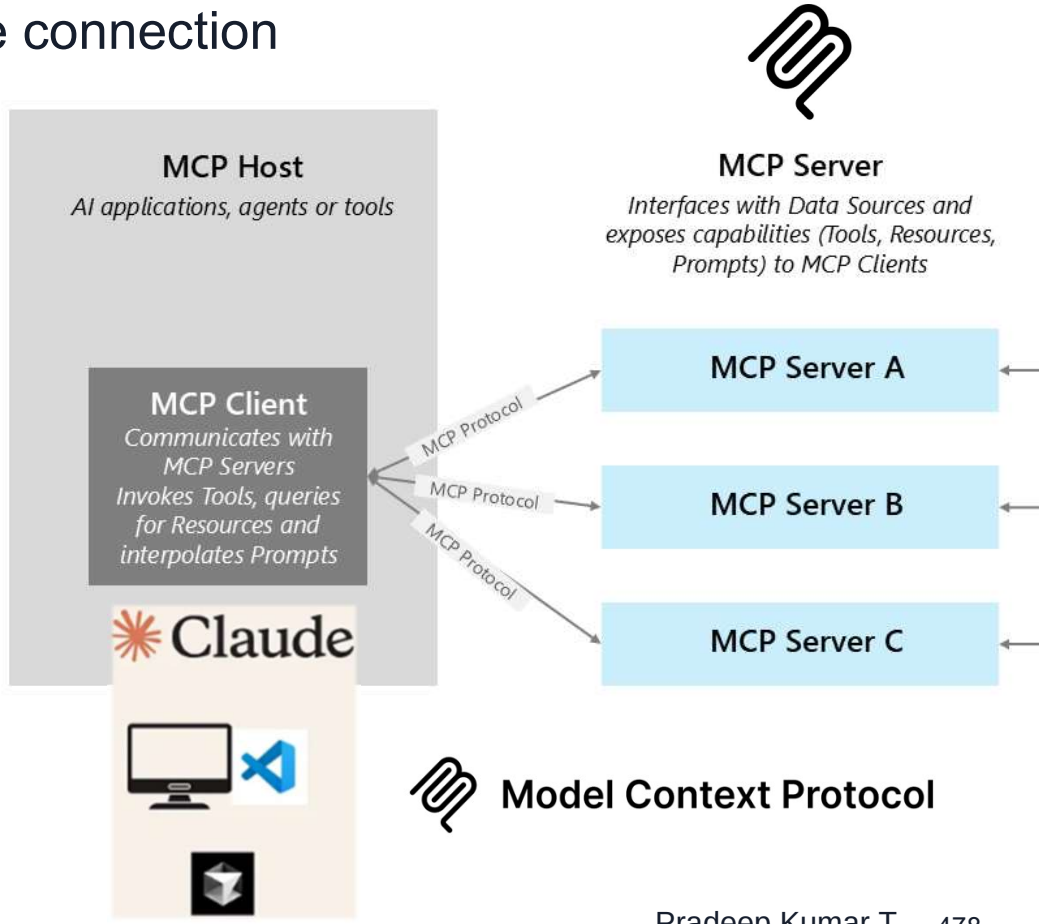


Component 1: The MCP Host / Client

What is it? The application that initiates the connection

Examples:
Claude Desktop App: A consumer client
Cursor IDE: A developer client
Your Custom Agent: An enterprise client

Responsibilities:
It maintains the Connection (1:1 or 1:Many)
It provides the LLM (The Brain)
It decides When to call a tool



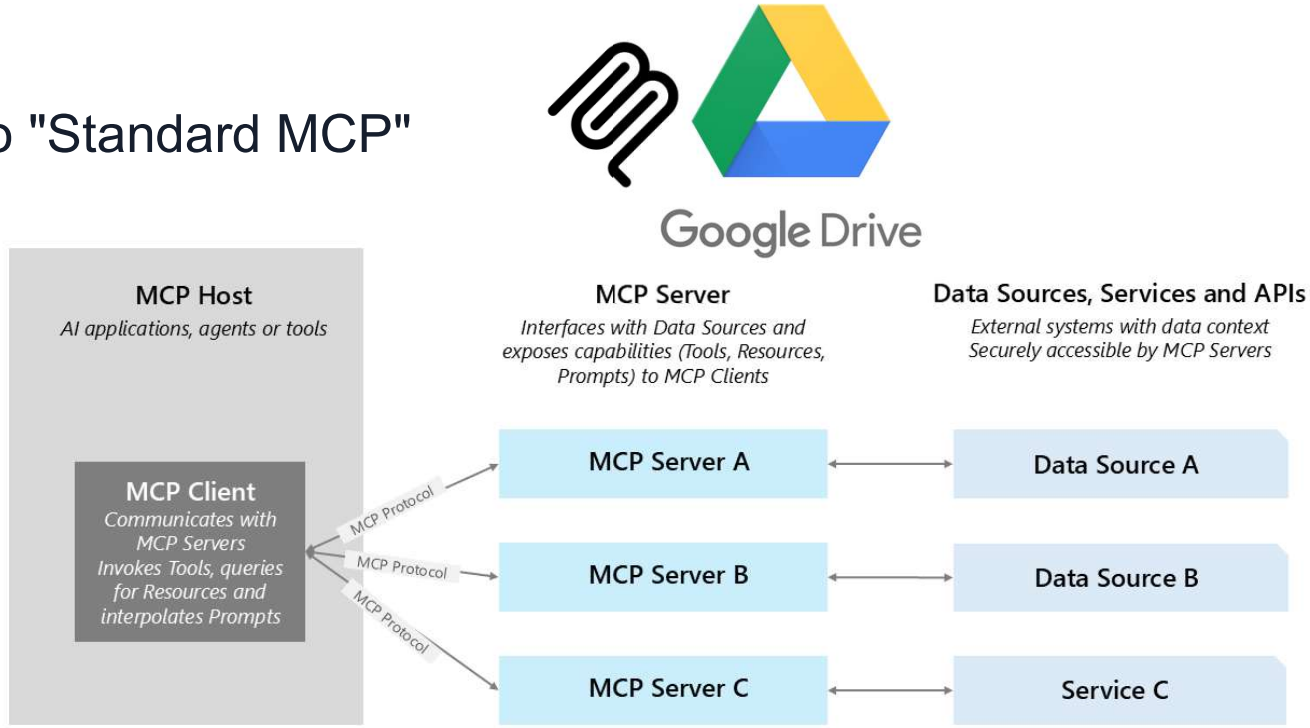
Component 2: The MCP Server (The "Adapter")

What is it? A lightweight service that exposes capabilities

The "Universal Adapter":
It wraps a specific Data Source
It translates "Messy Reality" into "Standard MCP"

Key Characteristic:
It does NOT contain an LLM
It is Deterministic (Code, SQL)

Example:
The "Google Drive MCP Server knows how to use the Google Drive API



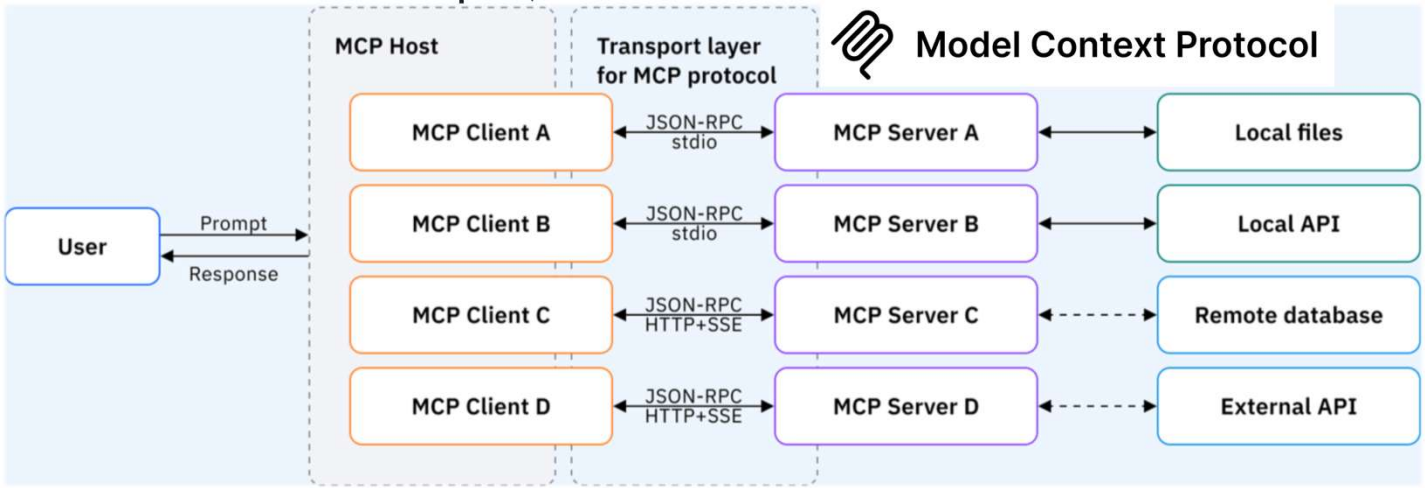
Component 3: The Protocol (The "Language")

What is it? The standard rules for the conversation

Based on: JSON-RPC 2.0

Communication Channels (Transports):
Stdio: Direct process piping (Great for local apps)
SSE (Server-Sent Events): HTTP streaming (Great for remote servers)

The "Contract":
Defines how to send Prompts, Resources and Tools



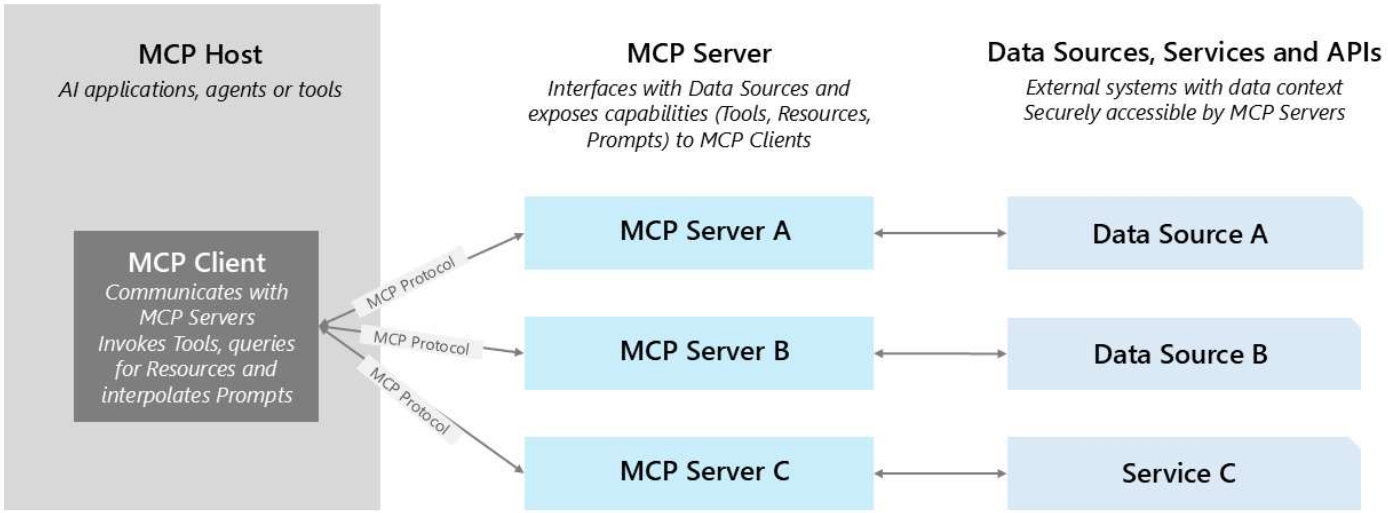
Component 4: The Data Source (The "Reality")

What is it? The system of record

Examples:
A PostgreSQL Database, The Slack API, A Local File System

The Flow:
Client (Ask) -> Protocol -> Server (Translate) -> Data Source (Fetch)

Server holds the API Keys for the Data Source, keeping them safe from the Client



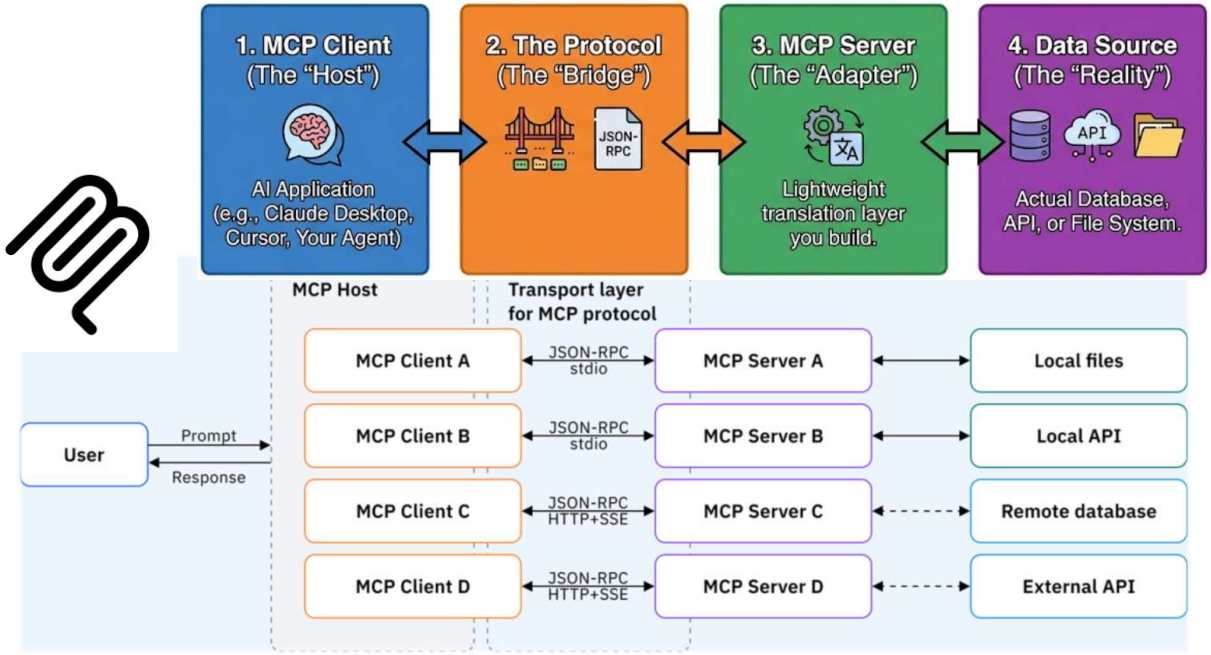


Summary: The MCP Architecture in Action

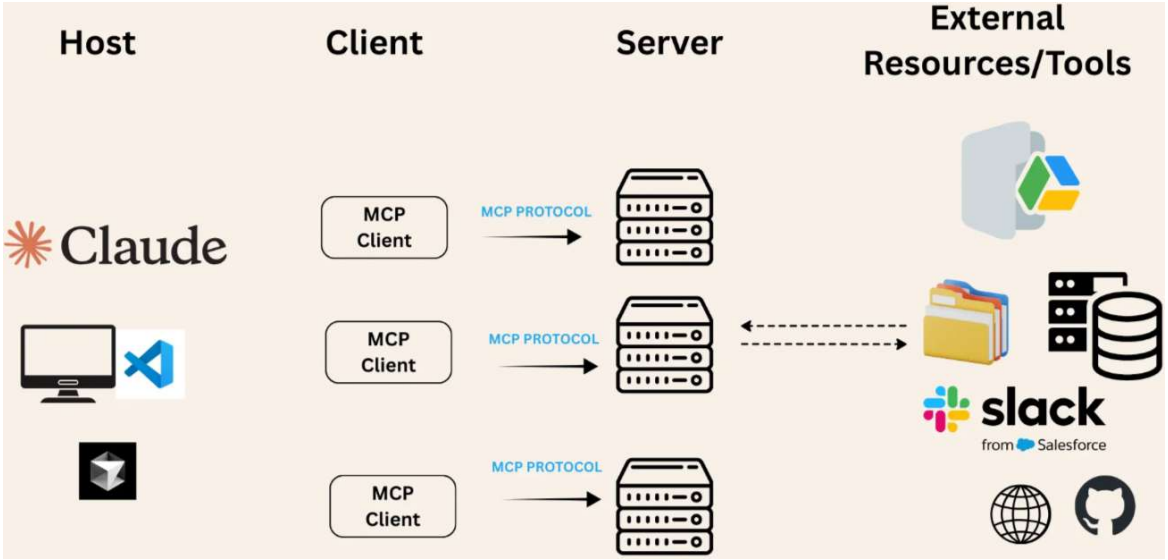
The MCP Architecture is a linear pipeline

The Client provides the Brain and initiates the request
The Protocol provides the standard Language
The Server provides the Hands and translates the request
The Data Source provides the Reality

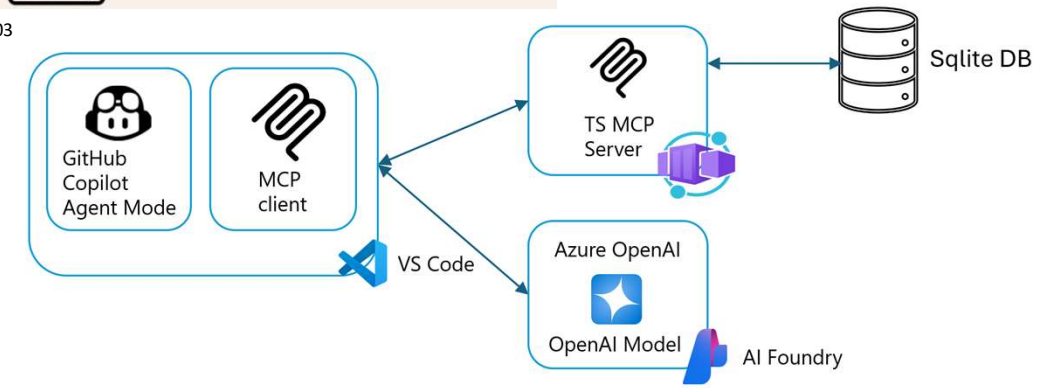
Model Context Protocol



Example - MCP Architecture with the 4-Layer



https://medium.com/aimonks/mcp-architecture-all-you-need-2cafe6c7d803



https://learn.microsoft.com/en-us/azure/developer/ai/intro-agents-mcp

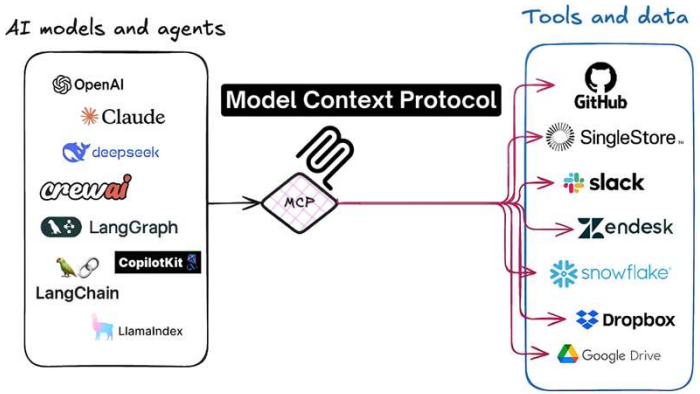
Introduction - MCP Core Concepts & Definitions

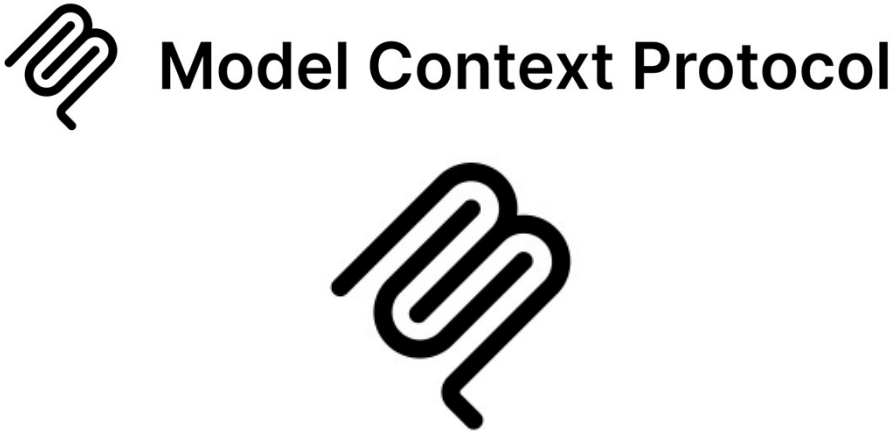
We established **why MCP** is the **critical** protocol for **enterprise integration**

Move from the **high-level 'why'** to the **technical 'what'**

As **architects**, we must **understand** the **blueprint** before we can build the house

We are going to do a **complete**, **field-by-field deep dive** into the **MCP** specification



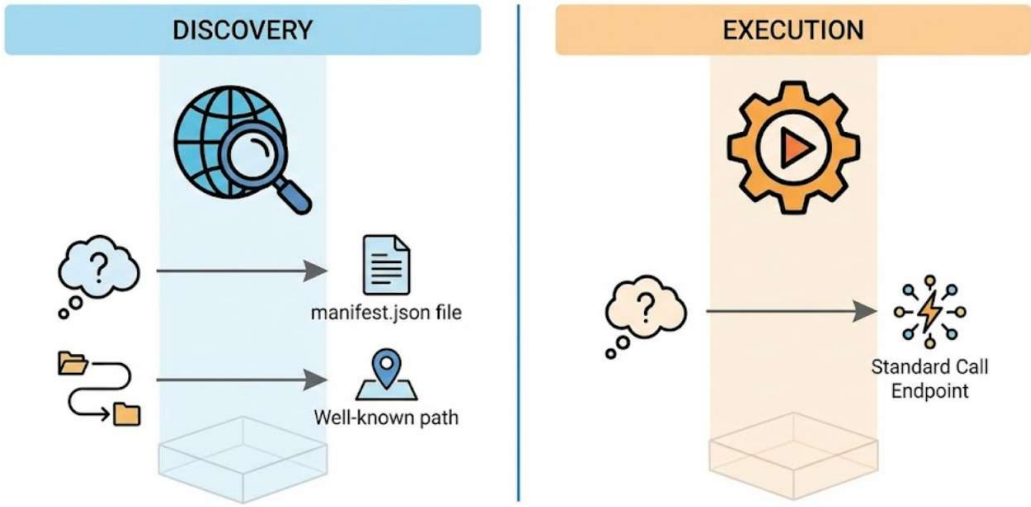


The Two Pillars of the MCP Specification

Architecturally, the entire spec is built on **2 simple pillars:**

Pillar 1: DISCOVERY
Question: What can this tool do?
Mechanism: A manifest.json file
Location: A standard, "well-known" path

Pillar 2: EXECUTION
Question: How do I run this tool?
Mechanism: A standard call endpoint



We'll start with the **foundation of discovery**: **the 'well-known' path**

Core Concept 1: The /.mcp/ Well-Known Path

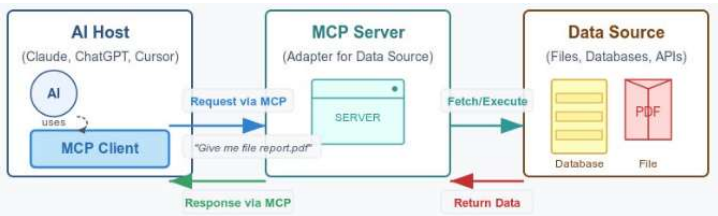
The Path: GET **https://api.my-eshop.com/.mcp/manifest.json**

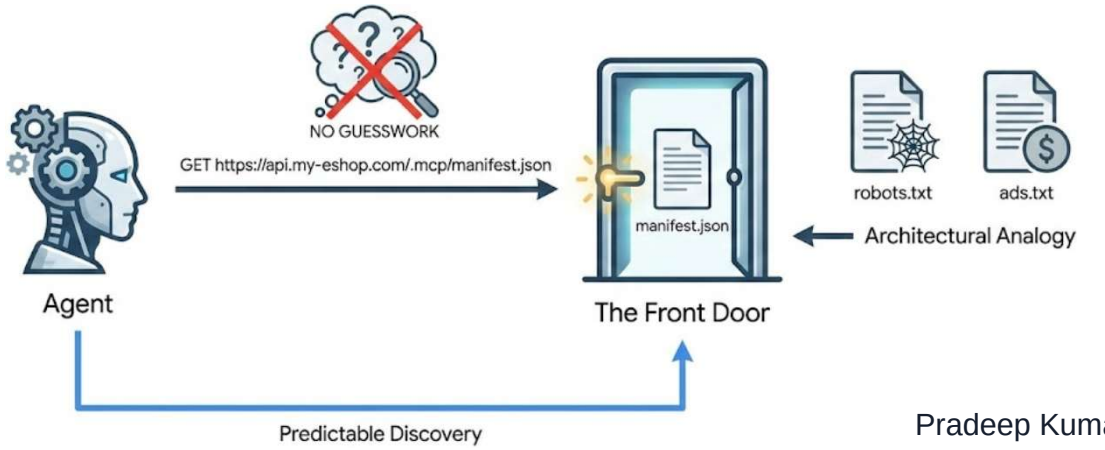
What is it? A **single, standard, predictable location** for an **agent** to **find** the **tool's manifest**

Why? It's the **"front door"** It **removes** all **guesswork**

The **agent doesn't need** to be **told where** the **manifest is**; it knows where to look

This is the **robots.txt** or **ads.txt** for **agents**. A known, standard entry point for **programmatic discovery**

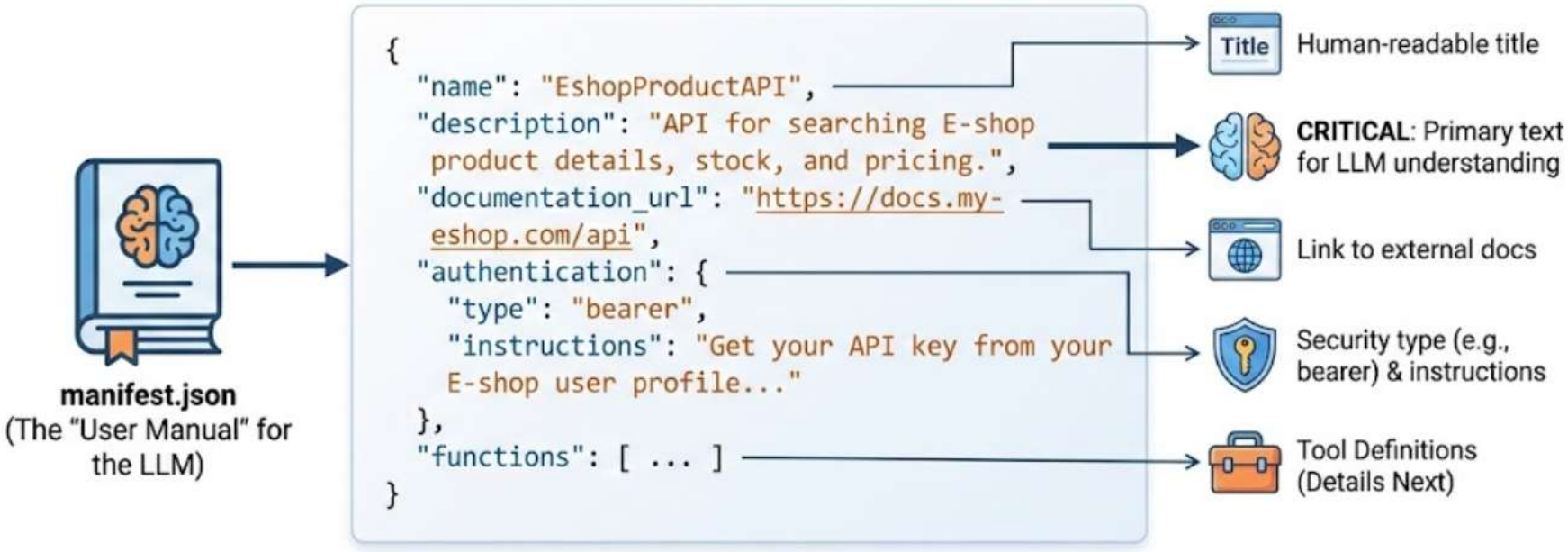




Concept 2: The manifest.json Schema (The "Cover Page")

This is the "**User Manual**" for the LLM

Let's look at the **top-level fields 'cover page':**



Top-level fields define the overall purpose and access.

Concept 2: The manifest.json Schema (The "Functions")

The **functions** array **defines** each **individual tool**

